Python による
はじめての
銀河シミュレーション

テスト粒子で学ぶ重力・軌道・銀河相互作用

見出し統一版 v2.5

馬場淳一（鹿児島大学・国立天文台）

2026 年 10 月 8 日

目次

0 計算環境を準備する **1**
0.1 配布物と必要なソフトウェアを確認する 1
0.2 Python 環境を準備する 2
0.3 Jupyter Notebook を使う 3
0.4 完成版と学生版 Notebook を使い分ける 3
0.5 最初に動作を確認する 4

1 Python で天体の運動を計算する **5**
1.1 数値シミュレーションの目的を考える 5
1.2 中心重力場で Newton の運動方程式を立てる 6
1.3 運動方程式を 1 階の連立方程式にする 7
1.4 Euler 法で時間を進める 8
1.5 leapfrog 法で位置と速度を更新する 9
1.6 4 次 Runge–Kutta 法で運動方程式を解く 11
1.7 Python で軌道を計算する 14
1.8 軌道を図とアニメーションで見る 17
1.9 エネルギーと角運動量を確かめる 20
1.10 時間刻みを変えて誤差を調べる 22
1.11 計算コストと長時間の振る舞いを比較する 24
1.12 本章のまとめ 27
演習 27

2 2 体問題を計算する **29**
2.1 一方を固定する近似を外す 29
2.2 二つの加速度を同じ位置から求める 30
2.3 重心運動と相対運動を分けて考える 31
2.4 初期速度から軌道を選ぶ 33
2.5 Python で初期条件と重力を実装する 34
2.6 KDK で両天体を同時に進める 37
2.7 重心系と相対座標で軌道を見る 39
2.8 2 体系の保存量を確かめる 41
2.9 軌道の時間変化と刻みを確認する 44
2.10 近点距離を指定して接近させる 48
2.11 条件を一つずつ変えて確かめる 49

2.12 本章のまとめ 50
演習 51

3 銀河の中で星を動かす 52
3.1 質点のモデルを広がった銀河へ拡張する 52
3.2 密度から質量と重力を求める 53
3.3 Plummer モデルを導入する 55
3.4 台形則で内側の質量を求める 57
3.5 数値積分を解析式と比べる 59
3.6 二分法で半質量半径を求める 62
3.7 求根の残差と半径の誤差を区別する 66
3.8 Plummer 球の重力で星を動かす 67
3.9 KDK で円軌道と非円軌道を計算する 70
3.10 保存量と時間刻みを確認する 73
3.11 三つの数値計算の役割を区別する 76
3.12 本章のまとめ 77
演習 77

4 テスト粒子で円盤銀河を作る 79
4.1 重力を作る球と表示する円盤を区別する 79
4.2 指数円盤を半径の確率分布へ直す 80
4.3 台形則で累積分布関数を作る 82
4.4 線形補間で分点間の値を求める 84
4.5 一様乱数を円盤の半径へ変換する 86
4.6 円運動速度を与えて円盤を作る 89
4.7 生成した分布を確かめる 91
4.8 1 粒子の式を多数粒子へ拡張する 93
4.9 KDK で円盤を時間発展させる 94
4.10 分布の維持と軌道の精度を確認する 96
4.11 数値実験の条件を一つずつ変える 99
4.12 本章のまとめ 101
演習 101

5 2 つの銀河を接近させる 103
5.1 これまでの計算要素を一つにまとめる 103
5.2 動いている中心から重力を求める 104
5.3 近点距離に合った初期軌道を作る 106
5.4 二つの円盤を配置する 108
5.5 全体の配列を作り、同じ時刻へ進める 111

5.6 標準モデルを計算する . . . 113
5.7 まず銀河中心の軌道を確かめる . . . 115
5.8 潮汐テイルとブリッジを見る . . . 118
5.9 外へ移動した星を数える . . . 122
5.10 星のエネルギーが変化する理由を考える . . . 123
5.11 相互作用計算の時間刻みを確認する . . . 125
5.12 このモデルで分かることと限界を整理する . . . 127
5.13 本章のまとめ . . . 128
演習 . . . 128

6 銀河相互作用を調べる 130
6.1 計算を数値実験として組み立てる . . . 130
6.2 第 5 章の基準と測定方法を引き継ぐ . . . 131
6.3 円盤の回転方向を変える . . . 134
6.4 近点距離を変える . . . 137
6.5 質量比を変える . . . 138
6.6 差が何に由来するかを確かめる . . . 140
6.7 小さな研究としてまとめる . . . 142
6.8 本章のまとめ . . . 143
演習 . . . 143

7 動力学的摩擦で軌道減衰を調べる 145
7.1 近接通過と軌道減衰の違いを考える . . . 145
7.2 Chandrasekhar 公式から摩擦加速度を求める . . . 146
7.3 RK4 で軌道減衰を調べる . . . 149
7.4 数値誤差とモデルの近似を区別する . . . 153
7.5 衛星の周りの星も動かす . . . 154
7.6 本章のまとめ . . . 156
演習 . . . 156

演習の解答・着眼点 158
第 1 章 Python で天体の運動を計算する . . . 158
第 2 章 2 体問題を計算する . . . 159
第 3 章 銀河の中で星を動かす . . . 160
第 4 章 テスト粒子で円盤銀河を作る . . . 161
第 5 章 2 つの銀河を接近させる . . . 162
第 6 章 銀河相互作用を調べる . . . 163
第 7 章 動力学的摩擦と軌道減衰 . . . 163

参考図書 165

第 0 章　計算環境を準備する

本書では、Python と Jupyter Notebook を使って数値計算を行います。本章では、第 1 章以降の Notebook を実行するために必要な計算環境を準備し、Jupyter Notebook の基本的な使い方を確認します。

Python の文法、NumPy、Matplotlib の一般的な使い方をここで解説することはしません。必要なコードは各章の物理・数値計算とともに示します。本章の目的は、**配布された Notebook を自分の計算機で開き、上から実行できる状態にすること**です。

0.1 配布物と必要なソフトウェアを確認する

0.1.1 本書のファイルを確認する

配布物には、本文のほかに完成版と学生版の Notebook があります。章番号 `XX` に対して、基本的な役割は表0.1のとおりです。

表 0.1　配布物の主なファイル。`XX` は `00` から `07` までの章番号です。

ファイル	役割
`notebook/complete/chapXX_complete.ipynb`	完成したコードと標準結果を確認する Notebook
`notebook/student/chapXX_student.ipynb`	TODO を埋めながら実装する Notebook
`code/requirements.txt`	本書の Notebook を実行するために必要な Python パッケージ

Notebook は章ごとに独立して実行できるようにしています。例えば第 4 章を実行するために、第 1–3 章の Notebook を先に実行しておく必要はありません。

0.1.2 必要なソフトウェアを確認する

本書の標準計算には GPU やスーパーコンピュータは必要ありません。一般的なノート PC で実行できる範囲の計算を扱います。必要なソフトウェアは次のとおりです。

- Python 3（3.10 以降を推奨）
- NumPy
- Matplotlib
- JupyterLab または Jupyter Notebook

すでに授業や研究室で指定された Python 環境がある場合は、その環境を使って構いません。新しく環境を作る場合は、次節の手順で配布物に含まれる `code/requirements.txt` を使います。

0.2 Python 環境を準備する

0.2.1 Python が使えるか確認する

まず端末を開き、Python が利用できるか確認します。macOS や Linux では、環境によって `python` または `python3` を使います。

端末で実行

```
python --version
```

版番号が表示されれば Python を実行できます。コマンドが見つからない場合は、授業で指定された方法で Python を導入してください。

0.2.2 仮想環境を作る

他の授業や研究で使う Python 環境と混ざらないよう、必要に応じて仮想環境を作ります。配布物の最上位フォルダで、例えば次を実行します。[7]

端末で実行

```
python -m venv .venv
```

仮想環境を有効にするコマンドは使用するシェルによって異なります。

環境	有効化するコマンド
macOS / Linux（bash, zsh）	`source .venv/bin/activate`
macOS / Linux（tcsh）	`source .venv/bin/activate.csh`
Windows PowerShell	`.venv\Scripts\Activate.ps1`

仮想環境を有効にした後、必要なパッケージをインストールします。

端末で実行

```
python -m pip install -r code/requirements.txt
```

すでに必要なパッケージが導入された環境を使う場合は、この操作を毎回行う必要はありません。

0.3 Jupyter Notebook を使う

0.3.1 Jupyter を起動する

配布物の最上位フォルダで、JupyterLab を使う場合は次を実行します。[8]

端末で実行

```
jupyter lab
```

Jupyter Notebook を使う場合は `jupyter notebook` とします。ブラウザが開いたら、`notebook/complete/` または `notebook/student/` から使う章の Notebook を開きます。

0.3.2 セルを実行する

Notebook には、文章を表示する Markdown セルと、Python を実行する Code セルがあります。Code セルを選び、通常は `Shift+Enter` で実行します。

Notebook は、セルを**実行した順序**に変数や関数の状態を保持します。そのため、標準結果を再現するときは、上から順にセルを実行してください。途中のセルだけを先に実行すると、画面に見えているコードと実際の計算状態が一致しないことがあります。

0.3.3 最初から計算し直す

結果が標準例と合わないときや、パラメータを変更して計算をやり直すときは、カーネルを再起動してから全セルを上から実行します。JupyterLab や Jupyter Notebook には、カーネルの再起動と全セル実行に相当する操作があります。メニュー名は版によって多少異なります。[8]

エラーが出た場合は、まずエラー表示の最後の数行を確認してください。`NameError`、`ValueError`、`ModuleNotFoundError` など、エラーの種類と原因を示す情報が表示されます。パッケージが見つからない場合は、Jupyter が使用している Python 環境と、パッケージをインストールした環境が同じか確認します。

0.4 完成版と学生版 Notebook を使い分ける

本書では、完成版と学生版の Notebook を次のように使い分けます。

完成版 Notebook 本文で説明した標準計算を実行し、コードと結果を確認します。

学生版 Notebook 一部のコードが TODO になっています。本文を参照して実装し、完成版と比較します。

最初は、次の順序で進めることを想定しています。

1. 本文で問題設定、数式、計算手順を確認します。

2. 完成版 Notebook を上から実行し、標準結果を再現します。
3. 学生版 Notebook の TODO に取り組みます。
4. 各章の演習では、指定された条件を変更して結果を比較します。

Notebook を編集するときは、元の配布ファイルを残して別名で保存しておくと、標準設定へ戻りやすくなります。

0.5 最初に動作を確認する

最後に、完成版の第 0 章 Notebook を開き、次のセルを実行します。

```
import sys
import numpy as np
import matplotlib

print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__)
print("Matplotlib:", matplotlib.__version__)
```

Python、NumPy、Matplotlib の版番号が表示され、エラーが出なければ準備は完了です。Jupyter 上でこのセルを実行できたことを確認したら、第 1 章へ進んでください。

第 1 章　Python で天体の運動を計算する

銀河相互作用の計算を始める前に、重力を受ける 1 個の粒子を動かしてみましょう。この章では、原点に固定した中心質量の周りを回る粒子を考えます。運動方程式を Python で計算できる形に直し、重力計算の標準法である leapfrog 法（KDK）を実装します。一般の常微分方程式に使える 4 次 Runge–Kutta 法（RK4）も学び、同じ軌道で両者を比較します。最後に、解析解、保存量、時間刻み、力評価回数から、計算結果と手法の使い分けを考えます。

この章で学ぶこと

- 位置・速度・加速度を用いて、運動方程式を 1 階の連立常微分方程式として書けます。
- leapfrog の KDK 更新と RK4 の 4 段階を説明し、両方を Python で実装できます。
- 軌道を図とアニメーションで表示し、同じ時刻の解析解と比較できます。
- エネルギー・角運動量・時間刻みを使って、計算結果を確認できます。
- 精度だけでなく力評価回数も比べ、重力計算と速度依存力で手法を使い分けられます。

1.1 数値シミュレーションの目的を考える

物体に働く力が分かれば、Newton の運動方程式を書くことができます。しかし、方程式を書いただけでは、しばらく後に物体がどこにいるかは分かりません。初めの位置と速度を与え、運動方程式を解く必要があります。このように、ある時刻の状態から後の状態を求める問題を**初期値問題**と呼びます。

ゼミでは、中心力のもとでの運動や保存量を学んできました。本章では、その知識を使って「粒子の位置と速度を、少し後の時刻へ更新する」という計算を作ります。これを繰り返すと、粒子の軌道が得られます。ここでいう**時間積分**とは、速度を積分して位置を、加速度を積分して速度を求め、状態を時間発展させることです。

最初の問題には、解析的に答えが分かる円軌道を選びます。答えが分かっている問題を数値的に解くのは、遠回りではありません。自分のコードが、正しい向きの力を計算し、適切な時刻に正しい場所へ粒子を運べるかを確認できるからです。これができてから、解析的に解くことが難しい銀河相互作用へ進みます。

本章の中心質量は、銀河全体の最終モデルではありません。第 3 章では質量に広がりを持たせた Plummer ポテンシャルへ、第 5 章では 2 つの移動する Plummer

ポテンシャルへ進みます。**重力場の式は変わっても、加速度を計算して leapfrog で進める基本手順は再利用します。**

1.2 中心重力場で Newton の運動方程式を立てる

1.2.1 計算するモデルを決める

原点に質量 M の天体を固定し、その周りを質量 m の粒子が運動するとします。粒子が中心天体に及ぼす重力は無視します。これは、例えば $m \ll M$ で中心天体の動きが小さい場合を考える近似です。粒子を単に質量ゼロと置いて運動方程式を消すのではなく、粒子から中心への反作用を計算しない、という意味です。

座標系は原点に中心天体を置いた慣性系とし、運動は x–y 平面内に限定します。重力以外の力、中心天体の大きさ、他の天体の影響は含めません。この中心質量は時間とともに変化せず、原点も動きません。2 つの天体がともに動く問題は、第 2 章で扱います。

粒子の位置、速度、加速度を、それぞれ

$$\boldsymbol{r} = (x, y), \qquad \boldsymbol{v} = \frac{d\boldsymbol{r}}{dt} = (v_x, v_y), \qquad \boldsymbol{a} = \frac{d\boldsymbol{v}}{dt} = (a_x, a_y) \tag{1.1}$$

と書きます。太字はベクトル、細字の $r = |\boldsymbol{r}| = \sqrt{x^2 + y^2}$ は距離を表します。

中心質量が粒子を引く力は、中心からの距離の 2 乗に反比例し、原点を向きます。大きさ GMm/r^2 に原点方向の単位ベクトル $-\boldsymbol{r}/r$ を掛けると、Newton の運動方程式は

$$m\frac{d^2\boldsymbol{r}}{dt^2} = -\frac{GMm}{r^3}\boldsymbol{r} \tag{1.2}$$

となります。両辺の m を消去し、$\mu \equiv GM$ と置くと、加速度は

$$\boxed{\boldsymbol{a}(\boldsymbol{r}) = -\frac{\mu}{r^3}\boldsymbol{r}} \tag{1.3}$$

です。与えられた重力場中では、粒子の軌道は m に依存しません。

1.2.2 ベクトルの式を成分で読む

式(1.3)を、そのままコードに移せる形にします。x 方向と y 方向の加速度は

$$a_x = -\frac{\mu x}{(x^2 + y^2)^{3/2}}, \qquad a_y = -\frac{\mu y}{(x^2 + y^2)^{3/2}} \tag{1.4}$$

です。例えば $x > 0,\ y = 0$ なら $a_x < 0,\ a_y = 0$ です。力が正しく原点へ向いていることを、このような簡単な配置で確認できます。

ここで分母が r^3 であることと、重力の大きさが r^{-2} に比例することは矛盾しません。分子に長さ r のベクトルがあるため、式全体の大きさは μ/r^2 です。

よくある失敗：重力の向きと原点

位置ベクトルを原点から粒子へ向けて定義したので、加速度には負号が必要です。また、点質量の重力は $r = 0$ で定義できません。本章では原点へ衝突する軌道を扱わず、分母へ根拠なく小さな数を足して発散を隠すこともしません。力の式を変えれば、別のモデルになるためです。

1.3 運動方程式を 1 階の連立方程式にする

1.3.1 位置と速度を一つの状態として扱う

数値積分では、2 階微分を含む式(1.2)を直接扱う代わりに、位置と速度を独立した変数として持ちます。位置の変化率は速度、速度の変化率は加速度ですから、

$$\frac{d\boldsymbol{r}}{dt} = \boldsymbol{v}, \qquad \frac{d\boldsymbol{v}}{dt} = \boldsymbol{a}(\boldsymbol{r}) \tag{1.5}$$

と書けます。もとの運動方程式と内容は同じで、未知変数を増やして微分の階数を下げただけです。

位置と速度をまとめた**状態ベクトル**を $\boldsymbol{y} = (\boldsymbol{r}, \boldsymbol{v})$ とします。縦に成分を並べれば、この問題は

$$\boldsymbol{y} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}, \qquad \frac{d\boldsymbol{y}}{dt} = \boldsymbol{f}(t, \boldsymbol{y}) = \begin{pmatrix} v_x \\ v_y \\ -\mu x/(x^2+y^2)^{3/2} \\ -\mu y/(x^2+y^2)^{3/2} \end{pmatrix} \tag{1.6}$$

です。太字の $\boldsymbol{y}$ は状態ベクトル、細字の y は位置の y 成分です。また、状態ベクトルの位置成分と速度成分は同じ単位ではありません。右辺の上 2 成分は速度、下 2 成分は加速度を表します。

今回の重力場には、時刻 t が直接は現れません。それでも $\boldsymbol{f}(t, \boldsymbol{y})$ という書き方にしておくと、後の章で重力場が時間変化する場合にも同じ形式を使えます。速度に依存する加速度が加わっても、状態ベクトルの構成は変わりません。一方、重力だけを leapfrog で解くときは、位置と速度を分けて持ち、順序を決めて交互に更新します。

1.3.2 単位と円軌道の初期条件を決める

計算を始める前に、単位をそろえます。本章では距離を kpc、時間を Gyr、質量を $M_\odot$ とします。**コード内部の速度は** $\mathrm{kpc\,Gyr^{-1}}$ **です**。速度を天文学でよく使う $\mathrm{km\,s^{-1}}$ で表示するときだけ、

$$1\,\mathrm{km\,s^{-1}} \simeq 1.022712165\,\mathrm{kpc\,Gyr^{-1}} \tag{1.7}$$

を使います。重力定数も同じ単位系へ変換します。教材では近似値

$$G = 4.30091 \times 10^{-6}\,\mathrm{kpc}\,(\mathrm{km\,s^{-1}})^2\,M_\odot^{-1}$$

を固定して使い、解析解にも同じ値を使います。このため、以下の精度試験は物理定数の測定精度ではなく、数値解法の誤差を調べています。

半径 r_0 の円運動では、重力加速度の大きさと向心加速度が等しくなります。この条件から、円運動速度 v_c、角速度 Ω、周期 T は

$$\frac{v_c^2}{r_0} = \frac{\mu}{r_0^2}, \qquad v_c = \sqrt{\frac{\mu}{r_0}}, \qquad \Omega = \sqrt{\frac{\mu}{r_0^3}}, \qquad T = \frac{2\pi}{\Omega} \tag{1.8}$$

となります。この円軌道を、数値計算の基準にします。

本章では、$t = 0$ で粒子を正の x 軸上へ置き、正の y 方向へ速度を与えます。

$$\boldsymbol{r}(0) = (r_0, 0), \qquad \boldsymbol{v}(0) = (0, v_c) \tag{1.9}$$

とすれば、反時計回りの円軌道です。採用する条件を表1.1にまとめます。数値はこの練習のための設定であり、実在する特定の銀河を表してはいません。

表 1.1　本章の標準計算条件。最終時刻は、初期条件から求めた解析的な 1 周期です。

量	設定または計算値
中心質量 M	$10^{11} M_\odot$
初期半径 r_0	$10\,\mathrm{kpc}$
初期速度の大きさ v_c	$207.386\,\mathrm{km\,s^{-1}}$
周期 T	$0.296242\,\mathrm{Gyr}$
最終時刻 t_{end}	T
ステップ数 N_{step}	200
時間刻み $\Delta t = T/N_{\mathrm{step}}$	$1.4812\,\mathrm{Myr}$

よくある失敗：表示する速度と計算する速度

$\mathrm{km\,s^{-1}}$ の数値を、そのまま Gyr 単位の時間刻みに掛けないようにします。位置の更新で使う $\boldsymbol{v}\Delta t$ が kpc になるよう、内部速度は $\mathrm{kpc\,Gyr^{-1}}$ にそろえます。

1.4 Euler 法で時間を進める

計算機では、時刻を $t_0, t_1, t_2, \ldots$ と区切って状態を求めます。本章では一定の時間刻み Δt を使い、$t_n = n\Delta t$ とします。数値計算で得た状態を $\boldsymbol{y}^n$ と書きます。上付きの n は時刻の番号であり、べき乗ではありません。

時間積分の考え方を、最も簡単な Euler 法で確認しましょう。小さな時間 Δt の間では、現在の変化率を一定と近似できます。Taylor 展開の最初の 2 項だけを残すと、

$$\boldsymbol{y}(t_n + \Delta t) = \boldsymbol{y}(t_n) + \Delta t\, \boldsymbol{f}(t_n, \boldsymbol{y}(t_n)) + O(\Delta t^2) \tag{1.10}$$

です。誤差項を省略して、次の更新式を得ます。

$$\boldsymbol{y}^{n+1} = \boldsymbol{y}^n + \Delta t\, \boldsymbol{f}(t_n, \boldsymbol{y}^n) \tag{1.11}$$

位置と速度に分ければ、$\boldsymbol{r}^{n+1} = \boldsymbol{r}^n + \Delta t \boldsymbol{v}^n$、$\boldsymbol{v}^{n+1} = \boldsymbol{v}^n + \Delta t \boldsymbol{a}(\boldsymbol{r}^n)$ です。両方の更新に、更新前の同じ状態を使います。

粒子は重力によって進行方向を変えますが、Euler 法の位置更新は、その時点の接線方向へ直進させる計算です。時間刻みが有限であれば、この近似にはずれが生じます。そのずれを次の初期値として計算を続けるため、誤差が蓄積します。円軌道が数値的には円でなくなる例を、後で実際に見ます。

Euler 法は本章では導入と比較のためだけに使います。次は、位置と速度の更新を対称に組み合わせる leapfrog 法へ進みます。

1.5 leapfrog 法で位置と速度を更新する

1.5.1 位置の更新と速度の更新を分ける

重力が粒子の速度を変え、速度が粒子の位置を変えます。加速度が位置だけで決まる式(1.5)では、この二つの働きを分けて計算できます。位置を固定して加速度を与える操作を **kick**、速度を固定して位置を動かす操作を **drift** と呼びます。ここでいう kick は計算上の速度更新であり、粒子が物理的な衝撃を受けるという意味ではありません。

長さ h の区間について、それぞれの操作は

$$\begin{aligned} \text{kick:} \quad & \boldsymbol{r}' = \boldsymbol{r}, \qquad \boldsymbol{v}' = \boldsymbol{v} + h\boldsymbol{a}(\boldsymbol{r}), \\ \text{drift:} \quad & \boldsymbol{r}' = \boldsymbol{r} + h\boldsymbol{v}, \qquad \boldsymbol{v}' = \boldsymbol{v} \end{aligned} \tag{1.12}$$

です。各操作では一方の変数を固定しているため、その部分の運動は正確に計算できます。ただし、実際の運動では位置も速度も同時に変わるので、二つの操作を組み合わせた結果は近似です。Euler 法のように片側の時刻だけを使うのではなく、前後を対称に組み合わせると精度が上がります。

1.5.2 kick–drift–kick の 3 段階で更新する

本書では、半ステップの kick、1 ステップの drift、半ステップの kick という順序を使います。これが **kick–drift–kick（KDK）形式の leapfrog 法**です。[5] 時刻 t_n の位置と速度を $\boldsymbol{r}^n, \boldsymbol{v}^n$、その位置での加速度を $\boldsymbol{a}^n = \boldsymbol{a}(\boldsymbol{r}^n)$ とします。更新式を実行する順に書くと、

$$\boldsymbol{v}^{n+1/2} = \boldsymbol{v}^n + \frac{\Delta t}{2}\boldsymbol{a}^n, \tag{1.13}$$

$$\boldsymbol{r}^{n+1} = \boldsymbol{r}^n + \Delta t \boldsymbol{v}^{n+1/2}, \tag{1.14}$$

$$\boldsymbol{a}^{n+1} = \boldsymbol{a}(\boldsymbol{r}^{n+1}), \tag{1.15}$$

$$\boldsymbol{v}^{n+1} = \boldsymbol{v}^{n+1/2} + \frac{\Delta t}{2}\boldsymbol{a}^{n+1} \tag{1.16}$$

です。最初の kick には古い位置での加速度を、最後の kick には**新しい位置での加速度**を使います。粒子を動かした後に重力の向きと強さを計算し直すことが重要です。

上付きの $n+1/2$ は、更新の途中で使う速度を表します。これは中間時刻の速度の近似ですが、中間時刻における厳密な解ではありません。第 1 の kick だけで、粒子の位置まで半ステップ進んだわけでもありません。位置と速度を交互に進める様子から、leapfrog（かえる跳び）という名前が付いています。KDK では最後の kick まで終えると、$\boldsymbol{r}^{n+1}$ と $\boldsymbol{v}^{n+1}$ が同じ整数時刻にそろいます。

図1.1を上から下へ読み、更新する量と保持する量を確認してください。最後に求めた加速度を次のステップへ渡す関係も示しています。

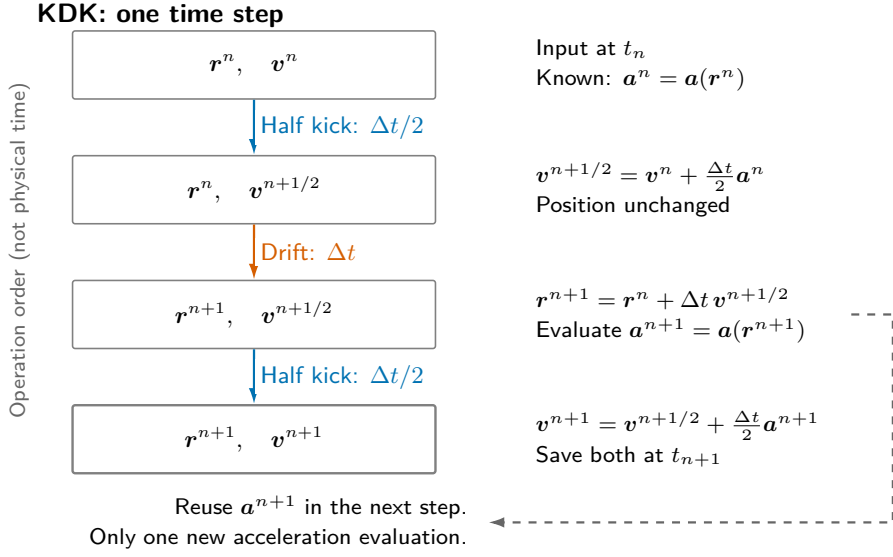


図 1.1　KDK の 1 ステップで受け渡す位置・速度・加速度。縦方向は計算操作の順序であり、物理時間軸ではありません。最後の半 kick を終えた位置と速度を整数時刻の状態として保存し、新しい加速度を次のステップへ引き継ぎます。

よくある失敗：半ステップの速度をそのまま保存しない

本書の履歴には、同じ整数時刻の位置と速度を保存します。エネルギーを計算するときに $\boldsymbol{r}^{n+1}$ と $\boldsymbol{v}^{n+1/2}$ を組み合わせると、異なる時刻の量を使ってしまいます。最後の kick を終えた $\boldsymbol{v}^{n+1}$ を用いてください。

1.5.3 2 次精度になる理由を考える

位置の更新式に最初の kick を代入すると、

$$\boldsymbol{r}^{n+1} = \boldsymbol{r}^n + \Delta t \boldsymbol{v}^n + \frac{\Delta t^2}{2}\boldsymbol{a}^n \tag{1.17}$$

となります。これは位置の Taylor 展開の 2 次までの項と一致します。Euler 法で省いていた、加速度によって軌道が曲がる効果が含まれています。

二つの kick をまとめると、速度は

$$\boldsymbol{v}^{n+1} = \boldsymbol{v}^{n} + \frac{\Delta t}{2}\left(\boldsymbol{a}^{n} + \boldsymbol{a}^{n+1}\right) \tag{1.18}$$

です。区間の両端の加速度を平均して使います。滑らかな問題では、この平均による速度更新も Taylor 展開の 2 次までと一致します。正しい初期値から 1 ステップだけ進めたときの誤差は $O(\Delta t^3)$ です。同じ有限の時刻まで計算した解の誤差は、十分小さい刻みでは $O(\Delta t^2)$ になります。この意味で leapfrog 法は **2 次精度**です。刻みを半分にすると、解の誤差はおよそ 1/4 になると期待できます。

1.5.4 時間反転と保存系での性質を理解する

KDK は、前後が同じ形になるように操作を並べています。式(1.16)から逆順に、符号を反転した $-\Delta t$ で操作すれば、もとの状態へ戻ります。固定刻みで、丸め誤差を除けば成り立つこの性質を**時間反転対称性**と呼びます。

また、本章のような静的なポテンシャル中の運動では、KDK は**シンプレクティック積分法**です。これは Hamilton 系の位相空間の幾何学的な構造を保つ方法という意味です。本書ではその数学的な証明には立ち入りませんが、**次数だけでなく、もとの運動方程式の構造を保つことにも価値がある**と理解してください。[5]

シンプレクティックであることは、物理的なエネルギーを毎ステップ厳密に保存することとは違います。固定刻みで軌道を十分に分解できている滑らかな保存系では、エネルギー誤差が一定の範囲で振動するなど、長時間積分で有利な振る舞いが期待できます。ただし、刻みが大きすぎる計算まで正しくなるわけではなく、軌道上の位置のずれも蓄積し得ます。後で、実際の数値解を用いて確かめます。

1.5.5 加速度を再利用する

式(1.15)で求めた $\boldsymbol{a}^{n+1}$ は、次のステップの最初の kick でも使います。この値を保存して渡せば、同じ位置で加速度を計算し直す必要はありません。最初の $\boldsymbol{a}^{0}$ を 1 回求めた後は、**1 ステップにつき新しい力評価は 1 回**です。N_{step} ステップでは、全体で $N_{\mathrm{step}}+1$ 回になります。2 回の kick を行うことと、2 回の新しい力評価を行うことは同じではありません。

N 体計算では、多数の粒子が作る重力場を求める処理が高価です。加速度を再利用できることは、保存系に適した性質とともに、重力計算で leapfrog 法を使う重要な理由です。[5] Python でも、この利点が実際に得られる形で実装します。

1.6 4 次 Runge–Kutta 法で運動方程式を解く

leapfrog は、加速度が位置から決まる重力計算を効率よく進める方法です。一方、速度に依存する力を扱うときには、状態全体の変化率を使う方法が便利です。本書では、そのための主要手法として RK4 も本文で学びます。第 6 章の動力学的摩擦を含む発展計算で、この方法を使います。

1.6.1 1 ステップの途中の状態を予測する

状態の変化を正確に書けば、次の時刻の状態は

$$\boldsymbol{y}(t_n + \Delta t) = \boldsymbol{y}(t_n) + \int_{t_n}^{t_n+\Delta t} \boldsymbol{f}(t, \boldsymbol{y}(t))\, dt \tag{1.19}$$

です。しかし、積分区間の途中の $\boldsymbol{y}(t)$ は、まだ分かっていません。RK4 では途中の状態を順に予測し、そこで右辺を評価します。高階の時間微分を自分で計算する代わりに、右辺関数 $\boldsymbol{f}$ を 4 回使います。[3]

本書で使う古典的 RK4 の 4 つの段階は、

$$\begin{aligned}
\boldsymbol{k}_1 &= \boldsymbol{f}(t_n, \boldsymbol{y}^n), \\
\boldsymbol{k}_2 &= \boldsymbol{f}\left(t_n + \frac{\Delta t}{2}, \boldsymbol{y}^n + \frac{\Delta t}{2}\boldsymbol{k}_1\right), \\
\boldsymbol{k}_3 &= \boldsymbol{f}\left(t_n + \frac{\Delta t}{2}, \boldsymbol{y}^n + \frac{\Delta t}{2}\boldsymbol{k}_2\right), \\
\boldsymbol{k}_4 &= \boldsymbol{f}\left(t_n + \Delta t, \boldsymbol{y}^n + \Delta t \boldsymbol{k}_3\right)
\end{aligned} \tag{1.20}$$

です。得られた 4 つの傾きを重み付きで平均し、

$$\boxed{\boldsymbol{y}^{n+1} = \boldsymbol{y}^n + \frac{\Delta t}{6}\left(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4\right)} \tag{1.21}$$

で 1 ステップを進めます。ここでは $\boldsymbol{k}_j$ を**状態の時間変化率**として定義し、Δt を含めていません。

各段階を、物理的な計算として読み直してみましょう。$\boldsymbol{k}_1$ は現在の状態での変化率です。それを使って半ステップ後の状態を予測し、$\boldsymbol{k}_2$ を求めます。次に $\boldsymbol{k}_2$ を使い、**もう一度、現在の状態 $\boldsymbol{y}^n$ から**半ステップ後を予測して、$\boldsymbol{k}_3$ を求めます。最後に $\boldsymbol{k}_3$ を使って、現在から 1 ステップ後の状態を予測し、$\boldsymbol{k}_4$ を求めます。

$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ の評価時刻は同じですが、使う予測状態は異なります。また、途中で得る状態は、その時刻の確定した数値解ではありません。1 ステップ全体を進めるための、一時的な予測値です。

図1.2では、予測状態と、そこで計算する傾きを分けて示しています。特に第 2・第 3 段階で、時刻は同じでも状態が異なる点を見てください。

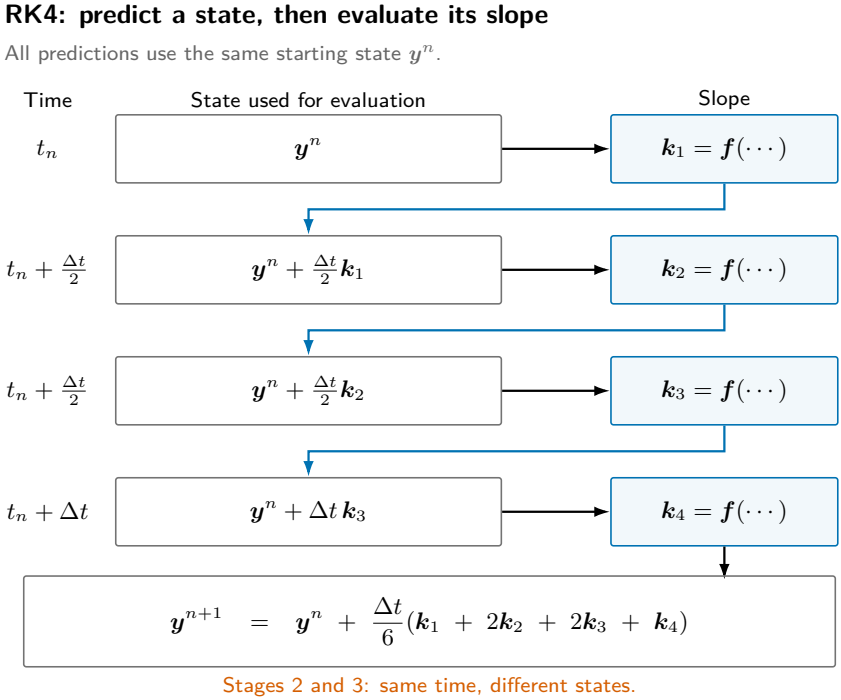


図 1.2　RK4 の予測状態と傾きの依存関係。各予測は同じ基準状態 $\boldsymbol{y}^n$ から作ります。$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ は評価時刻が同じでも、使う予測状態は異なります。矢印は計算の依存関係で、時間を 4 等分する図ではありません。

1.6.2 同じ予測状態から位置と速度の変化率を評価する

重力問題では、状態の上半分が位置、下半分が速度です。例えば $\boldsymbol{k}_1$ は $(\boldsymbol{v}^n, \boldsymbol{a}(\boldsymbol{r}^n))$ に対応します。第 2 段階で右辺を評価する位置と速度は、

$$\boldsymbol{r}_{(2)} = \boldsymbol{r}^n + \frac{\Delta t}{2}\boldsymbol{v}^n, \qquad \boldsymbol{v}_{(2)} = \boldsymbol{v}^n + \frac{\Delta t}{2}\boldsymbol{a}(\boldsymbol{r}^n) \tag{1.22}$$

です。そして、$\boldsymbol{k}_2$ の位置成分の変化率は $\boldsymbol{v}_{(2)}$、速度成分の変化率は $\boldsymbol{a}(\boldsymbol{r}_{(2)})$ です。位置だけを予測して、速度は初めのまま使う計算ではありません。

式(1.21)の係数 $1, 2, 2, 1$ は、単に「中点を重くする」という思いつきで決めたものではありません。途中の予測状態の作り方と合わせて、真の解の Taylor 展開と 4 次まで一致するように選ばれています。一般の場合の次数条件の導出は、数値計算の専門書に譲ります。[3]

1.6.3 4 次精度の意味を確認する

正確な初期値から 1 ステップだけ進めたとき、十分滑らかな問題での RK4 の誤差は $O(\Delta t^5)$ です。一方、同じ有限の時間まで多数のステップを積み重ねたときの解の誤差は、十分小さな時間刻みでは $O(\Delta t^4)$ になります。この後者を指して、4 次精度と呼びます。

したがって、初期条件と最終時刻を固定して時間刻みを半分にすると、解の誤差はおよそ 1/16 になることが期待できます。これは誤差の主要な項が Δt^4 に比例する範囲での性質です。刻みが大きすぎる場合や、丸め誤差が目立つほど小さい場合まで保証する関係ではありません。また、エネルギーのような特定の量の誤差が、必ず同じべきで変わるとも限りません。

よくある失敗：RK4 の段階と時間刻みを混同する

本書の $\boldsymbol{k}_j$ に Δt は含まれません。別の資料で $\Delta t \boldsymbol{f}$ を $\boldsymbol{k}_j$ と定義している場合、その更新式をそのまま混ぜると時間刻みを二重に掛けてしまいます。また、4 段階で位置と速度を同じ予測状態から評価することを守ります。

1.6.4 速度に依存する力も同じ形で扱う

加速度が $\boldsymbol{a}(\boldsymbol{r}, \boldsymbol{v}, t)$ となっても、RK4 では各段階の位置と速度から右辺を評価します。状態ベクトルと 4 段階の更新式を変更する必要はありません。一方、ここまでの KDK では、位置を固定すれば kick の間の加速度も固定できることを使いました。速度依存力では kick の間にも加速度が変わるので、同じ陽的な更新をそのまま使うことはできません。

これは「leapfrog の考え方では速度依存力を一切扱えない」「RK4 だけが使える」という意味ではありません。分割法や陰的な速度更新を組み合わせる方法もありますが、本書では複数の方法を追加せず、RK4 へ切り替えます。RK4 を採用する理由は、速度依存力を含む運動方程式を、学習済みの同じ枠組みで扱えることです。

もう少し詳しく：本文で詳しく扱わない時間積分法

RK2 は、少ない段階で 2 次精度を得る Runge–Kutta 法です。**適応刻み RK 法**は、推定誤差に応じて時間刻みを変えます。**Bulirsch–Stoer 法**は、複数の刻みで計算した解から外挿を行う方法です。また、kick と drift の組合せから**高次のシンプレクティック法**を作ることもできます。本書では KDK と RK4 の基本的な性質に集中するため、高次の方法の詳細は数値計算法の参考図書に譲ります。高次のシンプレクティック法については文献 [5] も参照してください。

1.7 Python で軌道を計算する

1.7.1 状態を配列へ対応させる

まず、`import numpy as np`と`import matplotlib.pyplot as plt`を実行します。本章では 1 個の粒子でも、位置と速度をそれぞれ形が`(1, 2)`の配列に入れます。最初の軸が粒子、2 番目の軸が x, y の成分です。後の章で粒子数を N へ増やしても、`(N, 2)`という同じ約束を使います。

次のコードで、単位変換、初期条件、計算時間を設定します。leapfrog では`pos`と`vel`を分けて更新します。RK4 では`np.column_stack`で位置と速度を横に

並べ、[x, y, vx, vy]を 1 行に持つ形(1, 4)のstateを使います。これはデータの持ち方の違いであり、計算する運動方程式は同じです。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
mass = 1.0e11                        # 中心質量 [Msun]
r0 = 10.0                            # 初期半径 [kpc]
mu = G * mass
vc = np.sqrt(mu / r0)                # 円運動速度 [kpc/Gyr]
period = 2.0 * np.pi * r0 / vc       # 軌道周期 [Gyr]
pos0 = np.array([[r0, 0.0]])         # shape = (1, 2)
vel0 = np.array([[0.0, vc]])         # shape = (1, 2)
state0 = np.column_stack((pos0, vel0))  # RK4 用：[x, y, vx, vy]
n_steps = 200
t_end = period
dt = t_end / n_steps
```

np.array([[r0, 0.0]])の角括弧が二重なのは、粒子の行と成分の列を作るためです。pos0[0]で最初の粒子の位置ベクトルを取り出せます。式(1.4)を、まず 1 粒子の成分のまま計算します。

```
x, y = pos0[0]
r2 = x**2 + y**2
ax = -mu * x / r2**1.5
ay = -mu * y / r2**1.5
print(f"ax = {ax:.6f}, ay = {ay:.6f} [kpc/Gyr^2]")
```

初期位置では、出力される a_x は $-4.4985 \times 10^3\,\mathrm{kpc\,Gyr^{-2}}$、$a_y$ は 0 です。原点へ向かう加速度になっていることを確認してください。

1.7.2 加速度関数と右辺関数を分ける

今計算した式を、位置から加速度を返す関数にします。入力と返り値は、どちらも形が(1, 2)です。この関数は本章の 1 粒子専用で、多数粒子への一般化は第 4 章で行います。

```
def acceleration_kepler(pos, mu):
    """1 粒子の重力加速度。本文 eq:ch1-acceleration-components。"""
    x, y = pos[0]
    r2 = x**2 + y**2
    if r2 <= 0.0:
        raise ValueError("Point-mass gravity is undefined at r = 0.")
    ax = -mu * x / r2**1.5
    ay = -mu * y / r2**1.5
    return np.array([[ax, ay]])
```

leapfrog には、この加速度関数を渡します。Euler 法と RK4 には、位置と速度をまとめた状態の変化率を返す関数を渡します。同じ加速度関数を使い、式(1.6)の右辺を組み立てます。

```
def rhs_kepler(t, state, mu):
    """ 状態 [x, y, vx, vy] の右辺。本文 eq:ch1-state-ode。"""
    pos, vel = state[:, :2], state[:, 2:]
    acc = acceleration_kepler(pos, mu)
    return np.column_stack((vel, acc))
```

`state[:, :2]`が位置、`state[:, 2:]`が速度です。右辺の先頭 2 成分は速度、後半 2 成分は加速度です。`t`は本章では使いませんが、時間依存する問題にも使えるように残します。

1.7.3 KDK の 1 ステップを実装する

式(1.13)から式(1.16)までを、その順に書きます。引数`acc`は、現在の`pos`に対応する計算済みの加速度です。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, mu):
    """KDK で 1 ステップ。acc は現在位置での加速度です。"""
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, mu)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

`vel_half`は中間速度、`pos_new`と`vel_new`は次の整数時刻の状態です。返り値の`acc_new`を次の呼び出しへ渡します。関数内部の`acceleration(...)`は 1 回だけです。入力配列を直接上書きせず、新しい配列を返すため、比較計算でも初期値を保てます。

確認：最後の kick で`acc_new`ではなく古い`acc`を使うと、新しい位置で変化した重力を反映できません。また、`acc`と`pos`の対応がずれると、意図した KDK ではなくなります。

1.7.4 Euler 法と RK4 の 1 ステップを実装する

Euler 法は、状態全体に更新前の変化率を掛けるだけです。

```
def euler_step(rhs, t, state, dt, mu):
    return state + dt * rhs(t, state, mu)
```

RK4 は、式(1.20)と式(1.21)を順に実装します。

```
def rk4_step(rhs, t, state, dt, mu):
    """RK4 の 4 段階。本文 eq:ch1-rk4-stages, eq:ch1-rk4-update。"""
    k1 = rhs(t, state, mu)
    k2 = rhs(t + 0.5 * dt, state + 0.5 * dt * k1, mu)
    k3 = rhs(t + 0.5 * dt, state + 0.5 * dt * k2, mu)
    k4 = rhs(t + dt, state + dt * k3, mu)
    return state + dt * (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
```

`state + 0.5 * dt * k1`には、予測位置と予測速度が一緒に入ります。RK4 の段階全体で同じ予測状態を使う点は、位置と速度を分けて更新する KDK との違いです。

1.7.5 更新を繰り返して履歴を残す

KDK では、最初の加速度をループの外で 1 回だけ計算します。その後は、位置・速度・加速度を受け取り、次のステップへ渡します。

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, mu):
    """ 初回の力を計算し、以降は前ステップの acc_new を使います。"""
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, mu)  # 初回だけ、ループの前に計算します。
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, mu)
        history[n + 1] = np.column_stack((pos, vel))
        if not np.all(np.isfinite(history[n + 1])):
            raise FloatingPointError("Non-finite state; check the time step.")
    return times, history
```

`prepare_history`は、正のステップ数、有限の最終時刻、初期配列の形を確認する補助関数です。初期状態のコピー、$\Delta t = t_{\mathrm{end}}/N_{\mathrm{step}}$、$t_n = n\Delta t$、保存用配列を用意します。その実装は Notebook に示します。`history`の形は`(n_steps + 1, 1, 4)`です。初期状態を含むので、出力数はステップ数より 1 つ多くなります。KDK の履歴にも、**位置と最後の kick 後の速度**を同じ行へ保存します。

Euler 法と RK4 では、一般の右辺と 1 ステップ関数を渡して計算します。

```
def integrate(rhs, stepper, state0, t_end, n_steps, mu):
    """Euler 法・RK4 用。右辺関数を指定して等間隔で積分します。"""
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    for n in range(n_steps):
        state = stepper(rhs, times[n], state, dt, mu)
        if not np.all(np.isfinite(state)):
            raise FloatingPointError("Non-finite state; check the time step.")
        history[n + 1] = state
    return times, history
```

同じ初期条件から、3 つの方法を実行します。本章では変数`history`に標準法の leapfrog の結果を入れ、RK4 と Euler 法には別の変数を使います。

```
times, history = integrate_leapfrog(acceleration_kepler, state0, t_end, n_steps, mu)
_, history_euler = integrate(rhs_kepler, euler_step, state0, t_end, n_steps, mu)
_, history_rk4 = integrate(rhs_kepler, rk4_step, state0, t_end, n_steps, mu)
```

1.8 軌道を図とアニメーションで見る

円軌道を初期条件に選んだので、時刻 t における正解は、初期条件から独立に計算できます。角速度 Ω を使うと、位置と速度は

$$\begin{aligned} \boldsymbol{r}_{\mathrm{exact}}(t) &= r_0\bigl(\cos\Omega t,\, \sin\Omega t\bigr), \\ \boldsymbol{v}_{\mathrm{exact}}(t) &= r_0\Omega\bigl(-\sin\Omega t,\, \cos\Omega t\bigr) \end{aligned} \tag{1.23}$$

です。数値解と解析解を、必ず**同じ時刻**で比べます。単にどちらも円を描いているだけでは、1 周する速さが正しいかは分かりません。

解析解を計算するコードは次のようになります。ここでpos_historyの最初の軸は、粒子数ではなく、保存した時刻を表しています。

```
def exact_circle(times, r0, mu):
    omega = np.sqrt(mu / r0**3)
    phase = omega * np.asarray(times)
    pos_exact = np.column_stack((r0 * np.cos(phase), r0 * np.sin(phase)))
    vel_exact = np.column_stack((-r0 * omega * np.sin(phase), r0 * omega *
    ↪  np.cos(phase)))
    return pos_exact, vel_exact

pos_exact, vel_exact = exact_circle(times, r0, mu)
pos_history = history[:, 0, :2]  # 第 1 軸は出力時刻です。
vel_history = history[:, 0, 2:]
```

図1.3は、標準の 200 ステップで計算した leapfrog の軌道です。$0, T/4, T/2, 3T/4$ の位置も示しました。解析解と数値解は、この図の大きさではほぼ重なります。図を描くときはax.set_aspect("equal")を使い、縦横で同じ距離が同じ長さに見えるようにします。そうしないと、円を楕円のように表示してしまいます。

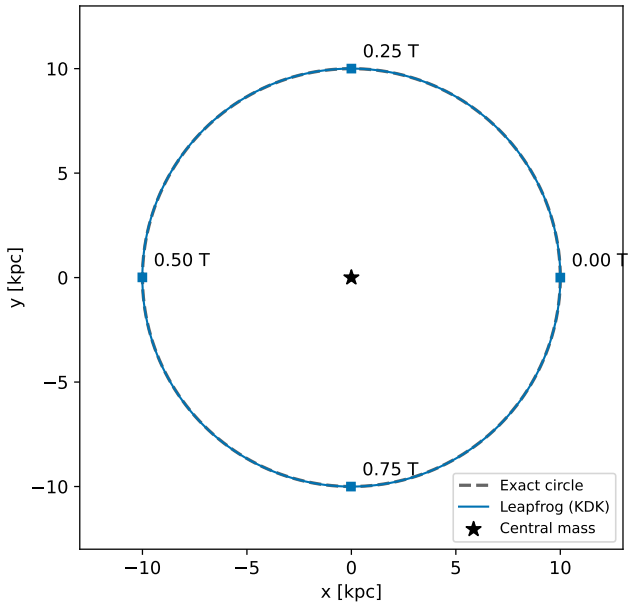


図 1.3　中心質量の周りを回る 1 粒子の軌道。標準条件の leapfrog（KDK）を青実線、解析的な円を灰色破線で示します。四角は指定した時刻の数値解、星印は固定した中心質量です。計算は反時計回りです。

描画の中心部分を次に示します。save_figureは Notebook に用意した保存用関数で、同じ図を PDF と PNG に保存して表示します。

```
fig, ax = plt.subplots(figsize=(5.6, 5.2), layout="constrained")
ax.plot(pos_exact[:, 0], pos_exact[:, 1], color=GRAY, ls="--", lw=2, label="Exact
↪  circle")
ax.plot(pos_history[:, 0], pos_history[:, 1], color=BLUE, lw=1.3, label="Leapfrog (KDK)")
ax.scatter([0.0], [0.0], marker="*", s=90, color="black", label="Central mass")
sample_ids = np.rint(np.arange(4) * n_steps / 4).astype(int)
for n in sample_ids:
    ax.plot(*pos_history[n], marker="s", color=BLUE, ms=5)
    label = f"{times[n] / period:.2f} T"
    ax.annotate(label, pos_history[n], xytext=(7, 7), textcoords="offset points")
ax.set(xlabel="x [kpc]", ylabel="y [kpc]", xlim=(-13, 13), ylim=(-13, 13))
ax.set_aspect("equal")
ax.legend(loc="lower right", fontsize=9)
save_figure(fig, "ch01_orbit")
```

同じ条件で Euler 法を使った結果を、図1.4に重ねました。本来は 1 周期で初期位置へ戻りますが、Euler 法では軌道が外側へずれ、1 周期後の位置も大きく異なります。これは別の初期速度を与えた結果ではなく、**同じ運動方程式と同じ初期条件を、異なる近似で時間積分した結果**です。

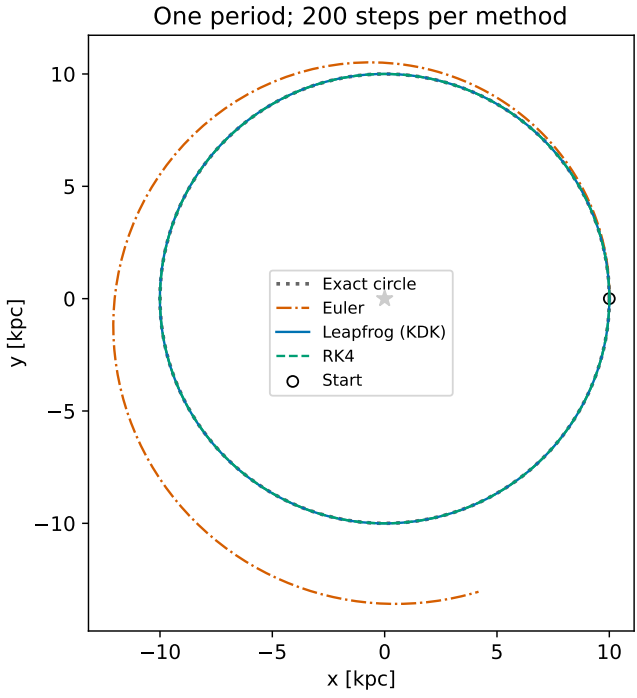


図 1.4　同じ時間刻み $\Delta t = T/200$ での Euler 法・leapfrog・RK4 の比較。計算時間は解析的な 1 周期です。朱色の一点鎖線が Euler 法、青実線が leapfrog、緑破線が RK4、灰色点線が解析解です。同じステップ数でも力評価回数は異なり、leapfrog と RK4 の軌道はこの縮尺ではほぼ重なります。

アニメーションと静止画　Notebook では、保存済みの`history`から粒子の位置を順に読み、アニメーションを作ります。表示のたびに運動方程式を解き直す必要はありません。再生間隔を決める`interval`は画面上の時間であり、物理的な時間刻み`dt`ではありません。再生速度を変えても計算した軌道は変わりません。本文では図1.3のように重要な時刻の位置を静止画としても残し、動画を見なくても結果が分かるようにします。

1.9 エネルギーと角運動量を確かめる

1.9.1 このモデルで保存される量を確認する

軌道の見た目だけでは、小さな誤差は分かりません。そこで、ゼミで学んだ保存量を数値解から計算します。中心質量は固定され、ポテンシャルは時間に依存しません。粒子の単位質量当たりのエネルギーは

$$\varepsilon = \frac{1}{2}(v_x^2 + v_y^2) - \frac{\mu}{r} \tag{1.24}$$

です。速度変化による運動エネルギーの変化率と、位置変化によるポテンシャルエネルギーの変化率が打ち消し合い、

$$\frac{d\varepsilon}{dt} = \boldsymbol{v}\cdot\boldsymbol{a} + \frac{\mu}{r^3}\boldsymbol{r}\cdot\boldsymbol{v} = 0 \tag{1.25}$$

となります。

また、重力は原点からの位置ベクトルと平行で、原点の周りのトルクは 0 です。運動面に垂直な単位質量当たりの角運動量は

$$\ell_z = x v_y - y v_x, \qquad \frac{d\ell_z}{dt} = x a_y - y a_x = 0 \tag{1.26}$$

となります。ここで確認するのは粒子の**比**エネルギーと**比**角運動量であり、粒子の質量 m は掛けません。これらは、中心力中の軌道を理解する基本的な保存量です。[1]

円軌道の初期値では、

$$\varepsilon_0 = -\frac{\mu}{2r_0}, \qquad \ell_{z,0} = r_0 v_c = \sqrt{\mu r_0} \tag{1.27}$$

です。負のエネルギーは、この粒子が中心質量に束縛されていることを表します。今回は ε_0 も $\ell_{z,0}$ も 0 ではないので、それぞれの初期値の絶対値で誤差を規格化できます。

1.9.2 数値解から保存量を計算する

履歴から位置と速度を取り出し、式(1.24)と式(1.26)を評価します。`np.sum(..., axis=1)` は、ここでは各時刻の 2 成分を足す操作です。例えば `np.sum(vel**2, axis=1)` は、各時刻の $v_x^2 + v_y^2$ を返します。

```python
def diagnostics(history, mu):
    pos = history[:, 0, :2]
    vel = history[:, 0, 2:]
    radius = np.sqrt(np.sum(pos**2, axis=1))
    energy = 0.5 * np.sum(vel**2, axis=1) - mu / radius
    angular_momentum = pos[:, 0] * vel[:, 1] - pos[:, 1] * vel[:, 0]
    return radius, energy, angular_momentum
```

時刻 t_n での誤差の大きさを、

$$e_\varepsilon^n = \frac{|\varepsilon^n - \varepsilon^0|}{|\varepsilon^0|}, \qquad e_\ell^n = \frac{|\ell_z^n - \ell_z^0|}{|\ell_z^0|} \tag{1.28}$$

と定義します。保存した全時刻での最大値を、それぞれ e_ε, e_ℓ とします。図1.5は、エネルギー誤差を時間の関数として示したものです。$t = 0$ では定義上誤差が 0 です。対数軸では 0 を表示できないため、その点だけを描いていません。保存した数値を、小さな正数へ置き換えているわけではありません。

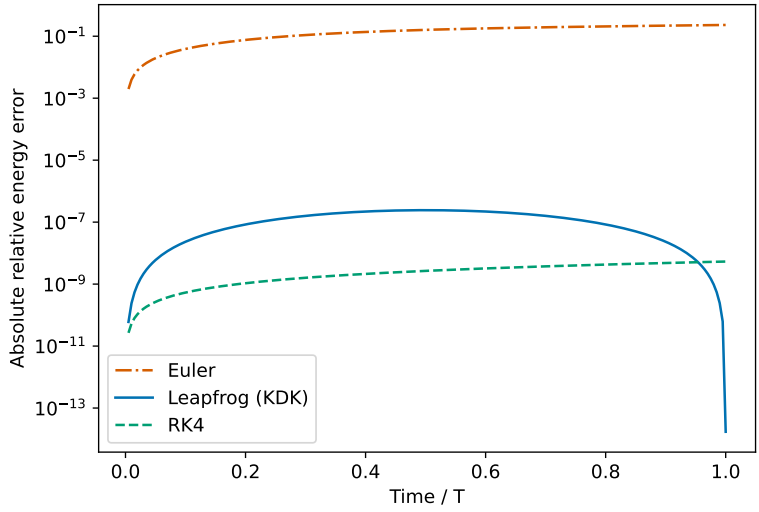


図 1.5　標準条件でのエネルギーの相対誤差。各方法の初期値からの変化の絶対値を示しています。縦軸は対数です。Euler 法、leapfrog、RK4 を比べています。leapfrog も RK4 もエネルギーを厳密に保存するわけではありません。

標準条件の誤差を表1.2にまとめました。位置の誤差 e_r は、解析解との位置の差を初期半径で割った最大値です。次節で詳しく定義します。

leapfrog のエネルギー誤差は約 2.43×10^{-7} です。一方、角運動量のずれは約 6.43×10^{-16} で、丸め誤差程度にとどまります。これは、中心力による kick と drift の各操作が、角運動量を保つためです。例えば kick では $\boldsymbol{r} \times \boldsymbol{a} = 0$ なので、速度に $h\boldsymbol{a}$ を加えても $\boldsymbol{r} \times \boldsymbol{v}$ は変わりません。drift でも、位置へ $h\boldsymbol{v}$ を加えた寄与は $h\boldsymbol{v} \times \boldsymbol{v} = 0$ です。この説明は本章の中心力についてのもので、任意の外力に角運動量保存を保証するものではありません。

表 1.2　同じ 1 周期・200 ステップでの比較。e_r は最大位置誤差、e_ε と e_ℓ は保存量の最大相対誤差です。本章の実行結果であり、末尾の桁は実行環境で多少変わります。

方法	力評価回数	e_r	e_ε	e_ℓ
Euler	200	1.43×10^{0}	2.31×10^{-1}	1.38×10^{-1}
Leapfrog (KDK)	201	2.16×10^{-3}	2.43×10^{-7}	6.43×10^{-16}
RK4	800	1.65×10^{-7}	5.34×10^{-9}	2.67×10^{-9}

RK4 の最大相対エネルギー誤差は 5.34×10^{-9}、角運動量誤差は 2.67×10^{-9} でした。短時間の位置とエネルギーの精度は非常に高い一方、エネルギーや角運動量を厳密に保存する手法ではありません。[3] また、どの方法でも、保存量のずれが小さいことだけで軌道上の位置が正しいとは判断できません。次節では、解析解に対する位置ベクトルの誤差を直接調べます。

よくある失敗：保存量はモデルを見て選ぶ

本章では、静的な中心重力場なのでエネルギーと角運動量を確認できます。後の章のように、重力場が時間変化する場合や摩擦がある場合には、粒子のエネルギーは一般に保存しません。「エネルギーが変わったからコードが間違っている」と決める前に、計算したモデルで何が保存されるかを確認します。

1.10 時間刻みを変えて誤差を調べる

1.10.1 位置ベクトルの誤差を測る

時間刻みの影響を分けて調べるため、中心質量、初期位置、初期速度、最終時刻 T を固定します。変えるのはステップ数だけです。ステップ数を 2 倍にすると、時間刻みは半分になります。数値解と解析解を同じ時刻で比較し、位置の差を r_0 で規格化します。保存した全時刻での最大値を

$$e_r = \max_{0 \le n \le N_{\mathrm{step}}} \frac{|\boldsymbol{r}^n - \boldsymbol{r}_{\mathrm{exact}}(t_n)|}{r_0} \tag{1.29}$$

と定義します。これは半径の差だけではなく、位置**ベクトル**の差です。円に沿っていても回転の速さがずれていれば、誤差に現れます。同様に、式(1.28)の最大値を e_ε, e_ℓ とします。

```
def error_metrics(times, history, mu, r0):
    pos_exact, _ = exact_circle(times, r0, mu)
    radius, energy, angular_momentum = diagnostics(history, mu)
    position_error = np.linalg.norm(history[:, 0, :2] - pos_exact, axis=1) / r0
    energy_error = np.abs(energy - energy[0]) / abs(energy[0])
    angular_error = np.abs(angular_momentum - angular_momentum[0]) /
    ↪  abs(angular_momentum[0])
    return np.max(position_error), np.max(energy_error), np.max(angular_error)
```

同じステップ数で leapfrog と RK4 を計算し、その結果を表にまとめます。

```
step_counts = [20, 40, 80, 160, 320]
rows_lf, rows_rk4 = [], []
for count in step_counts:
    t, h_lf = integrate_leapfrog(acceleration_kepler, state0, period, count, mu)
    _, h_rk4 = integrate(rhs_kepler, rk4_step, state0, period, count, mu)
    rows_lf.append([count, period / count, *error_metrics(t, h_lf, mu, r0)])
    rows_rk4.append([count, period / count, *error_metrics(t, h_rk4, mu, r0)])
convergence_lf, convergence_rk4 = np.array(rows_lf), np.array(rows_rk4)
orders_lf = np.log2(convergence_lf[:-1, 2] / convergence_lf[1:, 2])
orders_rk4 = np.log2(convergence_rk4[:-1, 2] / convergence_rk4[1:, 2])
```

表 1.3 時間刻みと最大位置誤差。すべて同じ初期条件から 1 周期 T まで計算しています。p_r は一つ前の行との誤差比から求めた見かけの次数です。保存量の誤差も、配布する CSV に保存しています。

N_{step}	Δt [Myr]	e_r (KDK)	p_r	e_r (RK4)	p_r
20	14.8121	2.09×10^{-1}	—	3.91×10^{-3}	—
40	7.4060	5.35×10^{-2}	1.967	1.66×10^{-4}	4.557
80	3.7030	1.35×10^{-2}	1.990	7.93×10^{-6}	4.387
160	1.8515	3.37×10^{-3}	1.998	4.19×10^{-7}	4.242
320	0.9258	8.44×10^{-4}	1.999	2.40×10^{-8}	4.129

1.10.2 2 次精度と 4 次精度を確認する

刻みを半分にした二つの計算から、見かけの次数を

$$p_r = \frac{\log\left[e_r(\Delta t)/e_r(\Delta t/2)\right]}{\log 2} \tag{1.30}$$

で求めます。leapfrog では 2 へ、RK4 では 4 へ近づくと期待できます。表1.3の最も細かい側では、それぞれ 1.999 と 4.129 です。図1.6では、同じ数値を両対数の図にしています。点線と一点鎖線は、それぞれ傾き 2 と 4 の参照線です。

粗い刻みでは高次の誤差項も影響するため、見かけの次数は 2 や 4 から離れます。一方、極端に細かい刻みでは、浮動小数点演算の丸め誤差が目立つことがあります。期待した誤差比は、刻みが十分小さく、かつ丸め誤差が支配しない範囲で確認します。

円軌道では、エネルギーなど特定の量の誤差に打ち消しが生じる場合があります。その量だけから「leapfrog は 4 次」「RK4 は 5 次」と判断してはいけません。本章では、同じ時刻の解析解に対する位置のずれを、解法の次数を確認する指標にしました。

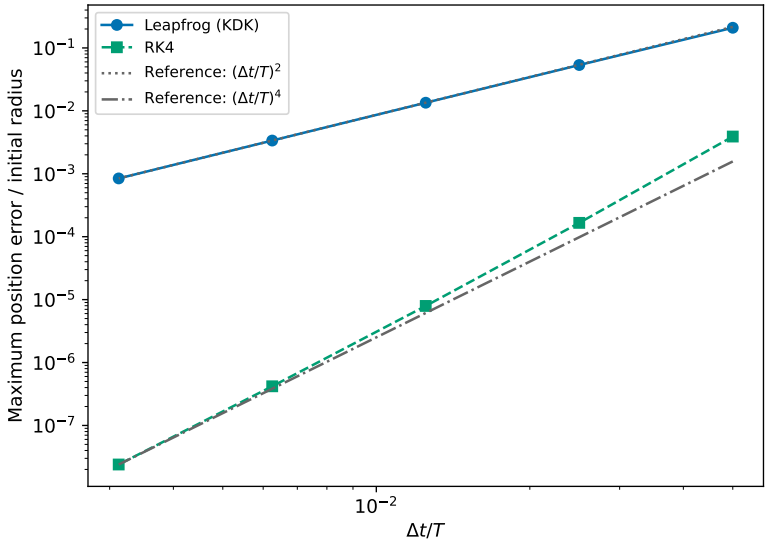


図 1.6　時間刻みと最大位置誤差の関係。青丸が leapfrog、緑四角が RK4 です。参照線は各方法の最も細かい刻みの実測点を通り、傾きを 2 または 4 に固定した線です。データに当てはめて求めた線ではありません。

1.11 計算コストと長時間の振る舞いを比較する

1.11.1 力評価回数をそろえる

同じ時間刻みで比べれば、RK4 の方が小さい位置誤差を与えました。ただし、RK4 は 1 ステップで右辺を 4 回評価し、各評価で重力を計算します。キャッシュ付き KDK は、最初の 1 回を除けば新しい重力評価が 1 回だけです。N_{step} ステップの力評価回数は

$$N_{\mathrm{force}} = \begin{cases} N_{\mathrm{step}} & \text{Euler}, \\ N_{\mathrm{step}} + 1 & \text{leapfrog（KDK）}, \\ 4N_{\mathrm{step}} & \text{RK4} \end{cases} \tag{1.31}$$

です。ここでは時間積分のための力評価だけを数え、保存量の計算や描画は含めません。200 ステップの場合の 200 回、201 回、800 回という値は、配布コードの独立試験でも関数の呼び出し数から確認しています。

力評価回数を 800 回にそろえる比較も行います。KDK は初回の 1 回が必要なので 799 ステップ、RK4 は 200 ステップです。いずれも同じ 1 周期 T まで進めます。

```
force_budget = 800
n_lf_cost = force_budget - 1
n_rk_cost = force_budget // 4
t_lf_cost, h_lf_cost = integrate_leapfrog(acceleration_kepler, state0, period, n_lf_cost, mu)
t_rk_cost, h_rk_cost = integrate(rhs_kepler, rk4_step, state0, period, n_rk_cost, mu)
cost_lf_errors = error_metrics(t_lf_cost, h_lf_cost, mu, r0)
cost_rk4_errors = error_metrics(t_rk_cost, h_rk_cost, mu, r0)
```

表 1.4 力評価回数を 800 回にそろえた 1 周期の比較。時間刻みは同じではありません。e_r は最大位置誤差、e_ε は最大相対エネルギー誤差です。

方法	N_{step}	力評価回数	e_r	e_ε
Leapfrog (KDK)	799	800	1.35×10^{-4}	9.56×10^{-10}
RK4	200	800	1.65×10^{-7}	5.34×10^{-9}

この滑らかな円軌道の短時間計算では、力評価回数をそろえても RK4 の位置誤差の方が小さくなりました。したがって、**力評価が少ないから、常に leapfrog の方が高精度になるわけではありません**。必要な精度、計算する時間の長さ、保存系としての性質を合わせて考えます。

直接 N 体計算では、1 回の重力評価に多数の粒子対の計算が含まれます。粒子数を N とすると、直接和の計算量は $O(N^2)$ です。本書のテスト粒子モデルは少数のポテンシャルから力を受けるので $O(N)$ ですが、どちらでも力評価の節約には意味があります。ただし、実行時間には配列の作成や Python の処理も含まれます。力評価回数だけから「実行時間が必ず 4 倍違う」とは言えません。Notebook には実行時間を測る例も用意し、環境ごとに確かめます。

1.11.2 同じ円軌道を 100 周期まで進める

次に、円軌道を長く追跡します。初期条件は同じですが、この実験だけ時間刻みを $\Delta t = T/100$、最終時刻を $100T$ とします。短時間の表と混同しないよう、条件の違いを明示しておきます。両方法とも 10000 ステップで、力評価回数は KDK が 10001 回、RK4 が 40000 回です。

エネルギーが一方向へずれるのか、増減を繰り返すのかを見るため、ここでは絶対値ではなく、**符号付き**の差 $[\varepsilon(t) - \varepsilon(0)]/|\varepsilon(0)|$ を描きます。

```
n_periods = 100
steps_per_period = 100
n_long = n_periods * steps_per_period
t_long, h_long_lf = integrate_leapfrog(acceleration_kepler, state0, n_periods * period, n_long, mu)
_, h_long_rk4 = integrate(rhs_kepler, rk4_step, state0, n_periods * period, n_long, mu)
_, e_long_lf, l_long_lf = diagnostics(h_long_lf, mu)
_, e_long_rk4, l_long_rk4 = diagnostics(h_long_rk4, mu)
signed_lf = (e_long_lf - e_long_lf[0]) / abs(e_long_lf[0])
signed_rk4 = (e_long_rk4 - e_long_rk4[0]) / abs(e_long_rk4[0])
```

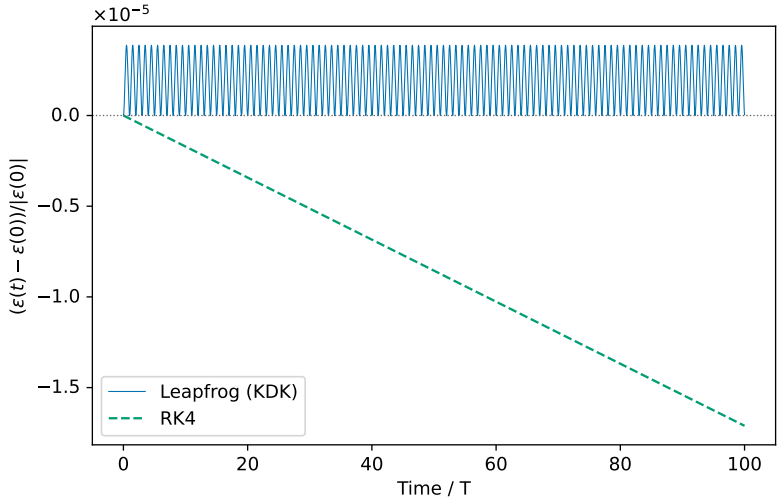


図 1.7 円軌道の 100 周期積分。両方法とも $\Delta t = T/100$ です。青実線が KDK、緑破線が RK4 です。KDK の細かい振動は、この縮尺では帯のように見えます。同じ時間刻みでの比較であり、同じ力評価回数での比較ではありません。

この条件では、KDK のエネルギー差は 0 付近から約 3.87×10^{-6} までの範囲で振動しました。RK4 の差は負の向きへ蓄積し、$100T$ で約 -1.71×10^{-5} となりました。これは本章の実行結果であり、任意の軌道や刻みで同じ数値や符号になるという主張ではありません。

静的な滑らかな保存系を固定刻みで解くとき、leapfrog の構造を保つ性質は、このような長時間の誤差の振る舞いに利点を持ちます。[5] ただし、エネルギー差が狭い範囲に収まっていても、軌道の位相が正確とは限りません。精度の確認には、保存量と位置の誤差を併用します。

1.11.3 本書での使い分けを考える

第 2 章から第 5 章までの重力計算では、leapfrog（KDK）を標準とします。計算コストを抑えながら重力系を追跡する方法として、実装を繰り返し使います。RK4 は、一般の常微分方程式を解くもう一つの主要手法として理解し、第 6 章の速度依存力を含む発展問題で用います。

そこでは Chandrasekhar の動力学的摩擦を、質量をもつ衛星銀河・銀河中心の運動へ加えます。質量ゼロの恒星テスト粒子そのものに摩擦を掛けるわけではありません。本章末の速度比例抵抗は、その前に速度依存力を扱うための小さな練習です。**手法は次数の高さだけでなく、方程式の形、必要な精度、計算コストに応じて選びます。**

1.12 本章のまとめ

本章では、固定した中心質量の周りを動く 1 個の粒子を計算しました。位置と速度を変数として持つと、Newton の運動方程式を 1 階の連立常微分方程式として扱えます。加速度を計算する部分と時間を進める部分を分けることで、同じ問題に異なる積分法を適用できました。

Euler 法で時間を刻む考え方を確認した後、重力計算の標準法として leapfrog（KDK）を実装しました。半 kick、drift、半 kick という順序、整数時刻にそろえた速度の保存、加速度の再利用が重要です。一般の常微分方程式へ進むため、RK4 の 4 段階も学びました。RK4 は、速度依存力を含む発展問題で再利用します。

両者を、軌道の見た目、解析解との位置の差、エネルギー、角運動量、時間刻み、力評価回数から比較しました。leapfrog は 2 次、RK4 は 4 次ですが、次数だけで優劣は決まりません。長時間の保存特性と、必要な精度を得るための計算コストも判断材料です。**計算が最後まで動いたことと、目的に必要な精度が得られたことは同じではありません**。次の章では leapfrog を使い、2 つの天体が互いの重力で動く問題へ進みます。

演習

演習 1.1　確認：初期値と単位　表1.1の条件から、$v_c, T, \varepsilon_0, \ell_{z,0}$ を計算してください。v_c は $\mathrm{km\,s^{-1}}$ と $\mathrm{kpc\,Gyr^{-1}}$ の両方で示し、保存量にも単位を付けてください。

演習 1.2　確認：KDK の更新と力評価　KDK の 3 段階を式で書き、各 kick で使う加速度がどの位置に対応するか説明してください。200 ステップの力評価回数を求めてください。また、$\boldsymbol{r}^{n+1}$ と $\boldsymbol{v}^{n+1/2}$ からエネルギーを計算する場合の問題点を説明してください。

演習 1.3　確認：RK4 の中間状態　$\boldsymbol{k}_2$ と $\boldsymbol{k}_3$ の評価時刻と予測状態を、それぞれ式で書いてください。$\boldsymbol{k}_2$ を求めるとき、位置成分の時間変化率に $\boldsymbol{v}^n$ をそのまま使うと、何が間違うでしょうか。

演習 1.4　応用：ステップ数の変更　初期条件と最終時刻 T を固定し、$N_{\mathrm{step}} = 50, 100, 200$ で leapfrog と RK4 を実行してください。最大位置誤差 e_r を求め、ステップ数を 2 倍にした誤差比を調べてください。軌道の図だけで精度を区別できるかも考えてください。

演習 1.5　応用：回転方向の変更　初期条件の v_y だけを $-v_c$ へ変え、leapfrog で同じ時間まで積分してください。軌道、エネルギー、角運動量はどのように変わるでしょうか。数値解が x 軸について鏡映した関係になるかを確認してください。

演習 1.6　応用：同じ計算コストでの比較　時間積分の力評価を両方とも 800 回にそろえ、1 周期を計算してください。両方法のステップ数、時間刻み、位置誤差

を比べてください。同じ刻みでの比較とは、何をそろえ、何を変えている点が違いますか。

演習 1.7　発展：速度に依存する力　重力加速度に速度比例抵抗 $-\gamma \boldsymbol{v}$ を追加し、$\gamma = 0.05/T$ として、400 ステップで 1 周期分の時間まで積分してください。右辺関数だけを変更し、RK4 の関数はそのまま使います。このモデルでは

$$\frac{d\varepsilon}{dt} = -\gamma v^2 \tag{1.32}$$

となります。エネルギー変化の向きは、この式と整合するでしょうか。この抵抗力は速度依存力の練習用であり、Chandrasekhar の動力学的摩擦公式ではありません。

第 2 章　2 体問題を計算する

第 1 章では、原点に固定した中心質量の周りを動く 1 個の粒子を計算しました。この章では、二つの天体が互いに重力を及ぼし、両方とも動く問題へ進みます。時間積分には、前章で学んだ leapfrog 法の kick–drift–kick（KDK）形式を使います。新しく作る中心的な部分は、二つの位置から両方の加速度を求める処理と、重心・相対運動を用いた初期条件です。

ゼミで学んだ 2 体問題を、長い解析解の導出からやり直すことはしません。計算に必要な式を短く整理し、円・楕円・放物線の軌道を作ります。その後、全エネルギー、全角運動量、全運動量、重心運動を確かめます。最後に、指定した近点距離へ向かう接近軌道を作り、第 5 章の銀河相互作用へ向けた準備をします。

この章で学ぶこと

- 2 天体の重力加速度を、作用・反作用を保つ形で計算できます。
- 重心運動と相対運動を区別し、質量比に応じた初期位置・速度を設定できます。
- 第 1 章の KDK を使って、両天体を同じ時刻へ更新できます。
- 初期速度を変えて円・楕円・放物線を作り、解析的な軌道の性質と比較できます。
- 保存量と位置の誤差を使い、時間刻みの影響を確認できます。

2.1 一方を固定する近似を外す

2.1.1 本章で計算する対象を整理する

質量 M_1, M_2 の 2 天体を考えます。両方に有限の質量があり、互いに重力を及ぼします。外部からの力はなく、質量は時間によらず一定です。慣性系の x–y 平面内で運動を追跡します。

本章では、各天体を大きさのない**点質量**として扱います。これは、2 体問題の軌道と保存量を確かめるための練習モデルです。後の章で使う銀河の有限サイズ、円盤のテスト粒子、動力学的摩擦は、まだ入れません。とくに、2 天体が同じ位置へ衝突する初期条件は扱いません。

第 1 章では、中心天体が動かないと近似しました。今回はその近似を外すので、重い天体も動きます。質量が同じなら二つは対称に動き、質量が異なれば、重い天体ほど重心の近くを小さく動きます。まずは質量を異ならせ、この違いを見える形にします。

2.1.2 位置と相対位置を区別する

天体 $k = 1, 2$ の位置と速度を $\boldsymbol{R}_k, \boldsymbol{V}_k$ と書きます。大文字を使うのは、後の章の銀河中心と記法をそろえるためです。天体 1 から天体 2 へ向かう相対位置と、その変化率を

$$\boldsymbol{D} = \boldsymbol{R}_2 - \boldsymbol{R}_1, \qquad D = |\boldsymbol{D}|, \qquad \boldsymbol{U} = \boldsymbol{V}_2 - \boldsymbol{V}_1 = \frac{d\boldsymbol{D}}{dt} \tag{2.1}$$

と定義します。D は **2 天体の間の距離**です。それぞれの天体から原点までの距離とは、一般に一致しません。

位置の原点は、どちらかの天体へ毎回移動させるのではなく、まず一つの慣性系に固定します。天体 1 を原点に置いたような相対軌道の図は、計算後に式(2.1)で座標を引き算して作ります。座標を変換して表示することと、一方の天体の運動を無視することは別です。

2.2 二つの加速度を同じ位置から求める

2.2.1 力と加速度の違いを確認する

天体 1 は天体 2 へ、天体 2 は天体 1 へ引かれます。式(2.1)の向きを使うと、Newton の運動方程式は

$$M_1 \frac{d\boldsymbol{V}_1}{dt} = +GM_1M_2 \frac{\boldsymbol{D}}{D^3}, \tag{2.2}$$

$$M_2 \frac{d\boldsymbol{V}_2}{dt} = -GM_1M_2 \frac{\boldsymbol{D}}{D^3}. \tag{2.3}$$

両辺をそれぞれの質量で割り、加速度 $\boldsymbol{A}_k = d\boldsymbol{V}_k/dt$ を求めると、

$$\boldsymbol{A}_1 = +GM_2 \frac{\boldsymbol{D}}{D^3}, \qquad \boldsymbol{A}_2 = -GM_1 \frac{\boldsymbol{D}}{D^3} \tag{2.4}$$

となります。自分自身の質量は消えますが、重力を作る相手の質量は残ります。両方とも同じ時刻の $\boldsymbol{R}_1, \boldsymbol{R}_2$ から計算します。これらは Ryden の 2 体問題で用いる運動方程式に対応します。[1]

作用・反作用の関係は

$$M_1\boldsymbol{A}_1 + M_2\boldsymbol{A}_2 = \boldsymbol{0} \tag{2.5}$$

です。$M_1 \neq M_2$ なら、$\boldsymbol{A}_1 = -\boldsymbol{A}_2$ ではありません。例えば $M_1 = 2M_2$ なら、天体 2 の加速度の大きさは天体 1 の 2 倍です。

2.2.2 成分で確かめてから関数にする

$\boldsymbol{D} = (d_x, d_y)$、$D^2 = d_x^2 + d_y^2$ と書けば、例えば x 成分は

$$A_{1x} = \frac{GM_2 d_x}{(d_x^2 + d_y^2)^{3/2}}, \qquad A_{2x} = -\frac{GM_1 d_x}{(d_x^2 + d_y^2)^{3/2}} \tag{2.6}$$

です。y 成分も分子の d_x を d_y へ替えるだけです。天体 2 が天体 1 の右側にあれば、$A_{1x} > 0$、$A_{2x} < 0$ となります。この単純な配置で符号を確認してから、軌道を計算します。

実装では $G\boldsymbol{D}/D^3$ を一度だけ求め、それに相手の質量と符号を掛けます。2 体問題なので、全粒子対を探索する二重ループは必要ありません。この段階では、一般的な N 体重力計算へ広げません。

よくある失敗：同じ大きさになるのは力です

二つの加速度を単に符号反転して作る実装は、等質量の場合にしか正しくありません。等質量のテストだけでは、この誤りを見落とします。本章では $M_1 : M_2 = 2 : 1$ を標準にして、$M_1\boldsymbol{A}_1 + M_2\boldsymbol{A}_2$ を確認します。

2.3 重心運動と相対運動を分けて考える

2.3.1 重心が等速直線運動することを確かめる

全質量を $M_{\mathrm{tot}} = M_1 + M_2$ とし、質量で重み付けした平均位置を重心とします。

$$\boldsymbol{R}_{\mathrm{cm}} = \frac{M_1\boldsymbol{R}_1 + M_2\boldsymbol{R}_2}{M_{\mathrm{tot}}}, \qquad \boldsymbol{V}_{\mathrm{cm}} = \frac{M_1\boldsymbol{V}_1 + M_2\boldsymbol{V}_2}{M_{\mathrm{tot}}}. \tag{2.7}$$

全運動量は $\boldsymbol{P} = M_{\mathrm{tot}}\boldsymbol{V}_{\mathrm{cm}}$ です。式(2.2)と式(2.3)を足すと、内力が打ち消し合います。

$$\frac{d\boldsymbol{P}}{dt} = \boldsymbol{0}, \qquad \frac{d\boldsymbol{V}_{\mathrm{cm}}}{dt} = \boldsymbol{0}. \tag{2.8}$$

したがって、$t = 0$ からの重心の位置は

$$\boldsymbol{R}_{\mathrm{cm}}(t) = \boldsymbol{R}_{\mathrm{cm}}(0) + t\boldsymbol{V}_{\mathrm{cm}}(0) \tag{2.9}$$

です。重心を静止させた座標系を重心系と呼びます。本章の標準例では、初期重心を原点、重心速度を 0 にします。ただし、実装の検証には、重心に速度を与えた場合も使います。

2.3.2 相対運動を第 1 章と同じ形に書く

今度は、天体 2 の加速度から天体 1 の加速度を引きます。式(2.4)から

$$\frac{d\boldsymbol{D}}{dt} = \boldsymbol{U}, \qquad \frac{d\boldsymbol{U}}{dt} = -\mu\frac{\boldsymbol{D}}{D^3}, \qquad \mu \equiv GM_{\mathrm{tot}} \tag{2.10}$$

を得ます。右辺の形は、第 1 章の固定中心質量による運動と同じです。ただし、そこでは $\mu = GM$ だったのに対し、ここでは**二つの質量の和**が入ります。[1]

相対位置だけをこの式で計算する方法もあります。本章の主計算では両天体を直接動かし、相対運動はその結果から求めます。このようにすると、作用・反作用、重心、各天体の運動を同時に確認できます。相対運動の式は、初期条件や解析的な比較値を作るために使います。

2.3.3 相対位置から各天体の位置へ戻す

重心と相対位置が分かれば、式(2.7)と式(2.1)を連立して、二つの位置を求められます。速度にも同じ関係が成り立ちます。

$$\boldsymbol{R}_1 = \boldsymbol{R}_{\rm cm} - \frac{M_2}{M_{\rm tot}}\boldsymbol{D}, \qquad \boldsymbol{R}_2 = \boldsymbol{R}_{\rm cm} + \frac{M_1}{M_{\rm tot}}\boldsymbol{D},$$
$$\boldsymbol{V}_1 = \boldsymbol{V}_{\rm cm} - \frac{M_2}{M_{\rm tot}}\boldsymbol{U}, \qquad \boldsymbol{V}_2 = \boldsymbol{V}_{\rm cm} + \frac{M_1}{M_{\rm tot}}\boldsymbol{U}. \tag{2.11}$$

重心系では $\boldsymbol{R}_1, \boldsymbol{R}_2$ は逆方向です。それぞれの重心からの距離は

$$|\boldsymbol{R}_1| = \frac{M_2}{M_{\rm tot}}D, \qquad |\boldsymbol{R}_2| = \frac{M_1}{M_{\rm tot}}D \tag{2.12}$$

です。二つの軌道の大きさの比は、質量比の逆数になります。どちらも同じ相対運動から作られるので、束縛された 2 体軌道では周期は共通です。[1]

図2.1では、原点を重心とは別の場所に置きました。$\boldsymbol{D}$ は二つの位置ベクトルの差であり、重心からの距離でも原点からの距離でもありません。重い天体ほど重心に近いことも確認してください。

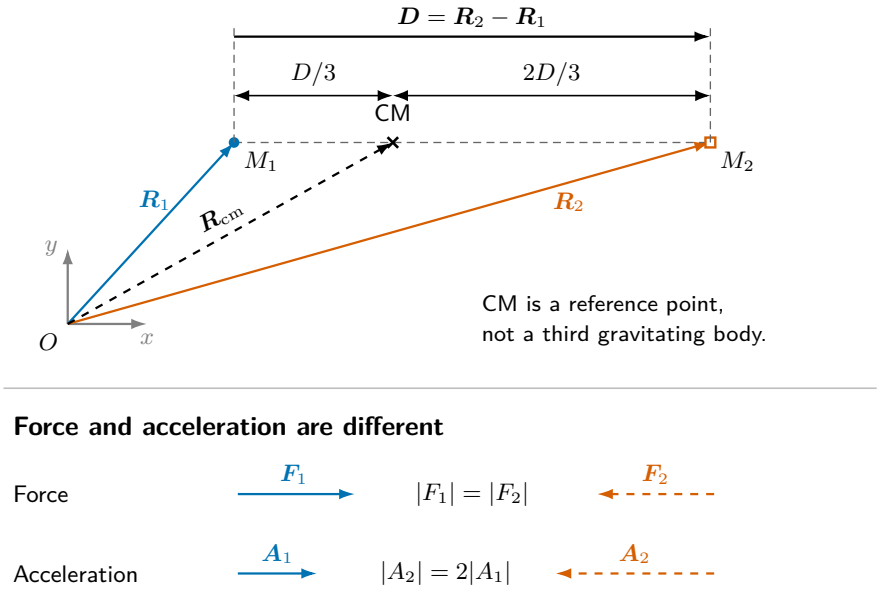


図 2.1　質量比 $M_1 : M_2 = 2 : 1$ の位置関係。任意の原点 O と重心 CM を区別しています。重心からの距離の比は 1 : 2 です。下段は力と加速度の違いで、力の大きさは等しくても加速度の大きさは異なります。段をまたいで矢印の長さを比較しません。

よくある失敗：単純な平均を重心にしない

$(\boldsymbol{R}_1+\boldsymbol{R}_2)/2$ が重心になるのは等質量のときだけです。同じく、$\boldsymbol{V}_1=-\boldsymbol{V}_2$ と置けば重心が静止するとは限りません。位置にも速度にも、式(2.11)の質量比を使います。

2.4 初期速度から軌道を選ぶ

2.4.1 円軌道を基準にする

相対距離が D_0 の円軌道を考えます。相対運動の向心加速度と重力加速度を等しくすると、相対円運動速度と周期は

$$v_{\rm c}=\sqrt{\frac{\mu}{D_0}},\qquad T_{\rm c}=2\pi\sqrt{\frac{D_0^3}{\mu}} \tag{2.13}$$

です。ここでの $v_{\rm c}$ は、一方の天体の速度ではなく、二つの**相対速度**の大きさです。重心系での各速度の大きさは、それぞれ $(M_2/M_{\rm tot})v_{\rm c}$、$(M_1/M_{\rm tot})v_{\rm c}$ です。

初期位置と速度を

$$\boldsymbol{D}(0)=(D_0,0),\qquad \boldsymbol{U}(0)=(0,\eta v_{\rm c}) \tag{2.14}$$

と置きます。η は接線方向の初期速度を調整する無次元の数です。ここでは動径方向の初期速度は 0 で、$\eta>0$ なら相対運動は反時計回りに始まります。

2.4.2 比エネルギーで軌道を分類する

相対運動の比エネルギーと比角運動量を

$$\varepsilon=\frac{U^2}{2}-\frac{\mu}{D},\qquad \ell_z=D_xU_y-D_yU_x \tag{2.15}$$

とします。どちらも式(2.10)で保存されます。式(2.14)の初期値を代入すると、

$$\varepsilon_0=\frac{\mu}{D_0}\left(\frac{\eta^2}{2}-1\right),\qquad \ell_{z,0}=\eta\sqrt{\mu D_0} \tag{2.16}$$

です。初期速度を変えることは、同じ距離から始める軌道のエネルギーと角運動量を変えることに対応します。

点質量の Kepler 問題では、$\varepsilon<0$ の非衝突軌道は楕円、$\varepsilon=0$ は放物線、$\varepsilon>0$ は双曲線です。円は楕円の特別な場合です。[1] 本章の 3 例を表2.1にまとめます。

$0<\eta<1$ では初期速度が円運動速度より小さいため、距離は初期位置からいったん減少します。動径速度が 0 だった初期位置は遠点です。$\eta=\sqrt{2}$ では脱出速度に相当し、初期位置から遠ざかって戻らない放物線になります。この例の放物線は、離れた場所から近づいてくる途中ではなく、**近点から出発する軌道**です。内向きに始める接近軌道は後で別に作ります。

表 2.1 式(2.14)を使う 3 例。D_0 と質量は共通で、変えるのは η だけです。

η	軌道	ε_0	初期位置の意味
1	円	$-\mu/(2D_0)$	半径一定
0.8	楕円	$-0.68\mu/D_0$	遠点
$\sqrt{2}$	放物線	0	近点

2.4.3 楕円の比較値を初期条件から求める

数値結果を見る前に、楕円の大きさと周期を求めておきます。Kepler 軌道の軌道長半径を a_{orb}、離心率を e とすると、

$$a_{\mathrm{orb}} = -\frac{\mu}{2\varepsilon}, \qquad e^2 = 1 + \frac{2\varepsilon \ell_z^2}{\mu^2} \tag{2.17}$$

です。[1] 2 次元なので、角運動量の大きさの 2 乗は ℓ_z^2 です。$0 < \eta < 1$ の初期条件を代入すると、

$$a_{\mathrm{orb}} = \frac{D_0}{2-\eta^2}, \qquad e = 1-\eta^2, \tag{2.18}$$

$$D_{\mathrm{a}} = D_0, \qquad D_{\mathrm{p}} = \frac{\eta^2 D_0}{2-\eta^2}, \qquad T_{\mathrm{ell}} = 2\pi\sqrt{\frac{a_{\mathrm{orb}}^3}{\mu}}. \tag{2.19}$$

添字 a は遠点、p は近点です。a_{orb} は相対軌道の長半径であり、後の Plummer モデルのスケール長ではありません。

もう少し詳しく：軌道の形と時間の関係

楕円の形が分かっても、任意の時刻の位置を直ちに計算できるとは限りません。Kepler 方程式を解いて時刻と軌道上の位置を対応させる方法もありますが、本章ではその求根算法を追加しません。円の同時刻の解析解と、楕円の解析周期後の戻り位置を、独立な位置の確認に使います。詳しい軌道の記述は文献 [1] に譲ります。

2.5 Python で初期条件と重力を実装する

2.5.1 単位・配列・標準条件をそろえる

距離は kpc、時間は Gyr、質量は $M_\odot$ とし、内部速度を $\mathrm{kpc\,Gyr^{-1}}$ にそろえます。重力定数と速度の換算係数は第 1 章と同じです。以下では `import numpy as np` と `import matplotlib.pyplot as plt` を実行済みとします。

標準条件は本教材のために設定したもので、特定の実在天体の再現を目的としません。とくに、本章の天体は $10^{11} M_\odot$ 程度の質量を持っていても、銀河の内部構造を持たない点質量です。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
masses = np.array([1.0e11, 5.0e10])  # M1, M2 [Msun]
d0 = 20.0                            # 2 天体の初期距離 [kpc]
mtot = np.sum(masses)
mu = G * mtot
m_red = masses[0] * masses[1] / mtot
v_c = np.sqrt(mu / d0)               # 相対運動の円運動速度 [kpc/Gyr]
t_c = 2.0 * np.pi * np.sqrt(d0**3 / mu)
n_steps = 1000
t_end = t_c
dt = t_end / n_steps
eta_values = {"Circular": 1.0, "Elliptic": 0.8, "Parabolic": np.sqrt(2.0)}
```

表 2.2 本章の標準条件。3 ケースとも同じ T_{c} まで 1000 ステップで計算します。楕円の周期 T_{ell} とは区別します。数値は実行結果から生成しています。

量	値
M_1, M_2	$10^{11} M_\odot,\ 5\times10^{10} M_\odot$
D_0	$20\,\mathrm{kpc}$
v_{c}	$179.601851\,\mathrm{km\,s^{-1}}$
T_{c}	$0.684141\,\mathrm{Gyr}$
$\Delta t = T_{\mathrm{c}}/1000$	$0.684141\,\mathrm{Myr}$
η	$1,\ 0.8,\ \sqrt{2}$
$\boldsymbol{R}_{\mathrm{cm}}(0), \boldsymbol{V}_{\mathrm{cm}}(0)$	いずれも $\mathbf{0}$

`masses[0]` が M_1、`masses[1]` が M_2 です。位置配列 `pos` と速度配列 `vel` は形 `(2, 2)` で、行が天体、列が x, y 成分です。両者を横につないだ `state` は形 `(2, 4)` で、各行に `[x, y, vx, vy]` を入れます。第 1 章の約束を変えず、天体の行を 1 行から 2 行へ増やしています。

2.5.2 初期条件を関数にまとめる

式(2.11)を関数にします。入力の `separation` は相対位置ベクトル、`relative_velocity` は相対速度ベクトルです。どちらも 2 成分を渡します。重心位置・速度を省略したときは 0 とします。

```
def make_initial_state(masses, separation, relative_velocity, r_cm=None, v_cm=None):
    """ 相対位置・速度から 2 天体を配置します。本文 eq:ch2-inverse-transform。"""
    masses = check_masses(masses)
    separation = np.asarray(separation, dtype=float)
    relative_velocity = np.asarray(relative_velocity, dtype=float)
    r_cm = np.zeros(2) if r_cm is None else np.asarray(r_cm, dtype=float)
    v_cm = np.zeros(2) if v_cm is None else np.asarray(v_cm, dtype=float)
    for vector in (separation, relative_velocity, r_cm, v_cm):
        if vector.shape != (2,) or not np.all(np.isfinite(vector)):
            raise ValueError("Positions and velocities must be finite 2D vectors.")
    if np.linalg.norm(separation) == 0.0:
        raise ValueError("The two point masses must not coincide.")
    mtot = np.sum(masses)
    pos1 = r_cm - masses[1] / mtot * separation
    pos2 = r_cm + masses[0] / mtot * separation
    vel1 = v_cm - masses[1] / mtot * relative_velocity
    vel2 = v_cm + masses[0] / mtot * relative_velocity
    return np.column_stack((np.array([pos1, pos2]), np.array([vel1, vel2])))
```

`check_masses` は、二つの質量が有限かつ正であることを確かめる短い補助関数です。実装は Notebook に含めています。検証のための分岐と、初期値を作る物理の式を区別して読んでください。

$D_0 = 20\,\mathrm{kpc}$、質量比 $2:1$ では、重心系の初期位置は $(-20/3, 0)\,\mathrm{kpc}$ と $(40/3, 0)\,\mathrm{kpc}$ です。二つの距離を足すと $20\,\mathrm{kpc}$ になり、質量で重み付けした平均は 0 になります。

2.5.3 同じ粒子対を使って加速度を返す

最初に成分のまま計算し、向きと大きさを確認します。ここでは `state0` に上で作った初期値を入れたとします。

```
x1, y1 = state0[0, :2]
x2, y2 = state0[1, :2]
dx, dy = x2 - x1, y2 - y1
sep2 = dx**2 + dy**2
ax1, ay1 = G * masses[1] * dx / sep2**1.5, G * masses[1] * dy / sep2**1.5
ax2, ay2 = -G * masses[0] * dx / sep2**1.5, -G * masses[0] * dy / sep2**1.5
```

この初期位置で、天体 1 の x 加速度は約 $+562.31\,\mathrm{kpc\,Gyr^{-2}}$、天体 2 は約 $-1124.62\,\mathrm{kpc\,Gyr^{-2}}$ です。y 成分はいずれも 0 です。重い天体 1 の加速度が小さくなっています。

次は、同じ計算を 2 成分のベクトルでまとめます。

```
def acceleration_two_body(pos, masses):
    """2 天体の同じ時刻の位置から両方の加速度を返します。"""
    dr = pos[1] - pos[0]
    sep2 = np.dot(dr, dr)
    if not np.isfinite(sep2) or sep2 <= 0.0:
        raise ValueError("Point-mass separation must be finite and nonzero.")
    common = G * dr / sep2**1.5
    acc1 = masses[1] * common
    acc2 = -masses[0] * common
    return np.array([acc1, acc2])
```

`dr` が $\boldsymbol{D}$、`np.dot(dr, dr)` が D^2 です。返り値も形 `(2, 2)` にそろえます。分母へ小さな数を足して特異点を隠しません。点質量モデルで距離が 0 なら、その条件はこのコードの対象外として停止します。

確認：`masses[0] * acc[0] + masses[1] * acc[1]` が、丸め誤差の範囲で 0 になっているか確かめてください。これは軌道を積分する前にできる確認です。

2.6 KDK で両天体を同時に進める

2.6.1 積分器ではなく加速度関数を替える

時刻 t_n における 2 天体の状態が分かっているとします。KDK を、それぞれの天体 $k = 1, 2$ に対して

$$\begin{aligned} \boldsymbol{V}_k^{n+1/2} &= \boldsymbol{V}_k^n + \frac{\Delta t}{2}\boldsymbol{A}_k^n, \\ \boldsymbol{R}_k^{n+1} &= \boldsymbol{R}_k^n + \Delta t \boldsymbol{V}_k^{n+1/2}, \\ \boldsymbol{A}_k^{n+1} &= \boldsymbol{A}_k(\boldsymbol{R}_1^{n+1}, \boldsymbol{R}_2^{n+1}), \\ \boldsymbol{V}_k^{n+1} &= \boldsymbol{V}_k^{n+1/2} + \frac{\Delta t}{2}\boldsymbol{A}_k^{n+1} \end{aligned} \tag{2.20}$$

の順に行います。半整数ステップの速度は途中で使う値です。履歴には最後の kick を終えた、整数時刻の位置と速度を保存します。KDK の性質と導出は第 1 章で学んだ内容を使います。[5]

両天体の最初の kick を終えてから両天体を drift し、その**新しい 2 位置をそろえて**次の加速度を計算します。配列全体を更新することで、この順序を簡単に保てます。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, masses):
    """KDK。acc は現在の pos に対応する加速度です。"""
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, masses)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

第 1 章から変えたのは、最後の引数名を `mu` から `masses` へ替えた点です。今回は加速度関数に二つの質量を渡す必要があるためです。KDK の 4 行の計算と、加速度を返して再利用する構造は変わりません。

よくある失敗：片方だけを先に次の時刻へ進めない

天体 1 の位置を更新してから、その新しい位置と天体 2 の古い位置を使って天体 2 を更新すると、式(2.20)とは別の計算になります。同じ時刻に対する作用・反作用もそろいません。全天体の drift を終えてから、次の加速度を一度に求めます。

2.6.2 更新を繰り返して履歴を作る

最初の加速度だけをループの前で求めます。その後は、前ステップで返された加速度を次の最初の kick に使います。

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, masses):
    masses = check_masses(masses)
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, masses)  # 初回だけ計算します。
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, masses)
        history[n + 1] = np.column_stack((pos, vel))
        if not np.all(np.isfinite(history[n + 1])):
            raise FloatingPointError("Non-finite state; check the time step.")
    return times, history
```

`prepare_history` は第 1 章と同じ役割の補助関数ですが、今回は初期状態の形 `(2, 4)` を確認します。正のステップ数と最終時刻、有限の初期値を確認し、コピーと保存領域を用意します。履歴の形は `(n_steps + 1, 2, 4)` です。

`history[n, 1, :2]` は時刻 t_n における天体 2 の位置です。最初の軸が時間、次が天体、最後が位置・速度の成分であることを、確認してください。初期値も保存するので、出力時刻の数はステップ数より 1 つ多くなります。

本章でも、N_{step} ステップのための加速度関数の呼び出しは $N_{\mathrm{step}}+1$ 回です。1 回の呼び出しで**両方の加速度**を求めます。1000 ステップなら 1001 回です。粒子対の計算が 1 組なので、高速化手法や並列化は必要ありません。

2.6.3 3 ケースを同じ条件で計算する

各ケースの初期状態を作り直し、独立に積分します。一つの計算の最終状態を、次のケースの初期状態へ流用しないようにします。

```
cases = {}
start = perf_counter()
for name, eta in eta_values.items():
    initial = make_initial_state(masses, [d0, 0.0], [0.0, eta * v_c])
    times, history = integrate_leapfrog(acceleration_two_body, initial, t_end, n_steps,
    ↪  masses)
    cases[name] = {"eta": eta, "initial": initial, "times": times, "history": history}
```

本章の時間積分はすべて KDK です。RK4 や Euler 法との再比較は行いません。今回は積分法ではなく、初期条件、相対運動、保存量に注目します。

2.7 重心系と相対座標で軌道を見る

2.7.1 2 天体の軌道中心を確認する

まず $\eta = 0.8$ の楕円の場合に、二つの軌道を別々に描きます。青実線と丸が天体 1、朱色破線と四角が天体 2、黒い星印が重心です。両天体の軌道を同じ軸に重ねると、重い天体が重心の近くを動いていることが分かります。

```
history = cases["Elliptic"]["history"]
fig, ax = plt.subplots(figsize=(5.8, 5.2), layout="constrained")
ax.plot(history[:, 0, 0], history[:, 0, 1], color=BLUE, label="Body 1")
ax.plot(history[:, 1, 0], history[:, 1, 1], color=VERMILLION, ls="--", label="Body 2")
ax.plot(history[0, 0, 0], history[0, 0, 1], "o", color=BLUE, ms=6)
ax.plot(history[0, 1, 0], history[0, 1, 1], "s", color=VERMILLION, ms=6)
ax.scatter([0.0], [0.0], marker="*", color="black", s=100, label="Center of mass")
ax.set(xlabel="x [kpc]", ylabel="y [kpc]", title="Elliptic orbit; mass ratio 2:1")
ax.set_aspect("equal")
ax.legend(fontsize=9)
save_figure(fig, "ch02_barycentric")
```

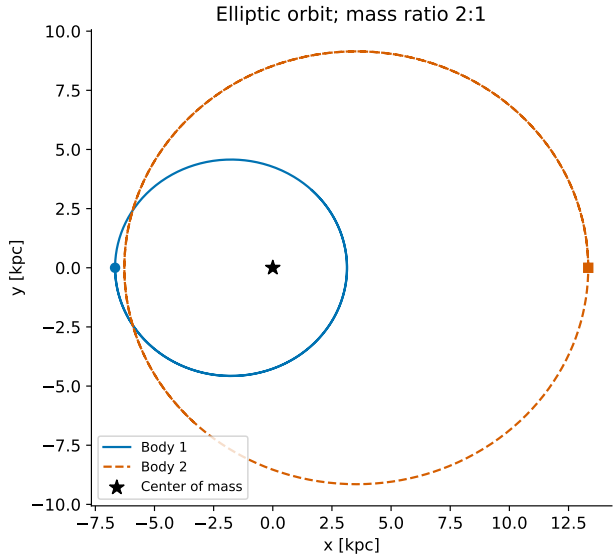


図 2.2 質量比 $M_1 : M_2 = 2 : 1$ の楕円軌道を重心系で表示した結果。丸・四角は初期位置です。二つの軌道は重心を共通の焦点とし、大きさの比は 1 : 2 です。計算時間は T_{c} で、楕円の約 1.586 周期に相当します。

原点に星印があるからといって、そこに第 3 の質量を置いたわけではありません。力を及ぼしているのは、動いている 2 天体だけです。重心は、二つの質量から定義する位置です。

Notebook ではこの履歴からアニメーションも作ります。画面の再生間隔を変えても、計算した軌道や物理的な時間は変わりません。本文の図も保存することで、動画なしでも軌道の違いを読めるようにします。

2.7.2 相対軌道の形を比較する

履歴の各時刻で天体 2 の位置から天体 1 の位置を引くと、相対軌道が得られます。式(2.10)を満たす軌道として、円・楕円・放物線を比べられます。

本章の接線速度から始める初期条件では、相対位置の極角を ϕ として、解析的な形を

$$D(\phi) = \frac{\eta^2 D_0}{1 + (\eta^2 - 1)\cos\phi} \tag{2.21}$$

と書けます。これは Kepler 軌道の式へ初期条件を代入し、$\phi = 0$ を初期位置に選んだものです。[1] $0 < \eta < 1$ ではその位置が遠点なので、通常の「近点を角度 0 にする」式と分母の符号が異なって見えます。

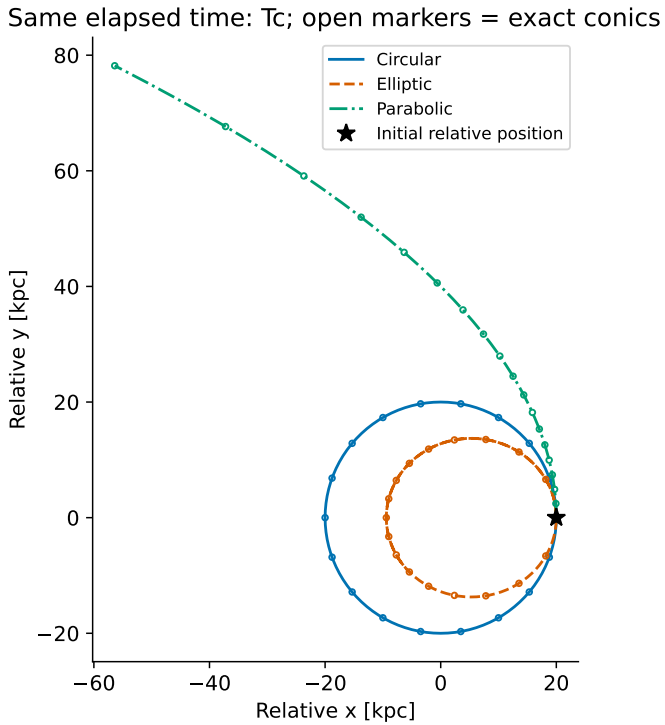


図 2.3 同じ D_0 から、初期接線速度だけを変えた相対軌道。3 ケースとも同じ T_c まで計算しています。白抜き丸は式(2.21)による解析的な軌道形状上の点で、等時間間隔の点ではありません。星印は共通の初期相対位置です。

図2.3の原点は相対座標での基準点です。この図だけでは、天体 1 が元の慣性系で静止しているようにも見えますが、実際の計算では図2.2のように両天体が動いています。

また、解析的な曲線に数値解が重なっても、時刻と位置の対応まで正しいとは限りません。軌道の形の確認と、同じ時刻における位置の確認を分けて行います。

2.8 2 体系の保存量を確かめる

2.8.1 全エネルギーと全角運動量を求める

第 1 章では 1 粒子の比エネルギーを計算しました。今回は二つの質量を掛けた**全エネルギー**を計算します。2 天体の運動エネルギーと、1 組の粒子対のポテンシャルエネルギーは

$$K = \frac{1}{2}M_1V_1^2 + \frac{1}{2}M_2V_2^2, \qquad W = -\frac{GM_1M_2}{D}, \qquad E = K + W \tag{2.22}$$

です。距離 D が変わると K と W はそれぞれ変わりますが、その和は一定です。例えば

$$\frac{dK}{dt} = -\frac{GM_1M_2}{D^3}\boldsymbol{D}\cdot\boldsymbol{U}, \qquad \frac{dW}{dt} = +\frac{GM_1M_2}{D^3}\boldsymbol{D}\cdot\boldsymbol{U} \tag{2.23}$$

となり、変化率が打ち消し合います。**片方の天体のエネルギーだけを保存量としない**ことが、第 1 章との違いです。

固定した原点の周りの全角運動量の z 成分は

$$L_z = \sum_{k=1}^{2} M_k(X_kV_{ky} - Y_kV_{kx}) \tag{2.24}$$

です。二つの内力は同じ直線上で逆向きに働くので、全トルクは 0 です。全角運動量も保存されます。全運動量 $\boldsymbol{P}$ と重心の等速直線運動は、式(2.8)、式(2.9)で確かめます。

よくある失敗：ポテンシャルエネルギーを 2 回足さない

粒子対のエネルギーは $-GM_1M_2/D$ を 1 回だけ数えます。$M_1\Phi_2(\boldsymbol{R}_1) + M_2\Phi_1(\boldsymbol{R}_2)$ をそのまま足すと、同じ相互作用を 2 回数えてしまいます。各天体で計算してから足す書き方では、全体に $1/2$ が必要です。

2.8.2 全エネルギーと相対運動の比エネルギーを区別する

式(2.15)の ε と、式(2.22)の E は別の量です。この関係を整理するため、換算質量を

$$m_{\mathrm{red}} = \frac{M_1M_2}{M_{\mathrm{tot}}} \tag{2.25}$$

とします。式(2.11)を K へ代入すると、重心速度と相対速度の交差項が消え、

$$E = \frac{1}{2}M_{\mathrm{tot}}V_{\mathrm{cm}}^2 + m_{\mathrm{red}}\varepsilon \tag{2.26}$$

です。同じように、

$$L_z = M_{\mathrm{tot}}(\boldsymbol{R}_{\mathrm{cm}} \times \boldsymbol{V}_{\mathrm{cm}})_z + m_{\mathrm{red}}\ell_z \tag{2.27}$$

となります。重心系では、$E = m_{\mathrm{red}}\varepsilon$、$L_z = m_{\mathrm{red}}\ell_z$ です。[1]

$\mu = GM_{\mathrm{tot}}$ は重力の強さを表すパラメータ、m_{red} は質量です。同じ記号で呼ばないようにします。ε の単位は $\mathrm{kpc}^2\,\mathrm{Gyr}^{-2}$、$E$ はそれに $M_\odot$ を掛けた単位です。また、束縛・非束縛の分類に使うのは**相対運動の** ε です。系全体を速く動かしたときの、任意の慣性系での E の符号ではありません。

2.8.3 履歴から数値を求める

位置と速度は、KDK の最後の kick まで終えた同じ時刻の値を使います。以下では履歴から各天体の量を取り出し、全エネルギー、全角運動量、全運動量、重心をまとめて返します。

```
def diagnostics(history, masses):
    pos, vel = history[:, :, :2], history[:, :, 2:]
    dr, dv = pos[:, 1] - pos[:, 0], vel[:, 1] - vel[:, 0]
    separation = np.linalg.norm(dr, axis=1)
    kinetic = 0.5 * masses[0] * np.sum(vel[:, 0]**2, axis=1)
    kinetic += 0.5 * masses[1] * np.sum(vel[:, 1]**2, axis=1)
    potential = -G * masses[0] * masses[1] / separation
    energy = kinetic + potential
    momentum = masses[0] * vel[:, 0] + masses[1] * vel[:, 1]
    r_cm = (masses[0] * pos[:, 0] + masses[1] * pos[:, 1]) / np.sum(masses)
    angular = masses[0] * (pos[:, 0, 0] * vel[:, 0, 1] - pos[:, 0, 1] * vel[:, 0, 0])
    angular += masses[1] * (pos[:, 1, 0] * vel[:, 1, 1] - pos[:, 1, 1] * vel[:, 1, 0])
    return {"D": separation, "dr": dr, "dv": dv, "K": kinetic, "W": potential,
            "E": energy, "L": angular, "P": momentum, "Rcm": r_cm}
```

この関数は各保存量を保存するように補正するものではありません。得られた履歴を読み取り、値を計算するだけです。結果を見て毎ステップ重心や速度を修正すると、もとの積分器の誤差を評価できなくなります。

2.8.4 初期値が 0 の量は相対誤差にしない

重心系では $\boldsymbol{P}(0) = \boldsymbol{0}$ です。放物線では、理論的に $E(0) = 0$ でもあります。これらを分母にした「相対誤差」は定義できません。浮動小数点計算で初期エネルギーが小さな非零値になっていても、それで割るのは不適切です。

そこで、本章では初期距離と相対円運動速度から、共通の正の基準量を作ります。

$$E_{\mathrm{scale}} = \frac{GM_1M_2}{D_0}, \qquad P_{\mathrm{scale}} = M_{\mathrm{tot}}v_{\mathrm{c}}. \tag{2.28}$$

保存した全時刻にわたる最大値を使い、

$$\begin{aligned} e_E &= \max_n \frac{|E^n - E^0|}{E_{\mathrm{scale}}}, & e_L &= \max_n \frac{|L_z^n - L_z^0|}{|L_z^0|}, \\ e_P &= \max_n \frac{|\boldsymbol{P}^n - \boldsymbol{P}^0|}{P_{\mathrm{scale}}}, & e_{\mathrm{cm}} &= \max_n \frac{|\boldsymbol{R}_{\mathrm{cm}}^n - \boldsymbol{R}_{\mathrm{cm}}^{\mathrm{exact}}(t_n)|}{D_0} \end{aligned} \tag{2.29}$$

とします。$\boldsymbol{R}_{\mathrm{cm}}^{\mathrm{exact}}$ は式(2.9)です。本章の初期条件では $L_z^0 \neq 0$ なので、角運動量にだけその初期値を使えます。動径軌道など $L_z^0 = 0$ の問題では、角運動量にも別の基準量が必要です。

```
def conservation_errors(times, history, masses):
    values = diagnostics(history, masses)
    d_initial = values["D"][0]
    vc_initial = np.sqrt(G * np.sum(masses) / d_initial)
    e_scale = G * masses[0] * masses[1] / d_initial
    p_scale = np.sum(masses) * vc_initial
    r_cm_exact = values["Rcm"][0] + times[:, None] * values["P"][0] / np.sum(masses)
    err_e = np.max(np.abs(values["E"] - values["E"][0])) / e_scale
    l_scale = abs(values["L"][0])
    if l_scale == 0.0:
        raise ValueError("This angular-momentum normalization requires nonzero L(0).")
    err_l = np.max(np.abs(values["L"] - values["L"][0])) / l_scale
    err_p = np.max(np.linalg.norm(values["P"] - values["P"][0], axis=1)) / p_scale
    err_cm = np.max(np.linalg.norm(values["Rcm"] - r_cm_exact, axis=1)) / d_initial
    return np.array([err_e, err_l, err_p, err_cm])
```

表 2.3 3 ケースの保存量・重心の確認。いずれも T_c まで 1000 ステップの KDK です。エネルギーは $|E(0)|$ ではなく、式(2.28)で規格化しています。

軌道	e_E	e_L	e_P	e_{cm}
円	1.95×10^{-10}	2.55×10^{-15}	0	0
楕円	5.72×10^{-5}	5.42×10^{-15}	0	0
放物線	5.32×10^{-6}	4.87×10^{-15}	0	0

表2.3は、配布コードの実行結果です。表示された 0 は、この質量比と対称な初期配置をこの実装で計算したとき、対応する引き算が浮動小数点演算でも 0 になったことを意味します。任意の質量比・初期値で誤差が必ず 0 になる、という主張ではありません。後で重心を動かした場合も確認します。

2.8.5 保存量ごとに誤差が異なる理由を考える

KDK では、同じ位置から求めた二つの力が打ち消し合うため、kick は全運動量を保ちます。drift は速度を変えません。全角運動量についても、kick での全トルクが 0 で、drift の追加項は $\boldsymbol{V}_k \times \boldsymbol{V}_k = 0$ です。このため、本章の中心力の粒子対を正しく計算すれば、全運動量と全角運動量のずれは丸め誤差程度になります。

一方、KDK は全エネルギーを毎ステップ厳密に保存する方法ではありません。図2.4のように、有限の刻みによる差が現れます。全運動量と全角運動量がよく保存されていても、エネルギーや軌道の位相まで同じ精度で正しいとは限りません。

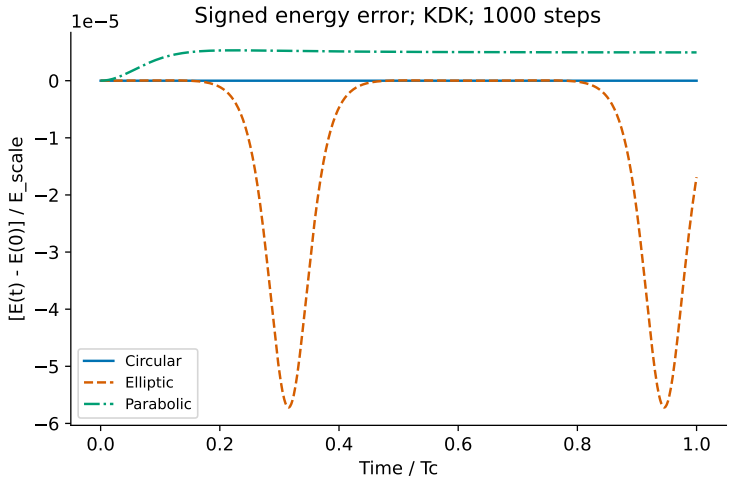


図 2.4 エネルギー差の時間変化。変化の向きも分かるよう、絶対値を取らない $[E(t) - E(0)]/E_{\rm scale}$ を描いています。楕円で差が大きくなる時期は、近点付近に対応します。

2.9 軌道の時間変化と刻みを確認する

2.9.1 近点と遠点を測る

式(2.18)と式(2.19)によれば、$\eta = 0.8$ の標準例は $e = 0.36$、$a_{\rm orb} = 14.705882\,{\rm kpc}$、$D_{\rm p} = 9.411765\,{\rm kpc}$ です。周期は $T_{\rm ell} = 0.431357\,{\rm Gyr}$ で、基準の円軌道周期 $T_{\rm c}$ より短くなります。

図2.5の円軌道では、距離はほぼ一定です。楕円では近点と遠点の間を往復し、放物線では近点から遠ざかります。楕円は $T_{\rm c}$ の間に 1 周以上するので、距離の谷が複数現れます。

標準計算で保存された楕円の最小距離は $9.412506\,{\rm kpc}$ でした。解析値との差には、時間積分の誤差に加えて、**真の近点通過時刻と保存時刻が一致しない**影響も含まれます。`np.argmin` で得るのは、保存された配列の最小要素です。連続時間の最小値そのものを解いているわけではありません。

2.9.2 保存量とは別に位置を比較する

円軌道には、第 1 章と同じ形の時刻に依存する解析解があります。

$$\boldsymbol{D}_{\rm circ}^{\rm exact}(t) = D_0\,(\cos\Omega t, \sin\Omega t)\,, \qquad \Omega = \sqrt{\frac{\mu}{D_0^3}}. \tag{2.30}$$

同じ時刻における数値相対位置との差を取り、

$$e_{\rm circ} = \max_n \frac{|\boldsymbol{D}^n - \boldsymbol{D}_{\rm circ}^{\rm exact}(t_n)|}{D_0} \tag{2.31}$$

を測ります。半径の差ではなく位置ベクトルの差なので、位相のずれも検出します。

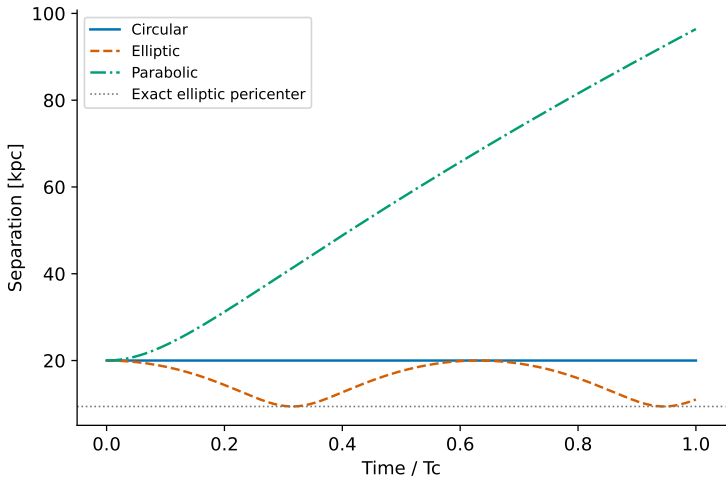


図 2.5 2 天体間距離の時間変化。横軸の $T_{\rm c}$ は 3 ケース共通の基準時間で、楕円の周期ではありません。点線は楕円の場合の解析的な近点距離です。

```
def exact_relative_circle(times, d0, mu):
    omega = np.sqrt(mu / d0**3)
    phase = omega * np.asarray(times)
    return np.column_stack((d0 * np.cos(phase), d0 * np.sin(phase)))
```

楕円では任意時刻の位置を解く代わりに、**解析的に求めた周期** $T_{\rm ell}$ を使います。ちょうど 1 周期後には初期位置へ戻るはずなので、

$$e_{\rm ret} = \frac{|\boldsymbol{D}^{N_{\rm step}} - \boldsymbol{D}(0)|}{D_0}, \qquad t_{N_{\rm step}} = T_{\rm ell} \tag{2.32}$$

を確認します。これは 1 周期後の戻り位置の誤差であり、途中も含めた最大誤差ではありません。数値解が戻った時刻を探してから、そこを周期と呼ぶ方法ではない点が重要です。

2.9.3 同じ最終時刻で刻みを細かくする

円軌道は $T_{\rm c}$ まで、楕円は $T_{\rm ell}$ まで計算し、それぞれの系列の中でステップ数だけを変えます。二つの系列の最終時刻は異なりますが、同じ系列では固定されています。二つの誤差の定義も異なるため、値の大小をそのまま「円と楕円の難しさ」の尺度にはしません。

```
step_counts = [100, 200, 400, 800]
convergence_rows = []
for count in step_counts:
    tc, hc = integrate_leapfrog(acceleration_two_body, cases["Circular"]["initial"], t_c, count, masses)
    dr_c = hc[:, 1, :2] - hc[:, 0, :2]
    exact_c = exact_relative_circle(tc, d0, mu)
    err_c = np.max(np.linalg.norm(dr_c - exact_c, axis=1)) / d0
    te, he = integrate_leapfrog(acceleration_two_body, cases["Elliptic"]["initial"], t_ell, count, masses)
    dr_end = he[-1, 1, :2] - he[-1, 0, :2]
    err_ell = np.linalg.norm(dr_end - np.array([d0, 0.0])) / d0
    err_energy_ell = conservation_errors(te, he, masses)[0]
    convergence_rows.append([count, t_c / count, t_ell / count, err_c, err_ell, err_energy_ell])
convergence = np.array(convergence_rows)
orders_c = np.log2(convergence[:-1, 3] / convergence[1:, 3])
orders_ell = np.log2(convergence[:-1, 4] / convergence[1:, 4])
```

表 2.4　KDK の時間刻み試験。円は解析的な 1 周期までの最大位置誤差、楕円は解析的な 1 周期後の戻り位置の誤差を示します。p は一つ前の行との誤差比から求めた推定次数です。

N_{step}	e_{circ}	p	e_{ret}	p
100	8.63×10^{-3}	—	9.13×10^{-3}	—
200	2.16×10^{-3}	1.9985	2.29×10^{-3}	1.9927
400	5.40×10^{-4}	1.9996	5.74×10^{-4}	1.9982
800	1.35×10^{-4}	1.9999	1.44×10^{-4}	1.9995

刻みを半分にしたときの誤差比から、

$$p = \frac{\log[e(\Delta t)/e(\Delta t/2)]}{\log 2} \tag{2.33}$$

を求めます。表2.4の最も細かい刻み側では、円が 1.9999、楕円が 1.9995 で、2 へ近づいています。第 1 章の 1 粒子の場合から 2 体へ変えても、KDK の 2 次精度を確認できます。

近点付近では距離が小さくなり、速度と加速度が大きく変わります。同じ刻みでも、円軌道より厳しい条件になることがあります。離心率をさらに大きくするときや、近点距離を小さくするときには、その新しい条件で刻み試験を行います。標準条件で得た誤差を、そのまま別の軌道の誤差とはみなしません。

もう少し詳しく：近点通過時刻の決め方

保存された距離の最小値を使う以外に、距離を時間の関数として補間したり、動径速度 $\boldsymbol{D}\cdot\boldsymbol{U}/D$ が 0 になる時刻を探したりする方法もあります。本章では新しい補間法・求根法を追加せず、最小値が保存時刻に依存することを明示します。補間と求根の基本は、第 3・4 章で別の具体例から学びます。

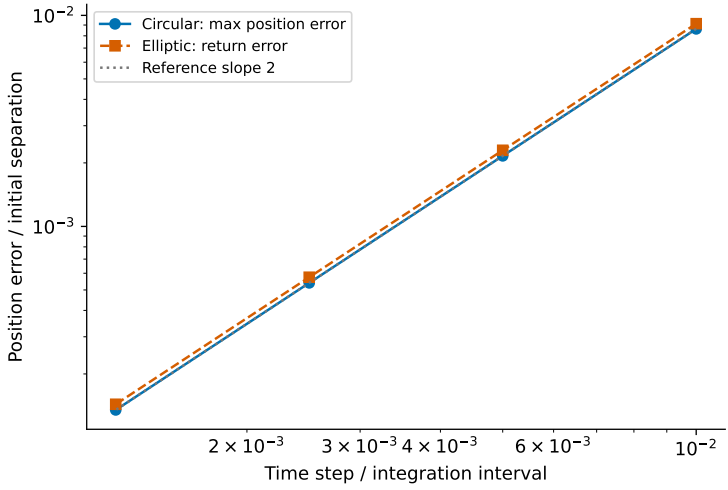


図 2.6 時間刻みと位置誤差の両対数表示。横軸は各系列の積分区間に対する刻みの比で、どちらも $1/N_{\mathrm{step}}$ です。点線は円の最も細かい刻みの点を通る傾き 2 の参照線です。数値データへの当てはめではありません。

2.10 近点距離を指定して接近させる

2.10.1 遠くから内向きに始める

これまでの放物線は、動径速度 0 の近点から始めました。銀河相互作用の準備では、離れた位置から互いに近づく初期条件が便利です。ここでは初期距離 D_0 と目標近点距離 D_p を指定し、$0 < D_\mathrm{p} < D_0$ とします。

放物線では $\varepsilon = 0$ です。近点では速度は接線方向だけなので、式(2.15)から

$$U_\mathrm{p}^2 = \frac{2\mu}{D_\mathrm{p}}, \qquad \ell_z = D_\mathrm{p} U_\mathrm{p} = \sqrt{2\mu D_\mathrm{p}} \tag{2.34}$$

です。角運動量保存より、初期距離での接線速度は $U_t = \ell_z / D_0$ です。初期位置でもエネルギー 0 とし、内向きの符号を選ぶと、

$$U_t = \frac{\sqrt{2\mu D_\mathrm{p}}}{D_0}, \qquad U_r = -\sqrt{\frac{2\mu}{D_0} - U_t^2} \tag{2.35}$$

になります。初期位置を $\boldsymbol{D} = (D_0, 0)$ とすれば、$\boldsymbol{U} = (U_r, U_t)$ をそのまま渡せます。ここでも原点を重心にした各天体の初期値へ、式(2.11)で変換します。

```
def make_parabolic_encounter(masses, d_start, d_peri):
    masses = check_masses(masses)
    if not (np.isfinite(d_start) and np.isfinite(d_peri) and 0.0 < d_peri < d_start):
        raise ValueError("Require finite distances with 0 < d_peri < d_start.")
    mu = G * np.sum(masses)
    ell = np.sqrt(2.0 * mu * d_peri)
    vt = ell / d_start
    vr = -np.sqrt(2.0 * mu / d_start - vt**2)
    return make_initial_state(masses, [d_start, 0.0], [vr, vt])

encounter_initial = make_parabolic_encounter(masses, 60.0, 8.0)
t_enc, h_enc = integrate_leapfrog(acceleration_two_body, encounter_initial, 0.60, 1200,
↪  masses)
enc_values = diagnostics(h_enc, masses)
i_peri = np.argmin(enc_values["D"])
```

2.10.2 指定した近点の再現性を確かめる

この追加実験では、質量は標準例のまま、$D_0 = 60\,\mathrm{kpc}$、$D_\mathrm{p} = 8\,\mathrm{kpc}$、最終時刻 $0.60\,\mathrm{Gyr}$、1200 ステップとします。刻みは $0.5\,\mathrm{Myr}$ です。3 ケース比較の D_0 や T_c と混同しないよう、条件を分けて記録します。

得られる初期相対速度は、$U_r = -136.518\,\mathrm{km\,s^{-1}}$、$U_t = 53.547\,\mathrm{km\,s^{-1}}$ です。内向きの動径速度が負であることを確認してください。計算後に保存された最小距離は $8.000330\,\mathrm{kpc}$、その保存時刻は $0.3145\,\mathrm{Gyr}$ でした。指定した $8\,\mathrm{kpc}$ へ近づき、その後は離れていきます。

近づいた後に離れていくことは、計算の失敗ではありません。摩擦のない点質量 2 体では相対エネルギーが保存されるため、放物線遭遇が自動的に束縛軌道へ変わることはありません。近接通過と合体は区別します。

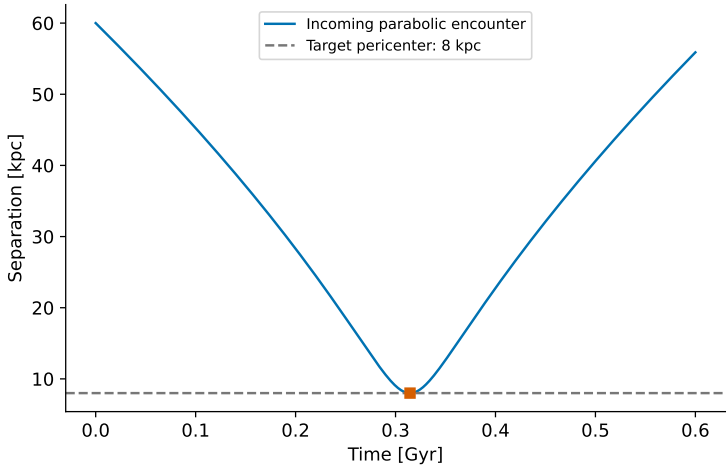


図 2.7 点質量 2 体の内向きに始まる放物線遭遇。破線は指定した近点距離、四角は保存された距離の最小値です。これは本章の質量比 2 : 1 の計算であり、後の等質量 Plummer 銀河の標準実験とは別の条件です。

本章の式(2.35)は、ポテンシャルが $-\mu/D$ の場合の式です。第 5 章で有限サイズを表す中心間重力へ替えたとき、μ だけを同じにしてこの速度を使えば、同じ近点距離が得られるとは限りません。**重力のモデルを変更したら、そのモデルに対応するエネルギーと初期条件を使う**という原則を引き継ぎます。

2.11 条件を一つずつ変えて確かめる

2.11.1 重心を等速で動かす

計算の原点や、系全体の等速運動を変えても、相対運動は変わらないはずです。式(2.11)の重心位置を $(5, -3)\,\mathrm{kpc}$、重心速度を $(30, -20)\,\mathrm{km\,s^{-1}}$ として、同じ楕円を計算します。

```
r_boost = np.array([5.0, -3.0])
v_boost = np.array([30.0, -20.0]) * KM_S_TO_KPC_GYR
boost_initial = make_initial_state(masses, [d0, 0.0], [0.0, 0.8 * v_c], r_boost, v_boost)
t_boost, h_boost = integrate_leapfrog(acceleration_two_body, boost_initial, t_c, n_steps,
    masses)
bv = diagnostics(h_boost, masses)
base = diagnostics(cases["Elliptic"]["history"], masses)
boost_relative_error = np.max(np.linalg.norm(bv["dr"] - base["dr"], axis=1)) / d0
boost_errors = conservation_errors(t_boost, h_boost, masses)
```

相対軌道の差を D_0 で割った最大値は 4.87×10^{-14}、重心が式(2.9)の直線から外れる量は D_0 単位で 4.06×10^{-15} でした。これらは本実装での丸め誤差程度の差です。重心系の計算でたまたま誤差が 0 になった場合だけでなく、移動する重心でも確認する意味があります。

このとき、任意の原点の周りの E や L_z の初期値は、重心系と同じとは限りません。比較すべきなのは、それぞれの系で保存する量と、式(2.26)・式(2.27)の関係です。

2.11.2 全質量を固定して質量比を変える

次は $M_{\rm tot}, D_0, \boldsymbol{U}(0)$ を固定し、$q = M_2/M_1$ だけを変えます。相対運動の式に現れるのは $\mu = GM_{\rm tot}$ なので、相対軌道は同じです。しかし、各天体が重心から離れる距離は変わります。

表 2.5 全質量を固定した質量比の比較。相対軌道の差は、標準の $q = 0.5$ との差の最大値です。初期相対速度は $0.8v_{\rm c}$ に固定しています。

$q = M_2/M_1$	$\lvert\boldsymbol{R}_1(0)\rvert$ [kpc]	$\lvert\boldsymbol{R}_2(0)\rvert$ [kpc]	相対軌道の差 $/D_0$
1.0	10.000000	10.000000	1.57×10^{-14}
0.5	6.666667	13.333333	0
0.1	1.818182	18.181818	4.04×10^{-14}

M_1 を固定したまま M_2 を変えると、$M_{\rm tot}$ も変わるので別の実験です。パラメータを変えたときには、何を固定し、何が連動して変わるかを記録します。これは銀河相互作用の比較実験にも必要な習慣です。

2.12 本章のまとめ

本章では、互いに重力を及ぼす 2 天体を KDK で動かしました。第 1 章から積分法を作り直すのではなく、2 天体の同じ時刻の位置から加速度を返す関数へ替えました。最後に評価した加速度を再利用し、両天体の位置と速度を整数時刻にそろえて保存しました。

重心は全質量による重み付き平均です。相対運動には $\mu = G(M_1 + M_2)$ が入り、各天体の軌道は重心運動と相対運動から作られます。初期接線速度を変えると、円・楕円・放物線を作れることも確認しました。

保存量は、今回は 2 体系の全エネルギー・全角運動量・全運動量です。放物線のように初期エネルギーが 0 の問題では、その値で誤差を割りません。保存量だけでなく、円の同時刻の位置、楕円の 1 周期後の戻り位置、刻みを変えた収束も確かめました。

最後に、初期距離と近点距離から内向きの接近軌道を作りました。ただし、本章の重力源はまだ点質量です。第 3 章では、質量に広がりをもつ球対称の Plummer モデルを導入し、その中の星の運動へ進みます。点質量の式と広がった銀河の式を、混同しないことが重要です。

演習

演習 2.1　確認：重心と作用・反作用　$M_1 = 10^{11} M_\odot$、$M_2 = 5 \times 10^{10} M_\odot$、$D_0 = 20\,\mathrm{kpc}$ の円軌道を重心系に置いてください。初期位置と各天体の速度を求め、全運動量が 0 になることを示してください。また、初期加速度の大きさの比と、力の大きさの比を求めてください。

演習 2.2　確認：楕円の形と周期　同じ D_0 で $\eta = 0.8$ としたとき、初期位置が遠点である理由を説明してください。$e, a_{\mathrm{orb}}, D_{\mathrm{p}}, T_{\mathrm{ell}}$ を初期条件から求め、数値計算の距離の時間変化と比較してください。保存された最小距離と解析的な近点距離が完全には一致しない理由も述べてください。

演習 2.3　確認：放物線のエネルギー誤差　$\eta = \sqrt{2}$ の計算で、$|E(t) - E(0)|/|E(0)|$ を誤差に使わない理由を説明してください。本章の E_{scale} で規格化すると、どの物理的な大きさに対するずれを測ることになるでしょうか。

演習 2.4　応用：全質量を固定する　表2.5と同じ条件で $q = 1, 0.5, 0.1$ を比較してください。各天体の重心からの距離が変わっても、相対軌道が同じになることを確認してください。M_1 だけを固定して q を変えた場合と、何が異なるか説明してください。

演習 2.5　応用：移動する重心　重心に位置 $(5, -3)\,\mathrm{kpc}$、速度 $(30, -20)\,\mathrm{km\,s^{-1}}$ を与えてください。重心が等速直線運動し、相対軌道がもとの楕円と同じになるか確かめてください。任意の慣性系での全エネルギー E の符号だけでは、束縛軌道かどうかを判断できない理由も説明してください。

演習 2.6　応用：KDK の時間反転　標準の楕円を T_{c} まで計算し、その最終位置を保ったまま、**両天体の速度**を符号反転してください。同じ正の時間刻みでさらに T_{c} だけ進めたとき、初期位置と初期速度の符号反転へ戻るか確認してください。時間反転の試験に通ることだけで、軌道の位相誤差が小さいと言えるでしょうか。

演習 2.7　発展：脱出速度を超える軌道　$\eta = 1.6$ として、ほかは 3 ケース比較と同じ条件で計算してください。相対エネルギーの符号と離心率を求め、軌道が双曲線になることを確認してください。T_{c} 後の距離を放物線の場合と比べ、全エネルギーの誤差も確認してください。積分器の変更は必要ありません。

第 3 章　銀河の中で星を動かす

第 2 章では、二つの天体を点質量として動かしました。ここからは、銀河の内部へ目を向けます。銀河の質量は中心の 1 点だけに集まっているわけではありません。中心からの距離によって、内側に含まれる質量も、星が受ける重力も変わります。本章では、その広がりを球対称の Plummer モデルで表します。

新しく学ぶ数値計算法は、**台形則**と**二分法**です。密度を積分して質量分布を求め、その質量の半分を含む半径を探します。さらに、第 1・2 章で使った KDK leapfrog で、銀河の重力場中の星を動かします。数値積分、求根、時間積分を、一つの銀河モデルの中で区別して使うことが目標です。

この章で学ぶこと

- 球対称の密度、内側の質量、ポテンシャル、重力加速度の関係を説明できます。
- 台形則を実装し、Plummer 球の質量分布を解析式と比較できます。
- 二分法で半質量半径を求め、求根の残差と半径の誤差を区別できます。
- Plummer ポテンシャルに対応する円運動速度を与え、KDK で円軌道と非円軌道を計算できます。
- 積分範囲、空間の分割、求根の許容幅、時間刻みが、それぞれ何に影響するかを確認できます。

3.1 質点のモデルを広がった銀河へ拡張する

3.1.1 星を表す 1 粒子を動かす

原点に中心を持つ、静的で球対称な銀河を考えます。その中を動く星を 1 個のテスト粒子で表します。粒子は銀河の重力を受けますが、銀河の密度やポテンシャルを変えません。銀河自身の粒子を多数用意して、その自己重力を計算するわけではありません。

第 2 章のように二つの有限質量が互いに動く問題とは違い、今回は重力場を固定します。これは銀河相互作用を始める前に、1 個の銀河の重力を理解するための段階です。銀河中心が動く場合は第 5 章で扱います。

粒子の位置を $\boldsymbol{r} = (x, y)$、速度を $\boldsymbol{v} = (v_x, v_y)$ とし、$r = |\boldsymbol{r}|$ を原点からの距離とします。銀河は 3 次元の球ですが、初期位置と初期速度を x–y 平面内に選べば、中心力による運動はその平面内にとどまります。**2 次元で軌道を描くことと、2 次元の質量分布を仮定することは別です。**[2]

3.1.2 球の大きさと計算する範囲を区別する

本章では、球全体の質量を M、その広がりを表すスケール長を a とします。a は第 2 章で使った Kepler 楕円の軌道長半径 $a_{\rm orb}$ ではありません。また、加速度ベクトル $\boldsymbol{a}$ とも区別します。

Plummer モデルの密度は、有限の半径で突然 0 にはなりません。外側ほど小さくなり、無限遠まで続きますが、全質量は有限です。後で使う $r_{\rm tab}$ は、質量の表を作る範囲の上限です。$r = r_{\rm tab}$ で密度や重力を切断するという意味ではありません。

3.2 密度から質量と重力を求める

3.2.1 薄い球殻に含まれる質量を求める

球対称なら、密度 $\rho(r)$ は半径だけの関数です。半径 r から $r + dr$ までの薄い球殻の体積は $4\pi r^2 dr$ ですから、その質量は

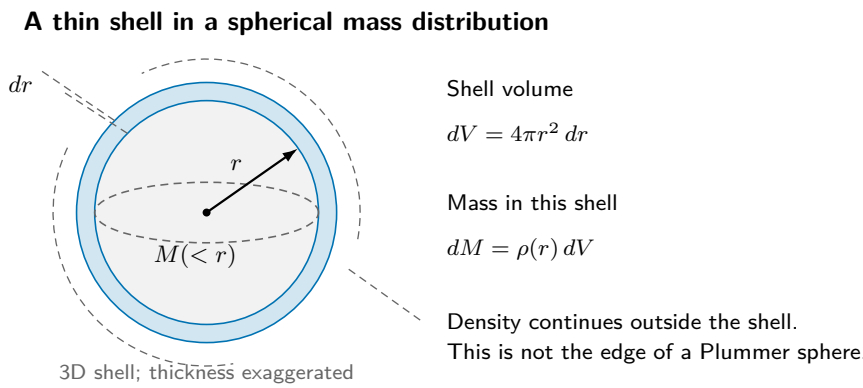
$$dM = 4\pi r^2 \rho(r)\, dr \tag{3.1}$$

です。中心から半径 r まで足し合わせると、内側の質量は

$$M(<r) = 4\pi \int_0^r r'^2 \rho(r')\, dr' \tag{3.2}$$

となります。r' は積分の途中で使う半径です。$M(<r)$ は r に依存する量、M は球全体の一定の質量として使い分けます。

図3.1では、薄い球殻とその内側を分けて示しています。図は断面ですが、積分する質量には 3 次元の球殻の体積を使います。



$$M(< r) = 4\pi \int_0^r s^2 \rho(s)\, ds$$

図 3.1 3 次元の球殻を断面で示した図。色の付いた殻の厚さ dr を強調しており、殻の体積は $4\pi r^2 dr$ です。内側の薄灰色の領域が $M(<r)$ に対応します。殻の外にも密度は続き、この円を Plummer 球の外縁とはみなしません。

よくある失敗：軌道が平面内でも、球殻の体積は変わりません

式(3.1)に必要なのは $4\pi r^2$ です。図が 2 次元だからといって $2\pi r$ へ替えません。後者は、面密度から円環内の質量を求める場合の因子です。第 4 章の円盤では、体積密度 ρ とは異なる面密度 Σ を使います。

3.2.2 内側の質量から重力加速度を求める

球対称な殻が内部へ及ぼす重力は、各方向からの寄与が打ち消し合います。一方、殻の外では、その質量を中心へ集めた場合と同じ重力になります。Newton の球殻に関する定理を使うと、半径 $r > 0$ での加速度は、内側の質量だけで決まります。[2]

$$\boldsymbol{a}(\boldsymbol{r}) = -\frac{GM(<r)}{r^3}\boldsymbol{r}. \tag{3.3}$$

向きは中心向き、大きさは $GM(<r)/r^2$ です。点質量の場合と似ていますが、ここでは分子の $M(<r)$ が半径に応じて変わります。

円運動をする星では、重力加速度と向心加速度が等しくなります。円運動速度 v_c は

$$\frac{v_c^2(r)}{r} = \frac{GM(<r)}{r^2}, \qquad v_c^2(r) = \frac{GM(<r)}{r} = r\frac{d\Phi}{dr} \tag{3.4}$$

です。Φ は単位質量当たりの重力ポテンシャルで、$\boldsymbol{a} = -\nabla\Phi$ と定義します。

もう少し詳しく：内側の質量だけでポテンシャルも書けるでしょうか

一般の球対称分布では、$\Phi(r) = -GM(<r)/r$ とは書けません。外側の殻は、内部の力には寄与しなくても、ポテンシャルには寄与するためです。$\Phi(\infty) = 0$ と選ぶと、$r > 0$ について

$$\Phi(r) = -G\left[\frac{M(<r)}{r} + 4\pi\int_r^\infty r'\rho(r')\,dr'\right]$$

です。詳しい導出は文献 [2] の §2.2.1 に譲ります。本章では、次に示す Plummer ポテンシャルの解析式を使います。

3.3 Plummer モデルを導入する

3.3.1 密度とポテンシャルを対応させる

中心で有限の密度を持ち、外側で滑らかに薄くなる簡単なモデルとして、Plummer 球を選びます。密度とポテンシャルは

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2}, \tag{3.5}$$

$$\Phi(r) = -\frac{GM}{\sqrt{r^2 + a^2}} \tag{3.6}$$

です。これらは同じ質量分布を表す一組の式です。文献 [2] の Plummer スケール長 b を、本書では a と書きます。どちらかの式だけで a を変更すると、異なるモデルを混ぜてしまいます。

中心密度は $\rho(0) = 3M/(4\pi a^3)$ です。全質量を固定して a を大きくすると、中心密度が下がり、質量が広く分布します。外側では $\rho \propto r^{-5}$ となり、Φ は点質量の $-GM/r$ へ近づきます。a は銀河の外縁ではなく、中心側と外側の振る舞いが変わる長さの尺度です。

3.3.2 内側の質量を解析的に求める

数値積分を始める前に、比較に使う解析式を求めます。式(3.5)を式(3.2)へ代入すると、

$$M(<r) = 3Ma^2\int_0^r \frac{r'^2}{(r'^2 + a^2)^{5/2}}\,dr' \tag{3.7}$$

です。ここで

$$\frac{d}{dr'}\left[\frac{r'^3}{(r'^2 + a^2)^{3/2}}\right] = \frac{3a^2 r'^2}{(r'^2 + a^2)^{5/2}} \tag{3.8}$$

を使うと、積分の結果は

$$\boxed{M(<r) = M\frac{r^3}{(r^2 + a^2)^{3/2}}} \tag{3.9}$$

になります。この式は、採用した密度から本章で直接計算したものです。$r = 0$ で 0、$r \to \infty$ で M になり、$r > 0$ では単調に増えます。$r = a$ では $M(< a)/M = 2^{-3/2} \simeq 0.3536$ なので、a は半質量半径でもありません。

3.3.3 単位と標準パラメータを決める

前章と同じく、内部単位は kpc、Gyr、$M_\odot$ です。速度は内部では $\mathrm{kpc\,Gyr^{-1}}$ を使い、図や数値の表示でだけ $\mathrm{km\,s^{-1}}$ へ換算します。以下では NumPy を `np`、Matplotlib の `pyplot` を `plt` として読み込んでいるものとします。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
mass = 1.0e11                        # 球全体の質量 [Msun]
a = 3.0                              # Plummer スケール長 [kpc]
mu = G * mass
r_tab = 5.0 * a                       # 質量表の範囲 [kpc]。球の外縁ではありません。
n_mass = 256                         # 台形則の区間数。分点数は n_mass + 1 です。
root_xtol = 1.0e-8                    # 二分法の半区間幅の許容値 [kpc]
r0 = 2.0 * a                         # 軌道の初期半径 [kpc]
```

本章の $M = 10^{11} M_\odot$、$a = 3\,\mathrm{kpc}$ は教材用の設定です。特定の実在銀河を再現する値ではありません。$r_{\mathrm{tab}} = 5a = 15\,\mathrm{kpc}$ までの質量表を、標準では 256 区間、257 分点で作ります。`root_xtol` の役割は、二分法の節で説明します。

密度、ポテンシャル、解析的な質量を、それぞれ短い関数にします。`radius` には 1 個の半径も、半径を並べた配列も渡せます。

```
def density_plummer(radius, mass, a):
    """3 次元の質量密度 [Msun/kpc^3]。本文 eq:ch3-density。"""
    radius = np.asarray(radius, dtype=float)
    return 3.0 * mass / (4.0 * np.pi * a**3) * (1.0 + (radius / a)**2)**(-2.5)


def potential_plummer(radius, mass, a):
    """ 単位質量当たりのポテンシャル [kpc^2/Gyr^2]。"""
    return -G * mass / np.sqrt(np.asarray(radius)**2 + a**2)


def mass_plummer_exact(radius, mass, a):
    """ 半径 radius 以内の解析的な質量。本文 eq:ch3-mass-exact。"""
    radius = np.asarray(radius, dtype=float)
    return mass * radius**3 / (radius**2 + a**2)**1.5
```

ここで半径の配列を使うのは、**同じ銀河の半径方向の表**を作るためです。複数の星の運動を同時に計算しているわけではありません。

3.4 台形則で内側の質量を求める

3.4.1 積分する関数を決める

式(3.9)を使えば質量はすぐ求まります。それでも数値積分を行うのは、正解の分かる問題で算法と実装を確かめるためです。解析的に積分できない密度や、分点上の値だけが与えられる場合にも、同じ考え方を使えます。

積分する関数を

$$g(r) = 4\pi r^2 \rho(r), \qquad M(<r) = \int_0^r g(r')\,dr' \tag{3.10}$$

と置きます。g の単位は $M_\odot\,\mathrm{kpc}^{-1}$ で、半径方向に積分すると質量になります。密度 ρ だけを dr で積分するのではありません。

第 1・2 章の時間積分は、未知の位置や速度を次の時刻へ更新する計算でした。今回は、与えられた関数の**定積分**を近似します。どちらも積分ですが、積分する変数と未知量を区別してください。

3.4.2 一つの小区間を台形で近似する

$0 = r_0 < r_1 < \cdots < r_{N_r} = r_{\mathrm{tab}}$ という分点を用意し、$g_j = g(r_j)$ とします。この節の下付き j は半径の分点番号です。軌道の粒子番号や時刻番号ではありません。

小区間 $[r_j, r_{j+1}]$ では、両端の値を結ぶ直線で関数を近似します。幅を $\Delta r_j = r_{j+1} - r_j$ とすると、その直線は

$$g(r) \simeq g_j + \frac{g_{j+1} - g_j}{\Delta r_j}(r - r_j)$$

です。これを小区間で積分すれば、

$$\int_{r_j}^{r_{j+1}} g(r)\,dr \simeq \frac{g_j + g_{j+1}}{2}\Delta r_j \tag{3.11}$$

となります。両端の平均の高さに幅を掛ける、台形の面積です。これが台形則の基本です。[3]

図3.2の注記から、一つの台形の面積を式(3.11)へ対応させてください。

3.4.3 各半径までの質量を順に足す

すべての台形を最後に足すだけでなく、途中の和も保存します。数値積分で得た質量を $\widehat{M}_j$ と書くと、

$$\widehat{M}_0 = 0, \qquad \widehat{M}_{j+1} = \widehat{M}_j + \frac{g_j + g_{j+1}}{2}(r_{j+1} - r_j) \tag{3.12}$$

です。最後の値だけでなく、各分点に対応する累積質量の表が得られます。ハットはここでは数値近似を示すもので、無次元量を意味しません。

まずループで、式をそのまま実装します。

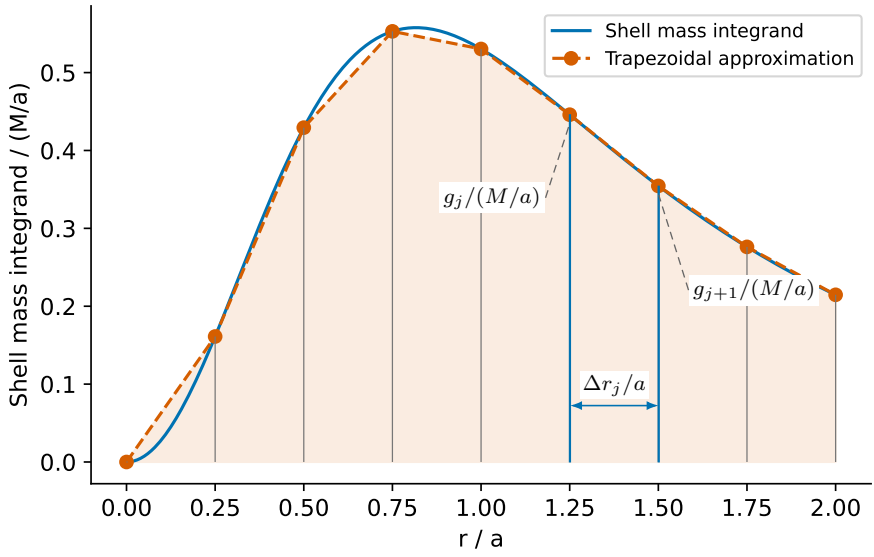


図 3.2 台形則で近似する球殻の質量の被積分関数。説明用に $0 \le r \le 2a$ を 8 区間へ分けています。縦軸は $g/(M/a) = 3(r/a)^2[1+(r/a)^2]^{-5/2}$ です。標準の 256 区間の計算とは分割数が異なります。一つの台形の高さ g_j, g_{j+1} と幅 Δr_j を注記しています。図の軸に合わせ、高さを M/a、幅を a で規格化しています。

```
def cumulative_trapezoid(x, values):
    """ 各分点までの累積積分。本文 eq:ch3-cumulative-trapezoid。"""
    x, values = check_grid(x, values)
    integral = np.zeros_like(x)
    for j in range(len(x) - 1):
        area = 0.5 * (values[j] + values[j + 1]) * (x[j + 1] - x[j])
        integral[j + 1] = integral[j] + area
    return integral
```

`check_grid` は、分点と関数値が同じ長さの 1 次元配列で、値が有限かつ分点が狭義単調増加していることを確認する補助関数です。実装は Notebook に置きます。積分の本体は、台形の面積を計算し、前の和へ足す 2 行です。

この関数へ、半径の表と g の値を渡します。

```
r_grid = np.linspace(0.0, r_tab, n_mass + 1)
rho_grid = density_plummer(r_grid, mass, a)
grid_shell = 4.0 * np.pi * r_grid**2 * rho_grid
mass_numeric = cumulative_trapezoid(r_grid, grid_shell)
mass_exact = mass_plummer_exact(r_grid, mass, a)
```

`mass_numeric[j]` が r_j 以内の質量です。`np.linspace` の最後の引数は分点数なので、区間数に 1 を加えます。

Notebook には、隣接する値をスライスで取り出し、`np.cumsum` で累積和を作る書き方も示します。計算している台形はループ版と同じです。まずは式と各配列要素の対応を理解し、コードが短いことだけを目標にしないようにします。

3.4.4 2 次精度の意味を確認する

滑らかな関数について、一つの幅 Δr の小区間を台形で近似した誤差は $O(\Delta r^3)$ です。固定した範囲を等分すると、小区間の個数は Δr^{-1} に比例します。そのため、小区間の誤差を足した合成台形則の誤差は、通常 $O(\Delta r^2)$ になります。[3]

分割数を 2 倍にして幅を半分にすると、誤差がおよそ $1/4$ になることを期待します。ただし、これは関数が滑らかで刻みが十分小さく、丸め誤差が支配しない範囲での話です。多項式を何次まで厳密に積分できるかという「代数的精度」とは、別の数え方です。本章では、**幅に対する誤差の変化**を確認します。

もう少し詳しく：ほかの数値積分法

Simpson 則は、小区間の関数を放物線で近似して積分する方法です。滑らかな問題では台形則より高い次数を得られます。Gauss 求積のように、分点と重みの選び方を工夫する方法もあります。本章では、表から累積積分を作りやすい台形則を使います。ほかの方法の導出・実装は数値計算の教科書に譲ります。台形則と Simpson 則の比較は、文献 [3] の第 3 章を参照してください。

3.5 数値積分を解析式と比べる

3.5.1 原点でも使える誤差を定義する

数値積分と解析式を同じ分点で比較します。原点では $M(<0)=0$ なので、各半径の質量で割った相対誤差は使いません。球全体の質量 M で規格化し、

$$e_M = \max_{0\le j\le N_r} \frac{|\widehat{M}_j - M(<r_j)|}{M} \tag{3.13}$$

とします。これなら原点を含む表全体を、一つの基準で評価できます。

標準条件の最大誤差は $e_M = 3.67\times10^{-5}$ でした。図3.3がほぼ重なっても、この差は 0 ではありません。図の見た目とは別に、数値で確認することが必要です。

3.5.2 積分範囲を固定して分割する

ここでは $r_{\mathrm{tab}} = 5a$ を変えず、区間数 N_r だけを増やします。

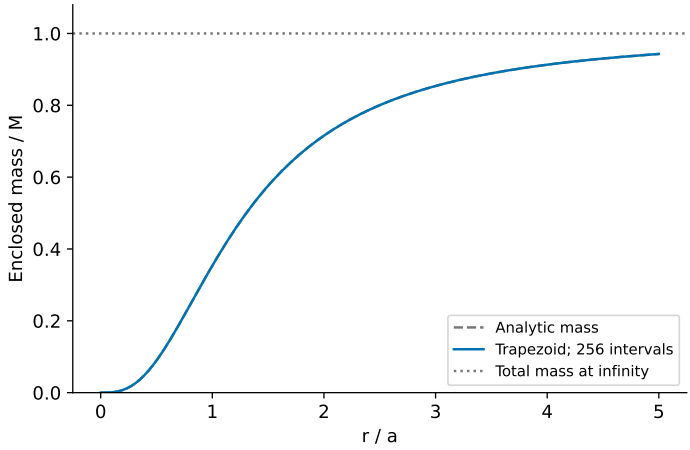


図 3.3　Plummer 球の累積質量。灰色破線が解析式、青実線が $0 \le r \le 5a$ を 256 区間で積分した結果です。両者はこの縮尺ではほぼ重なります。上の点線は球全体の質量であり、$5a$ の内側だけでは到達しません。

```
panel_counts = [20, 40, 80, 160, 320]
mass_rows = []
for count in panel_counts:
    r = np.linspace(0.0, r_tab, count + 1)
    shell = 4.0 * np.pi * r**2 * density_plummer(r, mass, a)
    calculated = cumulative_trapezoid(r, shell)
    expected = mass_plummer_exact(r, mass, a)
    error = np.max(np.abs(calculated - expected)) / mass
    mass_rows.append([count, r_tab / count, error])
mass_convergence = np.asarray(mass_rows)
mass_orders = np.log2(mass_convergence[:-1, 2] / mass_convergence[1:, 2])
```

表 3.1　台形則の分割数試験。積分範囲は全行で $[0, 5a]$ です。p_M は前の行との誤差比から求めた推定次数で、最初の行には比較相手がありません。

N_r	Δr [kpc]	e_M	p_M
20	0.750000	5.87×10^{-3}	—
40	0.375000	1.46×10^{-3}	2.0037
80	0.187500	3.76×10^{-4}	1.9602
160	0.093750	9.40×10^{-5}	2.0014
320	0.046875	2.35×10^{-5}	2.0004

推定次数は、第 1・2 章の時間刻みの試験と同様に、

$$p_M = \frac{\log[e_M(\Delta r)/e_M(\Delta r/2)]}{\log 2} \tag{3.14}$$

で求めます。表3.1の最も細かい側では $p_M = 2.0004$ となり、2 次の収束を確認できます。最大誤差を取る分点の位置も分割数とともに変わるため、粗い側の推定次数が滑らかに 2 へ近づくとは限りません。

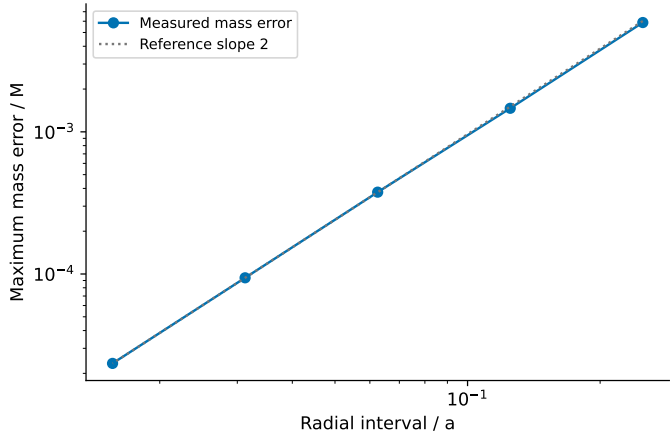


図 3.4　半径方向の幅と質量誤差。点線は、最も細かい分割の実測値を通る傾き 2 の参照線です。実測点を当てはめて傾きを決めた線ではありません。

3.5.3 積分範囲の効果と積分誤差を区別する

同じ $[0, 5a]$ をいくら細かく分けても、得られるのは $M(<5a)$ です。全質量 M にはなりません。式(3.9)から求めた、いくつかの範囲内の質量を表3.2に示します。

表 3.2　有限半径の内側と外側の質量。すべて解析式の値です。範囲外にある質量と、有限刻みによる数値積分誤差を区別します。

r_{tab}/a	$M(<r_{\mathrm{tab}})/M$	範囲外の質量 $/M$
5	0.942866034	0.057133966
10	0.985185337	0.014814663
20	0.996261685	0.003738315

例えば $5a$ より外には、全質量の約 5.7% が残ります。これは台形則の失敗ではありません。全質量まで近づけたいときには積分範囲も広げます。ただし、区間数を固定したまま範囲だけ広げると、中心付近の分割が粗くなります。**範囲を広げることと、同じ範囲を細かく分けることは、別の操作です。**

よくある失敗：最後の値を無条件に全質量へそろえない

`mass_numeric[-1]` を M に合わせて表全体を再規格化すると、積分範囲の外の質量を内側へ配り直してしまいます。本章の M は無限遠まで含めた全

質量です。有限半径で切断した別のモデルを作るときは、その定義を改めて示す必要があります。

3.6 二分法で半質量半径を求める

3.6.1 半径を未知数にする

ここまでは半径を指定して、その内側の質量を求めました。次は、質量を指定して半径を求めます。**半質量半径** r_h は、球全体の質量の半分が入る半径です。

$$M(<r_h) = \frac{M}{2}. \tag{3.15}$$

Plummer モデルでは、解析式を使って比較値も求められます。式(3.9)を M で割って $1/2$ と置き、両辺を $2/3$ 乗すると、

$$\frac{r_h^2}{r_h^2 + a^2} = 2^{-2/3}.$$

これを r_h について整理すると、

$$r_h = \frac{a}{\sqrt{2^{2/3} - 1}} \simeq 1.304766\, a \tag{3.16}$$

です。本章の $a = 3\,\mathrm{kpc}$ では $r_h = 3.914298080\,\mathrm{kpc}$ です。今回も、この解析式だけで答えを出すのではなく、数値的な求め方を検証します。

3.6.2 積分と求根を分ける

二分法は、関数の値が 0 になる場所を探す**求根法**です。本章で計算する関数を

$$F(r) = \frac{\widehat{M}(<r)}{M} - \frac{1}{2} \tag{3.17}$$

とします。$\widehat{M}$ は台形則の結果です。$F < 0$ なら内側の質量が半分より少なく、$F > 0$ なら多いことを意味します。

先ほどの質量表は決まった分点の値だけを持っていますが、二分法ではその間の半径も試します。そこで本章では、**半径を試すたびに $[0, r]$ を同じ区間数で積分し直す**実装にします。表の補間をまだ必要とせず、二分法が呼ぶ関数の意味を直接追えます。

```
def enclosed_mass_trapezoid(radius, mass, a, n_panels):
    """[0, radius] を n_panels 区間で積分します。"""
    check_panels(n_panels)
    if not np.isfinite(radius) or radius < 0.0:
        raise ValueError("radius must be finite and nonnegative.")
    if radius == 0.0:
        return 0.0
    r = np.linspace(0.0, radius, n_panels + 1)
    shell = 4.0 * np.pi * r**2 * density_plummer(r, mass, a)
    return cumulative_trapezoid(r, shell)[-1]


def half_mass_residual(radius, n_panels=n_mass):
    return enclosed_mass_trapezoid(radius, mass, a, n_panels) / mass - 0.5
```

`enclosed_mass_trapezoid` は積分、`half_mass_residual` は求根する方程式を担当します。$r = 0$ なら積分値は 0 として返します。$r > 0$ では、区間幅は r/N_r です。求根の途中では r が変わるので、分割幅も変わります。

3.6.3 符号の変わる区間を半分にする

連続な関数 F について、端点 $r_L < r_U$ で符号が異なれば、その間に少なくとも一つの根があります。Plummer 球の真の累積質量は単調増加するので、半質量半径は一つです。標準の数値積分でも、$[0, 5a]$ の両端で $F(0) < 0$、$F(5a) > 0$ になります。

まず中点を計算します。

$$r_C = \frac{r_L + r_U}{2}. \tag{3.18}$$

$F(r_C)$ が $F(r_L)$ と同符号なら、根は右半分にあるので $r_L \leftarrow r_C$ とします。異符号なら左半分を残し、$r_U \leftarrow r_C$ とします。これを繰り返すのが二分法です。導関数は必要ありません。[3]

根が残る側を選んでいるので、区間の幅は毎回半分になります。ただし、符号が変わるだけで十分なのは、関数がその区間で連続な場合です。不連続な発散点を挟んで符号が違っても、そこに根があるとは限りません。

図3.5で、端点と中点の符号から、次に残す半区間をたどってください。太い線の範囲が次の行の探索区間になります。

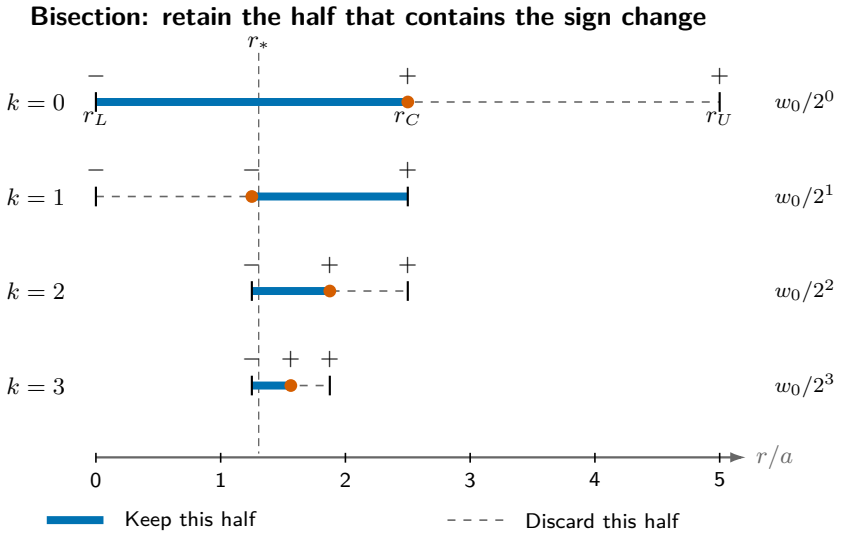


図 3.5 初期区間 $[0, 5a]$ からの二分法の例。点が中点、$+, -$ が評価した F の符号、太い線が次に残す半区間です。k はそれまでの半減回数です。破線 r_* は評価している数値関数の根であり、元の密度の真の半質量半径と必ずしも一致しません。

3.6.4 二分法の終了条件を決める

初期区間の幅を w_0 とします。k 回区間を半分にした後の幅は $w_k = w_0/2^k$ です。その区間の中点を根の近似値とすれば、区間が正しい根を含む限り、中点から根までの距離は半区間幅以下です。

$$|r_C - r_*| \le \frac{w_k}{2} = \frac{w_0}{2^{k+1}}. \tag{3.19}$$

ここで r_* は、**実際に評価している関数 F の根**です。まだ、元の連続的な密度の真の半質量半径 r_h と同じとは限りません。

本章では、半区間幅が許容値 τ_r 以下になったら終了します。τ_r は半径と同じ単位を持ちます。例えば `root_xtol = 1e-8` は $10^{-8}\,\mathrm{kpc}$ です。無次元の残差 $|F|$ の許容値ではありません。

符号を保つ更新をコードにします。

```
def bisection(func, lower, upper, xtol, max_iter=100):
    """ 半区間幅が xtol 以下になるまで二分します。本文 eq:ch3-bisection。"""
    f_lower, f_upper = check_bisection(func, lower, upper, xtol, max_iter)
    if f_lower == 0.0 or f_upper == 0.0:
        endpoint = lower if f_lower == 0.0 else upper
        return endpoint, np.array([[0, endpoint, endpoint, endpoint, 0.0, 0.0]])
    trace = []
    for k in range(max_iter):
        half_width = 0.5 * (upper - lower)
        midpoint = lower + half_width
        f_mid = finite_value(func, midpoint)
        trace.append([k, lower, upper, midpoint, f_mid, half_width])
        if f_mid == 0.0 or half_width <= xtol:
            return midpoint, np.asarray(trace)
        if midpoint == lower or midpoint == upper:
            raise RuntimeError("The bracket cannot be reduced in floating-point
            ↪  arithmetic.")
        if np.signbit(f_mid) == np.signbit(f_lower):
            lower, f_lower = midpoint, f_mid
        else:
            upper, f_upper = midpoint, f_mid
    raise RuntimeError("Bisection did not reach xtol within max_iter.")
```

`check_bisection` は、区間、許容幅、反復回数、端点の符号を確認します。`finite_value` は、関数値が有限であることを確認します。両者の完全なコードは Notebook に含めます。`np.signbit` は負の符号を持つかどうかを返し、関数値どうしの積を作らずに同符号かを判定できます。

`trace` には、反復番号、左右の端点、中点、残差、半区間幅を保存します。関数値が計算上ちょうど 0 になった場合も終了します。通常の浮動小数点演算を使うため、これは丸めを含めて厳密に根を保証する区間演算の実装ではありません。更新が丸めの限界に達したときや反復上限を超えたときには、未収束として停止します。

3.6.5 実際に半質量半径を求める

```
r_half, root_trace = bisection(half_mass_residual, 0.0, 5.0 * a, root_xtol)
r_half_exact = a / np.sqrt(2.0**(2.0 / 3.0) - 1.0)
print(f"Numerical half-mass radius = {r_half:.9f} kpc")
print(f"Analytic half-mass radius  = {r_half_exact:.9f} kpc")
print(f"Residual of numerical F = {half_mass_residual(r_half):.6e}")
print(f"Residual using exact M = {mass_plummer_exact(r_half, mass, a) / mass - 0.5:.6e}")
```

標準の 256 区間では、数値結果は $\widehat{r}_h = 3.914303805\,\mathrm{kpc}$ でした。区間を半分にした回数は 30 回、中点を評価した回数は 31 回です。最初の区間の中点も評価しているため、二つの回数は 1 だけ異なります。終了時の半区間幅は $6.98 \times 10^{-9}\,\mathrm{kpc}$ です。

もう少し詳しく：ほかの求根法

Newton 法は関数値と導関数から次の近似値を決め、適切な条件では二分法より速く収束しますが、初期値や関数の性質に注意が必要です。割線法は導関数の代わりに二つの点から傾きを見積もります。Brent 法は区間を保つ操作と補間を組み合わせる方法です。本章では、根を挟む区間を追いやすい二分法に集中します。二分法、Newton 法、割線法は文献 [3] の第 9 章を参照してください。Brent 法など、ここで扱わない方法の詳細は数値計算の専門書に譲ります。

3.7 求根の残差と半径の誤差を区別する

3.7.1 二つの誤差を区別する

二分法を終えたときの $|F(\widehat{r}_h)|$ は 3.93×10^{-10} でした。一方、同じ半径を解析的な質量式へ代入したときの残差は、

$$\left| \frac{M(<\widehat{r}_h)}{M} - \frac{1}{2} \right| = 8.12 \times 10^{-7}$$

です。二つの値は一致しません。二分法は台形則で定義した F を 0 へ近づけていますが、その F 自体に積分の誤差が含まれるためです。

実際、真の半径との差は $|\widehat{r}_h - r_h| = 5.73 \times 10^{-6}\,\mathrm{kpc}$ で、二分法の最終半区間幅より大きくなっています。式(3.19)が誤っているのではありません。そこに現れる r_* が、台形則で作った関数の根だからです。

3.7.2 まず積分の分割数を変える

求根の初期区間と許容幅を固定し、質量積分の区間数だけを変えます。

```
root_rows = []
for count in [32, 64, 128, 256, 512]:
    def residual(radius):
        return half_mass_residual(radius, n_panels=count)
    root, trace = bisection(residual, 0.0, 5.0 * a, root_xtol)
    actual_error = abs(root - r_half_exact)
    root_rows.append([count, root, actual_error, abs(residual(root)), trace[-1, 5]])
root_convergence = np.asarray(root_rows)
root_orders = np.log2(root_convergence[:-1, 2] / root_convergence[1:, 2])
```

表3.3では、積分を細かくすると真の半径との差も小さくなります。本例では、その差はおよそ区間数の 2 乗に反比例して減っています。求根の終了条件を厳しくする前に、その関数値をどの精度で計算しているかを見る必要があります。

表 3.3　積分の区間数による半質量半径の変化。二分法の許容半区間幅は全行で $10^{-8}\,\mathrm{kpc}$ です。小さな数値残差が得られていても、真の半径との差には区間数への依存が残ります。

N_r	$\widehat{r}_h$ [kpc]	$\lvert\widehat{r}_h - r_h\rvert$ [kpc]	$\lvert F(\widehat{r}_h)\rvert$
32	3.914664479	3.66×10^{-4}	7.95×10^{-10}
64	3.914389650	9.16×10^{-5}	5.68×10^{-10}
128	3.914320974	2.29×10^{-5}	3.60×10^{-10}
256	3.914303805	5.73×10^{-6}	3.93×10^{-10}
512	3.914299516	1.44×10^{-6}	8.84×10^{-10}

3.7.3 次に求根の許容幅だけを変える

今度は積分を 64 区間に固定し、τ_r だけを変えます。

表 3.4　積分を 64 区間に固定した許容幅の試験。求根を細かく続けても、積分から来る半径のずれは取り除けません。

τ_r [kpc]	最終半区間幅 [kpc]	真の半径との差 [kpc]	$\lvert F\rvert$
1.00×10^{-2}	7.32×10^{-3}	4.16×10^{-3}	5.76×10^{-4}
1.00×10^{-4}	5.72×10^{-5}	9.63×10^{-5}	6.70×10^{-7}
1.00×10^{-6}	8.94×10^{-7}	9.18×10^{-5}	3.61×10^{-8}
1.00×10^{-8}	6.98×10^{-9}	9.16×10^{-5}	5.68×10^{-10}

許容幅が十分小さくなると、真の半径との差は約 $9.16 \times 10^{-5}\,\mathrm{kpc}$ の付近でほぼ変わらなくなります。これは計算機の丸め誤差まで到達したからではなく、この実験では質量積分の誤差が残っているからです。

よくある失敗：求根の桁数だけで結果の精度を判断しない

数値的な関数の残差が小さくても、もとの方程式を同じ精度で解けたとは限りません。積分の分割数と求根の許容幅を別々に変更して確認します。また、$M/2$ を $\widehat{M}(<r_{\mathrm{tab}})/2$ へ替えると、有限の範囲内にある質量の半分を求める、別の問題になります。

3.8 Plummer 球の重力で星を動かす

3.8.1 加速度は解析式から与える

ここからは、星を表す 1 粒子の時間発展を計算します。質量積分と求根の誤差を軌道へ混ぜないため、**本章の標準軌道計算には解析的な Plummer 加速度を使います**。数値的な質量表は、ここまでの積分・求根の学習と検証に使ったものです。

式(3.6)を微分すると、

$$\frac{d\Phi}{dr} = \frac{GMr}{(r^2 + a^2)^{3/2}}$$

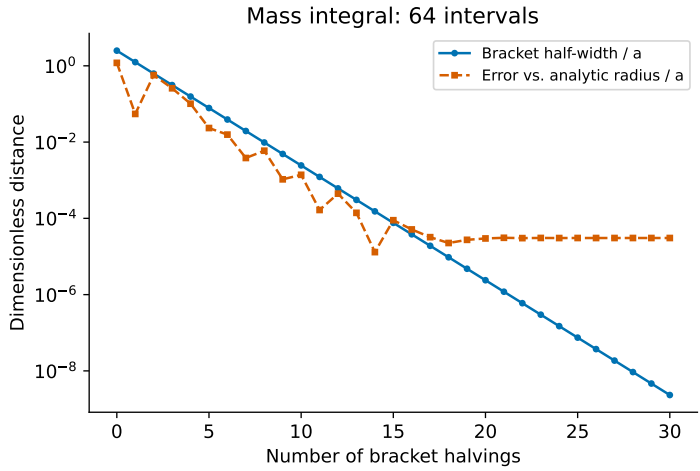


図 3.6 64 区間の質量積分を使った二分法の履歴。青は半区間幅、朱色は中点と解析的な半質量半径の差です。半区間幅は半減を続けますが、真の半径との差は積分誤差の大きさ付近に残ります。どちらも a で規格化しています。

なので、加速度は

$$\boxed{\boldsymbol{a}(\boldsymbol{r}) = -\frac{GM\boldsymbol{r}}{(r^2+a^2)^{3/2}}} \tag{3.20}$$

です。これは式(3.3)へ解析的な $M(<r)$ を代入しても得られます。二つの求め方が一致することが、密度・質量・ポテンシャルの整合性の確認になります。

原点では $\boldsymbol{a}=0$ です。$r \ll a$ では

$$\boldsymbol{a} \simeq -\frac{GM}{a^3}\boldsymbol{r}, \tag{3.21}$$

となり、位置に比例する調和振動子の形になります。一方、$r \gg a$ では点質量の重力へ近づきます。中心で加速度が発散しないのは、質量分布を広げたモデルの性質であり、計算エラーを隠す処置ではありません。

3.8.2 円運動速度も同じモデルから求める

式(3.4)に代入すると、

$$v_c^2(r) = \frac{GMr^2}{(r^2+a^2)^{3/2}}, \qquad \Omega_c(r) = \frac{v_c(r)}{r} \tag{3.22}$$

です。$r>0$ の円軌道周期は $T_c(r) = 2\pi/\Omega_c(r)$ になります。中心近くでは $v_c \propto r$、十分外では $v_c \propto r^{-1/2}$ です。[2]

まずは成分で加速度を計算します。

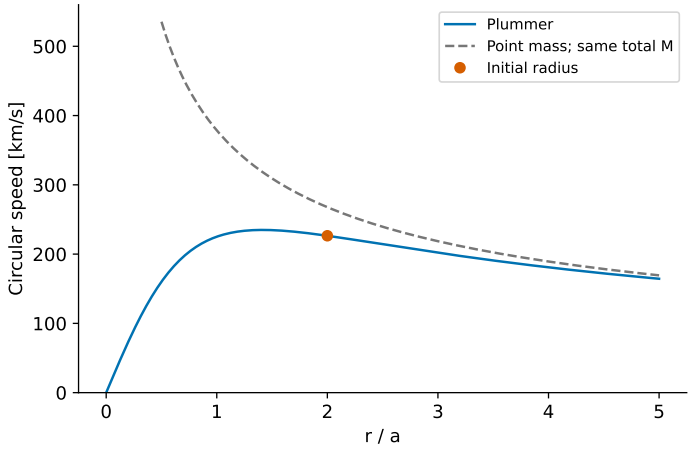


図 3.7　同じ全質量を持つ Plummer 球と点質量の円運動速度。丸は本章の初期半径 $r_0 = 2a$ です。点質量の曲線は $r \geq 0.5a$ だけを示しています。原点に向かって発散する点質量の速度と、中心で 0 へ向かう Plummer 球の速度は異なります。

```
x, y = r0, 0.0
r2 = x**2 + y**2
ax = -G * mass * x / (r2 + a**2)**1.5
ay = -G * mass * y / (r2 + a**2)**1.5
print(f"ax = {ax:.6f}, ay = {ay:.6f} [kpc/Gyr^2]")
```

本章の初期位置では $\boldsymbol{a} \simeq (-8.9413 \times 10^3, 0)\,\mathrm{kpc\,Gyr^{-2}}$ です。この計算を短い関数にします。

```
def acceleration_plummer(pos, mass, a):
    """ 原点に固定した Plummer 球。1 粒子の位置・加速度は (1, 2) です。"""
    x, y = pos[0]
    r2 = x**2 + y**2
    ax = -G * mass * x / (r2 + a**2)**1.5
    ay = -G * mass * y / (r2 + a**2)**1.5
    return np.array([[ax, ay]])


def circular_speed_plummer(radius, mass, a):
    radius = np.asarray(radius, dtype=float)
    return np.sqrt(G * mass * radius**2 / (radius**2 + a**2)**1.5)
```

`pos` と返り値は、1 粒子なので形が `(1, 2)` です。多数の粒子への一般化は、第 4 章で行います。円運動速度の関数では、半径ごとの表を描けるよう配列も受け取ります。

よくある失敗：点質量の円運動速度を流用しない

Plummer 球の内部では、全質量 M を使った $\sqrt{GM/r}$ は円運動速度ではありません。力のモデルを変えたら、初期速度も同じモデルから計算し直します。a を変えて円軌道を比較するときも同様です。

3.9 KDK で円軌道と非円軌道を計算する

3.9.1 変える初期値を一つにする

初期半径を $r_0 = 2a = 6\,\mathrm{kpc}$ とし、初期位置と速度を

$$\boldsymbol{r}(0) = (r_0, 0), \qquad \boldsymbol{v}(0) = (0, \eta v_c(r_0)) \tag{3.23}$$

と置きます。$\eta = 1$ が円軌道、$\eta = 0.7$ が比較する非円軌道です。どちらも動径方向の初期速度は 0 で、反時計回りに動き始めます。

$0 < \eta < 1$ では、円運動を保つために必要な接線速度より遅いため、初期半径から内側へ動き始めます。この初期位置は遠点です。ただし、第 2 章の Kepler 問題とは違い、その後の軌道は一般には閉じた楕円になりません。**非円軌道を最初から「楕円」と呼ばないようにします。**

```
v_c = circular_speed_plummer(r0, mass, a)
t_c = 2.0 * np.pi * r0 / v_c
n_periods = 5
steps_per_period = 1000
n_steps = n_periods * steps_per_period
t_end = n_periods * t_c
dt = t_end / n_steps
eta_values = {"Circular": 1.0, "Noncircular": 0.7}
```

表 3.5　本章の標準軌道条件。二つのケースで共通の基準周期 T_c を使います。これは非円軌道の動径振動の周期を表す記号ではありません。

量	設定または計算値
M, a	$10^{11} M_\odot$, $3\,\mathrm{kpc}$
r_0	$6\,\mathrm{kpc} = 2a$
$v_c(r_0)$	$226.475925\,\mathrm{km\,s^{-1}}$
$T_c = 2\pi r_0 / v_c(r_0)$	$0.162763\,\mathrm{Gyr}$
η	1, 0.7
t_{end}	$5T_c = 0.813815\,\mathrm{Gyr}$
N_{step}, Δt	5000, $T_c/1000 = 0.162763\,\mathrm{Myr}$

初期状態を作る関数は、位置と速度を横につないだ `(1, 4)` の配列を返します。

```
def make_initial_state(radius, eta, mass, a):
    check_model(mass, a)
    if not np.isfinite(radius) or radius <= 0.0 or not np.isfinite(eta):
        raise ValueError("Initial radius must be positive and eta finite.")
    pos = np.array([[radius, 0.0]])
    vel = np.array([[0.0, eta * circular_speed_plummer(radius, mass, a)]])
    return np.column_stack((pos, vel))
```

`check_model` は、M と a が有限かつ正であることを確認する関数です。KDK では、まとめた状態から位置と速度を分けて取り出し、更新します。

3.9.2 前章の積分法を再利用する

KDK の計算は、半ステップの速度更新、1 ステップの位置更新、新しい位置での加速度評価、残りの半ステップの速度更新です。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, mass, a):
    """KDK。最後の加速度を次のステップへ渡します。"""
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, mass, a)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

前章から変えたのは、重力の関数へ渡すパラメータです。今回は二つの質量の配列ではなく、1 個の銀河の `mass` と `a` を渡します。KDK の 4 行と、最後の加速度を再利用する構造は同じです。

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, mass, a):
    check_model(mass, a)
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, mass, a)
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, mass, a)
        history[n + 1] = np.column_stack((pos, vel))
        if not np.all(np.isfinite(history[n + 1])):
            raise FloatingPointError("Non-finite state; check the model and time step.")
    return times, history
```

`prepare_history` は、初期値をコピーし、時刻の配列と保存領域を作る補助関数です。前章と同じ役割ですが、今回は初期状態が `(1, 4)` であることを確認します。履歴は `(n_steps + 1, 1, 4)` です。保存する速度は、最後の半ステップの更新を終えた整数時刻の値です。

5000 ステップなら、加速度関数の評価は初回を含めて 5001 回です。台形則の区間数や二分法の反復回数は、この回数に含めません。いま評価している重力には解析式を使っています。

3.9.3 同じ時間まで独立に計算する

```
cases = {}
start = perf_counter()
for name, eta in eta_values.items():
    initial = make_initial_state(r0, eta, mass, a)
    times, history = integrate_leapfrog(acceleration_plummer, initial, t_end, n_steps,
    ↪  mass, a)
    cases[name] = {"eta": eta, "initial": initial, "times": times, "history": history}
```

一つのケースの最終状態を、別のケースの初期状態に流用しません。いずれも式(3.23)から作り直します。実行時間は環境に依存するので、Notebook で `perf_counter` を使って測ります。

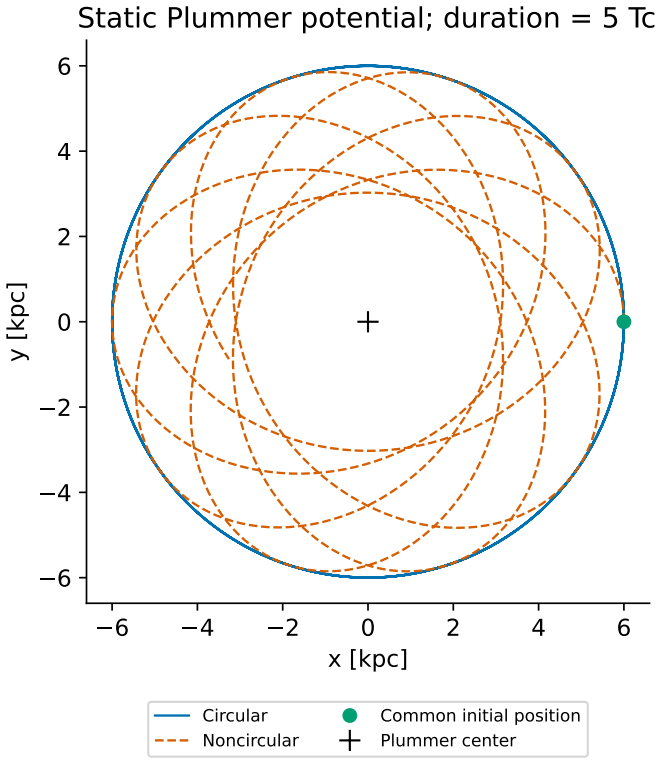


図 3.8 静的な Plummer 球中の円軌道と非円軌道。青実線が $\eta = 1$、朱色破線が $\eta = 0.7$ で、どちらも $5T_c$ まで計算しています。中心の十字はポテンシャルの中心であり、追加した点質量ではありません。丸が共通の初期位置です。

円軌道では半径がほぼ一定ですが、非円軌道では半径が増減し、近点・遠点の方向も変わります。このような軌道はロゼット軌道と呼ばれます。球対称でも、動径方向の振動と方位角方向の回転が、Kepler 問題と同じ周期関係になるとは限りま

せん。[2] 数値誤差による軌道のずれと区別するため、後で時間刻みを細かくした結果も比べます。

Notebook には、保存済みの履歴を使うアニメーションも含めます。画面上の再生間隔は物理的な時間刻みとは別で、動画の速度を変えても数値軌道は変わりません。

3.10 保存量と時間刻みを確認する

3.10.1 確認するエネルギーを決める

固定した銀河ポテンシャル中のテスト粒子なので、確認するのは粒子の**単位質量当たり**のエネルギーと角運動量です。

$$\varepsilon = \frac{1}{2}(v_x^2 + v_y^2) - \frac{GM}{\sqrt{r^2 + a^2}}, \qquad \ell_z = x v_y - y v_x. \tag{3.24}$$

Φ は時間に依存しないので、

$$\frac{d\varepsilon}{dt} = \boldsymbol{v} \cdot \boldsymbol{a} + \nabla\Phi \cdot \boldsymbol{v} = 0$$

です。力は中心向きなので、$d\ell_z/dt = x a_y - y a_x = 0$ でもあります。第 2 章の二つの質量を掛けた全エネルギーや、Plummer 球自身の自己重力エネルギーを計算しているわけではありません。

```
def diagnostics(history, mass, a):
    pos, vel = history[:, 0, :2], history[:, 0, 2:]
    radius = np.sqrt(np.sum(pos**2, axis=1))
    energy = 0.5 * np.sum(vel**2, axis=1) + potential_plummer(radius, mass, a)
    angular = pos[:, 0] * vel[:, 1] - pos[:, 1] * vel[:, 0]
    return radius, energy, angular
```

位置と速度を同じ保存時刻から取り出します。Plummer ポテンシャルを使って運動した結果に、点質量の $-GM/r$ を足してエネルギーを診断しないようにします。

3.10.2 有限刻みによる差を測る

今回の 2 例では $\varepsilon(0)$ と $\ell_z(0)$ がともに 0 ではないので、その絶対値で規格化できます。保存した全時刻での最大値を

$$e_\varepsilon = \max_n \frac{|\varepsilon^n - \varepsilon^0|}{|\varepsilon^0|}, \qquad e_\ell = \max_n \frac{|\ell_z^n - \ell_z^0|}{|\ell_z^0|} \tag{3.25}$$

とします。初期値が 0 の別の軌道では、この規格化は使わず、第 2 章のように正の基準量を選ぶ必要があります。

KDK では中心力による kick も、速度を固定する drift も角運動量を保つため、e_ℓ は丸め誤差程度になります。一方、エネルギーを毎ステップ厳密に保つ方法ではないので、円軌道では $e_\varepsilon = 1.62 \times 10^{-10}$、非円軌道では 1.83×10^{-5} という有限の差が現れました。保存量が小さくずれることと、軌道上の位置が同じ精度で正しいことは、別に調べます。

表 3.6　標準条件の保存量と最小保存半径。両例とも $5T_c$ まで 5000 ステップで計算しています。角運動量の末尾の桁は実行環境で多少変わります。

軌道	e_ε	e_ℓ	保存された最小半径 [kpc]
円	1.62×10^{-10}	1.08×10^{-14}	6.000000
非円	1.83×10^{-5}	9.47×10^{-15}	3.027208

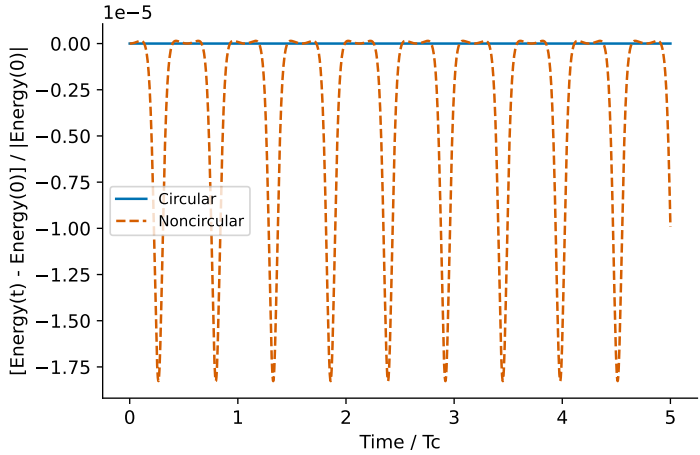


図 3.9　粒子の比エネルギーの初期値からの符号付きの差。円軌道の差は小さく、この縮尺では 0 付近に重なります。非円軌道では半径の増減に伴い差が変動します。これは摩擦によるエネルギー損失を入れた計算ではありません。

3.10.3 円軌道を同じ時刻の解析位置と比べる

円軌道では、$\Omega_c = v_c(r_0)/r_0$ から解析的な位置を

$$\boldsymbol{r}_{\mathrm{exact}}(t) = r_0(\cos \Omega_c t, \sin \Omega_c t) \tag{3.26}$$

と書けます。同じ時刻の数値解との最大の距離を、初期半径で規格化します。

$$e_r = \max_n \frac{|\boldsymbol{r}^n - \boldsymbol{r}_{\mathrm{exact}}(t_n)|}{r_0}. \tag{3.27}$$

半径の差だけではないので、回転の位相がずれた場合も検出できます。

この試験だけ最終時刻を T_c とし、100、200、400、800 ステップで比較します。$5T_c$ の標準図とは計算時間が異なりますが、試験系列の中では同じ時刻まで計算します。

```
def exact_circle(times, radius, mass, a):
    omega = circular_speed_plummer(radius, mass, a) / radius
    phase = omega * np.asarray(times)
    return np.column_stack((radius * np.cos(phase), radius * np.sin(phase)))


orbit_rows = []
initial_circle = cases["Circular"]["initial"]
for count in [100, 200, 400, 800]:
    t, h = integrate_leapfrog(acceleration_plummer, initial_circle, t_c, count, mass, a)
    exact = exact_circle(t, r0, mass, a)
    position_error = np.max(np.linalg.norm(h[:, 0, :2] - exact, axis=1)) / r0
    orbit_rows.append([count, t_c / count, position_error])
orbit_convergence = np.asarray(orbit_rows)
orbit_orders = np.log2(orbit_convergence[:-1, 2] / orbit_convergence[1:, 2])
```

表 3.7 Plummer 球中の円軌道の時間刻み試験。最終時刻は全行で解析的な円軌道周期 T_c です。p_r は前の行との位置誤差の比から求めます。

N_{step}	Δt [Myr]	e_r	p_r
100	1.627630	3.25×10^{-3}	—
200	0.813815	8.12×10^{-4}	2.00052
400	0.406907	2.03×10^{-4}	2.00020
800	0.203454	5.07×10^{-5}	2.00005

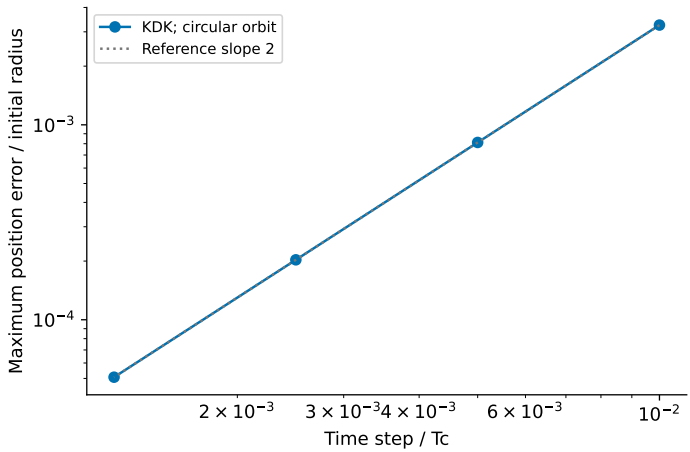


図 3.10 Plummer 球中の円軌道に対する位置誤差。横軸は $\Delta t/T_c$ です。点線は、最も細かい刻みの実測点を通る傾き 2 の参照線です。

最も細かい側の推定次数は $p_r = 2.00005$ で、KDK の 2 次精度を確認できました。重力場を点質量から Plummer 球へ変えても、滑らかな軌道を十分小さい刻み

で追う場合の次数は同じです。円軌道のエネルギー誤差には特殊な打ち消しが生じる場合があるので、その誤差だけから解法の次数を判断しません。

3.10.4 閉じない非円軌道の精度を確かめる

非円軌道では、T_c や $5T_c$ 後に初期位置へ戻るとは限りません。初期位置へ戻らない量を、そのまま誤差にしてはいけません。本章では、同じ $5T_c$ まで異なる刻みで計算した終点を比較します。

刻み h と $h/2$ による終点を使い、

$$d_h = \frac{|\boldsymbol{r}_h(t_{\mathrm{end}}) - \boldsymbol{r}_{h/2}(t_{\mathrm{end}})|}{r_0} \tag{3.28}$$

とします。もし $\boldsymbol{r}_h = \boldsymbol{r}_{\mathrm{true}} + \boldsymbol{C}h^2 + \cdots$ で主要な誤差を表せるなら、差を取ると $(3/4)\boldsymbol{C}h^2$ が残るので、$d_h/d_{h/2}$ は約 4 になります。

表 3.8 非円軌道の終点の自己収束試験。すべて $5T_c$ まで計算します。各行は、隣接する二つの刻みの数値解の差です。真の解との誤差ではありません。

粗い計算のステップ数	細かい計算のステップ数	終点の差 $/r_0$	前の行との比
1250	2500	8.98×10^{-4}	—
2500	5000	2.25×10^{-4}	3.99969
5000	10000	5.61×10^{-5}	3.99992

細かい側の比は 3.99992 で、およそ 4 でした。図3.8のロゼット状の運動が、刻みを細かくすると Kepler 楕円へ変わるわけではありません。ただし、自己収束の確認だけで、力の式そのものが正しいと証明できるわけでもありません。解析的な円軌道、密度と力の関係、保存量と合わせて判断します。

3.11 三つの数値計算の役割を区別する

ここまでの計算では、同じ Plummer モデルに対して、異なる目的の算法を使いました。どの入力を変えると、どの誤差を調べることになるかを整理しておきます。

操作	固定するもの	確認するもの
台形則の区間を細かくする	積分範囲、密度モデル	解析的な質量との差
二分法の許容幅を小さくする	質量積分の区間数、求根区間	数値的な関数の根の精度
KDK の時間刻みを小さくする	初期条件、重力場、最終時刻	軌道位置と保存量の変化

a を変えることは、これらの数値精度を調整することとは違います。密度分布と重力そのものを変更する、**物理モデルの変更**です。例えば M と r_0 を固定して a を大きくすると、内側の質量が減り、円運動速度は小さくなります。円軌道を比較

するのか、同じ位置・速度から異なるポテンシャルでの応答を比べるのかによって、そろえる初期条件も変わります。

また、本章では軌道に解析的な力を使っているので、台形則の区間数を変えても、そのまま軌道の計算結果が変わることはありません。数値的な質量表から力を計算する実装へ拡張した場合には、空間の積分・補間誤差と時間積分誤差を、改めて分けて確かめる必要があります。そこまでの拡張は本章の必須課題にはしません。

3.12 本章のまとめ

本章では、質量に広がりを持つ静的な銀河を、球対称の Plummer モデルで表しました。密度を球殻の体積で重み付けして積分すると、内側の質量が得られます。球対称の重力加速度と円運動速度は、その内側の質量から決まります。一方、ポテンシャルには外側の質量も寄与するため、力と同じ置き換えを無条件に行いません。

台形則では、小区間の両端を結ぶ直線の下の面積を足しました。積分範囲を固定して幅を半分にすると、質量の誤差がおよそ $1/4$ になることを確認しました。二分法では、半質量半径を挟む区間を保って半分にしました。ただし、二分法の残差を小さくしても、関数値に含まれる質量積分の誤差までは消えません。

最後に、解析的な Plummer 加速度を KDK へ渡して、円軌道と非円軌道を計算しました。固定した球対称場では、粒子の比エネルギーと比角運動量を確認できます。円軌道の解析位置と、非円軌道の刻みを変えた比較も使い、保存量だけに頼らず結果を確かめました。

次の第 4 章では、1 個のテスト粒子から多数の粒子へ進みます。円盤上の分布を作るために、台形則による累積積分を再利用し、線形補間と乱数を組み合わせます。見た目に円盤状の粒子を配置しても、その粒子自身の自己重力を計算するわけではない、という区別を引き継ぎます。

演習

演習 3.1　確認：密度・質量・円運動速度　$M = 10^{11} M_\odot$、$a = 3\,\mathrm{kpc}$ について、中心密度 $\rho(0)$、$M(<a)/M$、半質量半径 r_h を計算してください。また、$r = 2a$ での円運動速度を $\mathrm{km\,s^{-1}}$ と内部単位の両方で示してください。軌道を 2 次元で計算する場合でも、質量積分に $4\pi r^2$ が必要な理由を説明してください。

演習 3.2　確認：台形則の配列　$[0, 5a]$ を $N_r = 40$ 区間に分けたときの分点数と区間幅を求めてください。`cumulative_trapezoid` のループで、`integral[j + 1]` が何を表すか説明してください。密度ではなく $g = 4\pi r^2 \rho$ を渡す理由を、単位からも確かめてください。

演習 3.3　応用：範囲と分割数を分ける　$[0, 5a]$ の区間数を増やす実験と、Δr を固定して範囲を $5a, 10a, 20a$ へ広げる実験を行ってください。数値積分の誤差と範

囲外の質量を別々に示してください。全質量 M と $M(<5a)$ の違いを、台形則の誤差だけで説明できるでしょうか。

演習 3.4　確認：二分法の反復回数　初期区間を $[0, 15]\,\mathrm{kpc}$ としたとき、半区間幅を $10^{-8}\,\mathrm{kpc}$ 以下にするには何回の半減が必要か求めてください。半減回数と中点評価回数の違いも示してください。また、$|F(\widehat{r}_h)|$ が小さくても、$|\widehat{r}_h - r_h|$ が同じ程度には小さくならない理由を説明してください。

演習 3.5　応用：球全体の半分と、範囲内の半分　求根式の右辺を $M/2$ から $M(<5a)/2$ へ変えて、二つの半径を比較してください。先に解析式で違いを確認し、その後、数値積分で作る関数を二分法で解いてください。どちらが小さくなるでしょうか。この変更が単なる数値精度の改善ではない理由を述べてください。

演習 3.6　応用：スケール長を変える　M と $r_0 = 6\,\mathrm{kpc}$ を固定し、$a = 1.5, 3, 6\,\mathrm{kpc}$ の円運動速度と半質量半径を求めてください。それぞれに対応する円運動速度で初期条件を作り、円軌道を計算してください。すべてに同じ初期速度を与えた場合とは、何を比較している点が違うでしょうか。

演習 3.7　発展：非円軌道の近点を求根で調べる　$\eta = 0.7$ の初期比エネルギー ε_0、比角運動量 $\ell_{z,0}$ を用いて、

$$\Phi(r) + \frac{\ell_{z,0}^2}{2r^2} - \varepsilon_0 = 0 \tag{3.29}$$

の根を二分法で求めてください。この式は、動径速度が 0 になる条件です。初期遠点 r_0 ではない内側の根を探すため、区間を $[0.05r_0, 0.95r_0]$ とします。関数の符号を確認してから求根し、軌道の保存された最小半径と比較してください。両者の違いには何が含まれるでしょうか。

第 4 章　テスト粒子で円盤銀河を作る

第 3 章では、固定した Plummer ポテンシャル中で 1 個の星を動かしました。この章では、同じ重力場の中に多数の星を配置し、回転する円盤を作ります。新しく学ぶ数値計算法は線形補間です。前章の台形則と乱数を組み合わせ、指定した分布に従う粒子の位置を生成します。時間発展には、引き続き leapfrog 法の kick–drift–kick（KDK）形式を使います。

ここで作るのは、自己重力で支え合う円盤ではありません。重力は Plummer 球が作り、星を表すテスト粒子はその重力を受けるだけです。粒子を多数に増やすことと、粒子同士の重力を計算することを区別します。まず一つの銀河の中で円盤が保たれることを確かめ、第 5 章の相互作用へ進む準備をします。

この章で学ぶこと

- 円盤の面密度と、粒子の半径の確率密度の違いを説明できます。
- 台形則で累積分布関数を作り、線形補間で逆変換できます。
- 乱数から位置を生成し、同じ Plummer モデルの円運動速度を与えられます。
- 1 粒子の加速度を NumPy で多数粒子へ拡張し、KDK で時間発展させられます。
- 分布表の誤差、有限粒子数のばらつき、軌道積分の誤差を分けて確認できます。

4.1 重力を作る球と表示する円盤を区別する

4.1.1 本章のモデルを定義する

原点に質量 M、スケール長 a の Plummer 球を固定します。ポテンシャルは前章と同じ

$$\Phi(r) = -\frac{GM}{\sqrt{r^2 + a^2}} \tag{4.1}$$

です。[2] 球の質量は時間によらず一定で、粒子の配置を変えても変わりません。質量をもたないテスト粒子を x–y 平面に置き、初期速度もその平面内で与えます。外部からの力、ガス、星形成、動力学的摩擦は含めません。

粒子 i の位置と速度は $\boldsymbol{r}_i = (x_i, y_i)$、$\boldsymbol{v}_i = (v_{x,i}, v_{y,i})$ です。円盤の中心からの半径を $R = \sqrt{x^2 + y^2}$ と書きます。重力を作る球の半径 $r = \sqrt{x^2 + y^2 + z^2}$ とは定義が異なりますが、本章では $z = 0$ なので $r = R$ です。R はここでは円盤の半径であり、後の銀河中心の位置ベクトル $\boldsymbol{R}_k$ ではありません。

4.1.2 円盤の粒子に質量を割り当てない

画面上に円盤が現れても、その粒子分布から重力場を計算しているわけではありません。Plummer 球の質量 M を N 等分して粒子へ配る操作も行いません。M は背景ポテンシャルのパラメータで、N は円盤を描くために追跡する粒子数です。

本章では円盤の形を**粒子数面密度** Σ_N で定義します。これは面積当たりの粒子数です。各粒子に同じ表示上の重みを与えれば恒星分布の形を表せますが、その重みを加速度計算へ入れることはしません。このため、円盤の分布と Plummer 球の密度は独立に選べます。自分自身の重力まで含めた平衡銀河を構成する問題とは、区別してください。

よくある失敗：多数の粒子があっても、相互重力はまだありません

各粒子は同じ解析的ポテンシャルから独立に力を受けます。粒子数を 2 倍にしても、すでに置いた 1 粒子への力は変わりません。本章は多数のテスト粒子の計算であり、自己重力円盤の N 体計算ではありません。

4.2 指数円盤を半径の確率分布へ直す

4.2.1 初期の配置範囲を決める

中心ほど粒子が多い円盤を作るため、初期の粒子数面密度を

$$\Sigma_N(R) = \Sigma_{N,0}\exp(-R/R_d), \qquad 0 \le R \le R_{\max} \tag{4.2}$$

とします。R_d は指数分布のスケール長、$R_{\max}$ は初期粒子を配置する外縁です。この形は構成案で選んだ教材用モデルです。本章では $R_d = a = 3\,\mathrm{kpc}$ としますが、二つは同じ物理量ではなく、等しくする必要もありません。

第 3 章の r_{tab} は、無限遠まで続く Plummer 球の質量表を作る範囲でした。一方、今回は初期粒子分布を $R_{\max}$ で切断すると定義しています。ただし、Plummer 球の密度や重力をそこで切断しません。また、時間積分中に $R_{\max}$ を越えた粒子を取り除いたり、跳ね返したりもしません。初期の配置範囲と、運動の境界条件は別です。

4.2.2 円環の面積を含める

半径 R から $R + dR$ までの薄い円環の面積は $2\pi R\,dR$ です。その中の期待粒子数は

$$dN = 2\pi R \Sigma_N(R)\,dR \tag{4.3}$$

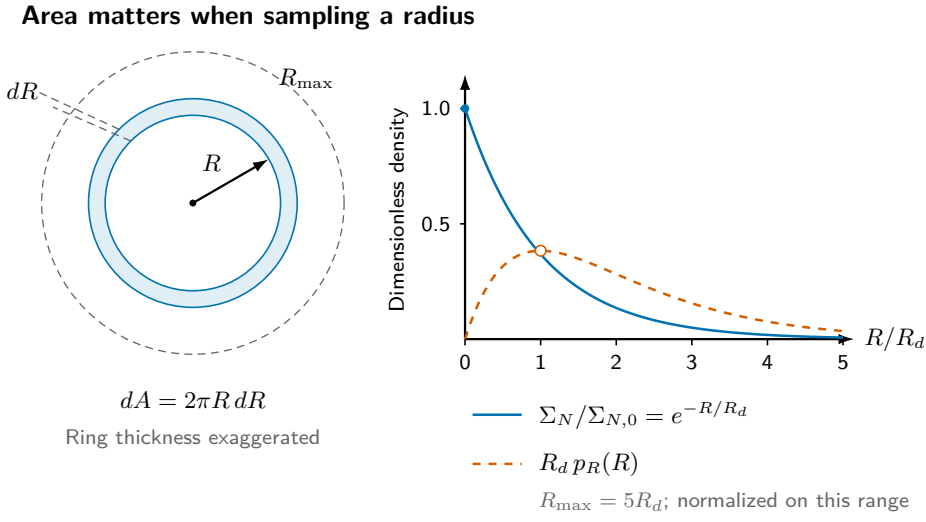
となります。全粒子数で割れば、半径の確率密度 $p_R(R)$ が得られます。

$$p_R(R) = \frac{2\pi R\Sigma_N(R)}{N} = \frac{R\exp(-R/R_d)}{\displaystyle\int_0^{R_{\max}} R'\exp(-R'/R_d)\,dR'}. \tag{4.4}$$

p_R の単位は長さの逆数です。$p_R(R)\,dR$ は無次元の確率であり、面密度とは異なります。第 3 章の球殻には $4\pi r^2$ を使いましたが、ここで粒子を配置するのは円盤なので $2\pi R$ です。

Σ_N は中心で最大ですが、$p_R(0) = 0$ です。中心に近い円環ほど面積が小さいからです。$R_{\max} > R_d$ なら、$R\exp(-R/R_d)$ を微分すると、半径の確率密度は $R = R_d$ で最大になります。中心に面密度のピークがあることと、半径のヒストグラムのピークが中心にあることは同じではありません。

図4.1では、円環の面積と二つの分布の違いを対応させています。第 3 章の球殻（図3.1）と比べ、今回は体積ではなく面積を掛けることを確認してください。



$$dN \quad = \quad \Sigma_N(R)\,dA \quad = \quad 2\pi R\,\Sigma_N(R)\,dR$$

図 4.1　円環の面積と半径の確率密度の関係。面密度に $2\pi R$ を掛けて半径の分布へ直します。右側は $R_{\max} = 5R_d$ で規格化した解析的な入力分布で、標本化した粒子の結果ではありません。面密度は中心で最大でも、半径の確率密度は中心で 0 です。

よくある失敗：半径を指数乱数にすればよい、とは限りません

$p_R \propto \exp(-R/R_d)$ としてしまうと、円環の面積に由来する R を落としてしまいます。また、R を一様乱数で選んでも、面積当たり一様な円盤にはなりません。何に対する密度を指定しているかを、最初に確認します。

4.3 台形則で累積分布関数を作る

4.3.1 半径以内に入る確率を求める

半径 R 以内に入る確率を、累積分布関数（cumulative distribution function; CDF）と呼びます。ここでは

$$F(R) = \int_0^R p_R(R')\,dR' = \frac{\displaystyle\int_0^R R' \exp(-R'/R_d)\,dR'}{\displaystyle\int_0^{R_{\max}} R' \exp(-R'/R_d)\,dR'} \tag{4.5}$$

です。$F(0) = 0$、$F(R_{\max}) = 1$ で、その間で単調に増えます。累積分布とその逆関数から目的の分布に従う乱数を作る考え方は、文献 [4] の第 23 章で扱われています。

解析的な比較値も用意できます。$x = R/R_d$、$x_{\max} = R_{\max}/R_d$ として

$$C(x) \equiv 1 - (1 + x)e^{-x} \tag{4.6}$$

と置きます。$R' \exp(-R'/R_d)$ を積分すると $R_d^2 C(R/R_d)$ になるので、

$$F(R) = \frac{C(R/R_d)}{C(x_{\max})}, \qquad p_R(R) = \frac{R \exp(-R/R_d)}{R_d^2 C(x_{\max})} \tag{4.7}$$

です。初期分布の規格化定数は $\Sigma_{N,0} = N/[2\pi R_d^2 C(x_{\max})]$ です。この式は本章の面密度を積分して得た比較用の式であり、数値 CDF を作る処理には使いません。

$x_{\max} = 5$ では $C(x_{\max}) = 0.959572318$ です。無限に延ばした同じ指数分布のうち、約 4.04% が $5R_d$ より外側にあります。本章ではそこを初期配置の対象から外し、内側の条件付き分布を確率 1 へ規格化します。Plummer 球の $5a$ 以内の質量割合とは別の数です。

4.3.2 前章の累積積分を再利用する

分点を R_j、被積分関数を $g_j = R_j \exp(-R_j/R_d)$ として、台形の面積を順に足します。

$$I_0 = 0, \qquad I_{j+1} = I_j + \frac{g_j + g_{j+1}}{2}(R_{j+1} - R_j), \tag{4.8}$$

$$\widehat{F}_j = \frac{I_j}{I_{N_R}}. \tag{4.9}$$

N_R は区間数で、分点数は $N_R + 1$ です。ハットは数値近似を表します。I_j の単位は長さの 2 乗ですが、最後の値で割った $\widehat{F}_j$ は無次元です。

第 3 章では、有限範囲の累積質量を無条件に全質量 M へそろえませんでした。ここで最後の値で割ってよいのは、式(4.5)そのものが有限範囲で規格化した確率だからです。表を都合よく補正する操作ではありません。

```
def cumulative_trapezoid(x, values):
    """ 第 3 章と同じ、各分点までの台形の和です。"""
    x, values = check_grid(x, values)
    integral = np.zeros_like(x)
    for j in range(len(x) - 1):
        area = 0.5 * (values[j] + values[j + 1]) * (x[j + 1] - x[j])
        integral[j + 1] = integral[j] + area
    return integral
```

4.3.3 標準条件で CDF 表を実装する

内部単位は kpc、Gyr、$M_\odot$ です。速度は内部では $\mathrm{kpc\,Gyr^{-1}}$ を使い、表示するときだけ $\mathrm{km\,s^{-1}}$ へ換算します。質量・Plummer スケール長は第 3 章と共通です。その他の条件は、この章の孤立円盤を確認するために設定しました。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
mass, a = 1.0e11, 3.0                # Plummer 球の質量 [Msun] とスケール長 [kpc]
rd, rmax = 3.0, 15.0                 # 粒子分布のスケール長と初期外縁 [kpc]
n_particles = 1000
n_cdf = 4096                        # CDF 表の区間数。分点数は 4097 です。
seed, spin = 42, +1                  # +1 は反時計回り、-1 は時計回りです。
t_end, n_steps = 0.60, 3000          # [Gyr]。dt = 0.2 Myr です。
save_every = 10                      # 時間刻みではなく、履歴の保存間隔です。
dt = t_end / n_steps
```

表 4.1 第 4 章の標準条件。N_R、N、N_{step} はそれぞれ別の個数です。$R_{\max}$ は Plummer 球の外縁ではありません。

量	設定
Plummer 球	$M = 10^{11} M_\odot$, $a = 3\,\mathrm{kpc}$
粒子の初期分布	$R_d = 3\,\mathrm{kpc}$, $R_{\max} = 15\,\mathrm{kpc} = 5R_d$
粒子数・乱数 seed	$N = 1000$, seed $= 42$
CDF の区間数・分点数	$N_R = 4096$, 4097 分点
回転方向	$s_{\mathrm{rot}} = +1$（反時計回り）
終時刻・ステップ数	$t_{\mathrm{end}} = 0.60\,\mathrm{Gyr}$, $N_{\mathrm{step}} = 3000$
時間刻み・保存間隔	$\Delta t = 0.2\,\mathrm{Myr}$、10 ステップごと

```
def build_radial_cdf(rd, rmax, n_panels):
    check_positive(rd, "rd")
    check_positive(rmax, "rmax")
    check_integer(n_panels, "n_panels")
    radius_grid = np.linspace(0.0, rmax, n_panels + 1)
    weight = radius_grid * np.exp(-radius_grid / rd)
    cumulative = cumulative_trapezoid(radius_grid, weight)
    if not np.isfinite(cumulative[-1]) or cumulative[-1] <= 0.0:
        raise ValueError("The normalization must be positive and finite.")
    cdf = cumulative / cumulative[-1]
    if np.any(np.diff(cdf) <= 0.0):
        raise ValueError("The inverse CDF needs strictly increasing values; revise the
        ↪  grid/range.")
    return radius_grid, cdf
```

`check_positive` と `check_integer` は、正で有限な値、正の整数であることを確認する補助関数です。完全なコードは Notebook に含めます。`cdf` を後で逆変換するため、隣接する値がすべて増えていることも確認します。標準条件では、$R = 0$ の被積分関数が 0 でも最初の台形の面積は正なので、CDF の最初の区間も増えます。

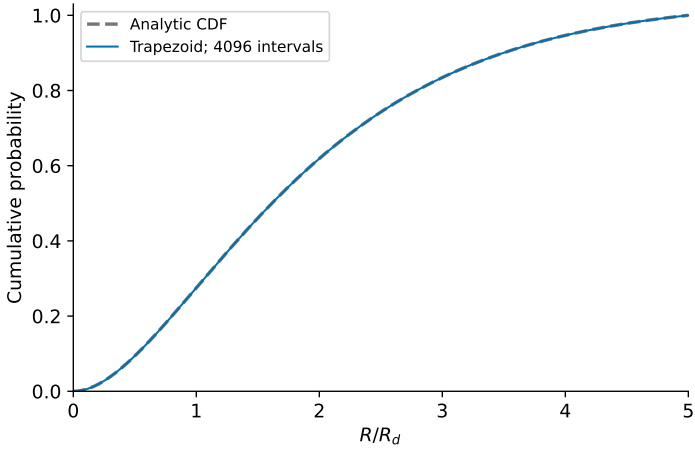


図 4.2 初期円盤の CDF。灰色破線は式(4.7)、青実線は台形則による数値表です。有限の初期配置範囲 $[0, 5R_d]$ で規格化しています。

分点上で解析式と比較した最大差は 9.29×10^{-8} でした。図4.2ではほぼ重なりますが、この近似誤差が 0 という意味ではありません。

4.4 線形補間で分点間の値を求める

4.4.1 2 点間の関数を直線で近似する

CDF 表は、決めた分点の値しか持ちません。分点の間の値を求めるため、二つの点を結ぶ直線で近似します。この操作が線形補間です。前章の台形則では同じ直線の下の面積を求めましたが、ここでは直線上の値そのものを使います。

一般の表 (x_j, y_j) について、$x_j \le x \le x_{j+1}$ の位置を

$$\lambda = \frac{x - x_j}{x_{j+1} - x_j} \tag{4.10}$$

で表します。左端なら $\lambda = 0$、右端なら $\lambda = 1$ です。二つの値をその割合で混ぜると、

$$y_{\mathrm{lin}}(x) = (1 - \lambda) y_j + \lambda y_{j+1} \tag{4.11}$$

となります。分点では表の値に一致し、その間では両端の値の間を取ります。傾きを用いた $y_j + (y_{j+1} - y_j)(x - x_j)/(x_{j+1} - x_j)$ とも同じ式です。補間と補間誤差の一般的な考え方は文献 [3] の §2.3 を参照してください。

4.4.2 一つの点を実装して確かめる

まず区間を順に探す、短い実装を作ります。区間が見つかったら、式(4.10)と式(4.11)を計算するだけです。入力する分点は狭義単調増加とします。

```
def linear_interp_one(x, xp, fp):
    """ 一つの点を補間します。本文 eq:ch4-linear-interpolation。"""
    xp, fp = check_grid(xp, fp)
    if not np.isfinite(x) or x < xp[0] or x > xp[-1]:
        raise ValueError("Interpolate only within the finite table range.")
    if x == xp[-1]:
        return float(fp[-1])
    for j in range(len(xp) - 1):
        if xp[j] <= x <= xp[j + 1]:
            fraction = (x - xp[j]) / (xp[j + 1] - xp[j])
            return (1.0 - fraction) * fp[j] + fraction * fp[j + 1]
    raise RuntimeError("No interpolation interval was found.")
```

この関数は表の外では停止します。補間は表の内側での近似であり、外側へ直線を延ばす外挿とは区別します。`check_grid` は第 3 章と同じく、分点・値の配列の形と有限性、分点の増加を確かめます。

多数の点を補間するときには、`np.interp(x, xp, fp)` を使います。`xp` が入力側の分点、`fp` がその値です。自作関数で原理を確認したうえで、同じ区分的な線形補間を行うライブラリ関数に置き換えます。[9] 全粒子について Python で表を繰り返し走査する必要はありません。

よくある失敗：増加していない表をそのまま渡さない

入力側の分点に重複があると、式(4.10)の分母が 0 になります。表を作る段階で、分点が有限で狭義単調増加かを確認します。また、`np.interp` は既定で

は表の外で端の値を返します。本章の自作関数とはこの扱いが違うので、呼び出し前に補間点が表の範囲内にあることを確認します。

4.4.3 線形補間の精度と表の誤差を区別する

真の関数 f が滑らかで、両端の値が正しく与えられている場合、線形補間の誤差は区間内のある点 ξ を使って

$$f(x) - f_{\mathrm{lin}}(x) = \frac{f''(\xi)}{2}(x - x_j)(x - x_{j+1}) \tag{4.12}$$

と書けます。区間幅を h とし、$|f''|$ が上から抑えられるなら、誤差の大きさは $\max|f''|\,h^2/8$ 以下です。これが、滑らかな関数を細かい表で線形補間したとき、通常は幅の 2 乗で誤差が減る理由です。式は多項式補間の誤差公式の 2 分点の場合です。[3]

ただし、これから使う CDF 表の値には台形則の誤差もあります。補間してもその誤差は消えません。さらに、CDF の逆関数は中心近くで傾きが大きくなるため、半径の誤差が全域で単純な同じ比例関係に従うとは限りません。本章では解析 CDF に戻した確率のずれを、まず確認します。

もう少し詳しく：ほかの補間法

多項式補間や三次スプラインは、より多くの分点を使って関数を近似します。高次の方法が有利な場合もありますが、点の間で振動したり、単調性を失ったりしないか確認が必要です。PCHIP は単調性を保つ区分的三次補間の代表例です。[10] 本章では、増加する表を扱いやすく実装も短い線形補間に集中します。詳細な理論と代替法の実装は数値計算の教科書に譲ります。

4.5 一様乱数を円盤の半径へ変換する

4.5.1 CDF の逆向きに表を読む

$0 \le u < 1$ の一様乱数を一つ選び、$F(R) = u$ となる半径を求めます。CDF は「半径から確率」を返しますが、ここでは「確率から半径」を求めます。この方法を逆変換サンプリングと呼びます。[4]

なぜこれで正しい分布になるのでしょうか。単調増加する F では、生成半径が R_* 以下になることと、一様乱数が $F(R_*)$ 以下になることが同値です。一様乱数がその範囲へ入る確率は $F(R_*)$ です。このため、生成半径の累積分布は目標の F になります。実装では、これを数値表の逆補間で近似します。

$\widehat{F}_j \le u \le \widehat{F}_{j+1}$ なら、入力側を $\widehat{F}$、出力側を R として

$$\widehat{R}(u) = R_j + \frac{u - \widehat{F}_j}{\widehat{F}_{j+1} - \widehat{F}_j}(R_{j+1} - R_j) \tag{4.13}$$

です。$\widehat{F}_j$ が増えているので、$\widehat{R}$ は必ず二つの分点の間に入ります。

```
# F -> R と表の役割を交換します。cdf は狭義単調増加でなければなりません。
u_demo = 0.5
radius_demo = linear_interp_one(u_demo, cdf, radius_grid)
radius_numpy = np.interp(u_demo, cdf, radius_grid)
print(f"Radius at CDF=0.5: manual={radius_demo:.9f}, NumPy={radius_numpy:.9f} kpc")
```

標準表で $u = 0.5$ を逆変換すると、半径は 4.843929332 kpc でした。自作関数と `np.interp` の値が一致することを確認します。これは初期円盤の粒子数の半分を含む半径で、Plummer 球の半質量半径ではありません。

図4.3の矢印を、縦軸の確率から横軸の半径へたどってください。

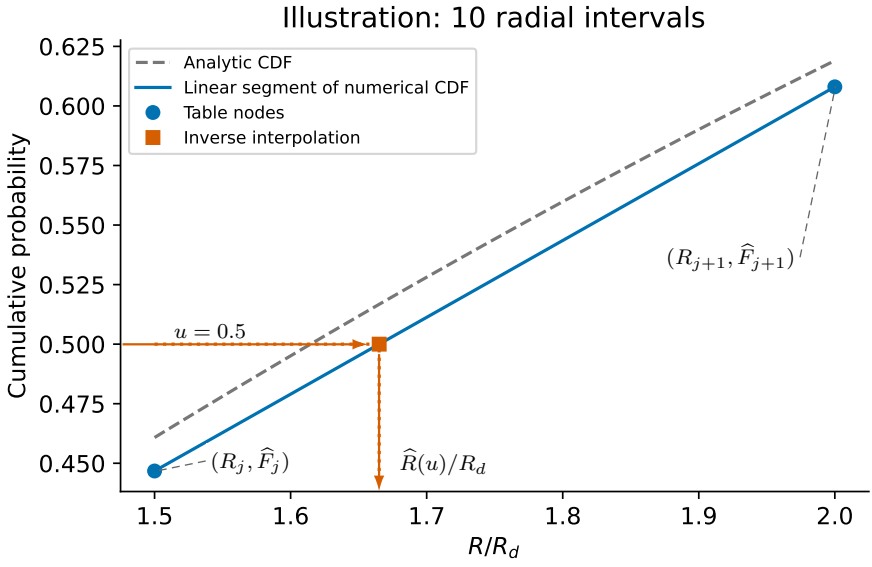


図 4.3 数値 CDF の一つの線分を逆向きに読む例。説明用に 10 区間だけで作った粗い表を使っています。丸が分点、四角が $u = 0.5$ の逆補間です。灰色破線の解析 CDF との差も示しています。標準の 4096 区間の結果とは区別してください。朱色の矢印は u から線分へ、そこから $\widehat{R}(u)$ へ読む順序です。表の組 $(R_j, \widehat{F}_j)$ を示し、横軸では R を R_d で規格化しています。

4.5.2 乱数を使う前に、逆変換を検証する

同じ数値表で $u \to \widehat{R} \to \widehat{F}$ と戻せば、線形補間の範囲ではもとの u にほぼ戻ります。しかし、それだけでは数値表が正しいかは分かりません。比較には、独立な解析 CDF である式(4.7)を使います。

まず分点上の誤差 $e_{F,\mathrm{node}}$ を、次に固定した確率の集合 $\mathcal{U}$ 上の逆変換の誤差 e_u を

$$e_{F,\mathrm{node}} = \max_j |\widehat{F}_j - F(R_j)|, \tag{4.14}$$

$$e_u = \max_{u\in\mathcal{U}} |F(\widehat{R}(u)) - u| \tag{4.15}$$

で定義します。本章の $\mathcal{U}$ は 0 から 1 までの 20001 個の等間隔の点です。まだ乱数は使いません。e_u は選んだ試験点上の最大値であって、連続な全確率範囲の最大誤差を厳密に保証する値ではありません。

```
u_check = np.linspace(0.0, 1.0, 20001)
cdf_rows = []
for count in [32, 64, 128, 256, 512]:
    nodes, values = build_radial_cdf(rd, rmax, count)
    node_error = np.max(np.abs(values - disk_cdf_exact(nodes, rd, rmax)))
    inverted = np.interp(u_check, values, nodes)
    probability_error = np.max(np.abs(disk_cdf_exact(inverted, rd, rmax) - u_check))
    cdf_rows.append([count, rmax / count, node_error, probability_error])
cdf_convergence = np.asarray(cdf_rows)
cdf_orders = np.log2(cdf_convergence[:-1, 3] / cdf_convergence[1:, 3])
```

表 4.2 CDF の分割数試験。範囲 $[0, 5R_d]$ と試験確率は固定しています。p_u は隣接する行の e_u の比から求めた推定次数です。

N_R	ΔR [kpc]	$e_{F,\mathrm{node}}$	e_u	p_u
32	0.468750	1.52×10^{-3}	2.45×10^{-3}	—
64	0.234375	3.80×10^{-4}	6.98×10^{-4}	1.8117
128	0.117188	9.51×10^{-5}	1.85×10^{-4}	1.9121
256	0.058594	2.38×10^{-5}	4.80×10^{-5}	1.9486
512	0.029297	5.94×10^{-6}	1.12×10^{-5}	2.0986

表4.2では、分割を細かくすると誤差が減っています。標準の 4096 区間での e_u は 1.76×10^{-7} でした。この数値誤差と、次に見る有限粒子数のばらつきを分けて考えます。表を細かくするだけで、乱数で配置した点が完全になめらかな分布になるわけではありません。

4.5.3 半径と方位角に乱数を与える

半径のための u_i とは別に、一様乱数 w_i を使って $\phi_i = 2\pi w_i$ とします。ここで ϕ は方位角です。二つを独立に選ぶことで、半径によらず方位角を一様にできます。位置は

$$x_i = R_i\cos\phi_i, \qquad y_i = R_i\sin\phi_i \tag{4.16}$$

です。同じ乱数一つを半径と角度に流用すると両者が関連し、意図した軸対称の分布を作れません。

Notebook では `rng = np.random.default_rng(seed)` で乱数生成器を用意します。`rng.random((N, 2))` の各行に、半径と角度に使う二つの一様乱数を入れ

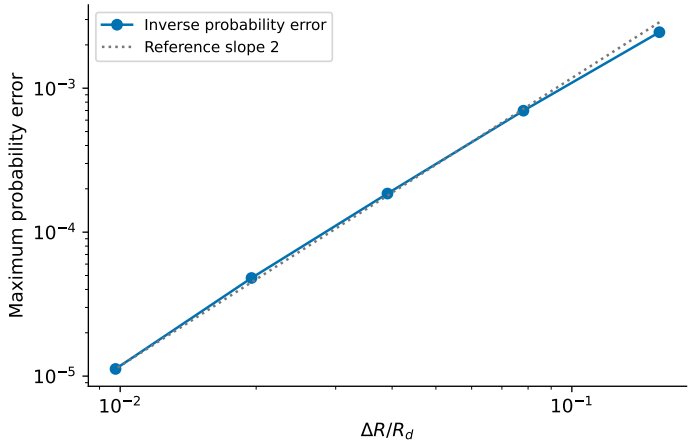


図 4.4　数値 CDF と逆補間による確率のずれ。点線は最も細かい分割の値を通る傾き 2 の参照線で、当てはめではありません。試験確率が有限個なので、最大誤差の位置の拾い方も推定次数に影響します。

ます。seed は実験を再現するための設定です。同じ実装・乱数生成環境・呼び出し順序なら同じ初期値を再現できますが、seed だけで異なる実装や将来のライブラリ環境まで同一の乱数列になると保証はしません。[9] 配布物には初期配列と使用した NumPy の版も記録します。

もう少し詳しく：ほかの標本化法

棄却法は、候補の乱数を生成し、目標分布に応じて採用するかを選ぶ方法です。逆関数を求めにくい場合にも使えます。本書では、既習の台形則と今回の補間を一つの計算へ結び付けるため、CDF の逆変換を使います。乱数による積分を含む Monte Carlo 法の一般論は、文献 [4] に譲ります。

4.6 円運動速度を与えて円盤を作る

4.6.1 同じ重力場から初期速度を求める

粒子を置くだけでは、円盤を回転させたことにはなりません。各半径で Plummer 球の円運動速度を与えます。第 3 章の式を再利用すると、

$$v_c^2(R) = \frac{GMR^2}{(R^2 + a^2)^{3/2}} \tag{4.17}$$

です。粒子の分布のスケール長 R_d を、この分母の a へ取り違えないようにします。位置の確率を決める式と、重力を決める式は別です。

反時計回りの接線方向の単位ベクトルは $(-\sin\phi, \cos\phi)$ です。回転方向を $s_{\mathrm{rot}} = +1$ または -1 として、

$$v_{x,i} = -s_{\mathrm{rot}} v_c(R_i)\sin\phi_i, \qquad v_{y,i} = s_{\mathrm{rot}} v_c(R_i)\cos\phi_i \tag{4.18}$$

と置きます。例えば正の x 軸上の粒子は、$s_{\mathrm{rot}} = +1$ なら正の y 方向へ動き始めます。すべての粒子に同じ速度の大きさを与えるのではなく、それぞれの R_i に対応する v_c を使います。

本章の孤立円盤には、もう一つの銀河の軌道はありません。そのため、ここでは反時計回り・時計回りと呼びます。遭遇軌道の向きとの関係で定義する prograde・retrograde は、第 5・6 章で扱います。

```
def circular_speed_plummer(radius, mass, a):
    radius = np.asarray(radius, dtype=float)
    return np.sqrt(G * mass * radius**2 / (radius**2 + a**2)**1.5)


def potential_plummer(radius, mass, a):
    return -G * mass / np.sqrt(np.asarray(radius)**2 + a**2)
```

4.6.2 位置と速度を一つの初期配列へまとめる

半径の標本化、方位角、接線速度を一つの関数にまとめます。位置と速度はそれぞれ形 $(N, 2)$ です。最後に横に並べ、各行に $[x, y, v_x, v_y]$ を持つ形 $(N, 4)$ の状態配列を返します。

```
def make_disk(n_particles, mass, a, rd, rmax, spin, rng, n_panels=4096):
    check_integer(n_particles, "n_particles")
    check_positive(mass, "mass")
    check_positive(a, "a")
    if isinstance(spin, (bool, np.bool_)) or spin not in (-1, +1):
        raise ValueError("spin must be +1 or -1.")
    nodes, values = build_radial_cdf(rd, rmax, n_panels)
    uniform = rng.random((n_particles, 2))
    radius = np.interp(uniform[:, 0], values, nodes)
    phi = 2.0 * np.pi * uniform[:, 1]
    vc = circular_speed_plummer(radius, mass, a)
    pos = np.column_stack((radius * np.cos(phi), radius * np.sin(phi)))
    vel = spin * np.column_stack((-vc * np.sin(phi), vc * np.cos(phi)))
    return np.column_stack((pos, vel)), radius, phi
```

標準例では、`rng = np.random.default_rng(seed)` を用意して `make_disk` に渡します。乱数生成器を関数の中で毎回同じ seed へ戻しません。比較実験でまったく同じ初期標本を使う場合だけ、同じ seed の生成器を改めて作ります。初期粒子の半径は、今回の標本では約 0.100153 から 14.950926 kpc でした。上限 15 kpc 以内であることを確認します。

4.6.3 標本の重心を無条件に原点へ戻さない

有限個の粒子を乱数で置けば、単純な平均位置が原点に厳密には一致しないことがあります。しかし、ここで重要なのは、各粒子の位置と速度が固定した *Plummer* 中心に対する円軌道になっていることです。

粒子の平均位置だけを引くと、各粒子の Plummer 中心からの距離が変わります。平均速度だけを引いても、円運動速度との対応が変わります。第 2 章の有限質量 2

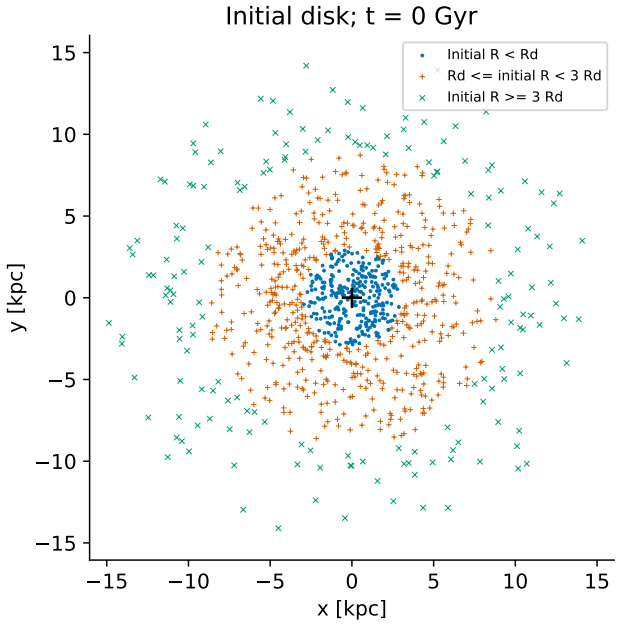


図 4.5 標準の初期円盤。初期半径による 3 群を色とマーカーで分けました。この分類は後の時刻でも固定します。中心の黒い十字は Plummer ポテンシャルの中心で、追加の質点ではありません。

体系を重心系へ移す操作と混同せず、本章では粒子だけを再中心化しません。原点を移すなら、背景ポテンシャルの中心も含めた座標変換を考える必要があります。

4.7 生成した分布を確かめる

4.7.1 累積分布で比べる

円盤らしく見えることだけでは、式(4.2)に従っているか判断できません。粒子半径を小さい順に並べ、

$$F_N(R) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(R_i \le R) \tag{4.19}$$

を作ります。$\mathbf{1}$ は条件が真なら 1、偽なら 0 です。これは、その半径までにある粒子の割合を返す**経験 CDF** です。

有限個の標本なので、F_N は幅 $1/N$ の階段になります。連続的な解析 CDF に完全には一致しません。図4.6では、この階段と解析式を比較します。

ずれの指標を $D_N = \sup_R |F_N(R) - F(R)|$ とします。並べた半径 $R_{(j)}$ における階段の直前と直後、つまり $(j-1)/N$ と j/N を両方比べれば計算できます。

$$D_N = \max_{1\le j\le N} \left\{ \left| \frac{j}{N} - F(R_{(j)}) \right|, \left| \frac{j-1}{N} - F(R_{(j)}) \right| \right\}. \tag{4.20}$$

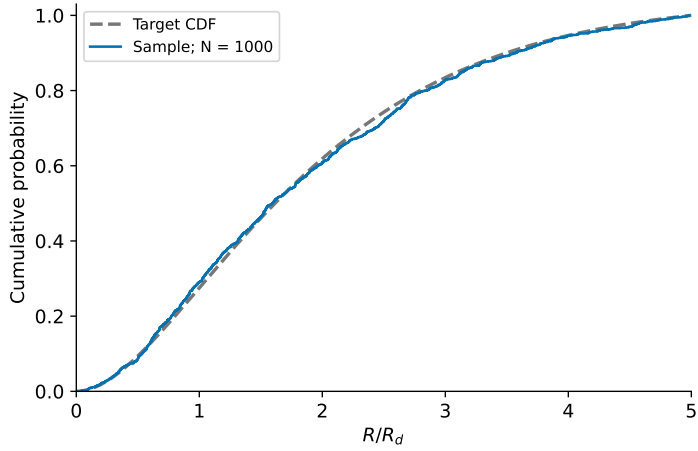


図 4.6 1000 粒子の経験 CDF と目標 CDF。数値 CDF 表の曲線ではなく、実際に生成した粒子から計算した割合です。有限標本のため、解析式の上下にずれます。

本章では分布のずれの大きさとして使い、統計的な仮説検定の合否へ直結させません。

```
def empirical_cdf_distance(radius, rd, rmax):
    sorted_radius = np.sort(np.asarray(radius))
    n = len(sorted_radius)
    target = disk_cdf_exact(sorted_radius, rd, rmax)
    right = np.arange(1, n + 1) / n
    left = np.arange(n) / n
    return max(np.max(np.abs(right - target)), np.max(np.abs(left - target)))
```

標準標本では $D_N = 0.029414$ でした。これは数値表の逆変換誤差 1.76×10^{-7} よりずっと大きい値です。表をさらに細かくしても、この標本のばらつきが同じ割合で減るわけではありません。

4.7.2 円環から粒子数面密度を求める

円環の内側と外側の半径を $R_{j,-}, R_{j,+}$ とし、その中の粒子数を N_j とします。円環の正確な面積を使うと、全粒子数で規格化した数面密度の推定値は

$$\frac{\widehat{\Sigma}_{N,j}}{N} = \frac{N_j}{N\pi(R_{j,+}^2 - R_{j,-}^2)} \tag{4.21}$$

です。単位は kpc^{-2} です。区間の幅だけで割った半径のヒストグラムとは違います。

```python
def surface_number_profile(radius, edges):
    """ 全粒子数で規格化した、円環ごとの数面密度 [kpc^-2]。"""
    counts, _ = np.histogram(radius, bins=edges)
    area = np.pi * (edges[1:]**2 - edges[:-1]**2)
    density = counts / (len(radius) * area)
    return density, counts, area
```

比較する解析値も、円環の中点の値だけでなく、同じ円環の平均

$$\left\langle \frac{\Sigma_N}{N} \right\rangle_j = \frac{F(R_{j,+}) - F(R_{j,-})}{\pi(R_{j,+}^2 - R_{j,-}^2)} \tag{4.22}$$

を使います。幅の広い円環では、点での値と平均値が異なるためです。時間発展後に、この同じ測り方で初期分布との比較も行います。

4.7.3 粒子数と seed を変える

CDF 表を固定したまま、$N = 250, 1000, 4000$ で標本化します。各 N について seed を 42 から 49 まで変え、8 回の D_N を比較した結果が表4.3です。これは同じ物理モデルを、異なる有限標本で表した比較です。

表 4.3 粒子数と seed による経験 CDF のずれ。平均・最小・最大は 8 個の seed での値です。数値積分法の収束次数を測った表ではありません。

N	D_N (seed 42)	平均	最小	最大
250	0.063392	0.052060	0.031292	0.066533
1000	0.029414	0.032565	0.017673	0.055907
4000	0.009382	0.012820	0.007856	0.018457

粒子数を増やすと、一般には分布のばらつきが小さくなります。ただし、個々の seed について D_N が必ず単調に減るとは限りません。ある領域に入る確率を p とすれば、独立な N 標本による粒子数の分散は $Np(1-p)$ です。割合のばらつきの尺度が $N^{-1/2}$ になることからも、粒子数と表の刻みは別の役割だと分かります。これは一つの領域の粒子割合に関する見積もりで、表の各 D_N が厳密に同じ係数で変わるという意味ではありません。

4.8 1 粒子の式を多数粒子へ拡張する

4.8.1 まず粒子ごとに計算する

Plummer 球の加速度は、第 3 章と同じ

$$\boldsymbol{a}_i = -\frac{GM\boldsymbol{r}_i}{(R_i^2 + a^2)^{3/2}} \tag{4.23}$$

です。新しいのは粒子番号 i が 1 個から N 個へ増える点だけです。1 粒子の成分計算を、まずループに入れます。

```
def acceleration_plummer_loop(pos, mass, a):
    acc = np.empty_like(pos)
    for i in range(len(pos)):
        x, y = pos[i]
        r2 = x**2 + y**2
        acc[i, 0] = -G * mass * x / (r2 + a**2)**1.5
        acc[i, 1] = -G * mass * y / (r2 + a**2)**1.5
    return acc
```

4.8.2 配列の軸を明示してまとめる

`pos` の形は $(N, 2)$ です。`np.sum(pos**2, axis=1)` は各粒子の $x_i^2 + y_i^2$ を返すので、`r2` の形は $(N,)$ になります。この分母を `[:, None]` で $(N, 1)$ へ変形し、各行の x, y 成分に同じ分母を使います。

```
def acceleration_plummer(pos, mass, a):
    """ 固定 Plummer 球から各粒子への加速度。本文 eq:ch4-acceleration。"""
    r2 = np.sum(pos**2, axis=1)
    acc = -G * mass * pos / (r2 + a**2)[:, None]**1.5
    return acc
```

`axis=0` で足すと、全粒子の x^2 の和と y^2 の和になり、別の量です。形が実行可能でも、意図した物理量になっているか確認します。独立なループ版との比較では、今回の差を GM/a^2 で規格化した最大値は 7.28×10^{-17} でした。演算の並べ方で末尾の桁が変わることはありますが、同じ加速度を計算しています。

この関数は $N \times N$ の距離表を作りません。背景の重力を粒子ごとに一度ずつ評価するため、1 回の加速度計算に必要な演算量は N に比例します。粒子対の重力を足す直接 N 体計算の $O(N^2)$ とは異なります。

よくある失敗：粒子番号と座標成分の軸を取り違えない

位置配列 $(N, 2)$ の第 1 軸は粒子、第 2 軸は成分です。`r2` は各粒子に一つの値を持ちます。分母を $(N, 1)$ にする操作は、同じ粒子の 2 成分へ値を対応させるためであり、粒子間相互作用を作る処理ではありません。

4.9 KDK で円盤を時間発展させる

4.9.1 積分器の計算は変えない

前章の KDK 関数は、位置と速度の配列へ加算・乗算する形でした。$(1, 2)$ を $(N, 2)$ へ増やしても、更新の 4 行は変わりません。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, mass, a):
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, mass, a)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

全粒子の最初の半 kick、全粒子の drift、新しい位置での加速度評価、最後の半 kick を順に行います。加速度を返し、次のステップへ再利用することも同じです。速度は最後の kick を終えた整数時刻の値を保存します。

4.9.2 時間刻みと出力間隔を分ける

多数粒子の位置と速度を毎ステップすべて保存すると、履歴のメモリが大きくなります。本章では時間積分は 3000 ステップすべて行いますが、履歴は 10 ステップごとに保存します。最後の時刻が保存間隔に乗らない場合も、最終状態は必ず保存します。

保存時刻の個数を N_{save} とすると、履歴の形は $(N_{\mathrm{save}}, N, 4)$ です。`float64` の各要素を 8 byte として、必要な領域は

$$B = 8 \times 4 \times N \times N_{\mathrm{save}} \quad \text{byte} \tag{4.24}$$

です。これは履歴配列だけの大きさで、一時配列や Notebook の画像のメモリは含みません。

表 4.4　標準の 1000 粒子・3000 ステップでの履歴配列。MB は 10^6 byte です。どちらも時間積分の力評価回数は 3001 回です。

保存方法	N_{save}	履歴の容量 [MB]
毎ステップ	3001	96.032
10 ステップごと	301	9.632

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, mass, a, save_every=10):
    check_positive(mass, "mass")
    check_positive(a, "a")
    state, saved_steps, times, history, dt = prepare_history(state0, t_end, n_steps,
    ↪  save_every)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, mass, a)
    output_index = 1
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, mass, a)
        if not np.all(np.isfinite(pos)) or not np.all(np.isfinite(vel)):
            raise FloatingPointError("Non-finite state; check the model and time step.")
        if n + 1 == saved_steps[output_index]:
            history[output_index] = np.column_stack((pos, vel))
            output_index += 1
    return times, history
```

`prepare_history` は初期値の形 $(N, 4)$、有限性、正のステップ数などを確認し、保存するステップ番号と配列を用意します。詳しいコードは Notebook に置きます。ループは保存の有無と無関係に毎ステップ進みます。標準の時間刻みは 0.2 Myr、保存間隔は 2 Myr です。`history[j]` の j は保存番号であり、積分した全ステップの番号ではありません。

4.9.3 孤立円盤を 0.6 Gyr 進める

初期配列を作った後、`integrate_leapfrog` へ加速度関数、終時刻、ステップ数、M, a を渡します。標準では初回の加速度と 3000 回の更新で、合計 3001 回の加速度関数の呼び出しです。各呼び出しで 1000 粒子分をまとめて評価します。

中心近くの角速度は $\Omega_0 = \sqrt{GM/a^3}$ に近づきます。対応する周期は $2\pi\sqrt{a^3/(GM)} \simeq 48.677\,\mathrm{Myr}$ です。円盤全体で一つの周期があるわけではありません。中心側の短い時間尺度も意識して刻みを選び、後の刻み試験で確認します。

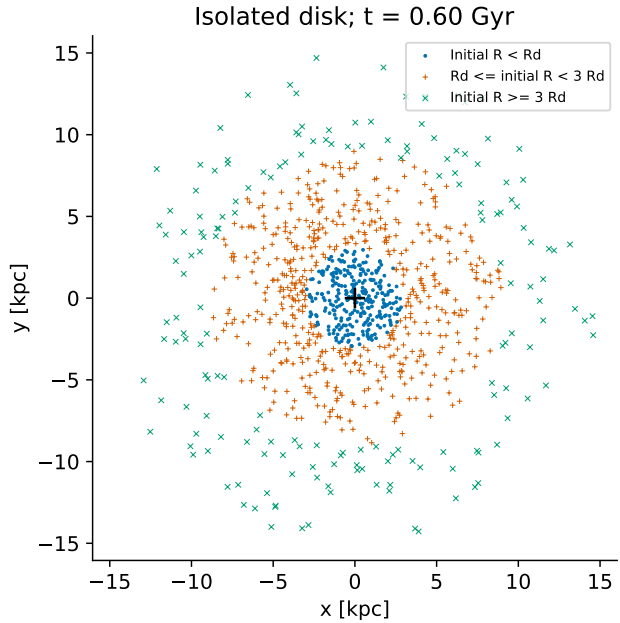


図 4.7 0.60 Gyr 後の孤立円盤。軸範囲、初期半径による色・マーカー分類は図4.5と同じです。各粒子の方位角は変わっていますが、半径方向の分布はほぼ保たれています。

円運動の初期条件では、連続的な軸対称の数面密度は時間に依存しません。ただし、角速度は半径によって異なるので、有限個の点の模様全体が一枚の板のように剛体回転するわけではありません。密度分布が定常であることと、個々の粒子が動かないことは別です。

4.10 分布の維持と軌道の精度を確認する

4.10.1 初期と終時刻の数面密度を比較する

初期と終時刻に同じ円環を用い、式(4.21)で数面密度を測ります。図4.8の灰色点線は、式(4.22)の円環平均です。初期標本からのばらつきが、時間積分中に新しく生じた数値誤差であるとは限りません。

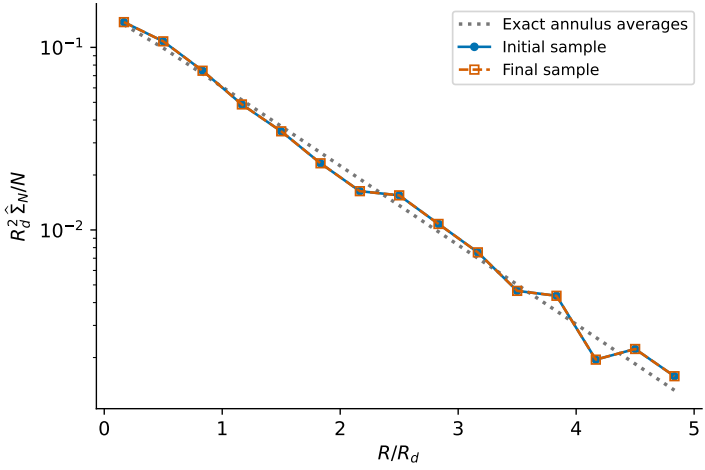


図 4.8 円環で測った数面密度。青丸が初期、朱色の白抜き四角が終時刻です。縦軸は $R_d^2\hat{\Sigma}_N/N$ です。粒子数の少ない外側では標本のばらつきが目立ちますが、初期と終時刻の曲線はほぼ重なります。

経験 CDF のずれは、初期で 0.029414、終時刻で 0.029420 でした。小さな違いはあっても、有限粒子数に由来する初期のずれが支配的です。ここで確かめているのは、固定ポテンシャル中の円軌道の集合が保たれることです。自己重力円盤の集団的な安定性を調べた結果とは解釈しません。

4.10.2 各粒子の保存量を計算する

固定した球対称場なので、各粒子の比エネルギーと比角運動量を使います。

$$\varepsilon_i = \frac{v_{x,i}^2 + v_{y,i}^2}{2} + \Phi(R_i), \qquad \ell_{z,i} = x_i v_{y,i} - y_i v_{x,i}. \tag{4.25}$$

ここでも粒子の質量を掛けません。Plummer 球の全エネルギーや、質量をもつ円盤全体のエネルギーを計算するのではありません。

```
def diagnostics(history, mass, a):
    pos, vel = history[:, :, :2], history[:, :, 2:]
    radius = np.sqrt(np.sum(pos**2, axis=2))
    energy = 0.5 * np.sum(vel**2, axis=2) + potential_plummer(radius, mass, a)
    angular = pos[:, :, 0] * vel[:, :, 1] - pos[:, :, 1] * vel[:, :, 0]
    return radius, energy, angular
```

履歴は 3 軸なので、成分を足す軸は `axis=2` です。半径・エネルギー・角運動量の返り値は、いずれも $(N_{\rm save}, N)$ です。全粒子のエネルギー変化を先に足すと、正負の誤差が相殺されることがあるため、粒子ごとの誤差を調べます。

本章の円軌道では全粒子の $\varepsilon_i^0 < 0$ なので、エネルギー誤差は各初期値の絶対値で割れます。一方、中心に近い粒子では $\ell_{z,i}^0$ が小さくなります。角運動量には共通の正の尺度 $\ell_* = R_d v_c(R_d)$ を使い、

$$e_\varepsilon = \max_{n,i} \frac{|\varepsilon_i^n - \varepsilon_i^0|}{|\varepsilon_i^0|}, \qquad e_\ell = \max_{n,i} \frac{|\ell_{z,i}^n - \ell_{z,i}^0|}{\ell_*} \tag{4.26}$$

とします。この e_ℓ は、各粒子の初期角運動量に対する相対誤差ではありません。

4.10.3 初期条件から同時刻の解析位置を作る

円盤が円く見えることだけでは、回転の速さまで正しいと分かりません。本章では全粒子に円運動を与えたので、初期半径から

$$\Omega_i = \frac{\sqrt{GM}}{(R_i(0)^2 + a^2)^{3/4}} \tag{4.27}$$

を求められます。解析的な方位角は $\phi_i(t) = \phi_i(0) + s_{\rm rot}\Omega_i t$ です。この関係で初期位置を回転させ、同じ時刻の数値位置と比較します。比較の途中で数値解の半径から Ω_i を更新してはいけません。

```
def exact_disk_positions(times, state0, mass, a, spin):
    """ 円運動の初期条件にだけ使う、同時刻の解析位置です。"""
    pos0 = state0[:, :2]
    radius2 = np.sum(pos0**2, axis=1)
    omega = np.sqrt(G * mass) / (radius2 + a**2)**0.75
    angle = spin * np.asarray(times)[:, None] * omega[None, :]
    c, s = np.cos(angle), np.sin(angle)
    x = pos0[None, :, 0] * c - pos0[None, :, 1] * s
    y = pos0[None, :, 0] * s + pos0[None, :, 1] * c
    return np.stack((x, y), axis=2)
```

式(4.27)は原点でも有限です。原点にちょうど置いた粒子は初期位置が 0 なので、この回転式でもそのまま 0 にとどまります。v_c/R をそのまま数値的に評価する場合の $0/0$ を作りません。

位置の誤差と半径だけの変化は、別に定義します。

$$e_{\rm pos} = \max_{n,i} \frac{|\boldsymbol{r}_i^n - \boldsymbol{r}_{i,\rm exact}(t_n)|}{R_d}, \qquad e_R = \max_{n,i} \frac{|R_i^n - R_i^0|}{R_d}. \tag{4.28}$$

すべての粒子に共通の R_d で規格化します。n は、ここでは検証に使う保存時刻を走る添字です。したがって、これらは保存時刻における最大値であり、保存していない時刻の最大誤差まで保証するものではありません。

```
def disk_error_metrics(times, history, state0, mass, a, rd, spin):
    radius, energy, angular = diagnostics(history, mass, a)
    exact = exact_disk_positions(times, state0, mass, a, spin)
    position_error = np.max(np.linalg.norm(history[:, :, :2] - exact, axis=2)) / rd
    radius_error = np.max(np.abs(radius - radius[0])) / rd
    energy_error = np.max(np.abs(energy - energy[0]) / np.abs(energy[0]))
    angular_scale = rd * circular_speed_plummer(rd, mass, a)
    angular_error = np.max(np.abs(angular - angular[0])) / angular_scale
    return np.array([position_error, radius_error, energy_error, angular_error])
```

表 4.5　標準の 1000 粒子・0.60 Gyr・3000 ステップの確認。301 個の保存時刻と全粒子について最大値を取りました。規格化は式(4.26)、式(4.28)です。

指標	値
位置 e_{pos}	5.46×10^{-4}
半径 e_R	4.73×10^{-5}
比エネルギー e_ε	2.19×10^{-9}
比角運動量 e_ℓ	5.00×10^{-14}

位置誤差は 5.46×10^{-4}、半径の変化は 4.73×10^{-5} でした。比エネルギーの相対誤差は 2.19×10^{-9} で、角運動量のずれは 5.00×10^{-14} です。中心力の kick と drift が角運動量を保つ性質は、各粒子にそのまま成り立ちます。一方、エネルギー差や位相の差は有限刻みで残ります。表4.5は本章の実測値で、任意の円盤・刻みで同じ値になるという主張ではありません。

4.11 数値実験の条件を一つずつ変える

4.11.1 時間刻みだけを変える

初期配列 `state0` を保存し、同じ終時刻 0.60 Gyr まで 750、1500、3000、6000 ステップで進めます。この比較では乱数を引き直しません。また、各計算の保存時刻を 151 個の同じ時刻にそろえ、式(4.28)を比べます。

```
orbit_rows = []
for count in [750, 1500, 3000, 6000]:
    t_check, h_check = integrate_leapfrog(acceleration_plummer, state0, t_end, count,
    ↪  mass, a, count // 150)
    errors = disk_error_metrics(t_check, h_check, state0, mass, a, rd, spin)
    orbit_rows.append([count, t_end / count, *errors])
orbit_convergence = np.asarray(orbit_rows)
orbit_orders = np.log2(orbit_convergence[:-1, 2] / orbit_convergence[1:, 2])
```

最も細かい側の推定次数は 2.00004 となり、2 次精度を確認できます。保存量が良好でも、同じ時刻の位置を独立に確かめることを続けます。円運動速度を変更した非円軌道の実験には、この円軌道の解析位置を使いません。

表 4.6　同じ初期粒子を使った KDK の時間刻み試験。全行で終時刻と 151 個の評価時刻をそろえています。p は隣接する行の e_{pos} から求めた推定次数です。

N_{step}	Δt [Myr]	e_{pos}	p	力評価回数
750	0.8000	8.74×10^{-3}	—	751
1500	0.4000	2.18×10^{-3}	2.00069	1501
3000	0.2000	5.46×10^{-4}	2.00017	3001
6000	0.1000	1.36×10^{-4}	2.00004	6001

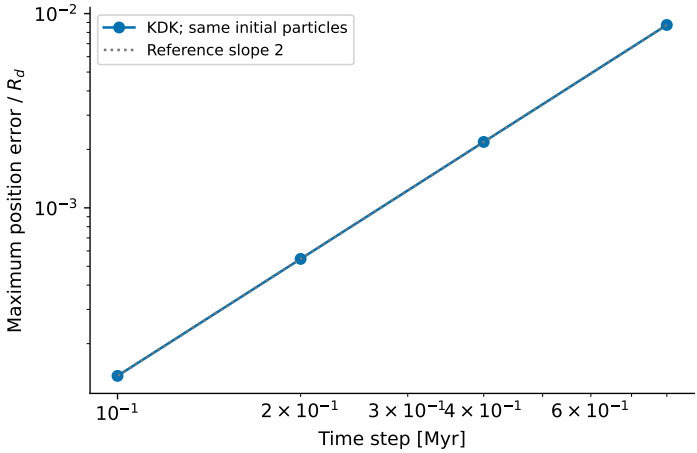


図 4.9　時間刻みと最大位置誤差。点線は最も細かい刻みの点を通る傾き 2 の参照線です。粒子数や CDF 表ではなく、軌道を進める刻みだけを変更しています。

4.11.2 粒子数・CDF 表・保存間隔の役割を区別する

本章には複数の「数」が出てきました。N_R を増やすと CDF の積分・逆補間の精度が変わります。N を増やすと分布を表す標本が増えます。N_{step} を増やすと、同じ終時刻へ進む時間刻みが細かくなります。`save_every` を変えると、履歴の容量と、保存された時刻で見つけられる変動が変わります。

保存の頻度だけを変えても、同じ初期値・同じ KDK 更新なら終点は変わりません。ただし、粗い出力では短い時間の変動や誤差のピークを見逃すことがあります。「図がなめらかになること」と「運動方程式を高精度で解くこと」を混同しないようにします。

4.11.3 回転方向を逆にする

同じ初期半径・方位角で $s_{\mathrm{rot}} = -1$ にすれば、初期位置は同じ、速度は逆になります。比エネルギーは同じで、角運動量は符号を反転します。固定した球対称場では、どちらの向きでも円軌道を作れます。本章の再計算でも、時計回りの解析位置に対する誤差は 5.46×10^{-4} でした。

一般の乱数配置について、初期位置をそのままに速度だけを反転しても、円盤のすべての点が同じ時刻に x 軸の鏡映になるわけではありません。対応するのは同じ初期位置から時間を逆向きにたどる運動です。個々の粒子は動いても、連続的な軸対称分布は両方で同じです。

4.11.4 静止画とアニメーションを比較する

Notebook では、保存済みの履歴から円盤のアニメーションを作ります。全粒子のうち 3 個だけを別のマーカーで追い、星が実際に動いていることも見られるようにしています。これらの粒子に特別な質量や力を与えてはいません。

再生間隔は画面表示の速度です。物理的な Δt とも、履歴の保存間隔とも異なります。動画を見るだけで検証を終えず、初期・終時刻の静止画、数面密度、保存量、位置誤差も残します。

4.12 本章のまとめ

本章では、静的な Plummer 球の中で回転するテスト粒子円盤を作りました。円盤の数面密度に円環の面積を掛けると、半径の確率密度が得られます。台形則で累積分布関数を作り、線形補間でその逆関係を読み、一様乱数を半径へ変換しました。方位角には独立な乱数を使います。

Plummer 球の円運動速度を半径ごとに与え、1 粒子の加速度を多数粒子の配列へ拡張しました。KDK の更新式はそのまま再利用し、計算済みの加速度を次のステップへ渡しました。多数粒子を置いても、粒子同士の重力は含めていません。

数値 CDF の誤差、有限粒子数のばらつき、軌道積分の誤差は、別の比較で確認しました。円盤の形が保たれることに加えて、各粒子の保存量と、同時刻の解析位置を調べました。出力を間引くことと、時間刻みを大きくすることも区別しました。

第 5 章では、この円盤を二つ用意します。第 2 章の銀河中心の運動、第 3 章の Plummer 重力、第 4 章の円盤初期条件を組み合わせ、互いに接近させます。今回確かめた孤立円盤を基準にすることで、相互作用による変化と、初期条件や数値誤差による変化を区別しやすくなります。

演習

演習 4.1　確認：面密度と半径の分布　$R_d = 3\,\mathrm{kpc}$、$R_{\max} = 5R_d$ について、$p_R(R)$ の規格化を確認してください。$R < R_d$ の粒子割合を求め、$N = 1000$ なら平均的に何個入るか答えてください。$p_R \propto e^{-R/R_d}$ とした場合に、何を落としているかも説明してください。

演習 4.2　確認：線形補間と逆補間　表 $(x_j, y_j) = (0,0), (1,2), (3,3)$ に対して、$x = 2$ の y を自作関数と `np.interp` で求めてください。次に $y = 2.5$ に対応する x を、表の入力側と出力側を交換して求めてください。$y = 4$ を渡す場合は、本章の自作関数と `np.interp` の既定の扱いにどのような違いがありますか。

演習 4.3　確認：有限範囲の規格化と中央値　CDF 表の最後の値を 1 にすることが、第 3 章の Plummer 累積質量を無条件に全質量へそろえることと異なる理由を述べてください。$F(R) = 0.5$ を数値表の逆補間で解き、解析 CDF を既習の二分法で解いた値と比較してください。両者の差と、実際に生成した 1000 粒子の半径の中央値のずれは、同じ原因でしょうか。

演習 4.4　応用：粒子数と乱数 seed　表4.3と同じ N、seed で経験 CDF のずれを計算してください。N を変える実験では、$M, a, R_d, R_{\max}, N_R$ を固定します。粒子数を増やしても 1 粒子の軌道計算の次数が上がるわけではない理由を説明してください。

演習 4.5　応用：時間刻みと保存間隔　まず同じ初期配列で時間刻みを変え、KDK の 2 次精度を確認してください。次に時間刻みを固定して、`save_every=1,10,50` を比べてください。各計算の最終配列、保存時刻数、履歴の容量はどうなりますか。粗い出力で求めた最大誤差を、全ステップの最大誤差と呼べるでしょうか。

演習 4.6　応用：回転方向　同じ seed・同じ粒子数で $s_{\mathrm{rot}} = +1, -1$ の初期条件を作ってください。位置が一致し、速度だけが符号反転すること、初期比エネルギーが一致し、比角運動量の符号が反転することを確認してください。両方を KDK で進め、それぞれに対応する円運動の解析位置と比較してください。

演習 4.7　発展：円運動速度の 80% で始める　初期位置を標準円盤と同じに保ち、初期速度だけを 0.8 倍してください。半径分布の時間変化と各粒子の保存量を調べてください。この実験を自己重力円盤の不安定性と呼んでよいでしょうか。また、標準の円運動の解析位置との差を、そのまま数値誤差と呼べない理由を説明してください。

第 5 章　2 つの銀河を接近させる

第 4 章では、一つの固定した Plummer ポテンシャル中に、回転するテスト粒子円盤を作りました。この章では、その円盤を二つ用意して互いに接近させます。第 2 章の重心と相対運動、第 3 章の Plummer 重力、第 4 章の円盤生成を組み合わせ、本書の中心となる銀河相互作用の計算を完成させます。

新しい数値積分法は導入しません。時間発展には、引き続き加速度を再利用する KDK leapfrog を使います。今回新しく考えるのは、**銀河中心も星も動く状況で、どの位置から、どの粒子への重力を計算するか**です。初期状態を作り、中心軌道を確認してから、潮汐テイルやブリッジを見ます。見た目の変化と数値誤差を区別するため、同じ初期粒子を使った時間刻みの比較も行います。

この章で学ぶこと

- 移動する Plummer ポテンシャルと、星を表すテスト粒子の役割を区別できます。
- 採用した中心間ポテンシャルに合わせて、近点距離を指定した初期軌道を作れます。
- 円盤の局所的な位置・速度を慣性系へ移し、銀河中心と星を同じ時刻へ更新できます。
- 中心軌道の保存量と、時間に依存する重力場中の粒子のエネルギー変化を区別できます。
- 潮汐構造を図と粒子割合で調べ、時間刻みによる結果の違いを確認できます。

5.1 これまでの計算要素を一つにまとめる

5.1.1 銀河を変形しない重力場として表す

各銀河 $k = 1, 2$ に、一定の全質量 M_k と Plummer スケール長 a_k を与えます。銀河中心の位置・速度を $\boldsymbol{R}_k, \boldsymbol{V}_k$ とし、その中心を動かします。一方、M_k と a_k 自体は時間とともに変えません。したがって、移動するのは**形を固定した解析的ポテンシャル**です。Plummer 球を多数の質量粒子で表し、その変形まで計算するわけではありません。

円盤の星は、質量ゼロのテスト粒子 $\boldsymbol{r}_i, \boldsymbol{v}_i$ です。星は両銀河の重力を受けますが、星同士の重力や、星から銀河中心への反作用は計算しません。粒子の表示上の重みを、Plummer 球の質量と足し合わせることもしません。第 4 章の粒子数面密度 Σ_N と、重力を作る質量分布の区別を引き継ぎます。

表 5.1　本章で計算する重力の関係。「力を受ける側」と「重力を作る側」を区別します。星には出身銀河だけでなく、両銀河の重力を与えます。

力を受ける側	計算する重力
銀河中心 1・2	中心間の有効対ポテンシャルから求める相互力
テスト粒子	二つの移動する Plummer ポテンシャルの重力の和
テスト粒子間	計算しません
テスト粒子から中心	計算しません

外部重力、ガス、星形成、動力学的摩擦は含めません。すべての位置と速度を一つの慣性系で計算し、運動は x–y 平面内に限定します。円盤が変形しても、背景の Plummer 球は同じ形です。ここで再現するのは、簡略化した重力モデルに対する星の軌道の応答です。

5.1.2 本章で統合する要素を確認する

第 2 章の相対位置・相対速度を

$$\boldsymbol{D} = \boldsymbol{R}_2 - \boldsymbol{R}_1, \qquad D = |\boldsymbol{D}|, \qquad \boldsymbol{U} = \boldsymbol{V}_2 - \boldsymbol{V}_1 \tag{5.1}$$

と書きます。銀河中心の軌道を決めた後、第 4 章の円盤を各中心の周りへ配置します。必要な操作は、重心系の初期軌道、円盤の初期位置・速度、加速度の計算、KDK の時間更新の四つです。

台形則と線形補間は円盤の CDF 生成で再利用します。求根法は第 3 章の半質量半径などを求める道具として既習ですが、**本章の標準計算では新たな求根を無理に挿入しません**。後述する中心軌道の初期速度は、保存量から直接求められるためです。各手法は、必要な役割を持つ場所で使います。

5.2 動いている中心から重力を求める

5.2.1 テスト粒子が受ける二つの Plummer 場を合成する

銀河 k の中心から星までの相対位置を

$$\boldsymbol{s}_{k,i} = \boldsymbol{r}_i - \boldsymbol{R}_k \tag{5.2}$$

とします。第 3・4 章の原点からの位置を、この相対位置へ置き換えます。銀河 k によるポテンシャルと加速度は

$$\Phi_k(\boldsymbol{r}_i, t) = -\frac{GM_k}{\sqrt{|\boldsymbol{s}_{k,i}|^2 + a_k^2}}, \tag{5.3}$$

$$\boldsymbol{a}_{k,i} = -GM_k \frac{\boldsymbol{s}_{k,i}}{(|\boldsymbol{s}_{k,i}|^2 + a_k^2)^{3/2}} \tag{5.4}$$

です。Plummer 球の形は固定でも、中心が動くため、慣性系ではポテンシャルが時間に依存します。Plummer 場の式そのものは前章と同じです。[2]

まず 1 粒子の x 成分だけを見ると、中心位置 X_k を使って

$$a_{x,k,i} = -\frac{GM_k(x_i - X_k)}{[(x_i - X_k)^2 + (y_i - Y_k)^2 + a_k^2]^{3/2}} \tag{5.5}$$

です。原点ではなく、動く銀河中心へ向かう加速度であることを確認してください。前章の配列計算では、最初に `pos - center` を計算するだけで同じ置き換えができます。

```
def acceleration_plummer(pos, center, mass, a):
    """ 任意の中心を持つ Plummer 場。pos は (N, 2)、center は (2,) です。"""
    relative = pos - center
    radius2 = np.sum(relative**2, axis=1)
    return -G * mass * relative / (radius2 + a**2)[:, None]**1.5
```

各星の運動方程式は

$$\frac{d\boldsymbol{r}_i}{dt} = \boldsymbol{v}_i, \qquad \frac{d\boldsymbol{v}_i}{dt} = \boldsymbol{a}_{1,i} + \boldsymbol{a}_{2,i} \tag{5.6}$$

です。どちらの円盤から出発した星も、同じ二つの場の和を受けます。出身銀河の番号は色分けと解析に使うだけで、重力源を選別する番号ではありません。

5.2.2 中心間の相互作用を別に定義する

中心の運動にも、有限の大きさを反映した滑らかな力を使います。本章では二つの中心の有効な対ポテンシャルエネルギーを

$$W_{\mathrm{c}}(D) = -\frac{GM_1M_2}{\sqrt{D^2 + b^2}} \tag{5.7}$$

と**モデルとして定義**します。b は中心間力に使う共通の有効長さです。a_1, a_2 は星が受ける各 Plummer 場のスケール長なので、記号を分けます。

式(5.7)は、**二つの広がった Plummer 密度分布どうしの厳密な相互作用エネルギーではありません**。その計算には、各分布の広がりにわたって相互作用を足す必要があります。本章はその積分を追加せず、中心軌道のための簡単な有効モデルを採用します。後で与える $b = 3\,\mathrm{kpc}$ も、この教材の選択です。

同じ W_{c} から両方の力を求めると、中心加速度は

$$\boldsymbol{A}_1 = +GM_2\frac{\boldsymbol{D}}{(D^2 + b^2)^{3/2}}, \qquad \boldsymbol{A}_2 = -GM_1\frac{\boldsymbol{D}}{(D^2 + b^2)^{3/2}} \tag{5.8}$$

になります。この二つは $M_1\boldsymbol{A}_1 + M_2\boldsymbol{A}_2 = 0$ を満たします。第 2 章と同じく、共通部分を一度だけ計算します。

```
def acceleration_centers(pos, masses, b):
    """ 共通の b を使う有効対ポテンシャルから、二つの加速度を返します。"""
    dr = pos[1] - pos[0]
    denom2 = np.dot(dr, dr) + b**2
    if not np.isfinite(denom2) or denom2 <= 0.0:
        raise ValueError("The effective force denominator must be positive and finite.")
    common = G * dr / denom2**1.5
    return np.array([masses[1] * common, -masses[0] * common])
```

b は両方の式で同じ値です。$M_1 \neq M_2$ の場合でも作用・反作用を保ちます。$a_1 \neq a_2$ へ拡張するとき、中心 1 へは a_2、中心 2 へは a_1 をそのまま代入すると、一般に二つの力の大きさが一致しません。その置き換えは行いません。本章の主実験では $a_1 = a_2 = b$ に固定します。

この選択には、力の対応を単純にする利点があります。$b = a_2$ なら、中心 1 への加速度は銀河 2 の Plummer 場を $\boldsymbol{R}_1$ で評価した値に一致します。中心 2 についても同じ対応を保つため、標準では $b = a_1 = a_2$ とします。その結果、星と出身中心が受ける相手の重力の差を、同じ場の二つの位置での評価として書けます。ただし、これは本章の有効モデル内での整合性であり、広がった二つの球の厳密な相互力を得たという意味ではありません。

よくある失敗：Plummer 場と中心間力を同じ意味にしない

星は式(5.3)の場を受け、中心は式(5.7)で動きます。形が似ていても、後者は本章で定義した有効な粒子対のモデルです。有限サイズの銀河を厳密に剛体として相互作用させた計算、と説明しないようにします。

図5.1に、このモデルで計算する作用をまとめました。中心間の有効長さ b と、星が受ける Plummer 場の a_k を区別し、矢印のない反作用を計算へ加えていないことを確認してください。

5.3 近点距離に合った初期軌道を作る

5.3.1 点質量の速度をそのまま使わない

銀河中心の相対加速度は、式(5.8)の差から

$$\frac{d\boldsymbol{U}}{dt} = -\frac{\mu \boldsymbol{D}}{(D^2 + b^2)^{3/2}}, \qquad \mu = G(M_1 + M_2) \tag{5.9}$$

です。その相対運動の比エネルギーと比角運動量は

$$\varepsilon = \frac{U^2}{2} - \frac{\mu}{\sqrt{D^2 + b^2}}, \qquad h = (\boldsymbol{D} \times \boldsymbol{U})_z \tag{5.10}$$

で、どちらも時間によらず一定になります。第 2 章の $-\mu/D$ とはポテンシャルが異なります。同じ μ と近点距離を選んでも、初期速度は改めて求めます。

本章では $\varepsilon = 0$ の内向きの遭遇を作ります。これは無限遠をポテンシャルの零点とした、束縛・非束縛の境界のエネルギーです。ただし、$b > 0$ では力が厳密な逆 2 乗則ではないため、**軌道の形を Kepler 放物線と呼ぶことはできません**。$b \to 0$ の極限で、第 2 章の放物線に対応します。

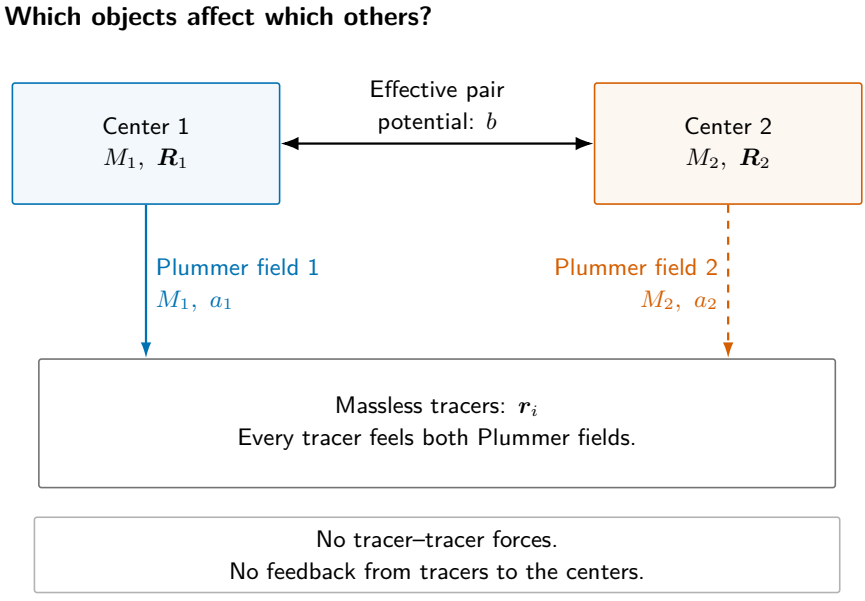


図 5.1 中心とテスト粒子の間で計算する作用の関係。横の双方向矢印は共通の有効長さ b を使う中心間相互作用、下向き矢印は各 Plummer 場から星への作用です。星から中心への反作用と星同士の重力は計算しません。矢印はモデルの依存関係であり、空間内の力の向きではありません。

5.3.2 近点から角運動量を決める

初期距離を D_0、目標の近点距離を D_p とし、$0 < D_p < D_0$ を選びます。近点では動径速度が 0 なので、速度の大きさは接線速度 U_p です。$\varepsilon = 0$ を使うと

$$U_p^2 = \frac{2\mu}{\sqrt{D_p^2 + b^2}}, \qquad h^2 = D_p^2 U_p^2 = \frac{2\mu D_p^2}{\sqrt{D_p^2 + b^2}} \tag{5.11}$$

です。ここでは $h > 0$ を選び、軌道の向きを反時計回りにします。

初期位置を $\boldsymbol{D}(0) = (D_0, 0)$ とすれば、角運動量から接線速度 $U_t = h/D_0$ が決まります。初期位置でも $\varepsilon = 0$ とし、動径速度の内向きの符号を選ぶと

$$U_t = \frac{h}{D_0}, \qquad U_r = -\sqrt{\frac{2\mu}{\sqrt{D_0^2 + b^2}} - U_t^2}, \qquad \boldsymbol{U}(0) = (U_r, U_t) \tag{5.12}$$

となります。これは採用した有効ポテンシャルから直接求めた初期条件で、近点を数値計算後に合わせ直す操作ではありません。

5.3.3 重心系の二つの位置・速度へ変換する

第 2 章の重心変換を再利用します。初期重心を原点、重心速度を 0 とすれば、$M_{\rm tot} = M_1 + M_2$ として

$$\boldsymbol{R}_1(0) = -\frac{M_2}{M_{\rm tot}}\boldsymbol{D}(0), \qquad \boldsymbol{R}_2(0) = +\frac{M_1}{M_{\rm tot}}\boldsymbol{D}(0),$$
$$\boldsymbol{V}_1(0) = -\frac{M_2}{M_{\rm tot}}\boldsymbol{U}(0), \qquad \boldsymbol{V}_2(0) = +\frac{M_1}{M_{\rm tot}}\boldsymbol{U}(0) \tag{5.13}$$

です。相対距離・相対速度を、それぞれの中心から原点までの距離・速度と取り違えないようにします。

```
def make_zero_energy_encounter(masses, d_start, d_peri, b):
    masses = np.asarray(masses, dtype=float)
    check_model(masses, np.ones(2), b)
    if not (np.isfinite(d_start) and np.isfinite(d_peri) and 0.0 < d_peri < d_start):
        raise ValueError("Require finite distances with 0 < d_peri < d_start.")
    mtot = np.sum(masses)
    mu = G * mtot
    h = np.sqrt(2.0 * mu * d_peri**2 / np.sqrt(d_peri**2 + b**2))
    vt = h / d_start
    vr2 = 2.0 * mu / np.sqrt(d_start**2 + b**2) - vt**2
    if vr2 <= 0.0:
        raise ValueError("The incoming radial speed must be real and nonzero.")
    relative_velocity = np.array([-np.sqrt(vr2), vt])
    separation = np.array([d_start, 0.0])
    weights = np.array([-masses[1], masses[0]]) / mtot
    pos = weights[:, None] * separation
    vel = weights[:, None] * relative_velocity
    return np.column_stack((pos, vel))
```

返り値は形 $(2, 4)$ の配列です。各行に銀河中心の $[X_k, Y_k, V_{x,k}, V_{y,k}]$ を入れます。`weights` は式(5.13)の二つの係数です。`[:, None]` で列を加え、各係数を x, y の両成分に掛けています。

5.4 二つの円盤を配置する

5.4.1 円盤の内部運動と銀河全体の運動を分ける

各円盤の粒子数面密度は、第 4 章と同じ切断指数分布です。

$$\Sigma_{N,k}(R) \propto e^{-R/R_{d,k}}, \qquad 0 \le R \le R_{{\rm max},k}. \tag{5.14}$$

この R は各銀河中心からの円盤半径です。全体系の原点から測る半径ではありません。CDF 表を台形則で作り、その逆関係を線形補間で読む処理は、前章のものを再利用します。

円盤の局所座標を $\boldsymbol{s}_i$、局所速度を $\boldsymbol{w}_i$ と書きます。粒子の半径 R での円運動速度は

$$v_{{\rm c},k}^2(R) = \frac{GM_kR^2}{(R^2 + a_k^2)^{3/2}} \tag{5.15}$$

です。これに局所の接線方向を掛け、各銀河の周りを回る速度を作ります。

```
def make_disk(n_particles, mass, a, rd, rmax, spin, rng, n_panels=4096):
    check_integer(n_particles, "n_particles")
    check_positive(mass, "mass")
    check_positive(a, "a")
    if isinstance(spin, (bool, np.bool_)) or spin not in (-1, +1):
        raise ValueError("spin must be +1 or -1.")
    nodes, values = build_radial_cdf(rd, rmax, n_panels)
    uniform = rng.random((n_particles, 2))
    radius = np.interp(uniform[:, 0], values, nodes)
    phi = 2.0 * np.pi * uniform[:, 1]
    vc = circular_speed_plummer(radius, mass, a)
    pos = np.column_stack((radius * np.cos(phi), radius * np.sin(phi)))
    vel = spin * np.column_stack((-vc * np.sin(phi), vc * np.cos(phi)))
    return np.column_stack((pos, vel)), radius, phi
```

慣性系へ移すときには、位置だけでなく速度にも銀河中心の値を足します。

$$\boldsymbol{r}_i(0) = \boldsymbol{R}_k(0) + \boldsymbol{s}_i, \qquad \boldsymbol{v}_i(0) = \boldsymbol{V}_k(0) + \boldsymbol{w}_i. \tag{5.16}$$

```
def place_disk(local_state, center_state):
    """ 局所の位置と速度を慣性系へ移します。入力を上書きしません。"""
    placed = local_state.copy()
    placed[:, :2] += center_state[:2]
    placed[:, 2:] += center_state[2:]
    return placed
```

この変換は、円盤粒子だけの平均位置を引く「再中心化」とは違います。第 4 章の各粒子と Plummer 中心の相対位置・速度をそのまま保ち、銀河全体へ並進運動を与えています。

図5.2の左右を対応させると、位置と速度にそれぞれ何を足すかが分かります。局所速度 $\boldsymbol{w}_i$ だけでは、銀河全体の並進運動は含まれません。

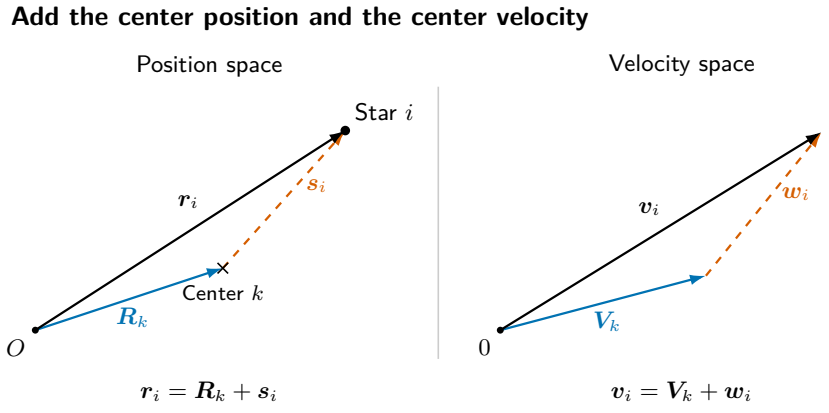


図 5.2　局所円盤を慣性系へ移す二つのベクトル加算。位置には中心位置 $\boldsymbol{R}_k$、速度には中心速度 $\boldsymbol{V}_k$ を加えます。朱色破線は中心に対する局所量です。左右は位置空間と速度空間の図なので、矢印の長さを相互に比較しません。

よくある失敗：中心速度も足します

位置にだけ $\boldsymbol{R}_k$ を足して、速度には $\boldsymbol{V}_k$ を足さないと、円盤が銀河中心と一緒に進みません。その後の崩れを潮汐相互作用の結果と判断してはいけません。初期値から $\boldsymbol{r}_i - \boldsymbol{R}_k$ と $\boldsymbol{v}_i - \boldsymbol{V}_k$ を計算し、局所円盤へ戻るか確認します。

5.4.2 prograde の意味を定める

第 4 章では円盤の向きを反時計回り・時計回りと呼びました。今回は二つの銀河の軌道があるので、円盤の角運動量と軌道の角運動量を比較できます。同じ向きの回転を **prograde**、逆向きを **retrograde** と呼びます。

標準軌道は $h > 0$ で反時計回りです。したがって、両円盤で `spin=+1` を選べば prograde–prograde の遭遇になります。x 軸の右側・左側のどちらに銀河があるかではなく、**中心に対する局所的な回転の向き**で判断します。本章はまずこの一例を完成させ、向きを変える比較は章末の演習と第 6 章へ残します。

図5.3は、軌道の向きを固定したまま円盤 2 の内部回転だけを変える比較です。中心の軌道方向を示す大きな矢印と、各円盤の回転を示す小さな矢印を区別してください。

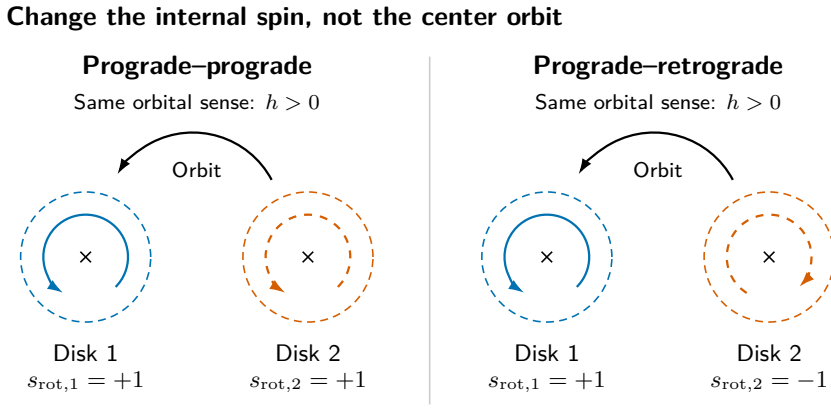


図 5.3 軌道の向きを $h > 0$ に固定した回転方向の比較。左は両円盤が prograde、右は円盤 2 の内部回転だけが retrograde です。大きな曲線矢印は軌道の回転の向きを示す記号で、中心軌道の形ではありません。回転は銀河中心に対する局所的な向きで判断します。

5.4.3 二銀河系の初期配置の近似を確認する

局所の円運動速度は、出身銀河だけの Plummer 場に対して決めています。有限の D_0 から計算を始めるので、相手の銀河の重力は時刻 0 から加わります。したがって、これは孤立円盤二つを初期条件として配置したものであり、二銀河の合成場に対する厳密な平衡円盤ではありません。

初期距離を十分離して選ぶのは、この初期の摂動を抑えるためでもあります。ただし、相手の力を初めの数ステップだけ無視したり、説明なく徐々に強めたりはしません。式(5.6)の力を最初から一貫して使います。

5.5 全体の配列を作り、同じ時刻へ進める

5.5.1 配列内の各行の役割を決める

テスト粒子の合計を $N = N_1 + N_2$ とします。`state` は形 $(2 + N, 4)$ で、最初の 2 行を銀河中心、その後を円盤 1 と円盤 2 の星にします。

Python の行	対象	列
`state[0]`	銀河中心 1	`[x, y, vx, vy]`
`state[1]`	銀河中心 2	同上
`state[2:2+N1]`	円盤 1 のテスト粒子	同上
`state[2+N1:]`	円盤 2 のテスト粒子	同上

星にだけ対応する長さ N の整数配列 `origin` も作ります。0 が銀河 1、1 が銀河 2 の出身です。Python の番号は 0 から始まるので、本文の銀河番号より 1 小さ

い点に注意します。この出身番号は時間とともに変えません。他方の銀河の近くへ移った粒子も、同じ色と番号で追跡します。

```
def make_initial_state(masses, scales, rds, rmaxs, counts, spins,
                       d_start, d_peri, b, seed, n_cdf=4096):
    check_model(masses, scales, b)
    for values in (rds, rmaxs, counts, spins):
        if np.shape(values) != (2,):
            raise ValueError("Each disk parameter array must have two entries.")
    centers = make_zero_energy_encounter(masses, d_start, d_peri, b)
    rng = np.random.default_rng(seed)
    disks, radii = [], []
    for k in range(2):
        local, radius, _ = make_disk(counts[k], masses[k], scales[k], rds[k], rmaxs[k],
                                     spins[k], rng, n_cdf)
        disks.append(place_disk(local, centers[k]))
        radii.append(radius)
    state0 = np.vstack((centers, disks[0], disks[1]))
    origin = np.repeat(np.arange(2), counts)
    return state0, origin, np.concatenate(radii)
```

乱数生成器は一つだけ作り、円盤 1、円盤 2 の順に使います。円盤ごとに同じ seed へ戻して、全く同じ点配置を二つ作ることはしません。以後の時間刻み試験では、ここで得た初期配列を保存し、その同じ配列を使います。

5.5.2 一度の呼び出しで全加速度を返す

中心への加速度と星への加速度を、一つの関数へまとめます。ただし、その計算内容は分けたままです。

```
def acceleration_system(pos, masses, scales, b):
    """ 先頭 2 行は銀河中心、それ以外は質量ゼロのテスト粒子です。"""
    acc = np.empty_like(pos)
    acc[:2] = acceleration_centers(pos[:2], masses, b)
    acc[2:] = acceleration_plummer(pos[2:], pos[0], masses[0], scales[0])
    acc[2:] += acceleration_plummer(pos[2:], pos[1], masses[1], scales[1])
    return acc
```

返り値は入力の `pos` と同じ形 $(2+N, 2)$ です。先頭 2 行は式(5.8)、残りは式(5.6)です。後半の星の計算に `origin` は使いません。星の出身にかかわらず、二つの重力を足すためです。

中心同士の計算は 1 組だけです。星には二つの解析的な場を評価するだけなので、1 回の全加速度計算は $O(N)$ です。$N \times N$ の距離表や、星の粒子対を走査する二重ループは作りません。

5.5.3 KDK の更新式をそのまま使う

第 4 章の KDK に、今作った加速度関数を渡します。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, masses, scales, b):
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, masses, scales, b)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

最初の半 kick で全行の速度を更新し、drift で全行の位置を更新します。その `pos_new` には、星だけでなく**新しい銀河中心の位置**も入っています。そのため、次の加速度は新しい中心と新しい星の位置から計算されます。最後の半 kick を終えると、全行の位置と速度が同じ整数時刻にそろいます。

よくある失敗：動く中心と星の時刻をずらさない

中心だけを先に 1 ステップ進め、星には古い中心位置を使う処理では、ここで定めた KDK になりません。全行の drift を終えてから、一つの位置配列を使って次の重力を求めます。描画用に間引いた中心の軌道を、時間積分の途中で代用することもしません。

力が位置から決まるので、この全行の更新を使えます。ただし、反作用を省いたテスト粒子を含む全配列を、通常の自己重力 N 体系と同一視しません。中心の二体系には保存量がありますが、星を加えた表示用のエネルギーの総和を、そのまま保存量にはできません。実際に確かめる量は5.7節と5.10節で分けます。

5.6 標準モデルを計算する

5.6.1 パラメータをそろえる

本章の標準条件を表5.2に示します。質量、Plummer スケール長、円盤分布は既存の標準モデルを引き継ぎます。二つの銀河を初期距離 60 kpc から、目標近点 8 kpc へ向かわせます。これらは教材用の値で、特定の観測銀河を再現するための調整ではありません。

第 4 章の孤立円盤では $\Delta t = 0.2\,\mathrm{Myr}$ を使いました。本章では近接遭遇の後の粒子軌道も追うため、**時間刻みを $0.05\,\mathrm{Myr}$ へ選び直します**。この選択は孤立円盤の誤差をそのまま流用したものではなく、5.11節の実際の刻み試験に基づきます。出力の間隔は 2 Myr に保ちます。

表 5.2　第 5 章の標準モデル。長さの役割を区別します。全質量は $M_1 + M_2 = 2 \times 10^{11} M_\odot$ で、テスト粒子に質量を加えません。

量	設定
重力源	$M_1 = M_2 = 10^{11} M_\odot$
各 Plummer 場	$a_1 = a_2 = 3\,\mathrm{kpc}$
中心間有効長さ	$b = 3\,\mathrm{kpc}$
円盤初期分布	両方 $R_d = 3\,\mathrm{kpc}$、$R_{\max} = 15\,\mathrm{kpc}$
粒子数・乱数	$N_1 = N_2 = 1000$、seed $= 42$
CDF 表	各円盤 4096 区間、4097 分点
中心軌道	$D_0 = 60\,\mathrm{kpc}$、$D_p = 8\,\mathrm{kpc}$、$\varepsilon = 0$
回転方向	軌道も両円盤も反時計回り
積分時間・時間刻み	$t_{\mathrm{end}} = 0.60\,\mathrm{Gyr}$、$\Delta t = 0.050\,\mathrm{Myr}$
ステップ数・保存	12000 ステップ、星は 40 ステップごと

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2
masses = np.array([1.0e11, 1.0e11])  # Plummer 球の全質量 [Msun]
scales = np.array([3.0, 3.0])         # a1, a2 [kpc]
b = 3.0                              # 中心間力の有効長さ [kpc]
rds = np.array([3.0, 3.0])
rmaxs = np.array([15.0, 15.0])
counts = np.array([1000, 1000])
spins = np.array([+1, +1])            # 今回は軌道も反時計回りなので両方 prograde
seed, n_cdf = 42, 4096
d_start, d_peri = 60.0, 8.0          # 初期距離・目標近点距離 [kpc]
t_end, n_steps, save_every = 0.60, 12000, 40
dt = t_end / n_steps                  # 0.05 Myr。出力は 2 Myr ごとです。
```

初期相対速度の成分は、式(5.12)から

$$U_r = -158.295116\,\mathrm{km\,s^{-1}}, \qquad U_t = 59.829889\,\mathrm{km\,s^{-1}} \tag{5.17}$$

となり、大きさは $169.224583\,\mathrm{km\,s^{-1}}$ です。等質量なので各中心の初期位置は $(-30, 0)$ kpc と $(30, 0)$ kpc です。速度はそれぞれ相対速度の $-1/2$ 倍、$+1/2$ 倍になります。コード内部では速度を kpc/Gyr へ換算しています。

5.6.2 中心と星で保存間隔を分ける

近点時刻や保存量を調べるため、銀河中心の 2 行だけは毎ステップ保存します。星を含む全配列は 40 ステップごとに保存します。これは時間積分を分けるという意味ではありません。**計算は全行で同じ 12000 ステップ**を実行し、履歴へ書き込む頻度だけを分けます。

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, masses, scales, b,
↪  save_every=10):
    masses, scales = check_model(masses, scales, b)
    state, saved_steps, times, history, dt = prepare_history(state0, t_end, n_steps,
    ↪  save_every)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, masses, scales, b)
    center_times = np.arange(n_steps + 1) * dt
    center_history = np.empty((n_steps + 1, 2, 4))
    center_history[0] = state[:2]
    output_index = 1
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, masses, scales, b)
        if not np.all(np.isfinite(pos)) or not np.all(np.isfinite(vel)):
            raise FloatingPointError("Non-finite state; check the model and time step.")
        center_history[n + 1] = np.column_stack((pos[:2], vel[:2]))
        if n + 1 == saved_steps[output_index]:
            history[output_index] = np.column_stack((pos, vel))
            output_index += 1
    return times, history, center_times, center_history
```

`prepare_history` は前章と同じ役割で、初期配列のコピー、保存ステップ番号、時間配列、保存領域を作ります。今回の初期配列は形 $(2+N, 4)$ です。最後の時刻が保存間隔に一致しない場合も、必ず最終状態を保存します。補助的な入力確認のコードは Notebook に収録しています。

標準設定では、全履歴は $(301, 2002, 4)$、中心の履歴は $(12001, 2, 4)$ です。`float64` の 1 要素を 8 byte として、配列だけの容量はそれぞれ 19.283264 MB、0.768064 MB です。MB は 10^6 byte で数えています。一時配列、比較計算、図や動画の容量は別です。

全加速度関数の呼び出しは、初回 1 回と各ステップ 1 回で **12001 回**です。1 回の呼び出しで、両中心と全テスト粒子の加速度を返します。粒子ごとの場の評価を数えた回数とは区別してください。実行時間は Notebook で測り、特定のノート PC でも同じ秒数になるとは仮定しません。

5.7 まず銀河中心の軌道を確かめる

5.7.1 中心二体系の保存量を求める

テスト粒子からの反作用がないので、中心軌道は星の位置に依存しません。その二体系の全エネルギーは

$$E_{\mathrm{c}} = \frac{1}{2}M_1V_1^2 + \frac{1}{2}M_2V_2^2 - \frac{GM_1M_2}{\sqrt{D^2+b^2}} \tag{5.18}$$

です。中心間のポテンシャルを 1 回だけ数えます。式(5.7)で動かした中心に、点質量の $-GM_1M_2/D$ を使ってエネルギーを診断しないようにします。

全運動量、原点の周りの全角運動量、重心位置は第 2 章と同じ

$$\begin{aligned}\boldsymbol{P}_{\mathrm{c}} &= M_1\boldsymbol{V}_1 + M_2\boldsymbol{V}_2,\\ L_{z,\mathrm{c}} &= M_1(\boldsymbol{R}_1\times\boldsymbol{V}_1)_z + M_2(\boldsymbol{R}_2\times\boldsymbol{V}_2)_z,\\ \boldsymbol{R}_{\mathrm{cm}} &= \frac{M_1\boldsymbol{R}_1 + M_2\boldsymbol{R}_2}{M_{\mathrm{tot}}}\end{aligned} \tag{5.19}$$

です。W_{c} は時間に陽には依存しない中心間のポテンシャルなので、E_{c} は一定です。作用・反作用と中心力の関係から、$\boldsymbol{P}_{\mathrm{c}}$ と $L_{z,\mathrm{c}}$ も保存され、重心は等速直線運動します。

```
def center_diagnostics(center_history, masses, b):
    pos, vel = center_history[:, :, :2], center_history[:, :, 2:]
    dr, dv = pos[:, 1] - pos[:, 0], vel[:, 1] - vel[:, 0]
    separation = np.linalg.norm(dr, axis=1)
    kinetic = 0.5 * np.sum(masses[None, :] * np.sum(vel**2, axis=2), axis=1)
    potential = -G * masses[0] * masses[1] / np.sqrt(separation**2 + b**2)
    momentum = np.sum(masses[None, :, None] * vel, axis=1)
    r_cm = np.sum(masses[None, :, None] * pos, axis=1) / np.sum(masses)
    cross = pos[:, :, 0] * vel[:, :, 1] - pos[:, :, 1] * vel[:, :, 0]
    angular = np.sum(masses[None, :] * cross, axis=1)
    return {"D": separation, "dr": dr, "dv": dv, "E": kinetic + potential,
            "L": angular, "P": momentum, "Rcm": r_cm}
```

5.7.2 初期値が 0 の量で割らない

重心系の初期エネルギーは理論的には $E_{\mathrm{c}}(0)=0$、全運動量も 0 です。そこで、共通の正の規格化尺度を

$$\begin{aligned}E_* &= \frac{GM_1M_2}{\sqrt{D_0^2+b^2}}, & V_* &= \sqrt{\frac{GM_{\mathrm{tot}}}{D_0}},\\ P_* &= M_{\mathrm{tot}}V_*, & L_* &= m_{\mathrm{red}}D_0V_*, & m_{\mathrm{red}} &= \frac{M_1M_2}{M_{\mathrm{tot}}}\end{aligned} \tag{5.20}$$

とします。V_* は単位をそろえるための速度の尺度で、有効ポテンシャル中の円運動速度という意味ではありません。

各時刻での初期値からの差の最大値を、E_*, L_*, P_* でそれぞれ割り、$e_{E,\mathrm{c}}, e_{L,\mathrm{c}}, e_{P,\mathrm{c}}$ とします。重心の誤差は

$$e_{\mathrm{cm}} = \max_n \frac{|\boldsymbol{R}^n_{\mathrm{cm}} - [\boldsymbol{R}_{\mathrm{cm}}(0) + t_n\boldsymbol{P}_{\mathrm{c}}(0)/M_{\mathrm{tot}}]|}{D_0} \tag{5.21}$$

です。最大値は、中心を保存した全ステップについて取ります。角運動量の誤差も今回は L_* で規格化するため、初期角運動量に対する相対誤差とは呼びません。

KDK の kick は一組の中心力から両方の速度を変えるため、全運動量と全角運動量を丸め誤差の範囲で保ちます。一方、エネルギーを厳密に保存する方法ではなく、$e_{E,\mathrm{c}} = 3.30\times10^{-6}$ のずれが残ります。表示される 0 は、この等質量・対称な初期条件と実装で得た結果であり、任意の条件でも厳密に 0 になるという意味ではありません。

表 5.3　標準計算での銀河中心の保存量・重心のずれ。規格化は式(5.20)と式(5.21)です。丸め誤差に近い数値の末尾は環境により変わります。

指標	実測値
エネルギー $e_{E,\mathrm{c}}$	3.30×10^{-6}
角運動量 $e_{L,\mathrm{c}}$	5.19×10^{-15}
運動量 $e_{P,\mathrm{c}}$	0
重心位置 e_{cm}	0

5.7.3 指定した近点への接近を確かめる

図5.4は銀河中心間距離の時間変化です。標準計算で保存された最小距離は 8.000004531 kpc、その時刻は 0.269900 Gyr でした。目標の 8 kpc へ近づいた後、両中心は離れていきます。

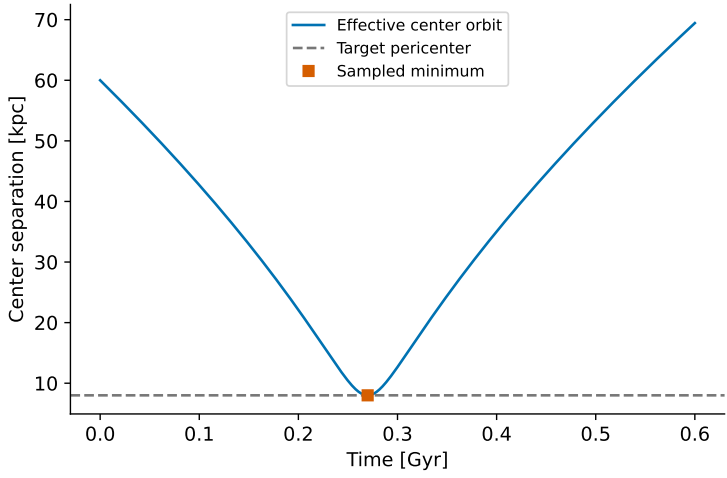


図 5.4　銀河中心間距離。破線は目標の近点 8 kpc、四角は毎ステップの保存値の最小です。有限刻みの積分誤差と、真の近点時刻が保存時刻の間にある影響をともに含みます。

中心の 2 行だけを取り出して、テスト粒子を全く置かずに再計算する試験も行います。今回の実装では中心の履歴は全要素で一致しました。星が多数あっても、中心へ反作用を返さないモデルになっていることを確認できます。

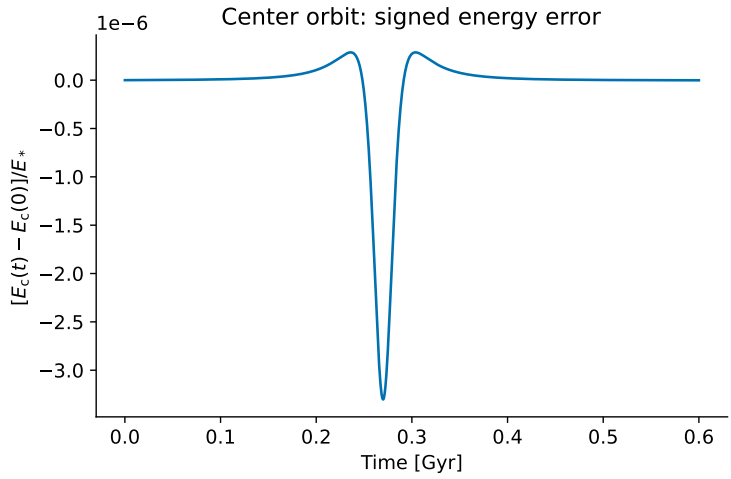


図 5.5 中心二体系の符号付きエネルギー差。近点付近で目立つ変化は KDK の有限刻みの効果です。動力学的摩擦を入れた軌道エネルギー損失ではありません。

5.8 潮汐テイルとブリッジを見る

本書の標準遭遇は、遠方の相手から弱い潮汐摂動を受ける場合ではなく、初期円盤の広がりが深く重なる強い近接遭遇です。目標近点距離 $D_{\mathrm{p}} = 8\,\mathrm{kpc}$ は、各円盤の初期外縁 $R_{\max} = 15\,\mathrm{kpc}$ より小さく、相手の中心が初期円盤の範囲内まで近づきます。この幾何学的な条件だけから、永久に失われる粒子割合は決まりません。以下では遠方展開ではなく、両 Plummer 場による運動を直接計算します。

5.8.1 初期状態から近点通過後までを並べる

図5.6に四つの時刻を示します。青の丸は銀河 1、朱色の三角は銀河 2 の出身粒子です。黒の $\times$ と $+$ がそれぞれの Plummer 中心です。中心の印は重力場の位置を示すもので、Plummer 場に追加した点質量ではありません。

初めはそれぞれの中心の周りに円盤があります。接近すると円盤が変形し、近点後には外側へ長く伸びる構造と、二つの中心の間をつなぐ構造が現れます。前者を**潮汐テイル**、後者を**潮汐ブリッジ**と呼びます。これらは本章の数値結果で粒子が集まって見える構造であり、ガスの衝撃波や新しく生まれた星を描いているのではありません。

5.8.2 相手の重力と潮汐力を区別する

銀河 1 の周りの星を考えます。相手の銀河 2 は、星だけでなく銀河 1 の中心も引きます。円盤の形を変えるのは、全体を同じように並進させる成分ではなく、**星と銀河中心が受ける加速度の差**です。ゼミで学んだ潮汐力の考え方に対応します。[1]

本章の標準条件 $a_1 = a_2 = b$ では、銀河 1 の中心加速度は、銀河 2 の Plummer 場を $\boldsymbol{R}_1$ で評価したものと一致します。$\boldsymbol{s} = \boldsymbol{r}_i - \boldsymbol{R}_1$ とすれば、中心に対する星の

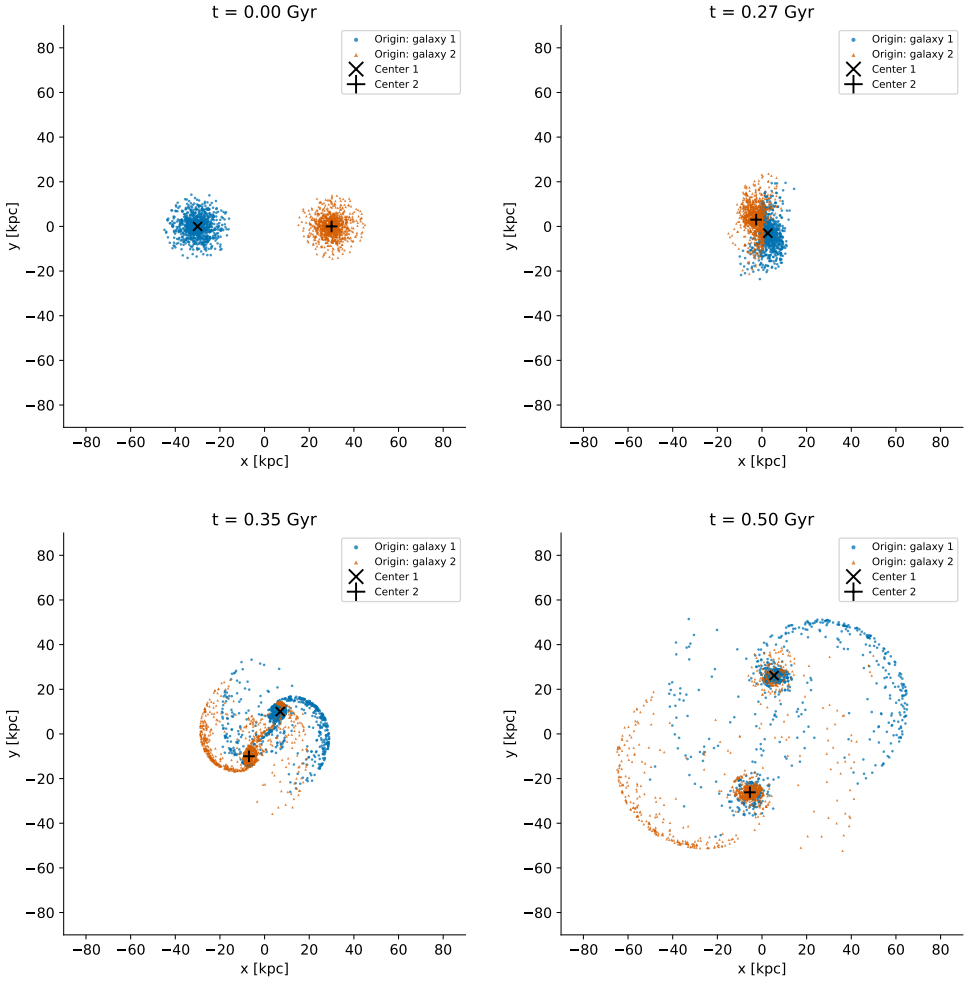


図 5.6　標準の prograde–prograde 遭遇。時刻は 0、0.27、0.35、0.50 Gyr で、全図の軸範囲は −90 から 90 kpc、縦横比は 1 です。近点後には、初期の円盤から長く伸びる潮汐構造が見えます。色とマーカーは出身銀河に固定しています。Notebook には 0.20、0.60 Gyr の図とアニメーションも保存しています。

相対加速度は

$$\frac{d^2\boldsymbol{s}}{dt^2} = \boldsymbol{a}_1(\boldsymbol{s}) + \underbrace{[\boldsymbol{a}_2(\boldsymbol{R}_1+\boldsymbol{s}) - \boldsymbol{a}_2(\boldsymbol{R}_1)]}_{\text{相手の銀河による加速度の差}} \tag{5.22}$$

です。ここで $\boldsymbol{a}_2$ は銀河 2 の中心位置も含めて評価する場を表しています。a_1, a_2, b を独立に変更した場合には、中心への有効力と中心位置での Plummer 場が一致するとは限らないので、この差の書き方をそのまま流用しません。

図5.7では、相手からの加速度そのものと、中心の加速度を引いた差を分けています。遠い側の差が左向きでも、重力が斥力になったわけではありません。この差を、計算済みの両銀河の重力へもう一度足す必要もありません。

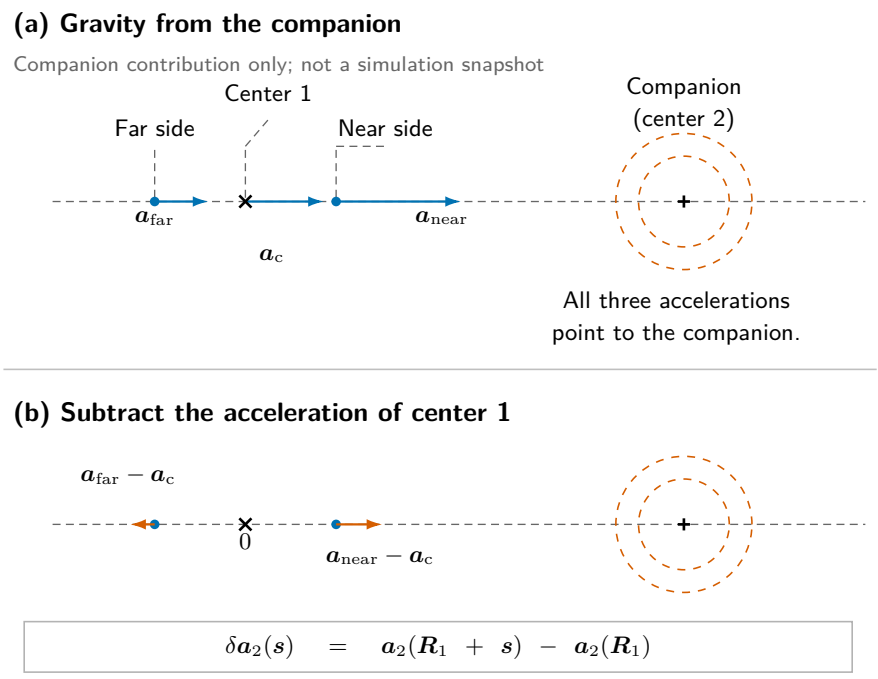


図 5.7　相手銀河が十分遠い配置での説明図。上段は相手からの加速度、下段は銀河 1 の中心加速度を引いた差です。銀河 1 自身の重力は描いていません。$a_1 = a_2 = b$ で中心加速度が対応する場合を考えています。近点 8 kpc の標準計算のスナップショットではありません。

相手が十分遠く、$|\boldsymbol{s}| \ll D$ かつ $D \gg a_2$ なら、加速度の差の尺度は

$$|\delta \boldsymbol{a}_2| \sim \frac{GM_2}{D^3}|\boldsymbol{s}| \tag{5.23}$$

です。同じ相手までの距離でも、円盤の中心から離れた星ほど差を受けやすいことが分かります。また、円盤外側ほど出身銀河による束縛も弱くなります。ただし、式(5.23)だけでテイルの長さや向きが決まるわけではありません。遭遇中の星の回転と、時間に伴って変わる重力を積分した結果を見ます。

よくある失敗：潮汐力をもう一度足さない

式(5.6)の二つの重力を使えば、星と中心が受ける重力の差はすでに含まれています。さらに「潮汐力」として別の加速度を足すと二重計算になります。また、本章の近点では円盤外縁 15 kpc が中心間距離 8 kpc より大きいため、円

盤全体に $|\boldsymbol{s}| \ll D$ の展開を使えません。実装は展開式ではなく、両 Plummer 場を直接評価します。

5.8.3 出身粒子を拡大図で追跡する

図5.8は同じ 0.35 Gyr の結果を拡大したものです。円盤の内側だけを残して新たに計算したのではありません。中心の間にブリッジがあり、その外側へ曲がったテイルが伸びています。

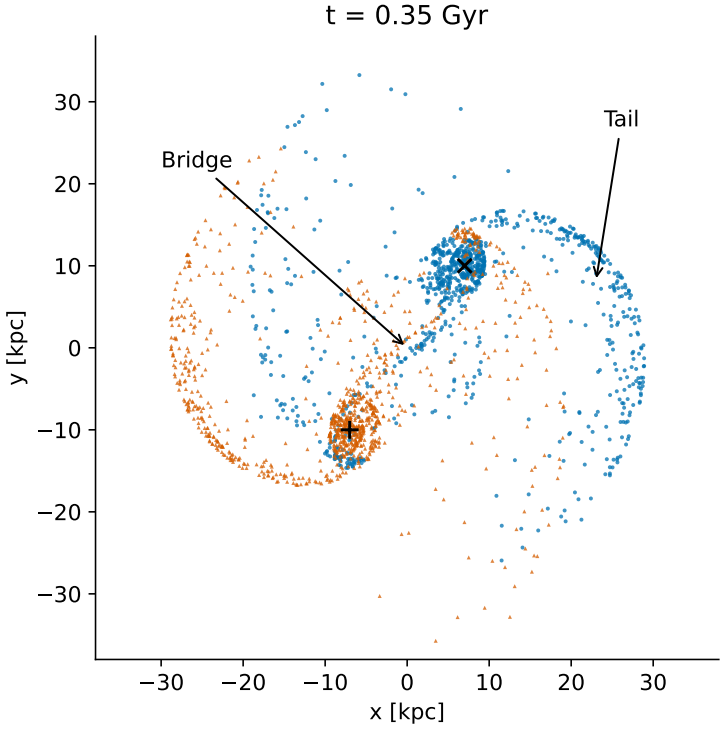


図 5.8　0.35 Gyr の拡大図。軸範囲は ±38 kpc で、図5.6とは異なります。矢印はブリッジとテイルの位置の例を示します。全体図と同じ粒子・同じ計算結果です。

他方の銀河中心の近くに、異なる出身色の星が見えることもあります。しかし、近くへ来たことだけで、その銀河に永久に捕獲されたとは判断できません。ここではまず出身ラベルを保ったまま、どこへ移動したかを確認します。複雑な束縛判定やテイルの自動抽出は、本章には追加しません。

5.9 外へ移動した星を数える

5.9.1 何からの距離を測るかを決める

図の見た目を補うため、出身銀河の中心から遠くへ移った粒子の割合を測ります。粒子 i の出身銀河を $k(i)$ と書くと、その距離は

$$s_i(t) = |\boldsymbol{r}_i(t) - \boldsymbol{R}_{k(i)}(t)| \tag{5.24}$$

です。原点や、その時刻に最も近い銀河中心からの距離ではありません。

本章では閾値を $s_{\rm cut} = 30\,\mathrm{kpc} = 2R_{\rm max}$ と定めます。各出身群 k について

$$f_k(t) = \frac{1}{N_k} \sum_{i:\, k(i)=k} \mathbf{1}\left(s_i(t) > s_{\rm cut}\right) \tag{5.25}$$

を計算します。$\mathbf{1}$ は条件が真なら 1、偽なら 0 です。初期配置の範囲は 15 kpc 以内なので、初期値では $f_1 = f_2 = 0$ です。

```
def origin_radii(history, origin):
    parent_centers = history[:, origin, :2]
    return np.linalg.norm(history[:, 2:, :2] - parent_centers, axis=2)


def outer_fractions(history, origin, threshold):
    radius = origin_radii(history, origin)
    fractions = np.empty((len(history), 2))
    for k in range(2):
        selected = origin == k
        if not np.any(selected):
            raise ValueError("Both origin groups must contain test particles.")
        fractions[:, k] = np.mean(radius[:, selected] > threshold, axis=1)
    return fractions
```

`history[:, origin, :2]` は、各テスト粒子の出身中心の位置を各時刻で取り出します。`history[:, 2:, :2]` と同じ $(N_{\rm save}, N, 2)$ の形になるので、その差から距離を測れます。粒子を除去して分母を変える操作はしません。

5.9.2 数値の意味と限界を確認する

標準計算の 0.60 Gyr では、$f_1 = 0.425$、$f_2 = 0.432$ でした。1000 粒子ずつの標本なので、各割合は 0.001 刻みです。二つは等質量・同じ分布モデルですが、有限の乱数標本が違うため、数値は完全には一致しません。

この指標は「**外へ移動した割合**」です。出身銀河に束縛されたまま大きな軌道を回る粒子、後で戻る粒子、他の銀河の近くへ移った粒子も含み得ます。閾値 30 kpc は本章で定めた測定条件であり、自然に一意に決まる潮汐半径ではありません。「剥ぎ取りが起こり得る様子」を見ることと、永久に失われた質量を定量的に予測することを区別します。

表 5.4　出身中心から 30 kpc を越えた粒子の割合。これは位置による指標で、非束縛粒子や永久に剥ぎ取られた質量の割合ではありません。

t [Gyr]	f_1	f_2
0.00	0.000	0.000
0.20	0.000	0.000
0.27	0.000	0.000
0.35	0.023	0.017
0.50	0.404	0.413
0.60	0.425	0.432

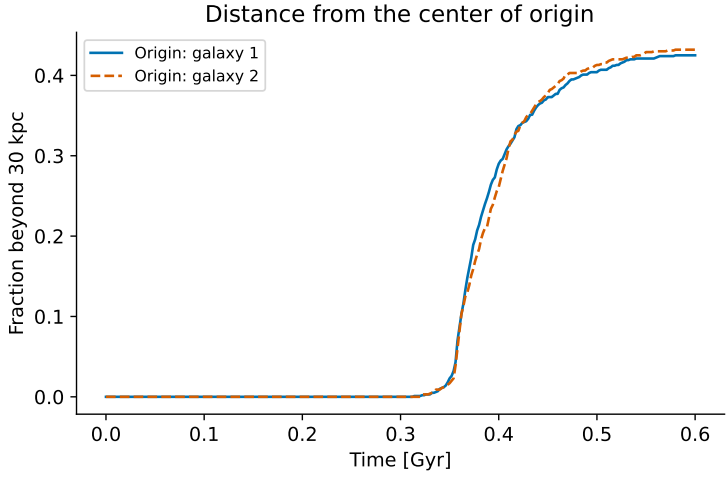


図 5.9　式(5.25)の時間変化。青実線と朱色破線が各出身群です。中心間距離が増えても、星の軌道はすぐに初期の円運動へ戻るわけではありません。

5.10 星のエネルギーが変化する理由を考える

第 4 章では固定した一つの Plummer 場を使ったため、各星の比エネルギーが保存されました。今回は両中心が動きます。慣性系での比エネルギーを

$$\varepsilon_i(t) = \frac{1}{2}|\boldsymbol{v}_i(t)|^2 + \Phi_1(\boldsymbol{r}_i, t) + \Phi_2(\boldsymbol{r}_i, t) \tag{5.26}$$

と定義しても、一般には一定ではありません。

運動方程式を使って時間微分すると、$\boldsymbol{v}_i \cdot \boldsymbol{a}_i$ と $\boldsymbol{v}_i \cdot \nabla\Phi$ が打ち消し合い、

$$\frac{d\varepsilon_i}{dt} = \frac{\partial \Phi}{\partial t} = \sum_{k=1}^{2} \boldsymbol{V}_k \cdot \boldsymbol{a}_{k,i} \tag{5.27}$$

が残ります。最後の等号は、Φ_k が $\boldsymbol{r}_i - \boldsymbol{R}_k(t)$ の関数で、M_k, a_k は一定であることから得られます。これは本章のモデルの式を微分した関係です。**動力学的摩擦を入れなくても、動く重力場は星のエネルギーを変えます。**

```
def tracer_energy(history, masses, scales):
    pos, vel = history[:, 2:, :2], history[:, 2:, 2:]
    energy = 0.5 * np.sum(vel**2, axis=2)
    for k in range(2):
        relative = pos - history[:, k:k+1, :2]
        radius2 = np.sum(relative**2, axis=2)
        energy -= G * masses[k] / np.sqrt(radius2 + scales[k]**2)
    return energy
```

図5.10では、銀河 1 の出身粒子のうち、初期半径が $3R_d = 9$ kpc に最も近い 1 粒子を選びました。その初期半径は 9.002224 kpc です。これは個別の軌道の例で、星全体の平均値ではありません。

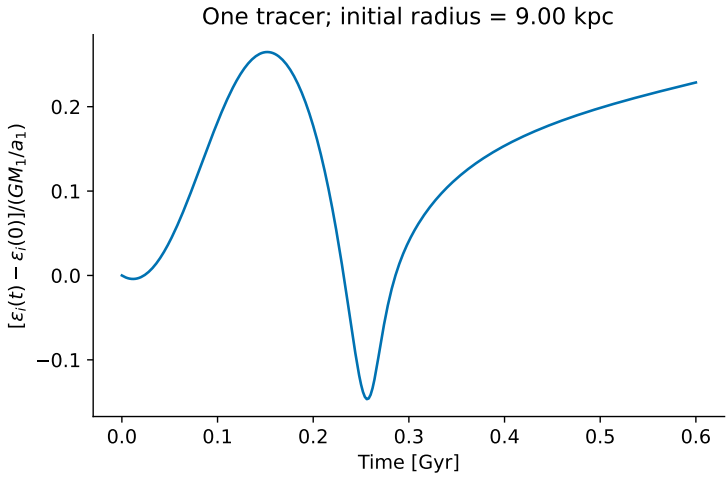


図 5.10 選んだ 1 粒子の比エネルギー変化。規格化は GM_1/a_1 です。図5.5の「中心の保存量の誤差」と違い、こちらの変化には時間依存する場の物理的な効果が含まれます。

星へのトルクも一般には 0 ではないので、慣性系の原点の周りの各星の角運動量も保存しません。また、星から中心へ反作用を返していないため、星の表示上の重みにエネルギーを掛け、その和を E_c へ足して「全系の保存量」とすることはできません。今回の確認では、**中心の二体系の保存量**と、**星の位置の時間刻み依存性**を分けます。

5.11 相互作用計算の時間刻みを確認する

5.11.1 初期粒子も評価時刻も固定する

二銀河が動く間の星には、第 4 章の孤立円盤の円軌道の解析位置は使えません。その位置からずれた量には、意図した相互作用の効果が入るためです。ここでは第 3 章の非円軌道と同じく、**同じモデルの、異なる刻みの数値解どうし**を比較します。

初期配列、終時刻 0.60 Gyr、評価する 151 個の時刻を固定します。ステップ数だけを 3000、6000、12000、24000 とし、刻みは 0.20、0.10、0.05、0.025 Myr です。比較時刻を 4 Myr ごとにそろえるため、各計算で `save_every = n_steps // 150` とします。この保存設定は、標準図の 301 時刻とは別の検証用設定です。

表 5.5　刻みごとの中心軌道と 0.60 Gyr の外側粒子割合。$D_{\min}$ と $e_{E,\mathrm{c}}$ は中心の全ステップから求めています。f_1, f_2 の閾値は 30 kpc です。

N_{step}	Δt [Myr]	$D_{\min}$ [kpc]	$e_{E,\mathrm{c}}$	f_1	f_2
3000	0.200	8.000109881	5.28×10^{-5}	0.425	0.432
6000	0.100	8.000014569	1.32×10^{-5}	0.425	0.432
12000	0.050	8.000004531	3.30×10^{-6}	0.425	0.432
24000	0.025	8.000001962	8.26×10^{-7}	0.425	0.432

中心の最小距離は 8 kpc へ近づき、エネルギー誤差も刻みを細かくするにつれて減ります。一方、この表では外側粒子の割合が全行で同じ値でした。しかし、その一致だけで各星の位置まで正確だとは判断できません。割合は粒子が閾値のどちら側にいるかだけを数え、個々の位置の違いを捨てる量だからです。

5.11.2 隣接する刻みの位置の差を測る

粗い刻みを Δt、細かい刻みを $\Delta t/2$ と書きます。評価する各時刻 t_j で位置の差を取り、中心と星について

$$\delta_{\mathrm{c}}(\Delta t) = \max_{j,k} \frac{|\boldsymbol{R}_{k,\Delta t}(t_j) - \boldsymbol{R}_{k,\Delta t/2}(t_j)|}{D_0},$$
$$\delta_{\mathrm{t}}(\Delta t) = \max_{j,i} \frac{|\boldsymbol{r}_{i,\Delta t}(t_j) - \boldsymbol{r}_{i,\Delta t/2}(t_j)|}{R_d} \tag{5.28}$$

を求めます。これは真の解との誤差ではなく、二つの数値解の差です。同じ時刻・同じ粒子番号で比較します。粒子を位置順に並べ替えてから対応させることはしません。

少数の軌道に大きな差が集中する場合を見分けるため、星については二乗平均平方根の差も求めます。

$$\delta_{\mathrm{rms}}(\Delta t) = \frac{1}{R_d}\sqrt{\frac{1}{N_{\mathrm{eval}}N}\sum_{j,i} |\boldsymbol{r}_{i,\Delta t}(t_j) - \boldsymbol{r}_{i,\Delta t/2}(t_j)|^2}. \tag{5.29}$$

$N_{\mathrm{eval}} = 151$、$N = 2000$ です。最大値と平均的な大きさの両方を残し、一方だけで精度を説明しないようにします。

```
step_counts = [3000, 6000, 12000, 24000]
resolution_runs, resolution_rows = [], []
for count in step_counts:
    tt, hh, ct, cc = integrate_leapfrog(acceleration_system, state0, t_end, count,
                                        masses, scales, b, count // 150)
    vv = center_diagnostics(cc, masses, b)
    ff = outer_fractions(hh[-1:], origin, threshold)[0]
    resolution_runs.append(hh)
    resolution_rows.append([count, t_end / count, np.min(vv["D"]),
                            *center_errors(ct, cc, masses, b), *ff])
self_rows = []
for j in range(len(step_counts) - 1):
    difference = resolution_runs[j][:, :, :2] - resolution_runs[j + 1][:, :, :2]
    delta_center = np.max(np.linalg.norm(difference[:, :2], axis=2)) / d_start
    delta_tracer = np.max(np.linalg.norm(difference[:, 2:], axis=2)) / rds[0]
    rms_tracer = np.sqrt(np.mean(np.sum(difference[:, 2:]**2, axis=2))) / rds[0]
    self_rows.append([step_counts[j], step_counts[j+1], t_end / step_counts[j],
                      delta_center, delta_tracer, rms_tracer])
self_convergence = np.asarray(self_rows)
self_ratios = self_convergence[:-1, 4] / self_convergence[1:, 4]
```

表 5.6 151 個の共通時刻での位置の自己収束。左の 2 列はステップ数です。前行との比は δ_{t} から求めています。δ_{rms} は星・評価時刻の両方にわたる二乗平均平方根です。

粗い計算	細かい計算	δ_{c}	δ_{t}	δ_{rms}	前行との比
3000	6000	3.90×10^{-6}	2.51×10^{-1}	2.68×10^{-3}	—
6000	12000	9.75×10^{-7}	6.41×10^{-2}	6.71×10^{-4}	3.92252
12000	24000	2.44×10^{-7}	1.61×10^{-2}	1.68×10^{-4}	3.97963

最も細かい比較では、最大差 $\delta_{\mathrm{t}} = 1.61\times10^{-2}$、二乗平均平方根は $\delta_{\mathrm{rms}} = 1.68\times10^{-4}$ でした。$R_d = 3$ kpc なので、最大の位置差はおよそ 0.048 kpc です。二つの値の違いは、一部の粒子が全体の平均より刻みに敏感であることを示します。

解の主な誤差が $C(\Delta t)^2$ で表せる範囲なら、隣接する解の差は $(3/4)C(\Delta t)^2$ です。その差どうしの比は約 4 になります。表5.6の細かい側では比が 3.97963 となり、4 へ近づいています。標準の 0.05 Myr は、この比較で潮汐形態と外側割合を調べるための基準とします。すべての粒子の真の軌道誤差が一定値以下だと保証する設定ではありません。より長い計算、より小さい近点、別の密度集中度へ変更したら、刻みも再確認します。

5.11.3 一致する数値の意味を確かめる

自己収束だけでは、加速度の式そのものが正しいことを証明できません。中心間力の作用・反作用、星への重力の和、中心軌道だけの独立計算、単独 Plummer 場中の円盤の検証を組み合わせます。これらの独立な確認コードも配布物に含めています。

全中心・全星に同じ初期位置 $(5, -3)$ kpc と速度 $(30, -20)$ km/s を加える試験も行います。相対位置は変わらないので、同じ相対運動になるはずです。今回の出身中心に対する粒子位置の差は、R_d で規格化した最大値で 4.50×10^{-10} でした。標

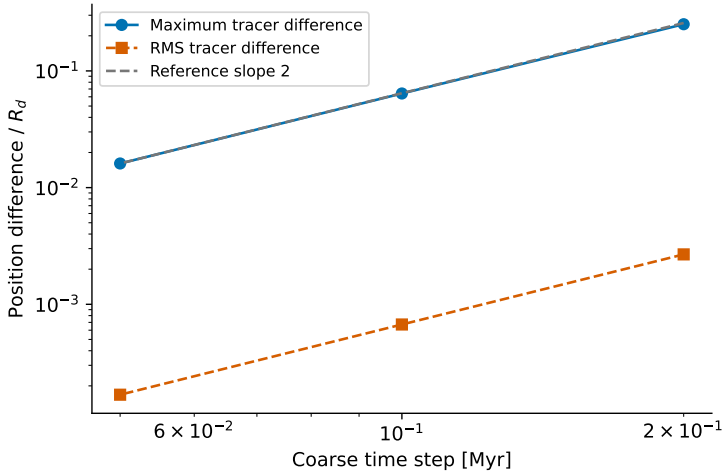


図 5.11　隣接する刻みの星の位置の差。青丸が最大値、朱色四角が二乗平均平方根です。灰色破線は最も細かい比較の最大値を通る傾き 2 の参照線で、実測点への当てはめではありません。

準の刻みを変えた差より十分小さくなっています。ただし、軌道が摂動に敏感な場合には丸めの違いも時間とともに増え得るので、任意の計算で同じ数値になるとは考えません。

5.12 このモデルで分かることと限界を整理する

本章の計算から、二つの銀河が接近すると、星の軌道が変わり、円盤からテイルやブリッジが生じる様子を確認できました。新しい星を生成したわけではなく、初めからあった星を別の位置へ運んだ結果です。また、星のエネルギーが変化しても、中心の二体系の軌道エネルギーは保存されるモデルだと確認しました。

中心が再び離れていくことは計算の失敗ではありません。$\varepsilon = 0$ の中心軌道に摩擦や星からの反作用がないので、近接通過後に自動的に合体へ切り替わる仕組みはありません。ポテンシャルの形も質量も固定なので、潮汐変形を受けて銀河の重力場自体が変わる過程、質量損失に伴う中心軌道の変化、合体残骸が落ち着く過程は扱っていません。

また、円盤と遭遇軌道は同じ平面に置いています。円盤の傾斜角、厚み、鉛直方向の運動は扱いません。したがって、後の prograde–retrograde の比較は共面配置で内部回転の向きだけを変える実験です。任意の傾斜を持つ銀河遭遇の形態まで、この二つの向きだけで分類することはできません。

次の第 6 章では、完成したコードを用い、近点距離、質量比、円盤の回転方向を一つずつ変えます。さらに発展では、衛星銀河の中心へ Chandrasekhar 動力学的摩擦を加え、軌道減衰を調べます。その速度依存力には RK4 を使います。摩擦を質量ゼロの星一つずつへ掛けるわけではありません。ここまでの重力だけの計算を基準として、何を追加したかを明確にします。

5.13 本章のまとめ

本章では、2 中心と多数のテスト粒子を一つの配列へまとめました。中心同士は共通の有効対ポテンシャルで動かし、星には二つの移動する Plummer 場を足しました。中心間の有効力と、広がった密度分布間の厳密な相互力は区別しました。

初期軌道は、採用したポテンシャルの比エネルギーと比角運動量から求めました。各円盤は第 4 章の方法で作り、位置と速度の両方に中心の運動を加えました。KDK では全中心・全粒子をそろえて更新し、新しい中心位置を使って星の加速度を求め直しました。

結果には潮汐テイルとブリッジが現れました。これらを重力の差による軌道の変化として読み、出身中心から遠くへ移動した星の割合も測りました。ただし、その割合を非束縛率や質量損失率とは呼びません。中心の保存量、星の位置の自己収束、異なる座標系での比較を使い、図の見た目だけで検証を終えないことを確認しました。

演習

演習 5.1　確認：どの重力を計算するか　中心 1、中心 2、銀河 1 出身の星、銀河 2 出身の星の加速度をそれぞれ書いてください。$M_1 \neq M_2$、$a_1 \neq a_2$ の場合でも、共通の b を使った中心間力が作用・反作用を満たすことを示してください。`origin` を星の重力源の選別に使ってはいけない理由も述べてください。

演習 5.2　確認：有効軌道の初期条件　表5.2の値から、h, U_t, U_r を求め、$\varepsilon(0) = 0$ を確認してください。同じ h で $D = D_p$ としたとき、動径速度が 0 になることも確かめてください。$b \to 0$ の極限で、第 2 章の放物線の式へ戻ることを示してください。

演習 5.3　確認：円盤の並進　`place_disk` の出力から中心位置と中心速度を引き、元の局所円盤へ戻ることを確認してください。中心速度を足し忘れた場合、各粒子の中心に対する速度にどの項が加わるでしょうか。その崩れを、正しい潮汐テイルの再現と判断できるでしょうか。

演習 5.4　応用：時間刻みと誤差指標　同じ初期配列で表5.5と表5.6を再計算してください。外側割合が同じ値でも最大位置差が 0 ではない理由を説明してください。自己収束の差を真の軌道誤差と区別し、標準設定を変更した場合に再確認すべきものを挙げてください。

演習 5.5　応用：保存間隔だけを変える　標準の時間刻みを固定します。`save_every` を 1、40、200 として比べてください。全粒子の最終状態、毎ステップの中心履歴、全履歴の容量はどう変わりますか。出力の細かさと時間積分の精度を区別してください。メモリが少ない場合は、粒子数を減らしてから三つの保存設定を比較して構いません。

演習 5.6　応用：系全体を等速で動かす　すべての中心と星に、同じ初期位置 $(5, -3)$ kpc と速度 $(30, -20)$ km/s を加えてください。出身中心からの相対位置と中心間距離を、もとの計算と比べてください。慣性系での各星のエネルギーの数値まで、もとの計算と一致する必要があるでしょうか。

演習 5.7　発展：片方の円盤を逆回転にする　標準の初期位置と中心の初期速度を保ち、銀河 2 出身の星の**中心に対する速度だけ**を反転してください。0.60 Gyr の粒子分布と f_1, f_2 を比べてください。星の速度全体へ単に -1 を掛けた場合とは何が違いますか。中心軌道と銀河 1 出身の星の軌道が変わらないことも確認してください。この比較を、自己重力銀河全体の逆回転合体の定量的予測とみなしてよいでしょうか。

第 6 章　銀河相互作用を調べる

第 5 章では、二つの移動する Plummer ポテンシャルとテスト粒子円盤を組み合わせ、銀河相互作用を計算しました。本章では、完成したコードを数値実験の道具として使います。円盤の回転方向、近点距離、質量比を別々に変え、何が変わり、何が変わらないかを調べます。新しい重力積分器を作るのではなく、前章の KDK leapfrog を再利用します。

この章で学ぶこと

- 変更するパラメータ、固定する条件、測定する量を、計算前に決められます。
- 同じ初期標本を使い、prograde・retrograde、近点距離、質量比の比較ができます。
- 近点からの経過時間をそろえ、粒子割合と形態の違いを説明できます。
- 時間刻み、乱数標本、測定条件への依存を、物理モデルの変更と区別できます。

6.1 計算を数値実験として組み立てる

6.1.1 まず問いを一つ決める

「銀河を接近させると何かが起きる」という確認から、もう一歩進みます。例えば、「円盤 2 の回転を逆向きにすると、出身中心から遠くへ移る星の割合は変わるでしょうか」という問いを立てます。この問いに答えるには、回転以外の条件をそろえる必要があります。

同時に、何をもって違いを判断するかも決めます。本章では、同じ範囲で描いた粒子分布と、第 5 章の外側粒子割合を使います。図を見てから都合のよい時刻や閾値だけを選ぶのではなく、標準の測定条件を先に定めます。別の条件でも確認するときには、それを追加の試験として区別します。

6.1.2 一つの変更が初期値に及ぼす影響を確認する

独立に指定するパラメータを一つ変えても、すべての配列要素が一つだけ変わるわけではありません。近点距離を変えれば、それを実現する初期接線速度と動径速度も変わります。銀河の質量を変えれば、中心軌道だけでなく、その銀河の円運動速度も変わります。

ここでいう「一つずつ変える」は、こうした連動を止めることではありません。変更する入力と、モデルの式から計算し直す値を区別することです。表6.1に、本章で行う三つの系列を示します。

表 6.1　基本編の比較系列。すべて第 5 章と同じ重力モデルを使います。質量比系列では全質量を固定しません。

系列	直接変更するもの	連動して作り直すもの
回転方向	円盤 2 の内部回転 $s_{\mathrm{rot},2} = +1, -1$	円盤 2 の中心に対する速度
近点距離	$D_{\mathrm{p}} = 5, 8, 12\,\mathrm{kpc}$	中心の初期相対速度
質量比	M_1 を固定した $q = M_2/M_1$	中心の初期位置・速度、円盤 2 の円運動速度

よくある失敗：名前だけを変えても、軌道は変わりません

計算済みの初期状態を保ったまま、変数 `d_peri` だけを書き換えても、実際の近点距離は変わりません。近点距離は積分器へ毎ステップ与える命令ではなく、初期速度を決めるための入力です。質量を変更した場合も、必要な初期値を同じモデルから作り直します。

6.2 第 5 章の基準と測定方法を引き継ぐ

6.2.1 基準モデルを再現する

基準モデルは、$M_1 = M_2 = 10^{11} M_\odot$、$a_1 = a_2 = b = 3\,\mathrm{kpc}$、各円盤 $N_k = 1000$ 粒子、$R_{\mathrm{d}} = 3\,\mathrm{kpc}$、$R_{\max} = 15\,\mathrm{kpc}$ です。中心間の初期距離は 60 kpc、目標近点距離は 8 kpc で、両円盤を prograde とします。円盤生成には前章と同じ seed、乱数の呼び出し順序、CDF 表を使います。

内部単位は kpc、Gyr、$M_\odot$ です。時間積分は 0.60 Gyr まで、刻みは 0.05 Myr です。全粒子は 2 Myr ごと、中心は毎ステップ保存します。KDK の新しい加速度評価は、1 計算につき $12000 + 1 = 12001$ 回です。各ステップで全中心・全粒子を同じ時刻へ進めます。

```
base_config = dict(masses=np.array([1e11, 1e11]), scales=np.array([3.0, 3.0]), b=3.0,
                   rds=np.array([3.0, 3.0]), rmaxs=np.array([15.0, 15.0]),
                   counts=np.array([1000, 1000]), spins=np.array([1, 1]),
                   d_start=60.0, d_peri=8.0, seed=42, n_cdf=4096)
t_end, n_steps, save_every = 0.60, 12000, 40
threshold, tau_target = 30.0, 0.20  # [kpc], [Gyr]
```

`dict` は、パラメータを名前と値の組でまとめる Python の辞書です。質量の配列は `base_config` からキー`"masses"` を指定して取り出します。新しいクラスは作りません。比較用の辞書は `dict(...)` で作り、変更しない配列はそのまま参照します。本章の関数は入力配列を上書きしないため、この使い方ができます。

コードは第 5 章の関数定義を Notebook 内にも収録しています。別の章の Notebook を先に実行する必要はありません。一方、本文では円盤の CDF、Plummer 加速度、KDK の導出を繰り返しません。式と実装の確認には前章までを参照します。（第5章参照）

6.2.2 出身中心から外へ移った割合を使う

粒子 i の出身銀河を $k(i)$ とし、出身中心からの距離を

$$s_i(t) = |\boldsymbol{r}_i(t) - \boldsymbol{R}_{k(i)}(t)| \tag{6.1}$$

とします。初期に付けた出身番号は、他方の中心へ近づいても変更しません。閾値を $s_{\mathrm{cut}} = 30\,\mathrm{kpc}$ とし、各出身群について

$$f_k(t) = \frac{1}{N_k} \sum_{i:k(i)=k} \mathbf{1}\big(s_i(t) > s_{\mathrm{cut}}\big) \tag{6.2}$$

を測ります。$\mathbf{1}$ は条件が真なら 1、偽なら 0 です。前章の `origin_radii` と `outer_fractions` を、そのまま使います。

この量は位置だけで定義した割合です。非束縛率、永久に剥ぎ取られた割合、質量損失率とは呼びません。N_k は重力源の質量を分割した数でもありません。例えば質量比 1:10 でも各円盤 1000 粒子を置き、両出身群を同じ標本数で表示できます。

6.2.3 近点からの経過時間をそろえる

近点距離や全質量を変えると、近点通過の時刻も変わります。同じ $t = 0.40\,\mathrm{Gyr}$ の図でも、一方は接近直後、他方はまだ接近中かもしれません。そこで、本章の主な比較には

$$\tau = t - t_{\mathrm{p}} \tag{6.3}$$

を使います。t_{p} は、毎ステップ保存した中心間距離の最小値に対応する時刻です。連続時間の厳密な近点時刻ではありません。

比較表では $\tau = 0.20\,\mathrm{Gyr}$ を目標にします。星の履歴は 2 Myr ごとなので、目標に最も近い保存時刻を使い、実際の t と τ も記録します。保存間隔が一定なら、この選択による目標時刻との差は 1 Myr 以下です。この上限は、中心の近点時刻自体の数値誤差を含む上限ではありません。

```
def aligned_index(times, center_times, center_history, tau_target):
    separation = np.linalg.norm(center_history[:, 1, :2] - center_history[:, 0, :2],
    ↪  axis=1)
    i_peri = int(np.argmin(separation))
    if i_peri == 0 or i_peri == len(center_times) - 1:
        raise ValueError("An interior pericenter is required for this comparison.")
    t_peri = center_times[i_peri]
    target = t_peri + tau_target
    if not times[0] <= target <= times[-1]:
        raise ValueError("The requested post-pericenter epoch is outside the stored
        ↪  range.")
    j = int(np.argmin(np.abs(times - target)))
    return j, t_peri, separation[i_peri], times[j] - t_peri
```

`np.argmin` が返すのは配列内の最小値の場所です。距離の最小が計算の端にある場合や、比較したい時刻が保存範囲を外れる場合には、そのまま比較を続けず停

止します。別のモデルで $\tau = 0.20\,\mathrm{Gyr}$ まで到達していなければ、計算時間を延ばすか、全系列の比較時刻を変更します。

図6.1で、二つの実験の近点時刻をそろえる操作を確認してください。目標の τ と実際に使った保存時刻は区別し、両者を記録します。

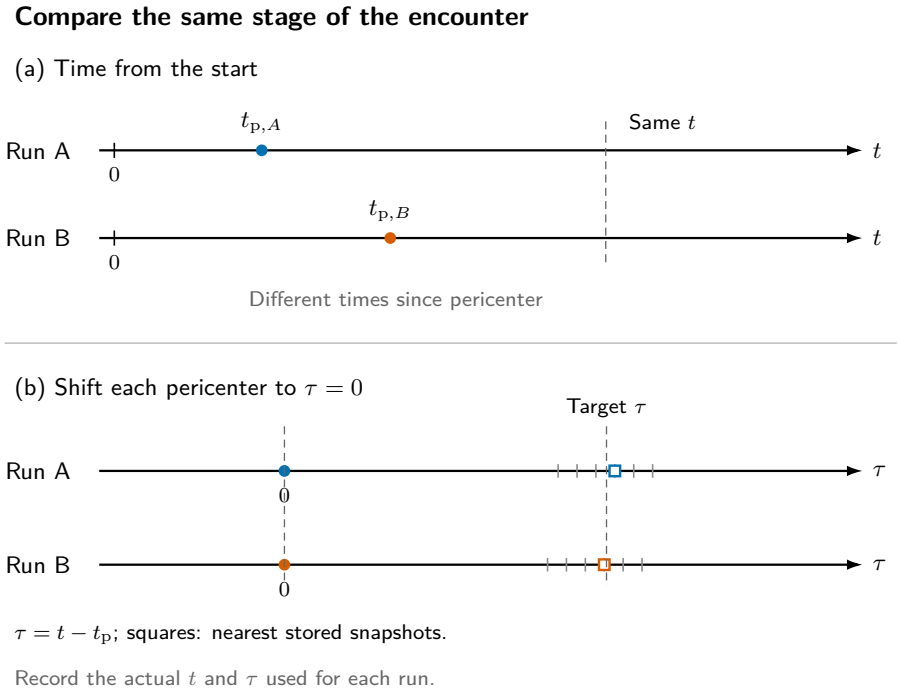


図 6.1 開始からの時刻 t と、近点からの経過時間 $\tau = t - t_{\mathrm{p}}$ の違い。下段では各近点を $\tau = 0$ へそろえています。小目盛りは保存時刻、四角は目標に最も近い保存時刻です。配置と目盛りは説明用で、実測の時間間隔を表していません。

6.2.4 一回の実験を短い関数にまとめる

初期値、KDK、診断という既習の順序を関数へまとめます。返す辞書には、設定と結果を一緒に入れます。比較後に、どの曲線がどの条件だったか分からなくなることを避けられます。

```
def run_case(config, n_steps=12000, save_every=40):
    initial, origin, radii = make_initial_state(**config)
    args = (config["masses"], config["scales"], config["b"])
    times, history, ct, ch = integrate_leapfrog(acceleration_system, initial, t_end,
    ↪  n_steps, *args, save_every)
    j, t_peri, d_min, tau_used = aligned_index(times, ct, ch, tau_target)
    fractions = outer_fractions(history, origin, threshold)
    return dict(config=config, initial=initial, origin=origin, radii=radii, times=times,
                history=history, center_times=ct, center_history=ch, fractions=fractions,
                index=j, t_peri=t_peri, d_min=d_min, tau_used=tau_used,
                errors=center_errors(ct, ch, config["masses"], config["b"]))
```

`make_initial_state(**config)` は、辞書内の名前付きパラメータを関数へ渡す記法です。`*args` は、ここでは質量、Plummer スケール、有効長さの三つを順に渡します。どちらも新しい数値計算法ではなく、引数の整理です。関数が返す位置や速度を、後から「保存則に合うように」修正する操作は行いません。

6.3 円盤の回転方向を変える

6.3.1 銀河中心の運動を固定する

回転方向の定義は図5.3を参照してください。ここでも変えるのは円盤 2 の内部回転だけです。

基準の prograde–prograde を PP、円盤 2 だけを retrograde にしたものを PR と呼びます。軌道の比角運動量は正で、反時計回りです。円盤 2 の回転を変えるときには、局所的な内部速度だけを反転します。

$$\boldsymbol{v}_i' = \boldsymbol{V}_2 - (\boldsymbol{v}_i - \boldsymbol{V}_2) = 2\boldsymbol{V}_2 - \boldsymbol{v}_i \quad \big(k(i) = 2\big). \tag{6.4}$$

中心の速度 $\boldsymbol{V}_2$ は変えません。全速度に単に -1 を掛けると、銀河全体の並進運動まで反転してしまいます。

```
retro_config = dict(base_config, spins=np.array([1, -1]))
runs["PR"] = run_case(retro_config)
retro = runs["PR"]
rows2 = 2 + np.flatnonzero(base["origin"] == 1)
manual = base["initial"].copy()
manual[rows2, 2:] = 2.0 * manual[1, 2:] - manual[rows2, 2:]
assert np.allclose(manual, retro["initial"], rtol=0.0, atol=1e-12)
assert np.array_equal(base["center_history"], retro["center_history"])
rows1 = 2 + np.flatnonzero(base["origin"] == 0)
assert np.array_equal(base["history"][:, rows1], retro["history"][:, rows1])
```

同じ seed・粒子数から円盤を生成すると、PP と PR の初期位置は同じで、円盤 2 の内部速度だけが反転します。配列から式(6.4)を直接計算した結果とも照合します。

本モデルでは、円盤粒子から中心への反作用がありません。また、円盤 2 の星から円盤 1 の星への力もありません。このため、PP と PR で中心軌道と円盤 1 の全粒子軌道は同じになります。配布コードでも配列の一致を確認します。これは一

般の自己重力銀河でも成り立つ性質ではなく、本書の近似に由来します。(第5章参照)

6.3.2 形と割合を合わせて読む

図6.2と図6.3は、$\tau \simeq 0.20\,\mathrm{Gyr}$ での比較です。位置の表示範囲、出身色、マーカー、粒子数をそろえています。PR の円盤 2 では、PP と比べて遠方まで伸びた粒子の数が少なく見えます。

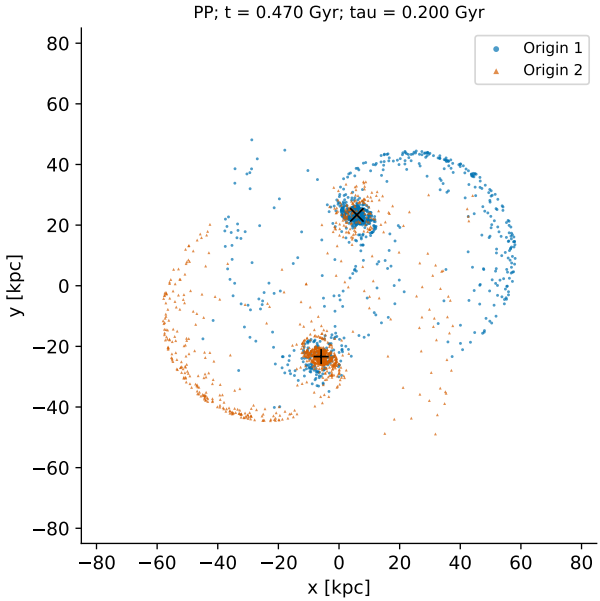


図 6.2 PP の粒子分布。青丸と朱色三角はそれぞれ銀河 1・2 の出身粒子です。黒い $\times, +$ は Plummer 中心です。実際の比較時刻は $t = 0.470\,\mathrm{Gyr}$、$\tau = 0.200100\,\mathrm{Gyr}$ です。

定量的には、円盤 2 の外側割合は PP で 0.401、PR で 0.055 となり、差は 0.346 でした。これは各 1000 粒子の一つの標本で得た結果です。第 5 章の $t = 0.60\,\mathrm{Gyr}$ における 0.432 と 0.065 とは測定時刻が違うので、値を混同しません。

6.3.3 回転方向によって相互作用が変わる理由を考える

潮汐変形に関係するのは、星と出身中心が受ける加速度の差です。[1] その差は時刻だけでなく、星が円盤のどこを動いているかにも依存します。したがって、中心軌道と初期の粒子位置が同じでも、内部速度の向きを変えれば、遭遇中に各星が受ける力の履歴は変わります。

順行では、軌道運動と同じ向きに回る星が相手の重力から受ける影響を、長く受け続ける場合があります。逆行では相手との相対的な角運動が違うため、力の向きが変わる速さも異なります。ここでは、これを計算結果の定性的な読み方とします。「すべての順行遭遇で、必ず同じ割合だけテイルが強くなる」という法則を、

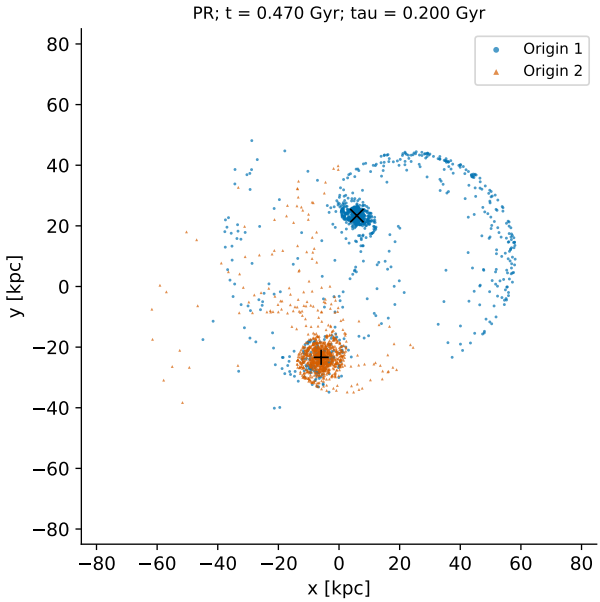


図 6.3 PR の粒子分布。図6.2と同じ時刻・同じ軸範囲です。円盤 2 の内部回転だけを変えています。円盤 1 と中心軌道は PP と同じです。

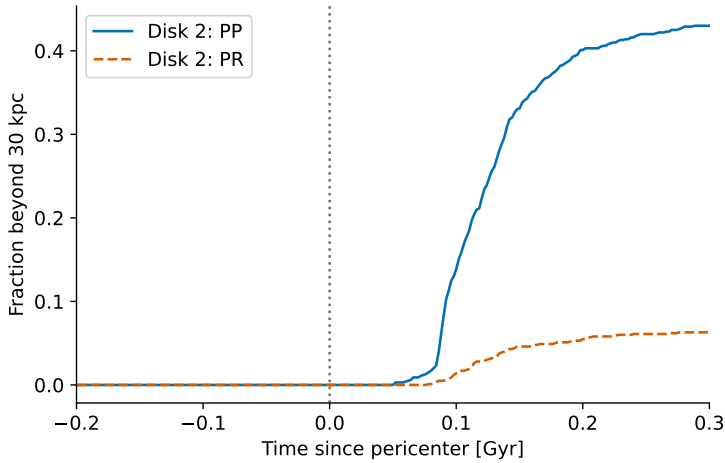


図 6.4 円盤 2 の外側割合の時間変化。横軸は近点からの経過時間です。PP を青実線、PR を朱色破線で示します。粒子が出身中心から 30 kpc を越えたかを数えており、非束縛判定ではありません。

この二例から導くことはできません。厳密な共鳴条件の導出は本章の範囲外です。また、ここでは円盤の傾斜角を固定して共面に限定しています。回転符号の比較を、任意の三次元的な向きの比較へ一般化しません。

6.4 近点距離を変える

等質量、同じ円盤、両円盤 prograde を保ち、$D_{\rm p} = 5, 8, 12\,\mathrm{kpc}$ を比べます。初期距離 $D_0 = 60\,\mathrm{kpc}$ と中心相対エネルギー $\varepsilon = 0$ は共通です。第 5 章の式から、各近点に合う中心初期速度を計算し直します。

```
for d_peri in [5.0, 12.0]:
    config = dict(base_config, d_peri=d_peri)
    runs[f"P{d_peri:g}"] = run_case(config)
```

$D_{\rm p}$ を小さくすれば、最接近時の重力の差が大きくなり得ます。しかし、近点での速度や星の応答時間も変わります。さらに、本書の最接近では円盤の大きさが中心間距離に比べて十分小さいとは限りません。遠方での見積もり $|\delta\boldsymbol{a}| \sim GM_2|\boldsymbol{s}|/D^3$ を、円盤全体の厳密な応答式には使いません。（第5章参照）

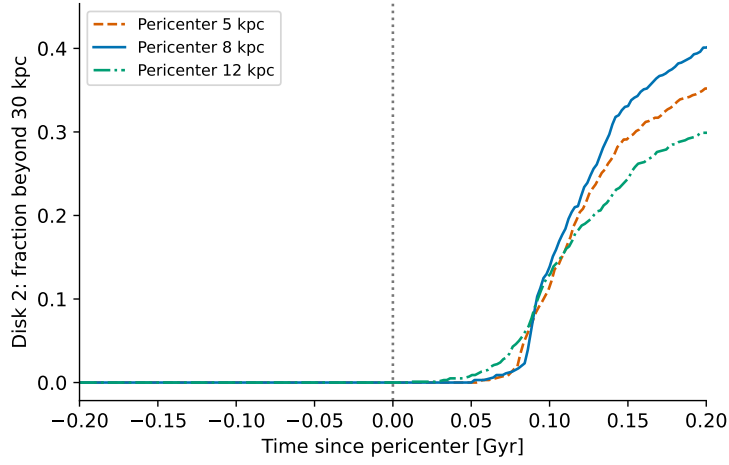


図 6.5　近点距離を変えた円盤 2 の外側割合。各曲線の近点時刻を横軸の 0 へそろえています。円盤・質量・回転方向・seed は共通です。

今回の $\tau \simeq 0.20\,\mathrm{Gyr}$ では、円盤 2 の割合は近点 5、8、12 kpc の順に $0.352, 0.401, 0.299$ でした。**この指標は、近点を小さくするほど単調に大きくなったわけではありません**。したがって、「より近いほど、30 kpc より外の粒子割合が必ず増える」とは結論できません。

この非単調性を、ただちに数値計算の失敗とすることもできません。測っているのは特定時刻・特定閾値での粒子数です。星の位相、遭遇の速さ、その後の運動などをまとめて反映します。どの効果が主な原因かを分けるには、さらに軌道やトル

クの履歴を調べる必要があります。本章では、まず同じ条件で得た結果を記録し、予想と違うことを消さない姿勢を優先します。

6.4.1 中心間の有効長さも確認する

近点距離を比べる際には、中心間力の有効長さ b も無視できません。同じ D と相手の質量で、中心が受ける有効加速度の大きさ A_{eff} と点質量の加速度 A_{point} を比べると、

$$\frac{A_{\mathrm{eff}}}{A_{\mathrm{point}}} = \frac{GMD/(D^2+b^2)^{3/2}}{GM/D^2} = \frac{D^3}{(D^2+b^2)^{3/2}} \tag{6.5}$$

です。表6.2は、既存の系列で使った $b = 3\,\mathrm{kpc}$ に対してこの比を計算したものです。$D = 5\,\mathrm{kpc}$ では比は約 0.631 で、点質量の値より小さくなっています。

表 6.2　同じ距離での中心間加速度の比較。軌道の追加計算ではなく、式(6.5)の評価です。$b = 3\,\mathrm{kpc}$ を固定しています。

D [kpc]	$\sqrt{D^2+b^2}$ [kpc]	$A_{\mathrm{eff}}/A_{\mathrm{point}}$
5	5.831	0.631
8	8.544	0.821
12	12.369	0.913

ただし、これは中心の**並進加速度**の比です。星と中心の加速度差を表す潮汐力の比でも、遭遇を通じた力積やテイルの長さでもありません。b は中心軌道を、a_1, a_2 は星が受ける場を決めます。これらの有限長さは結果に関係する候補ですが、表だけから外側割合の非単調性の原因を特定することはできません。

本章では近点系列を変更せず、この力の比較を演習 6.9 にします。b を変えることは、時間刻みを細かくする操作ではなく、中心軌道の物理モデルを変える操作です。標準の $b = a_1 = a_2$ から b だけを変えれば、中心と星が同じ相手場を感じる対応も変わります。原因を切り分ける追加実験では、その変更まで記録する必要があります。

6.5 質量比を変える

6.5.1 主銀河の質量を固定する

$q = M_2/M_1$ とし、$M_1 = 10^{11} M_\odot$ を固定して $q = 1, 0.5, 0.1$ を比べます。第 2 章の「全質量を固定して質量比を変える実験」とは異なります。今回は全質量が変わるため、相対軌道に入る $G(M_1+M_2)$ も変わります。

円盤 2 の円運動速度は $\sqrt{M_2}$ に比例して変わります。初期相対速度を決めるときにも、新しい全質量を使います。一方、Plummer スケール長、指数円盤の大きさ、各粒子数は同じです。これは、一定のサイズで質量を変更した教材用の系列です。実際の銀河が従う質量とサイズの関係を再現した系列ではありません。

```
for q in [0.5, 0.1]:
    config = dict(base_config, masses=np.array([1.0e11, q * 1.0e11]))
    runs[f"Q{q:g}"] = run_case(config)
```

6.5.2 近点通過前後を同じ段階で比べる

表6.3に、全系列で実際に使った時刻を示します。$q = 0.1$ では近点が約 $0.363900\,\mathrm{Gyr}$ へ遅れています。$t = 0.47\,\mathrm{Gyr}$ だけを比較すれば、基準モデルより近点後の経過時間が短い状態を見てしまいます。

表 6.3　比較系列と測定時刻。P5・P12 は近点距離、Q0.5・Q0.1 は質量比を変更したものです。t_{used} は星の最寄り保存時刻です。

ケース	q	D_{p} [kpc]	t_{p} [Gyr]	t_{used} [Gyr]	τ_{used} [Gyr]
PP	1.0	8	0.26990	0.470	0.20010
PR	1.0	8	0.26990	0.470	0.20010
P5	1.0	5	0.25470	0.454	0.19930
P12	1.0	12	0.28750	0.488	0.20050
Q0.5	0.5	8	0.31165	0.512	0.20035
Q0.1	0.1	8	0.36390	0.564	0.20010

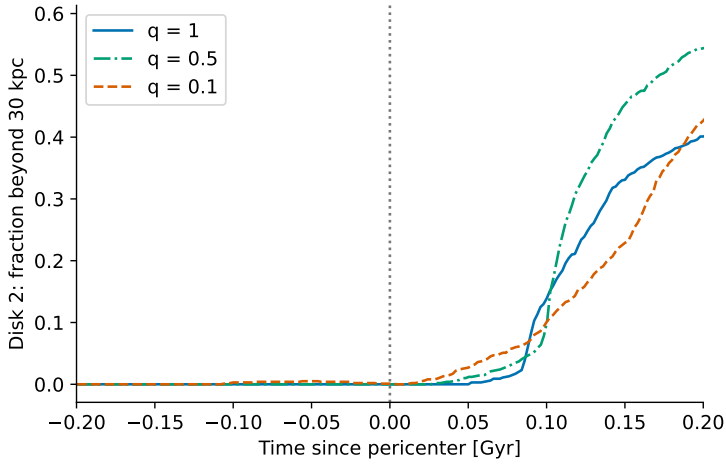


図 6.6　質量比系列の円盤 2 の外側割合。主銀河の質量は固定しています。粒子数は各円盤 1000 のままで、質量を粒子数で表していません。

6.5.3 一つの傾向で全結果を説明しない

表6.4では、主銀河側の f_1 は、この三つの質量比で $0.390, 0.225, 0.000$ でした。一方、円盤 2 の f_2 は $0.401, 0.544, 0.427$ で、やはり単調ではありません。小質量の銀河は自分の場による束縛が弱くなりますが、中心軌道や内部回転も同時に変わっ

ています。この系列だけで、質量比と外側割合の普遍的な関係を決めないようにします。

$f_1 = 0$ は、この標本・時刻で 30 kpc を越えた星がなかったことです。円盤 1 の星が全く摂動を受けなかったことや、円盤内部に何の変形もなかったことを意味しません。閾値で集約した量が捨てている情報も意識します。

表 6.4　外側粒子割合の比較。左 2 列は近点後の段階をそろえた値、右 2 列は開始から同じ 0.60 Gyr の値です。同じ行でも、異なる時刻の測定です。

ケース	$f_1(\tau \simeq 0.20)$	$f_2(\tau \simeq 0.20)$	$f_1(t = 0.60)$	$f_2(t = 0.60)$
PP	0.390	0.401	0.425	0.432
PR	0.390	0.055	0.425	0.065
P5	0.331	0.352	0.381	0.388
P12	0.304	0.299	0.330	0.322
Q0.5	0.225	0.544	0.272	0.584
Q0.1	0.000	0.427	0.002	0.508

6.6 差が何に由来するかを確かめる

6.6.1 測る閾値を変えてみる

同じ計算結果を使い、$s_{\mathrm{cut}} = 20, 30, 40\,\mathrm{kpc}$ で PP と PR を比べます。これは運動方程式の再計算ではなく、測定条件の変更です。時刻と粒子の軌道は同じに保ちます。

```
threshold_rows = []
for cut in [20.0, 30.0, 40.0]:
    f_pp = outer_fractions(base["history"][base["index"]:base["index"]+1],
    ↪  base["origin"], cut)[0]
    f_pr = outer_fractions(retro["history"][retro["index"]:retro["index"]+1],
    ↪  retro["origin"], cut)[0]
    threshold_rows.append([cut, f_pp[1], f_pr[1], f_pp[1] - f_pr[1]])
```

表 6.5　閾値を変えた PP–PR 比較。近点後約 0.20 Gyr、円盤 2 の出身粒子についての測定です。

s_{cut} [kpc]	$f_{2,\mathrm{PP}}$	$f_{2,\mathrm{PR}}$	PP − PR
20	0.447	0.092	0.355
30	0.401	0.055	0.346
40	0.314	0.038	0.276

今回の三つの閾値では、どれも PP の方が大きいという傾向は同じでした。しかし、差の大きさは変わります。報告するときは「PP の方が何% 大きい」とだけ書かず、出身中心からの距離、閾値、時刻、分母を明記します。

6.6.2 乱数標本を対にして比べる

次は seed を 42、43、44 へ変えます。各 seed について、PP と PR に同じ初期粒子位置を使います。二つの計算で別々の乱数を引くよりも、回転方向による差と標本の違いを分けやすくなります。

表 6.6 三つの seed での PP–PR 比較。各行が同じ標本から作った一組です。全行で $\tau \simeq 0.20\,\mathrm{Gyr}$、$s_{\mathrm{cut}} = 30\,\mathrm{kpc}$ です。

seed	$f_{2,\mathrm{PP}}$	$f_{2,\mathrm{PR}}$	PP − PR
42	0.401	0.055	0.346
43	0.409	0.060	0.349
44	0.415	0.054	0.361

この三組では、回転方向による差は seed を変えても残りました。ただし、三つの seed だけで統計分布や信頼区間を精密に推定したとは言いません。また、初期の有限標本を変える試験は、数値積分の刻みを変える試験でも、現実の銀河の多様性をすべて調べる試験でもありません。

6.6.3 新しい条件でも時間刻みを確認する

第 5 章で標準条件を検証していても、別の近点や質量比で同じ誤差になるとは限りません。本章では PP、近点 5 kpc、質量比 0.1 について、$\Delta t = 0.05$ と $0.025\,\mathrm{Myr}$ を比べます。初期配列は作り直さず、同じ配列を渡します。全 301 個の保存時刻も一致させます。

二つの数値解の位置差を、中心を除いた全粒子と全保存時刻について調べます。

$$\delta_{\max} = \frac{\max_{j,i} \left| \boldsymbol{r}_{i,h}(t_j) - \boldsymbol{r}_{i,h/2}(t_j) \right|}{R_{\mathrm{d}}}, \tag{6.6}$$

$$\delta_{\mathrm{rms}} = \frac{1}{R_{\mathrm{d}}} \sqrt{\frac{1}{N_{\mathrm{save}} N} \sum_{j,i} \left| \boldsymbol{r}_{i,h}(t_j) - \boldsymbol{r}_{i,h/2}(t_j) \right|^2}. \tag{6.7}$$

h は粗い側の刻みです。これらは真の解に対する誤差ではなく、二つの数値解の差です。Δf_k は同じ比較時刻での細かい計算の f_k から、粗い計算の f_k を引いた量とします。近点の推定時刻を細かい側で選び直して比較時刻まで動かすことはせず、粗い側で選んだ t_{used} にそろえます。

表 6.7 0.05/0.025 Myr の刻み比較。位置差は全 301 時刻、Δf_k は表6.3の同一時刻で測定します。$e_{E,\mathrm{c}}$ は第 5 章と同じ尺度で規格化した中心軌道のエネルギー誤差です。

ケース	$\delta_{\max}$	δ_{rms}	Δf_1	Δf_2	細かい側の $e_{E,\mathrm{c}}$
PP	1.61×10^{-2}	1.67×10^{-4}	+0.000	+0.000	8.26×10^{-7}
P5	1.25×10^{-2}	2.07×10^{-4}	+0.000	+0.000	3.98×10^{-6}
Q0.1	5.14×10^{-4}	1.19×10^{-5}	+0.000	+0.000	4.54×10^{-7}

今回の Δf_k はすべて 0 でした。しかし、位置差は 0 ではありません。最大位置差と二乗平均平方根にも大きな違いがあります。全体の割合だけなら一致していても、一部の星の軌道は刻みに敏感です。外側割合を議論するのか、一つの星の詳細な軌道を議論するのかによって、必要な確認の粒度が変わります。

この二つの刻みだけから収束次数を測ったとは言いません。第 5 章の複数刻みでの試験と、本章の変更条件での比較を組み合わせています。さらに小さな近点や長い計算を追加する場合は、必要に応じてもう一段細かい刻みでも確認してください。

よくある失敗：指標の一致を、すべての軌道の一致としない

f_k は閾値を越えたかどうかだけを数えます。同じ側にとどまる星の位置が変わっても値は変わりません。逆に、閾値に近い 1 粒子が反対側へ移るだけで $1/N_k$ 変わります。集約した指標、個別位置の差、中心の保存量を、それぞれの意味で確認します。

6.7 小さな研究としてまとめる

前半の最終課題は、三つの系列から一つを選び、基準と変更例の差を短いレポートへまとめることです。多くのパラメータを走査する必要はありません。まず一つの問いに対して、条件をそろえた比較ができることを目指します。

表 6.8　最終レポートに必要な内容。図を作るだけでなく、どのモデルの何を測った結果かを記録します。

項目	記述する内容
問い	どの入力を変えると、何が変わると考えたか
モデルと条件	重力源、テスト粒子近似、質量・長さ・単位、変更した入力と連動する初期値
数値設定	KDK、時間刻み、積分時間、粒子数、seed、保存間隔
結果	同じ軸範囲の静止画と、定義を示した一つの定量指標
確認	少なくとも一つの刻み試験、標本または測定条件への依存
解釈と限界	分かったこと、今回の計算だけでは分からないこと

例えば PP と PR の比較なら、「本書の固定形状 Plummer 場・同一平面のモデルでは、近点後約 0.20 Gyr に、円盤 2 の 30 kpc 外側割合が逆行の場合に小さくなった」と書けます。この範囲なら、実際に測ったことと結論が対応します。一方、「逆行銀河は合体しない」「実際の銀河でこの割合の質量が失われる」という結論は、このモデルからは導けません。

設定だけでなく初期配列も保存します。乱数環境や生成手順が変わった場合に、同じ seed という記録だけで完全な一致を仮定しないためです。本章の配布データには、各ケースの初期配列、出身番号、比較時刻のスナップショット、粒子割合、実行環境の版を含めています。

6.8 本章のまとめ

本章では、第 5 章で完成した KDK コードをそのまま使い、回転方向、近点距離、質量比を一つずつ変えて比較しました。入力を一つ変更すると、モデルの式から初期速度なども連動して作り直す必要があります。比較では、計算開始からの時刻だけでなく、近点からの経過時間をそろえました。

外側粒子割合は、図の見た目を補う簡単な指標です。ただし、閾値と測定時刻に依存し、非束縛率や質量損失率を直接意味しません。PP と PR の差は、今回試した閾値や seed を変えても残りました。一方、近点距離と質量比の系列では、この割合が単調に変わるとは限りませんでした。結果を予想に合わせて言い換えず、何を固定し、何を変え、何を測ったかに戻って解釈します。

本章の到達点は、完成したシミュレーションを「実行する」だけでなく、問いを立て、条件をそろえて比較し、数値的な確認とモデルの限界を含めて短い研究として説明することです。次の第7章では、比較実験の枠組みを保ちながら、モデルそのものへ新しい物理として動力学的摩擦を加えます。

演習

演習 6.1　確認：比較条件を整理する　回転方向、近点距離、質量比の系列について、直接変える入力、固定する入力、連動して計算し直す初期値を挙げてください。質量比系列が、第 2 章の全質量固定の実験とは異なる理由も説明してください。

演習 6.2　確認：内部回転だけを反転する　円盤 2 の星の速度を $2\boldsymbol{V}_2 - \boldsymbol{v}_i$ へ変える理由を説明してください。同じ seed から生成した PR と一致するか、中心軌道と円盤 1 の星が PP と同じかも確認してください。この一致を一般の自己重力銀河へ適用できるでしょうか。

演習 6.3　応用：時刻のそろえ方　表6.4の近点後約 0.20 Gyr と、開始から 0.60 Gyr の比較を再計算してください。$q = 0.1$ のモデルで、$t = 0.47\,\mathrm{Gyr}$ は近点後何 Gyr に相当するでしょうか。同じ保存番号を比べることと、同じ進化段階を比べることの違いを述べてください。

演習 6.4　応用：指標と標本を変える　PP–PR の円盤 2 について、表6.5と表6.6を再計算してください。閾値の変更と seed の変更が、それぞれ何を調べる操作か説明してください。三つの seed の結果だけから「すべての銀河でこの差が成り立つ」と結論してよいでしょうか。

演習 6.5　応用：刻みと結論の精度　近点 5 kpc のケースを 0.05、0.025 Myr で計算し、$\delta_{\max}$、δ_{rms}、Δf_k を比べてください。$\Delta f_k = 0$ でも粒子位置が一致しない理由を述べてください。さらに一段細かい刻みを加える場合には、比較する時刻と初期配列をどうそろえるか説明してください。

演習 6.6　最終課題：一つの問いを報告する　本章の三つの系列の一つを選び、表6.8の順で結果をまとめてください。同じ表示範囲の図、一つの定量指標、少なくとも一つの数値的な確認を含めます。結論には、用いた近似、比較時刻、指標の定義を反映させてください。

演習 6.7　応用：有効長さと中心間重力　$b = 3\,\mathrm{kpc}$ を固定し、$D = 5, 8, 12\,\mathrm{kpc}$ で式(6.5)を計算してください。$D \gg b$ の極限も確かめてください。この比だけで、星が受ける潮汐力の大きさや、近点系列の外側割合の順位を決められるでしょうか。また、b を小さくすることと、時間刻みを小さくすることの違いを説明してください。

第 7 章　動力学的摩擦で軌道減衰を調べる

第 6 章までは、二つの銀河中心を保存的な重力だけで動かしました。そのため、星の軌道が大きく変化しても、中心軌道のエネルギーを失わせる仕組みはなく、近接通過した二つの中心は再び離れていきます。本章では、ここへ新しい物理として Chandrasekhar 動力学的摩擦を加え、衛星銀河の軌道がどのように減衰するかを調べます。

ここでは第 5–6 章の等質量遭遇へ摩擦をそのまま追加しません。原点に固定した Plummer 主銀河と、質量比 1:10 の衛星銀河へモデルを切り替えます。衛星中心には主銀河の重力と速度依存の摩擦加速度を与え、時間積分には第 1 章で学んだ RK4 を使います。最後に衛星の周りへ質量ゼロのテスト粒子を置きますが、摩擦を星へ直接加えることはしません。

この章で学ぶこと

- Chandrasekhar 動力学的摩擦の各項の意味と、この近似で省いている背景応答を説明できます。
- 速度依存力を含む衛星中心の運動方程式を RK4 で積分し、摩擦あり・なしを比較できます。
- エネルギーと角運動量の物理的な減少を、保存系の数値誤差と区別できます。
- 時間刻み、Coulomb 対数、テスト粒子の追加を別々の変更として評価できます。

7.1 近接通過と軌道減衰の違いを考える

7.1.1 近接通過と軌道減衰を区別する

ここまでのモデルでは、星が遠くへ移動しても中心へ反作用を返しません。中心軌道のエネルギーも保存されるため、近接通過後に自動的に合体へ移る仕組みはありません。そこで本章では、主銀河の中を動く衛星銀河に、動力学的摩擦による減速を加えます。

動力学的摩擦は、単なる接触摩擦ではなく、背景の多数の粒子との重力相互作用に由来する効果です。Chandrasekhar の動力学的摩擦は、銀河力学で用いられる標準的な近似の一つです。[2] 本書では背景粒子の応答を直接計算せず、その効果を近似的な加速度として与えます。したがって、重力 wake の形や、背景が受け取るエネルギーを直接解いているわけではありません。

7.1.2 等質量の二銀河モデルから切り替える

本章では、原点に固定した Plummer 主銀河と、質量比 1:10 の衛星銀河を使います。これは本書の基本モデルとは別に用意した、軌道減衰を調べるための簡略モデルです。表7.1のように、第 5–6 章の二銀河遭遇とは初期軌道も重力源の扱いも違います。

表 7.1　本章の動力学的摩擦モデル。第 5–6 章の等質量・ゼロエネルギー遭遇とは別の設定です。

項目	設定
主銀河	$M_{\rm host} = 10^{11} M_\odot$、$a_{\rm host} = 3\,\mathrm{kpc}$、原点に固定
衛星銀河	$M_{\rm sat} = 10^{10} M_\odot$、$a_{\rm sat} = 1.5\,\mathrm{kpc}$、質量・形は一定
中心への重力	主銀河の場を衛星中心の位置で評価する近似
初期軌道	$r_{\rm s}(0) = 20\,\mathrm{kpc}$、主銀河場の円運動速度
摩擦の係数	$\ln\Lambda = \ln(11) = 2.397895\ldots$ を固定
中心の時間積分	RK4、1 Myr、1.50 Gyr まで（1500 ステップ）

主銀河は衛星から引かれて動くことも、加熱されることもありません。有限サイズの衛星への重力を、その密度全体で平均しているわけでもありません。このため、第 5 章の相互作用を持つ二つの中心を、そのまま固定したとは説明しません。本章の目的は衛星の軌道減衰を見ることであり、自己無撞着な minor merger の定量的予測ではありません。

星を置かずに衛星中心だけを動かす段階では、$a_{\rm sat}$ は中心軌道の式へ入りません。第7.5節の追加実習で、衛星の周りの星が受ける Plummer 場に使います。また、本章には第 5 章の中心間有効長さ b はありません。

```
df_params = dict(m_host=1.0e11, a_host=3.0, m_sat=1.0e10, a_sat=1.5,
                 ln_lambda=float(np.log(11.0)), friction=True)
df_r0, df_t_end, df_n_steps = 20.0, 1.50, 1500
df_vc = circular_speed_plummer(df_r0, df_params["m_host"], df_params["a_host"])
df_initial = np.array([[df_r0, 0.0, 0.0, df_vc]])
```

7.2 Chandrasekhar 公式から摩擦加速度を求める

7.2.1 密度と速度分散を同じ主銀河から与える

衛星の位置から主銀河中心までの距離を $r_{\rm s} = |\boldsymbol{R}_{\rm s}|$ とします。主銀河の密度には第 3 章の Plummer 式を使います。背景の 1 次元速度分散には、等方的な Plummer 球に対応する次の式を与えます。

$$\rho(r) = \frac{3M_{\rm host}}{4\pi a_{\rm host}^3}\left(1 + \frac{r^2}{a_{\rm host}^2}\right)^{-5/2}, \tag{7.1}$$

$$\sigma^2(r) = \frac{GM_{\rm host}}{6\sqrt{r^2 + a_{\rm host}^2}}. \tag{7.2}$$

密度・ポテンシャルの定義は既習のものです。[2] 速度分散は新しく与える背景モデルの情報であり、円盤の円運動速度でも、3 次元の速度の大きさでもありません。軌道は平面内でも、背景の速度分布は 3 次元で等方的とします。定義した式を用いることに集中し、分布関数からの導出は本章では行いません。

```
def host_density(radius, m_host, a_host):
    return 3.0 * m_host / (4.0 * np.pi * a_host**3) * (1.0 + (radius /
    ↪  a_host)**2)**(-2.5)


def host_sigma(radius, m_host, a_host):
    return np.sqrt(G * m_host / (6.0 * np.sqrt(radius**2 + a_host**2)))
```

局所の速度分布には、この σ を持つ Maxwell 分布を近似として使います。等方的 Plummer 球の速度分布全体が、各位置で厳密に Maxwell 分布になっているという意味ではありません。密度と速度分散を解析式で与える近似と、速度分布の形を置き換える近似を区別します。

7.2.2 衛星の速度と逆向きの摩擦を確認する

主銀河の背景は平均として静止しているとします。この座標系での衛星速度を $\boldsymbol{V}_{\rm s}$、その大きさを $V_{\rm s}$ と書きます。背景粒子の質量が衛星質量に比べて十分小さい場合の、局所 Maxwell 近似による Chandrasekhar 加速度を

$$\boldsymbol{a}_{\rm df} = -4\pi G^2 M_{\rm sat}\rho \ln\Lambda\, B(X)\frac{\boldsymbol{V}_{\rm s}}{V_{\rm s}^3}, \qquad X = \frac{V_{\rm s}}{\sqrt{2}\sigma} \tag{7.3}$$

とします。ここで

$$B(X) = \mathrm{erf}(X) - \frac{2X}{\sqrt{\pi}}e^{-X^2}, \qquad \mathrm{erf}(X) = \frac{2}{\sqrt{\pi}}\int_0^X e^{-u^2}\,du \tag{7.4}$$

です。本章ではこの標準形を短い Python 関数として実装し、誤差関数には標準ライブラリの `math.erf` を使います。[2]

ρ と σ は、各 RK4 段階での衛星位置から計算します。$\ln\Lambda$ は、重力散乱で考える距離範囲に関係する係数です。本章では、本章の標準設定として $\ln(1+M_{\rm host}/M_{\rm sat}) = \ln 11$ を代表値として固定します。この質量比による選択が、Plummer 球に対して厳密に導かれたものではありません。

もう少し詳しく：Coulomb 対数の処方は一つではありません

$\ln\Lambda$ を一定にする方法のほか、衛星の大きさ、距離、速度などで変える処方もあります。Coulomb 対数には、衛星の大きさ、距離、速度などを考慮した複数の処方があります。[2] 本章ではそれらを追加せず、式(7.3)の $\ln\Lambda$ を一定値として固定します。係数の違う表式を混ぜないことが重要です。

図7.1では、直接計算していない背景応答と、実装する加速度を別枠にしました。摩擦の矢印は衛星中心に置かれており、後で追加する星に直接掛ける項ではありません。

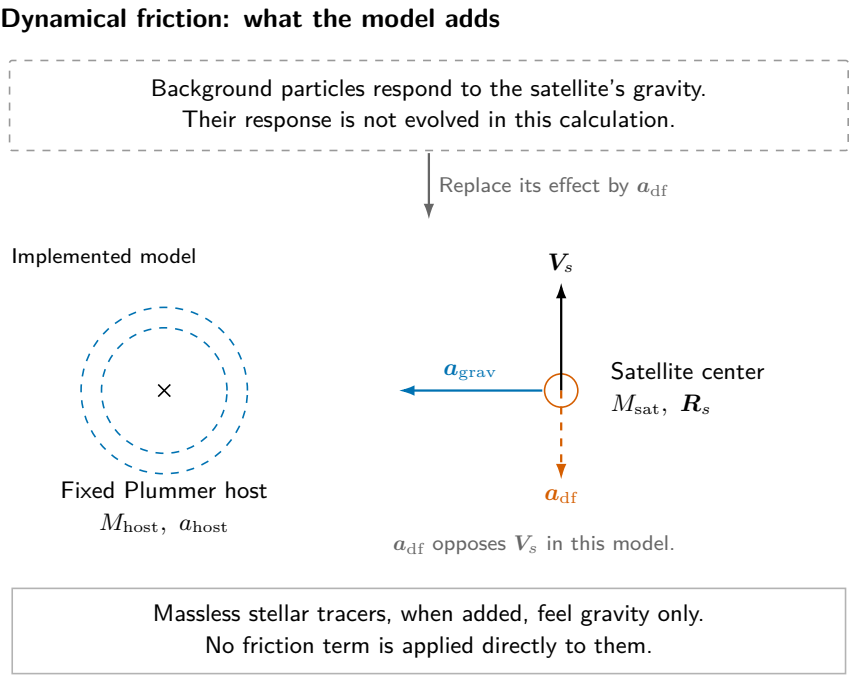


図 7.1　背景応答と実装する摩擦項の区別。上段の背景応答を直接計算する代わりに、下段では衛星中心へ $\boldsymbol{a}_{\rm df}$ を与えます。静止した平均背景に対して、摩擦は衛星速度と逆向きです。追加する質量ゼロの星には摩擦を直接掛けません。矢印の長さは力の実測値ではありません。

7.2.3 速度 0 の見かけの特異点を避ける

式(7.3)には $V_{\rm s}^{-3}$ がありますが、低速では $B(X)$ も X^3 に比例して 0 へ向かいます。式(7.4)を小さい X で展開すると

$$\frac{B(X)}{X^3} = \frac{4}{3\sqrt{\pi}}\left(1 - \frac{3X^2}{5} + \frac{3X^4}{14} + \cdots\right) \tag{7.5}$$

となります。したがって、低速極限では有限な係数を持つ $-\boldsymbol{V}_{\rm s}$ 型の加速度になり、$\boldsymbol{V}_{\rm s} = 0$ なら摩擦も 0 です。

コードでは B/X^3 を返す関数を用意し、$X < 10^{-3}$ でこの級数を使います。それ以外では誤差関数を直接評価します。これは摩擦の発散を隠すために速度へ小さな数を足す操作ではなく、同じ式の連続な低速極限を計算するためです。級数の導出全体や特殊関数の実装は、本書の主要課題にはしません。

```
def maxwell_factor_over_x3(x):
    """[erf(x)-2x exp(-x^2)/sqrt(pi)]/x^3 の連続な低速極限を使います。"""
    if not np.isfinite(x) or x < 0.0:
        raise ValueError("x must be finite and nonnegative.")
    if x < 1.0e-3:
        return 4.0 / (3.0 * np.sqrt(np.pi)) * (1.0 - 0.6 * x**2 + 3.0 * x**4 / 14.0)
    return (erf(x) - 2.0 * x * np.exp(-x*x) / np.sqrt(np.pi)) / x**3
```

```
def chandrasekhar_acceleration(pos, vel, params):
    radius, speed = np.linalg.norm(pos), np.linalg.norm(vel)
    rho = host_density(radius, params["m_host"], params["a_host"])
    sigma = host_sigma(radius, params["m_host"], params["a_host"])
    scale_v = np.sqrt(2.0) * sigma
    x = speed / scale_v
    gamma = 4.0 * np.pi * G**2 * params["m_sat"] * rho * params["ln_lambda"]
    gamma *= maxwell_factor_over_x3(x) / scale_v**3
    return -gamma * vel
```

gamma は $\boldsymbol{a}_{\rm df} = -\gamma \boldsymbol{V}_{\rm s}$ とまとめた係数で、一般には位置・速度に依存します。γ の単位は時間の逆数です。第 1 章の速度比例抵抗で使った一定の係数と、同じ関数形ではありません。scale_v= $\sqrt{2}\sigma$ を使い、$B/V^3 = (B/X^3)/(\sqrt{2}\sigma)^3$ として評価しています。

7.3 RK4 で軌道減衰を調べる

7.3.1 運動方程式の右辺に動力学的摩擦を加える

衛星中心の運動方程式は

$$\frac{d\boldsymbol{R}_{\rm s}}{dt} = \boldsymbol{V}_{\rm s}, \tag{7.6}$$

$$\frac{d\boldsymbol{V}_{\rm s}}{dt} = -\frac{GM_{\rm host}\boldsymbol{R}_{\rm s}}{(r_{\rm s}^2 + a_{\rm host}^2)^{3/2}} + \boldsymbol{a}_{\rm df}(\boldsymbol{R}_{\rm s}, \boldsymbol{V}_{\rm s}) \tag{7.7}$$

です。重力と摩擦を合わせた加速度が速度にも依存します。位置を固定して単純に $\Delta t \boldsymbol{a}$ を加える、既習の陽的 KDK の kick をそのまま流用しません。第 1 章の RK4 を使い、各段階で位置と速度を一緒に予測します。

```
def df_center_rhs(t, state, params):
    if state.shape != (1, 4):
        raise ValueError("The center-only RHS requires shape (1, 4).")
    pos, vel = state[0, :2], state[0, 2:]
    acc = acceleration_plummer(pos[None, :], np.zeros(2), params["m_host"],
    ↪  params["a_host"])[0]
    if params["friction"]:
        acc += chandrasekhar_acceleration(pos, vel, params)
    return np.array([[vel[0], vel[1], acc[0], acc[1]]])
```

```
def rk4_step(rhs, t, state, dt, params):
    k1 = rhs(t, state, params)
    k2 = rhs(t + 0.5 * dt, state + 0.5 * dt * k1, params)
    k3 = rhs(t + 0.5 * dt, state + 0.5 * dt * k2, params)
    k4 = rhs(t + dt, state + dt * k3, params)
    return state + dt * (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
```

RK4 の 4 段階の構造は第 1 章と同じです。最後の引数を背景・衛星のパラメータ辞書に変えただけです。中心だけの状態は形 $(1, 4)$ で、各行の順は $[x, y, v_x, v_y]$ です。摩擦を切る場合は、初期値と RK4 の刻みを保ち、`friction=False` とします。摩擦あり・なしの差へ、別の積分器の差を混ぜません。

速度依存力を扱えるのが RK4 だけ、という意味ではありません。分割法や陰的な更新を組み合わせる方法もありますが、ここでは既習の主要手法を再利用します。RK4 は 1 ステップで 4 回の右辺評価を行うので、中心だけの 1500 ステップでは 6000 回です。

7.3.2 摩擦なしの円軌道を基準にする

同じ初期値から、摩擦あり・なしを 1.50 Gyr まで計算します。摩擦なしでは、衛星中心は主銀河の静的な場の円軌道を回り、半径はほぼ 20 kpc のままです。摩擦ありでは、今回の計算の最終半径は 7.798196 kpc になりました。

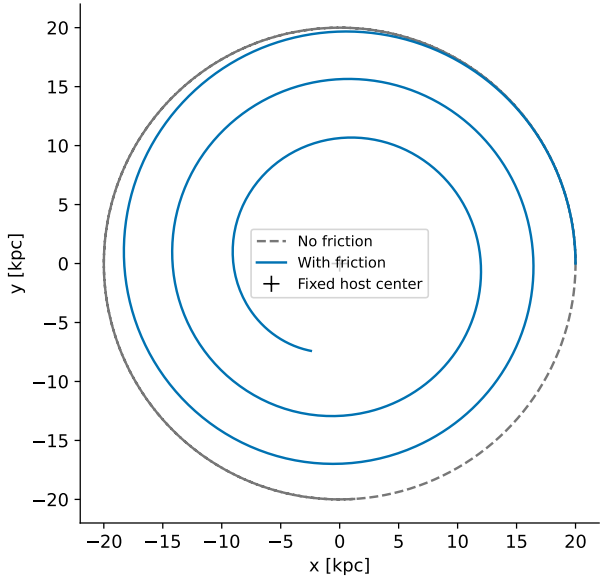


図 7.2　固定 Plummer 主銀河中の衛星中心軌道。灰色破線は摩擦なし、青実線は摩擦ありです。主銀河中心は固定で、主銀河への反作用は計算しません。

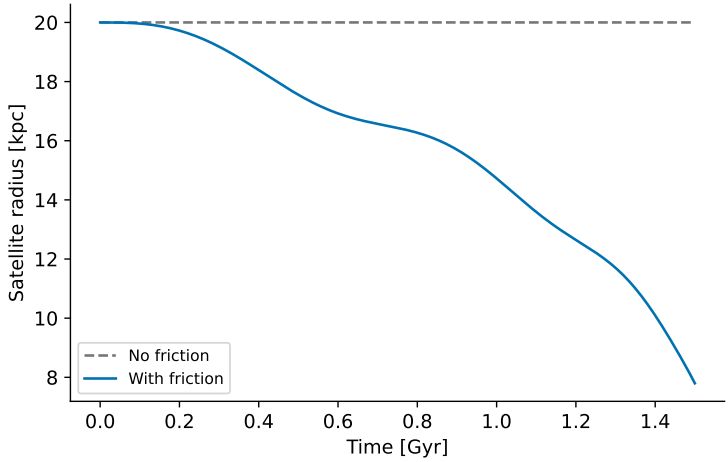


図 7.3　衛星中心半径の比較。$\ln\Lambda = \ln 11$、初期半径 20 kpc、RK4 の刻み 1 Myr です。この半径の減少はモデル内での軌道減衰であり、自己無撞着な銀河合体の完了判定ではありません。

本節では 1.50 Gyr で計算を終えます。$r_{\rm s} = 0$ に達するまで積分して、その時刻を合体時刻と呼ぶことはしません。衛星の質量損失、主銀河の変形、合体後の緩和が含まれていないからです。

7.3.3 保存しない量の変化率を調べる

衛星中心の比エネルギーと比角運動量を

$$\varepsilon_{\rm s} = \frac{V_{\rm s}^2}{2} - \frac{GM_{\rm host}}{\sqrt{r_{\rm s}^2 + a_{\rm host}^2}}, \qquad \ell_{z,{\rm s}} = X_{\rm s} V_{y,{\rm s}} - Y_{\rm s} V_{x,{\rm s}} \tag{7.8}$$

とします。主銀河のポテンシャルが時間に依存しないので、式(7.7)から

$$\frac{d\varepsilon_{\rm s}}{dt} = \boldsymbol{V}_{\rm s} \cdot \boldsymbol{a}_{\rm df} = -\gamma V_{\rm s}^2 \le 0, \qquad \frac{d\ell_{z,{\rm s}}}{dt} = -\gamma \ell_{z,{\rm s}} \tag{7.9}$$

を得ます。摩擦なしのときだけ両者は保存量です。$\ell_{z,{\rm s}} > 0$ の本例では、摩擦ありで角運動量も減少します。速度の大きさや運動エネルギー単独まで単調に減るとは限りません。中心へ近づくと、より深いポテンシャルの中を運動するためです。

```
def df_diagnostics(history, params):
    pos, vel = history[:, 0, :2], history[:, 0, 2:]
    radius = np.linalg.norm(pos, axis=1)
    potential = -G * params["m_host"] / np.sqrt(radius**2 + params["a_host"]**2)
    energy = 0.5 * np.sum(vel**2, axis=1) + potential
    angular = pos[:, 0] * vel[:, 1] - pos[:, 1] * vel[:, 0]
    power = np.zeros(len(history))
    if params["friction"]:
        for j in range(len(history)):
            power[j] = np.dot(vel[j], chandrasekhar_acceleration(pos[j], vel[j], params))
    return radius, energy, angular, power
```

摩擦なしの最大相対誤差は、エネルギーで 6.42×10^{-12}、角運動量で 3.35×10^{-12} でした。摩擦ありの終時刻では $\varepsilon_{\mathrm{s}}(t)/|\varepsilon_{\mathrm{s}}(0)| = -2.714378$、$\ell_{z,\mathrm{s}}(t)/\ell_{z,\mathrm{s}}(0) = 0.562525$ です。後者の変化を、保存系の数値誤差として扱わないことが重要です。

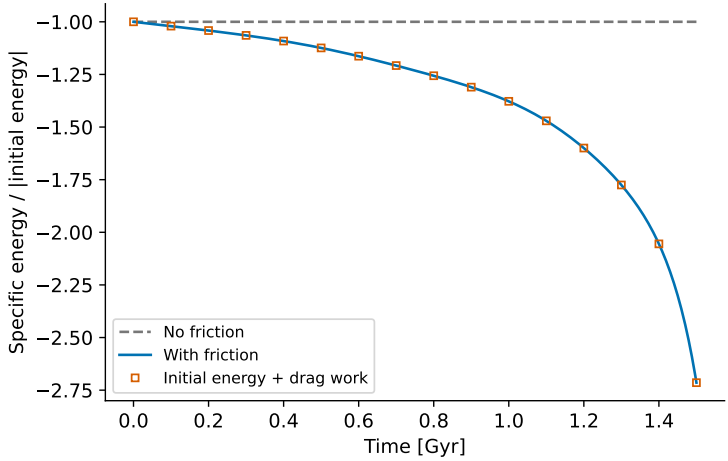


図 7.4　衛星の比エネルギー。四角は初期エネルギーへ、保存された摩擦の仕事率を台形則で積分した値を足したものです。減少の符号と大きさが式(7.9)と整合するかを確かめます。

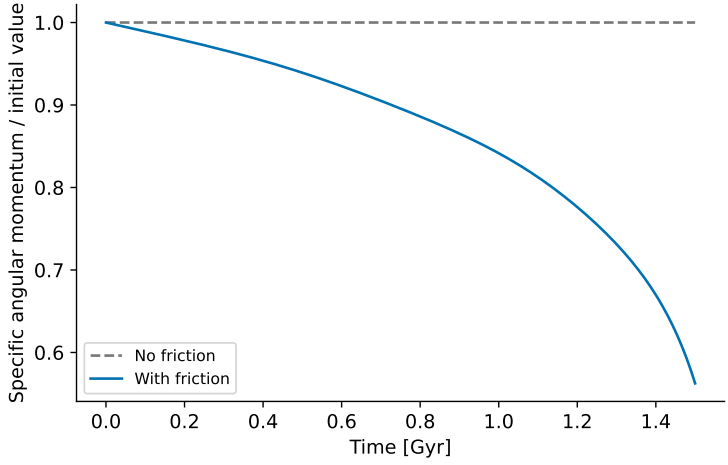


図 7.5　衛星の比角運動量。摩擦なしではほぼ一定、摩擦ありでは正の初期値から減少します。背景が受け取った角運動量を別の粒子で測定しているわけではありません。

ここでエネルギーを受け取る背景の自由度は、実際には計算していません。式(7.9)は衛星に課した運動方程式の収支です。「衛星と主銀河を合わせた全エネルギーを数値的に保存できた」と解釈しないようにします。

7.4 数値誤差とモデルの近似を区別する

7.4.1 摩擦ありでも刻みを細かくする

RK4 の刻みを 4、2、1、0.5 Myr にして、同じ初期値から同じ 1.50 Gyr まで計算します。隣接する二つの計算を粗い側の保存時刻にそろえ、衛星中心の位置差を

$$\delta_{\mathrm{s}}(h) = \frac{\max_j \left| \boldsymbol{R}_{\mathrm{s},h}(t_j) - \boldsymbol{R}_{\mathrm{s},h/2}(t_j) \right|}{20\,\mathrm{kpc}} \tag{7.10}$$

で測ります。主要な誤差が Ch^4 なら、隣接する差の比は 16 へ近づくはずです。

表 7.2　摩擦ありの RK4 の刻み試験。「次行との差」は式(7.10)です。最終行には比較相手がありません。e_{work} は後処理の仕事積分を含む別の指標です。

N_{step}	Δt [Myr]	最終半径 [kpc]	次行との δ_{s}	差の比	e_{work}
375	4.0	7.79814530	6.29×10^{-6}	—	1.91×10^{-4}
750	2.0	7.79819273	3.72×10^{-7}	16.91863	4.89×10^{-5}
1500	1.0	7.79819554	2.25×10^{-8}	16.48324	1.23×10^{-5}
3000	0.5	7.79819571	—	—	3.07×10^{-6}

細かい側の差の比は 16.48324 で、16 へ近づいています。一方、これらの差は真の軌道との誤差を直接測ったものではありません。また、数値解が収束しても、Chandrasekhar 近似自体の物理的な正確さが同じ精度で保証されるわけではありません。

7.4.2 仕事の数値積分に由来する誤差を評価する

式(7.9)を時間積分すれば

$$\varepsilon_{\mathrm{s}}(t) - \varepsilon_{\mathrm{s}}(0) = \int_0^t \boldsymbol{V}_{\mathrm{s}} \cdot \boldsymbol{a}_{\mathrm{df}}\, dt' \tag{7.11}$$

です。右辺を、保存した仕事率から既習の台形則で近似します。この累積仕事を $\widehat{W}(t)$ とし、

$$e_{\mathrm{work}} = \max_j \frac{\left| \varepsilon_{\mathrm{s}}(t_j) - \varepsilon_{\mathrm{s}}(0) - \widehat{W}(t_j) \right|}{\left| \varepsilon_{\mathrm{s}}(0) \right|} \tag{7.12}$$

を確認します。標準の 1 Myr では 1.23×10^{-5} でした。

表7.2では、この残差は刻みを半分にするとおよそ 1/4 になります。位置の RK4 誤差が 4 次でも、後処理の合成台形則は 2 次の誤差を持つためです。**仕事積分の残差だけを見て、RK4 が 2 次へ低下したとは判断しません**。第 3・4 章と同じく、どの処理がどの誤差を持つかを分けます。

7.4.3 摩擦係数の選び方による違いを比較する

$\ln\Lambda$ を 1、2、$\ln 11$ へ変え、その他を固定します。これは刻みの試験ではなく、摩擦モデルの強さを変えた比較です。

表 7.3　固定 Coulomb 対数を変更した中心軌道。初期半径、初期速度、主銀河、衛星質量、RK4 の刻み 1 Myr は共通です。

$\ln\Lambda$	1.50 Gyr 後の半径 [kpc]
1.000000	17.063407
2.000000	12.143812
2.397895	7.798196

今回の値では、係数が大きいほど 1.50 Gyr 後の半径は小さくなりました。しかし、刻みをいくら細かくしても、どの Coulomb 対数が物理的に最適かが自動的に決まるわけではありません。係数の処方、局所 Maxwell 近似、背景の固定、衛星の質量一定という仮定が残ります。

もう少し詳しく：中心部までの外挿には注意します

Plummer 球の中心側では、背景密度がほぼ一定になり、衛星の軌道半径も背景の代表的な長さに近づきます。本書の固定背景・局所 Maxwell 近似・固定 Coulomb 対数による Chandrasekhar 公式は、この領域まで自己無撞着な応答を計算するモデルではありません。したがって、中心到達時刻を高精度な合体時刻として予測しません。これは計算を細かくすることだけでは解決しない、モデルの適用範囲の問題です。

7.5 衛星の周りの星も動かす

中心の軌道減衰を確認した後、衛星の周りへ 500 個のテスト粒子を置けます。初期円盤は $R_{\mathrm{d,sat}} = 1\,\mathrm{kpc}$、$R_{\mathrm{max,sat}} = 4\,\mathrm{kpc}$ とし、衛星の Plummer 場に対する円運動速度を与えます。生成法は第 4 章と同じです。位置と速度の両方へ、衛星中心の初期値を足します。

先頭 1 行を衛星中心、残りを星とした状態配列を作ります。原点の主銀河は固定しているため、可動中心として配列へ追加しません。星の加速度は

$$\frac{d\boldsymbol{v}_i}{dt} = \boldsymbol{a}_{\mathrm{host}}(\boldsymbol{r}_i) + \boldsymbol{a}_{\mathrm{sat}}(\boldsymbol{r}_i - \boldsymbol{R}_{\mathrm{s}}) \tag{7.13}$$

です。Chandrasekhar 摩擦は、この式には直接加えません。摩擦が変えた衛星中心の運動を通じて、星が受ける時間依存の場が変わります。

```
def df_tracer_rhs(t, state, params):
    deriv = np.empty_like(state)
    deriv[:, :2] = state[:, 2:]
    deriv[0] = df_center_rhs(t, state[:1], params)[0]
    pos = state[1:, :2]
    deriv[1:, 2:] = acceleration_plummer(pos, np.zeros(2), params["m_host"],
    ↪  params["a_host"])
    deriv[1:, 2:] += acceleration_plummer(pos, state[0, :2], params["m_sat"],
    ↪  params["a_sat"])
    return deriv
```

`df_center_rhs` を先頭行へ使い、残りの行に二つの重力場を加えます。RK4 の各段階では、予測された衛星位置と星の位置を一緒に使います。衛星中心だけ RK4、星だけ KDK として同期させる方法も考えられますが、本書では別の同期処理を追加せず、本章の全配列を RK4 で進めます。

この追加実習では、中心だけの 1 Myr より細かい 0.25 Myr を標準にします。1.50 Gyr まで 6000 ステップ、10 Myr ごとに 151 状態を保存します。同じ初期値を 0.125 Myr でも計算したとき、全保存時刻・全星の最大位置差は $R_{\mathrm{d,sat}}$ 単位で 6.30×10^{-4}、二乗平均平方根は 7.70×10^{-6} でした。二つの数値解の差として記録し、自己無撞着な衛星崩壊の精度とは解釈しません。

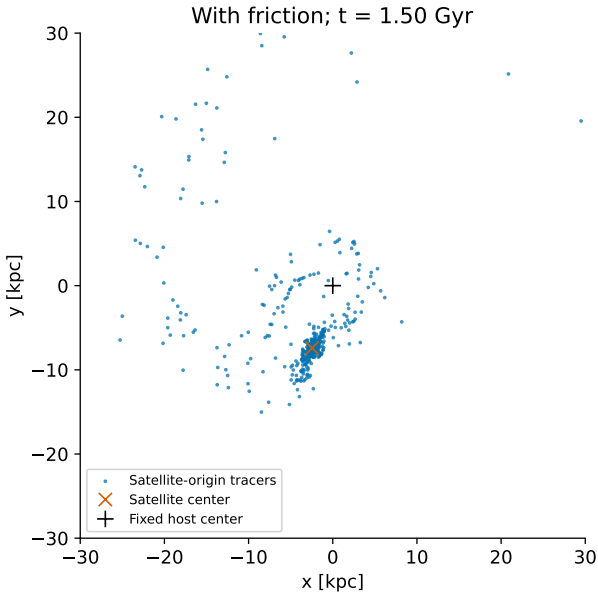


図 7.6　摩擦ありの衛星と、その出身星の 1.50 Gyr 後の位置。青点は質量ゼロのテスト粒子です。星が広がっても、衛星の M_{sat} や a_{sat} は変更していません。表示範囲は両軸とも ±30 kpc で、範囲外の粒子も計算には保持しています。

星が離れても、衛星質量が減り、それに応じて摩擦が弱まるという処理はありません。図7.6を衛星の自己無撞着な質量損失計算とは呼べません。さらに、有限距

離から与えた衛星円盤は、主銀河を含む合成場の厳密な平衡でもありません。この追加実習の主な到達点は、中心の速度依存力と、それに伴う移動ポテンシャルの更新を同じ RK4 の枠組みで扱うことです。

7.6 本章のまとめ

本章では、固定した Plummer 主銀河の中を回る衛星銀河に Chandrasekhar 動力学的摩擦を加えました。背景密度と速度分散は主銀河の Plummer モデルから与え、局所速度分布を Maxwell 分布で近似し、Coulomb 対数を一定値として扱いました。背景粒子や重力 wake そのものを計算したわけではありません。

摩擦加速度は速度にも依存するため、本書の標準的な KDK をそのまま使わず、第 1 章で学んだ RK4 へ戻りました。摩擦なしでは比エネルギーと比角運動量がほぼ保存され、摩擦ありでは両者が物理的に減少しました。仕事率とエネルギー変化の収支、時間刻みを変えた位置の自己収束を別々に確認しました。

Coulomb 対数を変えることは摩擦モデルの変更であり、時間刻みを細かくする数値精度の変更とは異なります。また、衛星の周りへ星を置いても、星の離脱に応じて衛星質量を減らす処理や、主銀河の変形は含めていません。本章の軌道減衰を、自己無撞着な合体時刻の予測とはみなしません。

演習

演習 7.1　確認：摩擦の向きと低速極限　式(7.3)を $\boldsymbol{a}_{\rm df} = -\gamma \boldsymbol{V}_{\rm s}$ の形にし、γ の単位を求めてください。$\ln\Lambda \geq 0$ で $\boldsymbol{V}_{\rm s}\cdot\boldsymbol{a}_{\rm df} \leq 0$ となること、速度を反転すると摩擦も反転すること、速度 0 で有限な 0 を返すことを、コードでも確認してください。

演習 7.2　確認：摩擦なしを基準にする　同じ初期値と RK4 の刻みで、摩擦あり・なしを計算してください。摩擦なしで半径、比エネルギー、比角運動量がほぼ一定になることを確認し、摩擦ありでそれらが変化する場合との意味の違いを説明してください。

演習 7.3　応用：軌道減衰と数値誤差　摩擦ありの仕事積分の残差と、隣接する RK4 解の位置差を時間刻みで比較してください。仕事残差が約 2 次、位置差が約 4 次の振る舞いを示し得る理由を、それぞれに使っている数値算法から説明してください。

演習 7.4　応用：Coulomb 対数を変える　$\ln\Lambda = 1, 2, \ln 11$ を比べ、1.50 Gyr 後の衛星半径を表7.3から確認してください。この変更を時間刻みの収束試験と呼べない理由を述べてください。また、係数を大きくするほど常に現実の銀河をよく再現する、と結論できるでしょうか。

演習 7.5　発展：衛星の星とモデルの限界　第7.5節の 500 粒子計算を再実行し、0.25 Myr と 0.125 Myr の結果を比較してください。星が衛星中心から離れても

$M_{\rm sat}$ を一定にしていることが、軌道減衰と質量損失の解釈にどのような制限を与えるか説明してください。主銀河の応答を計算していないことも含め、本章から決められない量を挙げてください。

演習の解答・着眼点

第 1 章から第 7 章の演習について、数値的な答えと考えるべき点をまとめます。完全な Python コードは完成版 Notebook で確認してください。条件を変更した場合は、ここに示す標準値をそのまま期待値にせず、その条件で計算と確認をやり直してください。

第 1 章　Python で天体の運動を計算する

1.1　$v_c = 207.386\,\mathrm{km\,s^{-1}} \simeq 212.097\,\mathrm{kpc\,Gyr^{-1}}$、$T = 0.296242\,\mathrm{Gyr}$ です。比エネルギーは $\varepsilon_0 = -2.2492 \times 10^4\,\mathrm{kpc^2\,Gyr^{-2}}$、比角運動量は $\ell_{z,0} = 2.1210 \times 10^3\,\mathrm{kpc^2\,Gyr^{-1}}$ です。

1.2　最初の kick には $\boldsymbol{a}(\boldsymbol{r}^n)$、最後の kick には $\boldsymbol{a}(\boldsymbol{r}^{n+1})$ を使います。最後の加速度を再利用するので、初回 1 回と各ステップ 1 回の計 201 回です。$\boldsymbol{v}^{n+1/2}$ を使うと位置と速度の時刻がそろいません。整数時刻の速度は、最後の半 kick を加えた $\boldsymbol{v}^{n+1}$ です。

1.3　評価時刻はどちらも $t_n + \Delta t/2$ ですが、予測状態はそれぞれ $\boldsymbol{y}^n + (\Delta t/2)\boldsymbol{k}_1$ と $\boldsymbol{y}^n + (\Delta t/2)\boldsymbol{k}_2$ です。第 2 段階では、位置の変化率にも式(1.22)で予測した速度を使います。

1.4　leapfrog の最大位置誤差は順に 3.44×10^{-2}、8.63×10^{-3}、2.16×10^{-3} でした。RK4 では 6.16×10^{-5}、3.05×10^{-6}、1.65×10^{-7} でした。細かい刻み側へ進むと、誤差比はそれぞれ 4 と 16 へ近づきます。図がほぼ重なっても、同じ時刻の位置を比べれば差を測れます。

1.5　軌道は x 軸について鏡映され、回転方向が逆になります。エネルギーは同じで、角運動量は符号が反転します。この対称性は、成分と符号を正しく扱っているかを確認する方法にもなります。

1.6　KDK は 799 ステップ、RK4 は 200 ステップです。位置誤差はそれぞれ 1.35×10^{-4} と 1.65×10^{-7} でした。同じ時間刻みでの比較とは異なり、ここでは力評価回数をそろえて刻みを変えています。力評価が少ないことだけから、同じコストでの精度まで決まるわけではありません。

1.7　$\gamma > 0$ なら $d\varepsilon/dt \le 0$ です。今回の条件では $\varepsilon(T)/|\varepsilon(0)| \simeq -1.10511$ となりました。初期値の -1 より小さく、エネルギーが減っています。これは抵抗がする仕事による変化で、保存系の数値誤差とは区別します。解答の完全なコードは完成版 Notebook で確認できます。

第 2 章　2 体問題を計算する

2.1　初期位置は $\boldsymbol{R}_1 = (-6.666667, 0)\,\mathrm{kpc}$、$\boldsymbol{R}_2 = (13.333333, 0)\,\mathrm{kpc}$ です。相対円運動速度は $179.601851\,\mathrm{km\,s^{-1}}$ なので、速度はそれぞれ $(0, -59.867284)$、$(0, 119.734568)\,\mathrm{km\,s^{-1}}$ です。内部単位へ換算してから時間積分します。加速度の大きさの比は $1:2$、力の大きさの比は $1:1$ です。

2.2　$e = 0.36$、$a_{\mathrm{orb}} = 14.705882\,\mathrm{kpc}$、$D_{\mathrm{p}} = 9.411765\,\mathrm{kpc}$、$T_{\mathrm{ell}} = 0.431357\,\mathrm{Gyr}$ です。初期動径速度が 0 で、その後に距離が減るため、初期位置は遠点です。標準計算の最小保存距離は $9.412506\,\mathrm{kpc}$ でした。時間積分誤差と、近点時刻を有限の時間間隔で保存する誤差を区別します。

2.3　理論値が $E(0) = 0$ なので、初期値に対する相対誤差は定義できません。丸めによって得た小さな非零値で割っても有効な比較にはなりません。$E_{\mathrm{scale}} = GM_1M_2/D_0$ は、初期の粒子対のポテンシャルエネルギーの大きさです。放物線でも有限な基準になります。

2.4　$q = 1, 0.5, 0.1$ での各天体の初期重心距離は、同じ順に

$$|\boldsymbol{R}_1(0)| = (10.000000, 6.666667, 1.818182)\,\mathrm{kpc},$$
$$|\boldsymbol{R}_2(0)| = (10.000000, 13.333333, 18.181818)\,\mathrm{kpc}$$

です。全質量を固定すれば μ が同じなので、同じ相対初期条件は同じ相対軌道を与えます。M_1 だけを固定する場合は、μ も変わります。

2.5　重心は式(2.9)に従います。配布コードでは、重心の直線からのずれは D_0 で割って約 4.06×10^{-15} でした。系全体の運動エネルギー $M_{\mathrm{tot}}V_{\mathrm{cm}}^2/2$ を増やせば、相対軌道が束縛されていても E を正にできます。束縛性は ε で判断します。

2.6　両天体の位置は初期位置へ、速度は初期速度の符号反転へ戻ります。本実装の最大ずれは、位置を D_0 で割って約 2.90×10^{-14}、速度を v_{c} で割って約 2.86×10^{-14} でした。これは更新の可逆性の確認であり、順方向の軌道が正確であることだけを検証する試験ではありません。位置の解析的な比較と刻み試験も必要です。

2.7　$\varepsilon_0 = (1.6^2/2 - 1)\mu/D_0 = 0.28\mu/D_0 > 0$、$e = 1.6^2 - 1 = 1.56$ です。標準の 1000 ステップで計算した T_{c} 後の距離は $130.721\,\mathrm{kpc}$ でした。双曲線は正の相対エネルギーを持ち、放物線と同じ初期距離から、より大きな速度で遠ざかります。完全な実装と数値出力は完成版 Notebook で確認できます。

第 3 章　銀河の中で星を動かす

3.1　$\rho(0) = 8.8419 \times 10^8 M_\odot\,\mathrm{kpc}^{-3}$、$M(<a)/M = 2^{-3/2} \simeq 0.353553$、$r_h = 3.914298080\,\mathrm{kpc}$ です。$r = 2a$ の円運動速度は $226.475925\,\mathrm{km\,s^{-1}}$ です。内部単位では $231.619684\,\mathrm{kpc\,Gyr^{-1}}$ です。平面内に選んだのは軌道であり、重力を作るモデルは 3 次元の球なので、球殻の体積を使います。

3.2　分点数は 41、幅は $15/40 = 0.375\,\mathrm{kpc}$ です。`integral[j + 1]` は、最初の分点から $j+1$ 番目の分点までの台形の面積の和です。g の単位 $M_\odot\,\mathrm{kpc}^{-1}$ に区間幅を掛けると質量になります。ρ だけを半径で積分しても、質量の単位にはなりません。

3.3　同じ $[0, 5a]$ で細かくすれば、解析的な $M(<5a)$ へ近づきます。一方、幅を固定して範囲を広げる実験では、分点数も増やして中心の分解能を保ちます。解析的な範囲外の質量は、$5a, 10a, 20a$ でそれぞれ約 $(0.057134, 0.014815, 0.003738)M$ です。有限半径の内側の質量が M より少ないことは、積分範囲の定義から生じます。

3.4　$15/2^{k+1} \le 10^{-8}$ より、必要な半減回数は 30 です。最初の区間も $k = 0$ で中点を評価するため、中点評価は 31 回です。数値積分で定義した関数の根と、連続密度の真の半質量半径との違いは、この半減だけでは取り除けません。

3.5　$f = M(<5a)/(2M)$ と置くと、解析的には $r = a/\sqrt{f^{-2/3} - 1}$ です。$f < 1/2$ なので、球全体の半質量半径より小さくなります。数値計算では右辺にも同じ範囲内の積分値を使うか、解析値を使うかを明示してください。前者は両辺の積分誤差を含む、別の検証になります。

3.6　$a = 1.5, 3, 6\,\mathrm{kpc}$ の順に、円運動速度は約 $255.834, 226.476, 159.196\,\mathrm{km\,s^{-1}}$、半質量半径は約 $1.957149, 3.914298, 7.828596\,\mathrm{kpc}$ です。各モデルで円にする場合は初期速度を調整します。同じ初期速度を固定すると、異なる重力場に同じ初期運動を与えた比較になり、一般には全部を円軌道にはできません。

3.7　$v^2 = v_r^2 + \ell_z^2/r^2$ を比エネルギーへ代入し、$v_r = 0$ とすると式(3.29)です。求めた内側の根は $r_p = 3.027147967\,\mathrm{kpc}$ でした。標準計算の最小保存半径は $3.027207825\,\mathrm{kpc}$ です。この差には KDK の時間積分誤差と、保存時刻が近点時刻に一致しない効果が含まれます。ここで使うのは初期値からの解析的な保存量で、数値軌道の途中のエネルギーを毎回使って根を合わせ直す方法ではありません。

完全な実装と数値出力は完成版 Notebook にあります。演習条件を変更した場合は、標準結果の数値をそのまま期待値にせず、その条件で確認をやり直してください。

第 4 章　テスト粒子で円盤銀河を作る

4.1　$\int_0^{R_{\max}} p_R\,dR = 1$ です。$F(R_d) = C(1)/C(5) = 0.275373844$ なので、期待粒子数は約 275 個です。実際の乱数標本ではこの値の周りにばらつきます。落としやすい因子は円環の面積の R です。

4.2　区間 $[1, 3]$ の中点なので $y = 2.5$ です。逆補間では $x = 2$ に戻ります。本章の自作関数へ $y = 4$ を渡すと範囲外として停止します。`np.interp` の既定値では最終点の $x = 3$ を返すので、それを方程式の解とみなしてはいけません。

4.3　今回の初期円盤は有限範囲で規格化すると定義しています。Plummer 球の全質量の定義を変更したわけではありません。標準表の逆補間は 4.843929332 kpc、解析 CDF の二分法による比較値は 4.843928453 kpc です。標本半径の中央値は今回 4.782418 kpc でした。最初の二つの差は表の積分・補間に由来し、標本の中央値は有限粒子数のばらつきも含みます。これは Plummer 球の半質量半径約 3.9143 kpc とは別の量です。

4.4　表4.3の値を再計算できます。ただし、異なる乱数生成環境では初期標本の照合も行います。粒子間重力を計算しないので、N は分布の標本数です。積分の次数は更新式と時間刻みによって決まり、N を増やしても KDK は 2 次のままです。

4.5　終点は同じ更新を通るので同じになります。標準の 3000 ステップでは、保存時刻数は $3001, 301, 61$ 個、履歴容量は $96.032, 9.632, 1.952$ MB です。初期状態と最終状態も含みます。保存しない時刻の誤差の極値を見逃し得るので、出力を間引いた最大値の評価範囲を明示します。

4.6　速度の 2 乗は変わらないので初期エネルギーは同じです。$\boldsymbol{r} \times \boldsymbol{v}$ の符号は反転します。どちらも円軌道なので、連続的な数面密度は定常です。乱数で置いた任意の点の集まりについて、同時刻の粒子配置全体が x 軸の鏡映になるとは限りません。

4.7　接線速度が不足するので、粒子は初期半径から内側へ動き始めます。その後の半径は一般に振動します。今回の 0.6 Gyr 後の半径の中央値は 4.025966 kpc、保存時刻での最大相対エネルギー誤差は 1.55×10^{-5} でした。これは非円軌道の集合であり、自己重力はありません。円軌道の解析位置は比較対象ではないので、保存量と刻みを変えた数値解どうしを使って精度を確認します。

完全な実装と実行結果は完成版 Notebook に含めています。条件を変更した場合は、ここに示した標準値がそのまま期待値になるとは限りません。何を固定し、何を変更したかを記録してください。

第 5 章　2 つの銀河を接近させる

5.1　中心は式(5.8)、星は出身にかかわらず式(5.6)です。$M_1\boldsymbol{A}_1$ と $M_2\boldsymbol{A}_2$ は同じ分母と質量積を持ち、逆符号になります。星は近くへ来た相手からも重力を受けるので、出身番号によって重力源を一つに制限しません。

5.2　相対速度は式(5.17)、角運動量は $h = D_0 U_t$ です。h の単位も内部単位へそろえます。$U_p = h/D_p$ を式(5.10)へ代入すれば、近点でも $\varepsilon = 0$ です。$b \to 0$ では $h^2 = 2\mu D_p$、$U_r^2 = 2\mu/D_0 - U_t^2$ となり、点質量の式へ戻ります。

5.3　正しい変換では元の $\boldsymbol{s}_i, \boldsymbol{w}_i$ へ戻ります。中心速度を足し忘れると、中心に対する速度は $\boldsymbol{w}_i - \boldsymbol{V}_k$ です。すべての星に意図しない一様な速度差が入り、孤立した円軌道の初期条件ではなくなります。

5.4　粒子割合は閾値の片側に何個あるかだけを数えるため、位置が変わっても粒子が閾値を越えなければ同じ値です。細かい側の最大位置差の比は今回 3.97963 でした。終時刻、近点、質量・スケール長などを変えたときには、中心の保存量だけでなく、星の位置と調べたい指標の刻み依存性を再確認します。

5.5　同じ更新を行うので、全粒子の終点と毎ステップの中心履歴は同じです。全履歴の保存時刻数は $12001, 301, 61$ 個になります。履歴の容量は $8 \times 4 \times 2002 \times N_{\mathrm{save}}$ byte なので、それぞれ約 $768.832, 19.283, 3.908$ MB です。粗い出力では、保存していない時刻に起きた星の変化の極値を見逃す可能性があります。

5.6　相対運動は同じで、慣性系全体の等速運動を加えただけです。今回の粒子相対位置の最大差は R_d 単位で 4.50×10^{-10} でした。一方、速度の 2 乗には並進速度との交差項が入るので、慣性系で測った個々のエネルギーの数値が同じである必要はありません。

5.7　星の初期速度を $\boldsymbol{v}_i' = \boldsymbol{V}_2 - (\boldsymbol{v}_i - \boldsymbol{V}_2) = 2\boldsymbol{V}_2 - \boldsymbol{v}_i$ とします。中心の運動は変更しません。今回の f_1, f_2 は $0.425, 0.065$ でした。出身銀河 1 の粒子は同じ二つの場を同じ初期条件で受け、銀河 2 の星から力を受けないので、軌道も変わりません。この違いは、固定形状のポテンシャルと質量ゼロの円盤という近似の中で得た応答です。自己重力、質量損失、ガスなどの効果を含む一般の合体結果まで保証しません。

完全な実装と数値出力は完成版 Notebook に含めています。数値条件を変えた場合は、この章に示した値をそのまま期待値にしないでください。

第 6 章　銀河相互作用を調べる

6.1　回転方向では円盤 2 の内部速度だけを反転します。近点距離では中心初期速度、質量比では中心の重心配分・相対速度と円盤 2 の円運動速度も変わります。本章の質量比系列は M_1 固定なので $M_1 + M_2$ が変わり、第 2 章の全質量固定の比較とは違います。

6.2　$\boldsymbol{v}_i = \boldsymbol{V}_2 + \boldsymbol{w}_i$ から、$\boldsymbol{w}_i$ だけを反転すると式(6.4)です。本実装では中心と円盤 1 の履歴が一致します。これは星からの反作用と星同士の重力を省いているためで、自己重力系への一般化はできません。

6.3　$q = 0.1$ では $t_{\mathrm{p}} = 0.363900\,\mathrm{Gyr}$ なので、$t = 0.47\,\mathrm{Gyr}$ は $\tau = 0.106100\,\mathrm{Gyr}$ です。基準 PP の同じ時刻は $\tau = 0.200100\,\mathrm{Gyr}$ で、約 94 Myr 違います。本章では目標経過時間に最も近い保存時刻を選び、選択のずれも記録します。

6.4　20、30、40 kpc の閾値では PP–PR の差は $0.355, 0.346, 0.276$ でした。seed 42、43、44 では、30 kpc の差は $0.346, 0.349, 0.361$ です。前者は測定方法、後者は有限標本を変えています。三組で傾向が残ることは確認できますが、観測銀河全体に対する統計的な保証ではありません。

6.5　標準出力では、近点 5 kpc の最大差は R_{d} 単位で約 1.25×10^{-2}、二乗平均平方根は約 2.07×10^{-4} です。Δf_k は両方 0 でした。粒子が閾値の同じ側にいれば割合は変わりません。追加計算でも初期配列を再利用し、保存時刻を共通にして粒子番号ごとに比較します。

6.6　一意の数値解答はありません。問い、比較条件、結果、確認、解釈の対応を重視します。非単調な結果が得られた場合も、値を選別せず、測定時刻と閾値の意味に戻って説明してください。数値計算の再現性と、物理モデルの適用範囲を両方示すことが到達目標です。

6.7　比は 5、8、12 kpc の順に $0.631, 0.821, 0.913$ です。$D \gg b$ なら 1 へ近づきます。中心に対する全加速度と、星と中心の加速度差は別の量です。また、軌道の速さや粒子の位相・測定時刻も変わるため、この比だけで f_k の大小は決まりません。b はモデル、Δt は同じモデルを解く数値精度の設定です。完成版 Notebook に、軌道を変更せずこの比を計算する短い実装を収録しています。

第 7 章　動力学的摩擦と軌道減衰

7.1　$\gamma = 4\pi G^2 M_{\mathrm{sat}} \rho \ln\Lambda\, B(X)/V_{\mathrm{s}}^3$ で、単位は Gyr^{-1} です。$B(0) = 0$ で、$X \geq 0$ では $B(X) \geq 0$ なので、$\ln\Lambda \geq 0$ なら $\boldsymbol{V}_{\mathrm{s}} \cdot \boldsymbol{a}_{\mathrm{df}} \leq 0$ です。低速では式(7.5)を使って γ は有限にとどまり、$\boldsymbol{V}_{\mathrm{s}} = 0$ なら加速度も 0 になります。

7.2　摩擦なしの最終半径は約 20.000000 kpc で、比エネルギーと比角運動量の最大相対誤差はそれぞれ 6.42×10^{-12}、3.35×10^{-12} です。摩擦ありの最終半径は 7.798196 kpc です。摩擦ありの保存量の変化を、摩擦なしの場合と同じ数値誤差として扱いません。

7.3　標準の 1 Myr では仕事積分の残差は 1.23×10^{-5} です。後処理に合成台形則を使うため、その残差には 2 次の積分誤差が入ります。一方、RK4 で求めた位置の隣接解差は、細かい側で比が 16.48324 となり、4 次法に対応する 16 へ近づきます。異なる処理の誤差次数を混同しません。

7.4　$\ln\Lambda = 1, 2, \ln 11$ に対する 1.50 Gyr 後の半径は 17.063, 12.144, 7.798 kpc です。Coulomb 対数は摩擦モデルの係数で、時間刻みを変える数値収束とは別です。大きい係数ほど本例では減衰が強くなりますが、どの値が現実的かは背景・衛星構造と摩擦モデルの仮定に依存します。

7.5　0.25 Myr と 0.125 Myr の全保存時刻・全星の最大位置差は $R_{\mathrm{d,sat}}$ 単位で 6.30×10^{-4}、二乗平均平方根は 7.70×10^{-6} でした。星が広がっても M_{sat} を減らさないため、質量損失によって摩擦が弱まる効果はありません。主銀河の変形や wake も直接計算していないので、自己無撞着な合体時刻や合体残骸の構造は本章から決められません。

参考図書

銀河力学・天体力学

[1] Barbara Ryden, *Celestial and Stellar Dynamics*, Cambridge University Press, 2025.
2 体問題、中心力、球対称ポテンシャル、潮汐などを学ぶための入門的な参考書です。本書では、学部生が物理的な意味を確認するときの主参考書として用います。

[2] J. Binney and S. Tremaine, *Galactic Dynamics*, 2nd ed., Princeton University Press, 2008.
銀河ポテンシャル、Plummer モデル、軌道論、恒星系の力学、動力学的摩擦などをより詳しく調べるための標準的な参考書です。

数値計算法

[3] 杉浦洋『数値計算の基礎と応用［新訂版］ 数値解析学への入門 』サイエンス社、2009 年。
浮動小数点数と丸め誤差、補間、台形則、二分法・Newton 法・割線法、Euler 法・Runge–Kutta 法など、本書で使う数値計算法の多くを扱っています。

[4] 伊理正夫・藤野和建『数値計算の常識』共立出版、1985 年。
数値計算で生じる誤差や注意点に加え、逆補間、乱数、特定の確率分布に従う乱数の生成を扱っています。第 4 章の逆変換サンプリングと直接関係します。

[5] 冨阪幸治・花輪知幸・牧野淳一郎編『シリーズ現代の天文学 14 シミュレーション天文学』日本評論社、2007 年。
天文学の数値計算を扱い、leapfrog 法の時間反転対称性・シンプレクティック性と、粒子系で広く使われる理由を説明しています。本書の重力計算における leapfrog の位置づけと対応します。

[6] 幸谷智紀『Python 数値計算プログラミング』講談社、2021 年。
Python 3 と NumPy・SciPy を用いて数値計算を実装しながら学ぶ参考書です。浮動小数点数、補間、数値積分、Euler 法・古典的 Runge–Kutta 法などを扱っており、本書の計算法を Python でさらに確認したい場合の補助参考書とします。

補足文献・公式ドキュメント

[7] Python Software Foundation, `venv`: Creation of virtual environments.
第 0 章の Python 環境構築を補う公式文書です。

[8] Project Jupyter, Installing Jupyter; JupyterLab Documentation.
第 0 章の Notebook とカーネル操作を補う公式文書です。

[9] NumPy Documentation, NumPy: the absolute basics for beginners; `numpy.interp`.
第 4 章で使う線形補間の実装仕様を確認するために参照します。

[10] SciPy Documentation, `PchipInterpolator`.
第 4 章の「もう少し詳しく」で、単調性を保つ区分的三次補間の例を紹介するときにのみ参照します。SciPy は本書の標準計算には必要ありません。

公式ドキュメントの URL は 2026 年 9 月 28 日に確認しました。本書では、数値計算法について和書で扱える範囲を優先して参考図書に挙げています。より高度な常微分方程式や幾何学的積分法へ進む場合は、必要に応じて専門書を追加してください。